B_t+\Delta t,F(B_t,R_t)
\right),
]

where ΠΑΠΑ is the admissibility projection.

“Repair mechanism” is too weak. “Projected dynamical system” or “admissibility projection” is better.
4. It is a hard constraint, not a soft regularizer

The projection makes the post-projection minimum gap exactly:

text

0.05

It is therefore a hard admissibility operation, even if the individual corrections appear small.

A better description is:

A hard constraint enforced through frequent, small numerical corrections.

Calling it a “soft constraint” or “soft regularizer” would be inaccurate.
5. Projection magnitude is small in code units, but not yet dimensionless

The reported magnitude is:
‖Bafter−Bbefore‖F.
‖Bafter−Bbefore‖F.

It should not be directly compared numerically with:
0.05−Gminpre,
0.05−Gminpre,

because those quantities have different meanings and units.

The current result supports:

Projection corrections are small in absolute matrix-norm units and show no evidence of rare, dramatically larger corrections.

But “gentle” should remain provisional until the correction is normalized.

For AIEF-1.4, record:
Mrelative=
Mrelative=

\frac{
|B_{\rm after}-B_{\rm before}|F
}{
\max(|B{\rm before}|_F,\epsilon)
}.
]

An additional useful quantity is:
Mupdate=
Mupdate=

\frac{
|B_{\rm after}-B_{\rm before}|F
}{
\max(|B{\rm before}-B_{\rm previous}|_F,\epsilon)
},
]

which measures how much of the unconstrained update is removed by projection.
6. Projection does not create sparsity by itself

The no-μ result is essential here:

projection remains active;
total weight remains near the cap;
but weights become diffuse across all edges.

Therefore:

the projection enforces a low-total-weight admissible envelope;
μμ creates concentration/pruning within that envelope.

A precise decomposition is:
gap projection → bounded coupling budget,
gap projection → bounded coupling budget,
μ → concentration into a sparse backbone.
μ → concentration into a sparse backbone.

Do not attribute sparse topology solely to the projection.
7. No-gap verifies envelope enforcement, not instability

The clean intervention result is:
Condition Violations Projection Final regime
Baseline Yes Yes Gmin=0.05Gmin=0.05
No-gap Yes No Gmin≪0Gmin≪0

This demonstrates that the projection causally enforces the prescribed envelope within the simulation.

It does not establish dynamical instability without the gap. The no-gap system remains mathematically bounded through decay but enters a very high-weight, dense regime outside the desired dominance set.
8. Shuffle is a driver intervention, not a randomized topology

The code shuffles resonance-driver assignments; it does not directly shuffle BB.

Its result is:

no gap pressure,
no projection,
extremely weak total coupling,
no threshold persistence.

The precise wording is:

Timestep-wise destruction of edge-specific driver identity prevents the adaptive network from reaching either the threshold-selection regime or the gap-constrained regime.

9. Some proposed “polish” is unnecessary

A few suggestions from the second review are not useful:

Displaying N/A for shuffle's conditional projection ratio is cosmetic; NaN is mathematically correct because there are no violations.
Hiding duplicate no-gap pre/post values would remove useful evidence that no projection occurred.
An immediate symmetry assert would stop the script before it could write a complete audit report. The current flag-plus-closure-failure behavior is preferable.
A temporal median projection norm cannot be recovered by the closure script because the simulator saved only per-seed mean and maximum norms. The displayed median is the median across seeds, not the median across timesteps.

Recommended frozen AIEF-1.3 statement

Across two independent 50-seed validation cohorts, AIEF-1.3 produces an exactly binary, temporally frozen baseline topology during the future evaluation window. The gap constraint operates as a hard admissibility projection on essentially every post-boundary update, maintaining the prescribed dominance envelope through small absolute matrix corrections. Removing the projection produces a dense, high-weight regime with strongly negative gap, while timestep-wise destruction of resonance-driver identity prevents the network from reaching the constrained or threshold-selective regime. Active deletion, rather than projection alone, concentrates the bounded coupling budget into a sparse backbone.

Freeze artifact

closure_status.json already provides the machine-readable pass/fail record. A shorter freeze_summary.json is optional and should be generated automatically rather than manually transcribed.

If added, include:

both configuration hashes;
scalar/NPZ file hashes;
validation seed count and overlap;
exact binary status;
projection frequency and magnitude;
no-gap minimum gap;
closure status;
workflow/source versions.

AIEF-1.4 implication

A split at HIST_END now tests maintenance and reconstruction of a mature frozen topology, not initial topology formation.

That should appear explicitly in the AIEF-1.4 protocol. Its primary outcomes should include:

weighted similarity to B_at_split;
late-window threshold retention;
topology reconstruction after resetting BB;
effects of resetting EMA or phase history;
effects of shuffling only the future driver;
relative projection magnitude.

AIEF-1.3 is ready to freeze. The next unresolved question is no longer whether topology persists—it demonstrably does—but which state variables are necessary to maintain or reconstruct that frozen topology.

Think about this: (deck@steamdeck SSRH)$ python AIEF_1_3_closure_diagnostics_updated.py --strict Auditing validation_A Auditing validation_B AIEF-1.3 freeze-grade closure diagnostics ========== CLOSURE PASSED: True Closure checks ---------- expected_cohort_count: True seed_coverage_complete: True cohorts_seed_disjoint: True baseline_matrices_square_and_finite: True baseline_matrices_symmetric: True baseline_diagonal_exact_zero: True baseline_unique_edges_exact_binary: True baseline_unique_edges_tolerant_binary: True baseline_full_matrix_exact_binary: True matrix_scalar_consistent: True projection_metrics_all_finite: True projection_activity_subset_of_violation: True Seed and artifact coverage ---------- validation_A / baseline: scalar=50, P_future=50, complete=True validation_A / shuffle_R: scalar=50, P_future=50, complete=True validation_A / no_delay: scalar=50, P_future=50, complete=True validation_A / no_mu: scalar=50, P_future=50, complete=True validation_A / no_gap: scalar=50, P_future=50, complete=True validation_B / baseline: scalar=50, P_future=50, complete=True validation_B / shuffle_R: scalar=50, P_future=50, complete=True validation_B / no_delay: scalar=50, P_future=50, complete=True validation_B / no_mu: scalar=50, P_future=50, complete=True validation_B / no_gap: scalar=50, P_future=50, complete=True Cross-cohort seed overlap ---------- validation_A vs validation_B: overlap=0 Baseline P_future audit ---------- validation_A: seeds=50, exact_binary=True, tolerant_binary=True, intermediate_exact=0, full_matrix_binary=True, symmetric=True, diagonal_zero=Tru e validation_B: seeds=50, exact_binary=True, tolerant_binary=True, intermediate_exact=0, full_matrix_binary=True, symmetric=True, diagonal_zero=Tru e Projection/gap diagnostics (pooled)

---------- baseline pre-gap violation fraction: mean=0.972186875, median=0.97218125, max=0.9725, nonzero_seed_fraction=1, valid=100/100 active projection fraction: mean=0.972186875, median=0.97218125, max=0.9725, nonzero_seed_fraction=1, valid=100/100 projection active given violation: mean=1, median=1, max=1, nonzero_seed_fraction=1, valid=100/100 mean active projection magnitude (Frobenius norm): mean=0.00229983349, median=0.00237959999, max=0.00284602532, nonzero_seed_fraction=1, valid= 100/100 maximum projection magnitude (Frobenius norm): mean=0.00236759039, median=0.00248372972, max=0.00295121331, nonzero_seed_fraction=1, valid=100/ 100 minimum pre-projection gap: mean=0.0472381751, median=0.0471425, max=0.0477425, nonzero_seed_fraction=1, valid=100/100 minimum post-projection gap: mean=0.05, median=0.05, max=0.05, nonzero_seed_fraction=1, valid=100/100 shuffle_R pre-gap violation fraction: mean=0, median=0, max=0, nonzero_seed_fraction=0, valid=100/100 active projection fraction: mean=0, median=0, max=0, nonzero_seed_fraction=0, valid=100/100 projection active given violation: mean=nan, median=nan, max=nan, nonzero_seed_fraction=nan, valid=0/100 mean active projection magnitude (Frobenius norm): mean=0, median=0, max=0, nonzero_seed_fraction=0, valid=100/100 maximum projection magnitude (Frobenius norm): mean=0, median=0, max=0, nonzero_seed_fraction=0, valid=100/100 minimum pre-projection gap: mean=0.953755168, median=0.953629945, max=0.958199038, nonzero_seed_fraction=1, valid=100/100 minimum post-projection gap: mean=0.953755168, median=0.953629945, max=0.958199038, nonzero_seed_fraction=1, valid=100/100 no_delay pre-gap violation fraction: mean=0.97210075, median=0.97211875, max=0.9725375, nonzero_seed_fraction=1, valid=100/100 active projection fraction: mean=0.97210075, median=0.97211875, max=0.9725375, nonzero_seed_fraction=1, valid=100/100 projection active given violation: mean=1, median=1, max=1, nonzero_seed_fraction=1, valid=100/100 mean active projection magnitude (Frobenius norm): mean=0.00241700264, median=0.00241561988, max=0.00244578022, nonzero_seed_fraction=1, valid= 100/100 maximum projection magnitude (Frobenius norm): mean=0.00248372972, median=0.00248372972, max=0.00248372972, nonzero_seed_fraction=1, valid=100/ 100 minimum pre-projection gap: mean=0.0471425, median=0.0471425, max=0.0471425, nonzero_seed_fraction=1, valid=100/100 minimum post-projection gap: mean=0.05, median=0.05, max=0.05, nonzero_seed_fraction=1, valid=100/100 no_mu pre-gap violation fraction: mean=0.988001625, median=0.988, max=0.988175, nonzero_seed_fraction=1, valid=100/100 active projection fraction: mean=0.988001625, median=0.988, max=0.988175, nonzero_seed_fraction=1, valid=100/100 projection active given violation: mean=1, median=1, max=1, nonzero_seed_fraction=1, valid=100/100 mean active projection magnitude (Frobenius norm): mean=0.00276261513, median=0.00287070364, max=0.00291838723, nonzero_seed_fraction=1, valid= 100/100 maximum projection magnitude (Frobenius norm): mean=0.00300432369, median=0.00311217891, max=0.00311223111, nonzero_seed_fraction=1, valid=100/ 100 minimum pre-projection gap: mean=0.0465752038, median=0.0466095236, max=0.046719795, nonzero_seed_fraction=1, valid=100/100 minimum post-projection gap: mean=0.05, median=0.05, max=0.05, nonzero_seed_fraction=1, valid=100/100 no_gap pre-gap violation fraction: mean=0.972186875, median=0.97218125, max=0.9725, nonzero_seed_fraction=1, valid=100/100 active projection fraction: mean=0, median=0, max=0, nonzero_seed_fraction=0, valid=100/100 projection active given violation: mean=0, median=0, max=0, nonzero_seed_fraction=0,

valid=100/100 mean active projection magnitude (Frobenius norm): mean=0, median=0, max=0, nonzero_seed_fraction=0, valid=100/100 maximum projection magnitude (Frobenius norm): mean=0, median=0, max=0, nonzero_seed_fraction=0, valid=100/100 minimum pre-projection gap: mean=-32.7844638, median=-42.916891, max=-14.9998349, nonzero_seed_fraction=1, valid=100/100 minimum post-projection gap: mean=-32.7844638, median=-42.916891, max=-14.9998349, nonzero_seed_fraction=1, valid=100/100 Interpretation notes ---------- * Unique upper-triangular edges are the scientific topology object. * Full-matrix, symmetry, and diagonal checks are storage/model invariants. * Exact binary and binary-within-tolerance are reported separately. * Projection magnitudes are Frobenius norms of numerical corrections, not physical work. * Conditional projection activity is active_fraction / violation_fraction when the denominator is nonzero. Saved closure artifacts to /home/deck/AI_Programs/SSRH/aief_1_3_closure_diagnostics

This is a clean closure result. The diagnostics are internally consistent, and several relationships provide useful cross-checks beyond the Boolean CLOSURE PASSED.
1. Baseline topology is fully frozen in the future window

Across all 100 independent validation seeds:

text

exact_binary=True
intermediate_exact=0
full_matrix_binary=True
matrix_scalar_consistent=True

Therefore every unique baseline edge is either:
Pij=0
Pij=0

or:
Pij=1.
Pij=1.

No baseline edge crosses the threshold during only part of the future window. By HIST_END, the network has entered a fixed threshold-topology regime.

This means the AIEF-1.3 primary observable is more precisely:

Association between historical resonance and membership in a future topology that remains fixed throughout evaluation.

It also means an intervention at HIST_END tests mature-topology maintenance or reconstruction, not initial selection.
2. The projection-onset timing is reproducible

The projection is active during approximately 97.22% of baseline steps. This implies the unconstrained trajectory first reaches the boundary after approximately:
$(1-0.9722)\times 800 \approx 22.2$
$(1-0.9722)\times 800 \approx 22.2$

simulation-time units.

For no-μ:
$(1-0.9880)\times 800 \approx 9.6$.
$(1-0.9880)\times 800 \approx 9.6$.

So removing μμ causes the system to exhaust its row-sum budget much earlier, consistent with widespread positive weight growth.

This timing interpretation is approximate because the reported value is averaged across seeds, but it is useful mechanistically.
3. Projection activation is completely deterministic once violated

For baseline, no-delay, and no-μ:

text

Projection active given violation = 1

Thus every pre-gap violation produces a correction above PROJECTION_NORM_TOL.

This is partly a validation of the implementation—the code only projects after detecting a violation—but it also confirms that no violations are too small to result in a recorded correction.

The accurate description is:

After entering the boundary regime, every outward update is projected back into the admissible set.

4. Baseline versus no-gap provides a particularly clean causal check

Baseline and no-gap have the same pre-gap violation fraction:

text

0.972186875

That is expected and reassuring:

Their dynamics are identical before the first point at which projection would be needed.
They therefore reach the boundary at the same time.
Baseline projects on every subsequent violating step.
No-gap applies no correction and remains outside the set.

The outcomes then diverge:

text

Baseline post-gap: 0.05
No-gap post-gap: −32.78 mean

This is a clean within-model intervention signature that the projection—not some earlier trajectory difference—is responsible for maintaining the prescribed envelope.
5. The correction magnitude is highly regular

Baseline:

text

mean active norm: 0.002300
mean per-seed maximum: 0.002368
largest observed max: 0.002951

No-delay and no-μ show similarly narrow ranges.

The closeness between mean and maximum suggests that projection magnitude is fairly uniform after boundary entry, rather than being dominated by rare large corrections.

However, these remain absolute Frobenius norms. “Gentle” is plausible but not yet dimensionless. In AIEF-1.4, record:
Mrelative=
Mrelative=

\frac{|B_{\rm projected}-B_{\rm pre}|F}
{\max(|B{\rm pre}|_F,\epsilon)}.
]

Also useful:
Mremoved=
Mremoved=

\frac{|B_{\rm projected}-B_{\rm pre}|F}
{\max(|B{\rm pre}-B_{\rm previous}|_F,\epsilon)}.
]

These distinguish a small correction relative to the network state from a correction that removes most of each proposed update.
6. No-μ pushes harder and reaches the constraint earlier

Compared with baseline, no-μ has:

earlier boundary entry;
higher projection frequency;
larger mean and maximum projection magnitude;
a more negative pre-projection gap.

This is consistent with the paired concentration results:

all edges become nonzero;
total weight approaches the cap;

weights are diffuse;
projection must continuously enforce the coupling budget.

This reinforces the division of roles:

μμ: pruning/concentration;
projection: total coupling-envelope enforcement.

7. Shuffle never approaches the admissibility boundary

Shuffle has:

text

Pre-gap violation fraction = 0
Projection active fraction = 0
G_min ≈ 0.954

So shuffle collapse is not caused by projection. Timestep-wise driver reassignment prevents enough edge-specific accumulation for the network even to approach the row-sum cap.

The conditional projection ratio is NaN because its denominator is zero. That is mathematically correct.

One minor labeling detail: projection_metrics_all_finite=True refers to the saved source metrics. The derived conditional ratio is intentionally undefined for shuffle. Renaming that status to:

text

source_projection_metrics_all_finite

would remove the apparent contradiction, but no calculation is wrong.
8. No-gap outcomes appear heterogeneous

For no-gap:

text

mean G_min: −32.78
median G_min: −42.92
least negative maximum: −15.00

The difference between mean and median suggests a skewed or multimodal no-gap distribution. This agrees with the earlier observation that approximately half the no-gap seeds have degenerate topology.

This does not block closure, but a histogram of no-gap G_min and total edge weight would be useful in the final results figure.
Closure decision

AIEF-1.3 can now be frozen. The audit verifies:

complete independent validation cohorts;
exact binary baseline future topology;
scalar/matrix consistency;
valid symmetric storage;
deterministic projection enforcement;
frequent boundary operation;
bounded constrained dynamics;
and clean no-gap/shuffle intervention signatures.

Implication for AIEF-1.4

The mature split-state test should proceed, but its question must be phrased carefully:

Which state variables are necessary to maintain or reconstruct an already-selected frozen topology?

If the goal also includes identifying how topology initially forms, add an earlier intervention split in a later stage. The current HIST_END split is intentionally a mature-topology intervention.

No additional AIEF-1.3 reruns or closure-script modifications are necessary before beginning the AIEF-1.4 implementation.

Can you produve the code nessasary for 1.4 implementation?
WriteAIEF_1_4_memory_ablation.py1007 lines
AIEF_1_4_memory_ablation.pyPYTHON

```
#!/usr/bin/env python3
"""AIEF-1.4 split-state memory-carrier ablation experiment.

One common baseline prefix is simulated per seed through HIST_END. The exact
split state is then deep-cloned into paired future branches that selectively
remove topology, EMA, phase/history, or future driver identity.

The experiment tests maintenance/reconstruction of an already mature topology;
it does not test initial topology formation.
"""

from __future__ import annotations

import argparse
import csv
import hashlib
import json
import os
import platform
import secrets
import time
from collections import deque
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable

import numpy as np



# ============================================================================
# Model and protocol
# ============================================================================

WORKFLOW_VERSION = "AIEF-1.4-memory-ablation-r1"
```

```
N = 12
T_SIM = 800.0
DT = 0.01
TAU = 0.5
OMEGA = np.array(
[
1.00, 1.02, 1.50, 2.00, 2.01, 1.01,
1.03, 1.51, 2.02, 2.00, 1.00, 1.04,
],
dtype=float,
)

A = 1.0
Q = 1.5
DELTA = 0.05
ETA = 0.04
GAMMA = 0.015
MU = 0.03
T_WINDOW = 25.0 / float(np.mean(OMEGA))
B_C = 0.12
B_INIT_SCALE = 0.0005
ALPHA = float(np.exp(-DT / T_WINDOW))

BURN_FRACTION = 0.20
HISTORY_END_FRACTION = 0.60
DELAY_STEPS = int(round(TAU / DT))
N_STEPS = int(round(T_SIM / DT))
BURN_END = int(BURN_FRACTION * N_STEPS)
HIST_END = int(HISTORY_END_FRACTION * N_STEPS)
FUTURE_STEPS = N_STEPS - HIST_END
WINDOW_STEPS = int(round(T_WINDOW / DT))
RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)
DIAGNOSTIC_STRIDE = 50

EDGES = np.triu_indices(N, k=1)
N_EDGES = len(EDGES[0])
CAP = (A - DELTA) / Q

GAP_TOL = 1e-9
ZERO_TOL = 1e-12
PROJECTION_NORM_TOL = 1e-14
```

```
DEFAULT_MASTER_SEED = 20260714
DEFAULT_N_SEEDS = 40
DEFAULT_RUN_ROOT = "runs"

BRANCHES = (
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_only",
"full_reset",
"future_shuffle",
)

HORIZONS = ("full", "immediate", "recovery", "late")

BRANCH_MATRIX_KEYS = (
"B_final",
"B_mean_late",
"P_full",
"P_immediate",
"P_recovery",
"P_late",
"R_physical_future",
"R_driver_future",
"sample_times",
"density_trajectory",
"similarity_trajectory",
)

PREFIX_KEYS = (
"B_at_split",
"R_physical_past",
"theta_at_split",
"coherence_acc_at_split",
"history_at_split",
"selected_at_split",
)
```

```
BRANCH_SCALAR_KEYS = (
"D_B_final",
"P_mean_full",
"P_mean_immediate",
"P_mean_recovery",
"P_mean_late",
"Weighted_similarity_final",
"Weighted_similarity_late",
"Jaccard_final",
"Retention_selected_late",
"Emergence_unselected_late",
"Retention_contrast_late",
"Corr_Rpast_Pfull",
"Corr_Rpast_Plate",
"Total_edge_weight",
"Mean_row_sum",
"Max_row_sum",
"Nonzero_edge_fraction",
"Effective_edges",
"Weight_gini",
"Weight_entropy",
"r_mean_future",
"r_mean_late",
"G_min_future",
"G_min_pre_future",
"Pre_gap_violation_fraction",
"Projection_active_fraction",
"Projection_active_given_violation",
"Projection_norm_mean_active",
"Projection_norm_max",
"Projection_relative_state_mean_active",
"Projection_relative_state_max",
"Projection_relative_update_mean_active",
"Projection_relative_update_max",
"Recovery_time_50pct_reference_density",
"Recovery_time_90pct_reference_density",
)



#
================================================================================
====
```

```
# State and accumulator classes
#
================================================================================


@dataclass
class DynamicState:
theta: np.ndarray
B: np.ndarray
coherence_acc: np.ndarray
history: deque[np.ndarray]


def clone(self) -> "DynamicState":
return DynamicState(
theta=self.theta.copy(),
B=self.B.copy(),
coherence_acc=self.coherence_acc.copy(),
history=deque(
(item.copy() for item in self.history),
maxlen=self.history.maxlen,
),
)



@dataclass
class ProjectionTotals:
steps: int = 0
violations: int = 0
active: int = 0
norm_sum: float = 0.0
norm_max: float = 0.0
relative_state_sum: float = 0.0
relative_state_max: float = 0.0
relative_update_sum: float = 0.0
relative_update_max: float = 0.0
g_min: float = np.inf
g_min_pre: float = np.inf


def update(self, info: dict[str, Any]) -> None:
self.steps += 1
```

```
self.violations += int(info["pre_violation"])
self.g_min = min(self.g_min, float(info["g_min"]))
self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))
if info["projection_active"]:
self.active += 1
norm = float(info["projection_norm"])
rel_state = float(info["projection_relative_state"])
rel_update = float(info["projection_relative_update"])
self.norm_sum += norm
self.norm_max = max(self.norm_max, norm)
self.relative_state_sum += rel_state
self.relative_state_max = max(self.relative_state_max, rel_state)
self.relative_update_sum += rel_update
self.relative_update_max = max(self.relative_update_max, rel_update)


def metrics(self) -> dict[str, float]:
active_denominator = max(self.active, 1)
return {
"G_min_future": float(self.g_min),
"G_min_pre_future": float(self.g_min_pre),
"Pre_gap_violation_fraction": float(self.violations / self.steps),
"Projection_active_fraction": float(self.active / self.steps),
"Projection_active_given_violation": float(self.active / self.violations)
if self.violations
else np.nan,
"Projection_norm_mean_active": float(
self.norm_sum / active_denominator
),
"Projection_norm_max": float(self.norm_max),
"Projection_relative_state_mean_active": float(
self.relative_state_sum / active_denominator
),
"Projection_relative_state_max": float(self.relative_state_max),
"Projection_relative_update_mean_active": float(
self.relative_update_sum / active_denominator
),
"Projection_relative_update_max": float(self.relative_update_max),
}
```

```
#
============================================================================
====
# Utility functions
#
============================================================================
====



def validate_configuration() -> None:
if OMEGA.shape != (N,) or not np.all(np.isfinite(OMEGA)):
raise ValueError("OMEGA must contain N finite frequencies")
if not np.isclose(DELAY_STEPS * DT, TAU, rtol=0.0, atol=1e-12):
raise ValueError("TAU/DT must be integral")
if not np.isclose(N_STEPS * DT, T_SIM, rtol=0.0, atol=1e-10):
raise ValueError("T_SIM/DT must be integral")
if not (0 <= BURN_END < HIST_END < N_STEPS):
raise ValueError("invalid analysis windows")
if not (0 < WINDOW_STEPS < RECOVERY_END < FUTURE_STEPS):
raise ValueError("future interval is too short for configured horizons")



def parse_master_seed(value: str) -> int:
if value.strip().lower() == "random":
return secrets.randbits(64)
try:
result = int(value, 0)
except ValueError as exc:
raise argparse.ArgumentTypeError(
"master seed must be a nonnegative integer or 'random'"
) from exc
if result < 0:
raise argparse.ArgumentTypeError("master seed must be nonnegative")
return result



def generate_seed_set(master_seed: int, count: int) -> list[int]:
if count <= 0:
raise ValueError("seed count must be positive")
children = np.random.SeedSequence(master_seed).spawn(count)
```

```
seeds = [
int(child.generate_state(1, dtype=np.uint64)[0])
for child in children
]
if len(set(seeds)) != len(seeds):
raise RuntimeError("duplicate generated seeds")
return seeds



def source_sha256() -> str | None:
try:
return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
except (NameError, OSError):
return None



def atomic_write_text(path: str, text: str) -> None:
temporary = f"{path}.tmp-{os.getpid()}-{time.time_ns()}"
try:
with open(temporary, "w", encoding="utf-8", newline="") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
os.replace(temporary, path)
finally:
if os.path.exists(temporary):
os.remove(temporary)



def save_npz_atomic(path: str, payload: dict[str, Any]) -> None:
temporary = f"{path}.tmp-{os.getpid()}-{time.time_ns()}.npz"
try:
np.savez_compressed(temporary, **payload)
os.replace(temporary, path)
finally:
if os.path.exists(temporary):
os.remove(temporary)
```

```
def safe_corr(x: np.ndarray, y: np.ndarray) -> float:
x = np.asarray(x, dtype=float)
y = np.asarray(y, dtype=float)
valid = np.isfinite(x) & np.isfinite(y)
x = x[valid]
y = y[valid]
if len(x) < 3 or np.std(x) < ZERO_TOL or np.std(y) < ZERO_TOL:
return np.nan
return float(np.corrcoef(x, y)[0, 1])



def cosine_edges(reference: np.ndarray, current: np.ndarray) -> float:
x = np.asarray(reference[EDGES], dtype=float)
y = np.asarray(current[EDGES], dtype=float)
denominator = float(np.linalg.norm(x) * np.linalg.norm(y))
if denominator <= ZERO_TOL:
return np.nan
return float(np.dot(x, y) / denominator)



def jaccard_edges(reference: np.ndarray, current: np.ndarray) -> float:
x = reference[EDGES] > B_C
y = current[EDGES] > B_C
union = np.count_nonzero(x | y)
if union == 0:
return np.nan
return float(np.count_nonzero(x & y) / union)



def effective_number(weights: np.ndarray) -> float:
weights = np.asarray(weights, dtype=float)
total = float(np.sum(weights))
sum_sq = float(np.sum(weights**2))
if total <= ZERO_TOL or sum_sq <= ZERO_TOL:
return 0.0
return total**2 / sum_sq



def gini_coefficient(weights: np.ndarray) -> float:
```

```
x = np.sort(np.asarray(weights, dtype=float))
total = float(np.sum(x))
if total <= ZERO_TOL:
return 0.0
n = len(x)
ranks = np.arange(1, n + 1, dtype=float)
return float(2.0 * np.dot(ranks, x) / (n * total) - (n + 1.0) / n)



def normalized_entropy(weights: np.ndarray) -> float:
weights = np.asarray(weights, dtype=float)
total = float(np.sum(weights))
if total <= ZERO_TOL:
return 0.0
probabilities = weights / total
positive = probabilities > 0.0
entropy = -float(
np.sum(probabilities[positive] * np.log(probabilities[positive]))
)
return entropy / float(np.log(len(weights)))



def horizon_name(relative_step: int) -> str:
if relative_step < WINDOW_STEPS:
return "immediate"
if relative_step < RECOVERY_END:
return "recovery"
return "late"



def compute_R(state: DynamicState) -> np.ndarray:
delayed = state.history[0]
dphi = (
state.theta[:, None]
- delayed[None, :]
- ((OMEGA[:, None] + OMEGA[None, :]) / 2.0) * TAU
)
z_inst = np.exp(1j * dphi)
state.coherence_acc[:] = (
ALPHA * state.coherence_acc + (1.0 - ALPHA) * z_inst
```

```
)
result = np.abs(state.coherence_acc)
result = 0.5 * (result + result.T)
np.fill_diagonal(result, 0.0)
return result



def advance_state(
state: DynamicState,
*,
shuffle_driver: bool,
rng: np.random.Generator,
) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
B_previous = state.B.copy()
R_physical = compute_R(state)
R_driver = R_physical


if shuffle_driver:
values = R_physical[EDGES].copy()
rng.shuffle(values)
R_driver = np.zeros_like(R_physical)
R_driver[EDGES] = values
R_driver[(EDGES[1], EDGES[0])] = values


B_unprojected = state.B + DT * (
(ETA + MU) * R_driver - MU - GAMMA * state.B
)
B_unprojected = np.maximum(B_unprojected, 0.0)
B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)
np.fill_diagonal(B_unprojected, 0.0)


proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))
row_sum_pre = np.sum(B_unprojected, axis=1)
g_min_pre = float(np.min(A - Q * row_sum_pre))
pre_violation = bool(g_min_pre < DELTA)


B_projected = B_unprojected
if pre_violation:
scale = np.minimum(1.0, CAP / (row_sum_pre + ZERO_TOL))
B_projected = B_unprojected * np.minimum.outer(scale, scale)
```

```
np.fill_diagonal(B_projected, 0.0)


correction_norm = float(np.linalg.norm(B_projected - B_unprojected))
projection_active = bool(correction_norm > PROJECTION_NORM_TOL)
relative_state = correction_norm / max(
float(np.linalg.norm(B_unprojected)), ZERO_TOL
)
relative_update = correction_norm / max(proposed_update_norm, ZERO_TOL)


state.B = B_projected
row_sum = np.sum(state.B, axis=1)
g_min = float(np.min(A - Q * row_sum))
if g_min < DELTA - GAP_TOL:
raise RuntimeError(f"gap projection failed: {g_min}")


delayed = state.history[0]
coupling = np.sum(
state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1
)
state.theta = np.mod(
state.theta + DT * (OMEGA + coupling), 2.0 * np.pi
)
state.history.append(state.theta.copy())


info = {
"pre_violation": pre_violation,
"projection_active": projection_active,
"projection_norm": correction_norm,
"projection_relative_state": relative_state,
"projection_relative_update": relative_update,
"g_min_pre": g_min_pre,
"g_min": g_min,
}
return R_physical, R_driver, info



#
==========================================================================
====
# Prefix and interventions
```

```
#
================================================================================
====



def simulate_prefix(seed: int) -> tuple[DynamicState, dict[str, np.ndarray]]:
root = np.random.SeedSequence(seed)
init_sequence, _ = root.spawn(2)
rng = np.random.default_rng(init_sequence)


theta = rng.random(N) * 2.0 * np.pi
B = B_INIT_SCALE * rng.random((N, N))
B = 0.5 * (B + B.T)
np.fill_diagonal(B, 0.0)
state = DynamicState(
theta=theta,
B=B,
coherence_acc=np.zeros((N, N), dtype=complex),
history=deque(
(theta.copy() for _ in range(DELAY_STEPS + 1)),
maxlen=DELAY_STEPS + 1,
),
)


R_past_sum = np.zeros((N, N), dtype=float)
R_past_count = 0
unused_rng = np.random.default_rng(0)


for step in range(HIST_END):
R_physical, _, _ = advance_state(
state,
shuffle_driver=False,
rng=unused_rng,
)
if BURN_END <= step < HIST_END:
R_past_sum += R_physical
R_past_count += 1


prefix = {
"B_at_split": state.B.copy(),
```

```
"R_physical_past": R_past_sum / R_past_count,
"theta_at_split": state.theta.copy(),
"coherence_acc_at_split": state.coherence_acc.copy(),
"history_at_split": np.stack(tuple(state.history)),
"selected_at_split": (state.B > B_C).astype(np.uint8),
}
return state, prefix



def apply_intervention(
state: DynamicState,
branch: str,
rng: np.random.Generator,
) -> None:
if branch in {"reset_ema", "reset_B_ema", "B_only", "full_reset"}:
state.coherence_acc.fill(0.0)

if branch in {"reset_B", "reset_B_ema", "full_reset"}:
state.B.fill(0.0)

if branch in {"B_only", "full_reset"}:
state.theta = rng.random(N) * 2.0 * np.pi
state.history = deque(
(state.theta.copy() for _ in range(DELAY_STEPS + 1)),
maxlen=DELAY_STEPS + 1,
)

state.B = 0.5 * (state.B + state.B.T)
np.fill_diagonal(state.B, 0.0)



#
============================================================================
====
# Future branch simulation
#
============================================================================
====
```

```
def simulate_future(
split_state: DynamicState,
prefix: dict[str, np.ndarray],
branch: str,
rng: np.random.Generator,
) -> dict[str, Any]:
state = split_state.clone()
apply_intervention(state, branch, rng)


P_sums = {name: np.zeros((N, N), dtype=float) for name in HORIZONS}
B_sums = {name: np.zeros((N, N), dtype=float) for name in HORIZONS}
counts = {name: 0 for name in HORIZONS}
R_physical_sum = np.zeros((N, N), dtype=float)
R_driver_sum = np.zeros((N, N), dtype=float)


projection = ProjectionTotals()
r_sum = 0.0
r_count = 0
r_late_sum = 0.0
r_late_count = 0


sample_times: list[float] = []
density_trajectory: list[float] = []
similarity_trajectory: list[float] = []


shuffle_driver = branch == "future_shuffle"
B_reference = prefix["B_at_split"]


for relative_step in range(FUTURE_STEPS):
R_physical, R_driver, projection_info = advance_state(
state,
shuffle_driver=shuffle_driver,
rng=rng,
)
projection.update(projection_info)


selected = state.B > B_C
name = horizon_name(relative_step)
```

```
for horizon in ("full", name):
P_sums[horizon] += selected
B_sums[horizon] += state.B
counts[horizon] += 1


R_physical_sum += R_physical
R_driver_sum += R_driver


if relative_step % DIAGNOSTIC_STRIDE == 0:
r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))
r_sum += r_value
r_count += 1
if name == "late":
r_late_sum += r_value
r_late_count += 1
sample_times.append((relative_step + 1) * DT)
density_trajectory.append(
float(np.count_nonzero(state.B[EDGES] > B_C) / N_EDGES)
)
similarity_trajectory.append(cosine_edges(B_reference, state.B))


P = {name: P_sums[name] / counts[name] for name in HORIZONS}
B_mean = {name: B_sums[name] / counts[name] for name in HORIZONS}
final_weights = state.B[EDGES]
final_rows = np.sum(state.B, axis=1)
split_selected = prefix["selected_at_split"][EDGES].astype(bool)
p_late_edges = P["late"][EDGES]


retention = (
float(np.mean(p_late_edges[split_selected]))
if np.any(split_selected)
else np.nan
)
emergence = (
float(np.mean(p_late_edges[~split_selected]))
if np.any(~split_selected)
else np.nan
)


result: dict[str, Any] = {
```

```
"D_B_final": float(np.count_nonzero(final_weights > B_C) / N_EDGES),
"P_mean_full": float(np.mean(P["full"][EDGES])),
"P_mean_immediate": float(np.mean(P["immediate"][EDGES])),
"P_mean_recovery": float(np.mean(P["recovery"][EDGES])),
"P_mean_late": float(np.mean(P["late"][EDGES])),
"Weighted_similarity_final": cosine_edges(B_reference, state.B),
"Weighted_similarity_late": cosine_edges(B_reference, B_mean["late"]),
"Jaccard_final": jaccard_edges(B_reference, state.B),
"Retention_selected_late": retention,
"Emergence_unselected_late": emergence,
"Retention_contrast_late": retention - emergence,
"Corr_Rpast_Pfull": safe_corr(
prefix["R_physical_past"][EDGES], P["full"][EDGES]
),
"Corr_Rpast_Plate": safe_corr(
prefix["R_physical_past"][EDGES], P["late"][EDGES]
),
"Total_edge_weight": float(np.sum(final_weights)),
"Mean_row_sum": float(np.mean(final_rows)),
"Max_row_sum": float(np.max(final_rows)),
"Nonzero_edge_fraction": float(np.mean(final_weights > ZERO_TOL)),
"Effective_edges": effective_number(final_weights),
"Weight_gini": gini_coefficient(final_weights),
"Weight_entropy": normalized_entropy(final_weights),
"r_mean_future": float(r_sum / r_count),
"r_mean_late": float(r_late_sum / r_late_count),
"Recovery_time_50pct_reference_density": np.nan,
"Recovery_time_90pct_reference_density": np.nan,
**projection.metrics(),
"B_final": state.B.copy(),
"B_mean_late": B_mean["late"],
"P_full": P["full"],
"P_immediate": P["immediate"],
"P_recovery": P["recovery"],
"P_late": P["late"],
"R_physical_future": R_physical_sum / FUTURE_STEPS,
"R_driver_future": R_driver_sum / FUTURE_STEPS,
"sample_times": np.asarray(sample_times, dtype=float),
"density_trajectory": np.asarray(density_trajectory, dtype=float),
"similarity_trajectory": np.asarray(similarity_trajectory, dtype=float),
}
return result
```

```
def recovery_time(
sample_times: np.ndarray,
density: np.ndarray,
target: float,
) -> float:
indices = np.flatnonzero(density >= target)
return float(sample_times[indices[0]]) if len(indices) else np.nan



def run_seed(
seed: int,
seed_index: int,
config_hash: str,
verify_continue: bool,
) -> dict[str, Any]:
split_state, prefix = simulate_prefix(seed)
immutable_snapshot = {
"theta": split_state.theta.copy(),
"B": split_state.B.copy(),
"coherence": split_state.coherence_acc.copy(),
"history": np.stack(tuple(split_state.history)),
}


_, branch_root = np.random.SeedSequence(seed).spawn(2)
branch_sequences = branch_root.spawn(len(BRANCHES))
branch_results: dict[str, dict[str, Any]] = {}


for branch, sequence in zip(BRANCHES, branch_sequences):
branch_results[branch] = simulate_future(
split_state,
prefix,
branch,
np.random.default_rng(sequence),
)


if not np.array_equal(split_state.theta, immutable_snapshot["theta"]):
raise RuntimeError("branch mutation leaked into split theta")
```

```
if not np.array_equal(split_state.B, immutable_snapshot["B"]):
raise RuntimeError("branch mutation leaked into split B")
if not np.array_equal(
split_state.coherence_acc, immutable_snapshot["coherence"]
):
raise RuntimeError("branch mutation leaked into split EMA")
if not np.array_equal(
np.stack(tuple(split_state.history)), immutable_snapshot["history"]
):
raise RuntimeError("branch mutation leaked into split history")


reference_density = branch_results["continue"]["D_B_final"]
for result in branch_results.values():
result["Recovery_time_50pct_reference_density"] = recovery_time(
result["sample_times"],
result["density_trajectory"],
0.5 * reference_density,
)
result["Recovery_time_90pct_reference_density"] = recovery_time(
result["sample_times"],
result["density_trajectory"],
0.9 * reference_density,
)


if verify_continue:
duplicate = simulate_future(
split_state,
prefix,
"continue",
np.random.default_rng(0),
)
for key in ("B_final", "P_full", "P_late"):
if not np.array_equal(branch_results["continue"][key], duplicate[key]):
raise RuntimeError(f"continue reproducibility failed for {key}")


return {
"seed": int(seed),
"seed_index": int(seed_index),
"config_hash": config_hash,
"prefix": prefix,
"branches": branch_results,
```

```
}



#
# ================================================================================
# Checkpoint serialization
#
# ================================================================================



def flatten_seed_result(result: dict[str, Any]) -> dict[str, Any]:
payload: dict[str, Any] = {
"seed": result["seed"],
"seed_index": result["seed_index"],
"config_hash": result["config_hash"],
}
for key, value in result["prefix"].items():
payload[f"prefix__{key}"] = value
for branch, branch_result in result["branches"].items():
for key, value in branch_result.items():
payload[f"branch__{branch}__{key}"] = value
return payload



def unflatten_seed_result(payload: dict[str, Any]) -> dict[str, Any]:
result: dict[str, Any] = {
"seed": int(payload["seed"]),
"seed_index": int(payload["seed_index"]),
"config_hash": str(payload["config_hash"]),
"prefix": {},
"branches": {branch: {} for branch in BRANCHES},
}
for key, value in payload.items():
if key.startswith("prefix__"):
result["prefix"][key.split("__", 1)[1]] = value
elif key.startswith("branch__"):
_, branch, metric = key.split("__", 2)
result["branches"][branch][metric] = value
```

```
return result



def load_npz(path: str) -> dict[str, Any]:
payload: dict[str, Any] = {}
with np.load(path, allow_pickle=False) as data:
for key in data.files:
value = data[key]
payload[key] = value.item() if value.ndim == 0 else value.copy()
return payload



def validate_seed_result(
result: dict[str, Any], seed: int, config_hash: str
) -> None:
if result["seed"] != seed or result["config_hash"] != config_hash:
raise RuntimeError("checkpoint identity/configuration mismatch")
missing_prefix = set(PREFIX_KEYS) - set(result["prefix"])
if missing_prefix:
raise RuntimeError(f"checkpoint missing prefix keys: {missing_prefix}")
if set(result["branches"]) != set(BRANCHES):
raise RuntimeError("checkpoint branch set mismatch")
for branch in BRANCHES:
missing = (set(BRANCH_SCALAR_KEYS) | set(BRANCH_MATRIX_KEYS)) - set(
result["branches"][branch]
)
if missing:
raise RuntimeError(f"{branch} checkpoint missing keys: {missing}")



#
================================================================================
====
# Manifest, output, and CLI
#
================================================================================
====
```

```
def build_manifest(master_seed: int, seeds: list[int]) -> dict[str, Any]:
return {
"workflow_version": WORKFLOW_VERSION,
"source_sha256": source_sha256(),
"model": {
"N": N, "T_sim": T_SIM, "dt": DT, "tau": TAU,
"omega": OMEGA.tolist(), "a": A, "q": Q, "delta": DELTA,
"eta": ETA, "gamma": GAMMA, "mu": MU,
"T_window": T_WINDOW, "alpha": ALPHA,
"B_c": B_C, "B_init_scale": B_INIT_SCALE,
},
"protocol": {
"purpose": "maintenance and reconstruction of mature topology",
"branches": list(BRANCHES),
"n_steps": N_STEPS,
"hist_end": HIST_END,
"future_steps": FUTURE_STEPS,
"window_steps": WINDOW_STEPS,
"recovery_end": RECOVERY_END,
"diagnostic_stride": DIAGNOSTIC_STRIDE,
"primary_horizon": "late",
},
"randomization": {
"master_seed": int(master_seed),
"paired_seeds": [int(seed) for seed in seeds],
"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",
"branch_generation": "second per-seed child sequence spawns branch streams",
"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,
},
"environment": {
"python": platform.python_version(),
"numpy": np.__version__,
"platform": platform.platform(),
},
"schema": {
"prefix_keys": list(PREFIX_KEYS),
"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),
"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),
},
}
```

```
def write_csv_atomic(path: str, rows: list[dict[str, Any]]) -> None:
if not rows:
raise RuntimeError("cannot write empty CSV")
temporary = f"{path}.tmp-{os.getpid()}-{time.time_ns()}"
try:
with open(temporary, "w", newline="", encoding="utf-8") as handle:
writer = csv.DictWriter(handle, fieldnames=list(rows[0]))
writer.writeheader()
writer.writerows(rows)
handle.flush()
os.fsync(handle.fileno())
os.replace(temporary, path)
finally:
if os.path.exists(temporary):
os.remove(temporary)



def summarize(values: Iterable[float]) -> tuple[float, float, int]:
x = np.asarray(list(values), dtype=float)
finite = x[np.isfinite(x)]
if not len(finite):
return np.nan, np.nan, 0
return float(np.mean(finite)), float(np.std(finite)), int(len(finite))



def print_summary(results: list[dict[str, Any]]) -> None:
print("=" * 140)
print("AIEF-1.4 MEMORY-CARRIER ABLATION — mean ± std over paired seeds")
print("=" * 140)
print(
f"{'Branch':<18} {'D_B':>13} {'P_late':>13} {'Sim_final':>13} "
f"{'Sim_late':>13} {'Retention':>13} {'Emergence':>13} "
f"{'Proj_frac':>13} {'Rel_proj':>13}"
)
print("-" * 140)
for branch in BRANCHES:
branch_runs = [result["branches"][branch] for result in results]
cells = []
for key in (
"D_B_final", "P_mean_late", "Weighted_similarity_final",
"Weighted_similarity_late", "Retention_selected_late",
```

```
"Emergence_unselected_late", "Projection_active_fraction",
"Projection_relative_state_mean_active",
):
mean, std, _ = summarize(run[key] for run in branch_runs)
cells.append("nan±nan" if not np.isfinite(mean) else f"{mean:.3f}±{std:.3f}")
print(f"{branch:<18}" + "".join(f" {cell:>13}" for cell in cells))
print("=" * 140)



def parse_arguments() -> argparse.Namespace:
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument(
"--master-seed", type=parse_master_seed, default=DEFAULT_MASTER_SEED,
help="fixed integer master seed or 'random' (default: 20260714)",
)
parser.add_argument(
"--n-seeds", type=int, default=DEFAULT_N_SEEDS,
help=f"paired seed count (default: {DEFAULT_N_SEEDS})",
)
parser.add_argument(
"--run-root", default=DEFAULT_RUN_ROOT,
help=f"output root (default: {DEFAULT_RUN_ROOT})",
)
parser.add_argument(
"--verify-continue", action="store_true",
help="repeat the continue branch and require exact equality (smoke tests)",
)
return parser.parse_args()



def main() -> None:
validate_configuration()
args = parse_arguments()
seeds = generate_seed_set(int(args.master_seed), args.n_seeds)
manifest_base = build_manifest(int(args.master_seed), seeds)
manifest_text = json.dumps(
manifest_base, sort_keys=True, separators=(",", ":"), allow_nan=False
)
config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]
run_dir = os.path.join(args.run_root, config_hash)
os.makedirs(run_dir, exist_ok=True)
```

```
manifest = {**manifest_base, "config_hash": config_hash}
atomic_write_text(
os.path.join(run_dir, "manifest.json"),
json.dumps(manifest, indent=2, sort_keys=True, allow_nan=False) + "\n",
)


print("Running AIEF-1.4 split-state memory-carrier ablation")
print(f" master seed: {args.master_seed}")
print(f" paired seeds: {args.n_seeds}")
print(f" configuration: {config_hash}")
print(f" output: {run_dir}")


all_results: list[dict[str, Any]] = []
started = time.time()
for seed_index, seed in enumerate(seeds):
checkpoint = os.path.join(run_dir, f"seed_{seed}.npz")
if os.path.exists(checkpoint):
try:
result = unflatten_seed_result(load_npz(checkpoint))
validate_seed_result(result, seed, config_hash)
status = "loaded"
except Exception as exc:
print(f" invalid checkpoint {checkpoint}: {exc}; recomputing")
os.remove(checkpoint)
result = run_seed(
seed, seed_index, config_hash, args.verify_continue
)
save_npz_atomic(checkpoint, flatten_seed_result(result))
status = "recomputed"
else:
result = run_seed(seed, seed_index, config_hash, args.verify_continue)
save_npz_atomic(checkpoint, flatten_seed_result(result))
status = "completed"
all_results.append(result)
print(f" seed {seed_index + 1}/{len(seeds)} {status}")


print(f"Runtime: {time.time() - started:.1f} s\n")


scalar_rows: list[dict[str, Any]] = []
```

```
matrix_payload: dict[str, Any] = {}
for result in all_results:
seed = result["seed"]
for key, value in result["prefix"].items():
matrix_payload[f"prefix_seed_{seed}_{key}"] = value
for branch in BRANCHES:
branch_result = result["branches"][branch]
scalar_rows.append(
{
"seed_index": result["seed_index"],
"seed": seed,
"branch": branch,
**{key: branch_result[key] for key in BRANCH_SCALAR_KEYS},
}
)
for key in BRANCH_MATRIX_KEYS:
matrix_payload[f"{branch}_seed_{seed}_{key}"] = branch_result[key]


csv_path = os.path.join(run_dir, f"AIEF_1_4_{config_hash}_scalar_summary.csv")
matrix_path = os.path.join(
run_dir, f"AIEF_1_4_{config_hash}_branch_diagnostics.npz"
)
write_csv_atomic(csv_path, scalar_rows)
save_npz_atomic(matrix_path, matrix_payload)
print_summary(all_results)
print(f"Saved {csv_path}")
print(f"Saved {matrix_path}")



if __name__ == "__main__":
main()
```
Open

Created the complete AIEF-1.4 split-state intervention driver:

AIEF_1_4_memory_ablation.py
Experimental structure

For each seed, the script:

Simulates one common baseline prefix through HIST_END.
Saves and deep-clones:
theta,
B,
coherence_acc,
complete delay history,
historical physical resonance,
selected topology at the split.
Runs seven paired future branches from that identical split state.

Branches
Branch Intervention
continue Preserve everything
reset_ema Zero the EMA accumulator
reset_B Zero all adaptive weights
reset_B_ema Zero weights and EMA
B_only Preserve BB, but reset EMA and randomize phase/history
full_reset Reset BB, EMA, phase, and delay history while preserving ωω
future_shuffle Preserve split state and shuffle only future driver identity

The code does not use the invalid ALPHA=0 instantaneous-resonance control.
Outcomes

It measures separate immediate, recovery, and late horizons:

text

Immediate: 0 to 1 T_WINDOW
Recovery: 1 to 5 T_WINDOW
Late: after 5 T_WINDOW

Primary branch diagnostics include:

final and late weighted similarity to B_at_split;
final threshold Jaccard similarity;
retention of split-selected edges;
emergence of previously unselected edges;
retention contrast;
full/immediate/recovery/late persistence;

historical-resonance correlation;
density and total weight;
effective edges, Gini, and entropy;
coherence;
gap behavior;
projection frequency and absolute magnitude;
projection magnitude relative to network state;
projection magnitude relative to the proposed unconstrained update;
density recovery times.

Run a smoke test first

Bash

```
python AIEF_1_4_memory_ablation.py \
--master-seed 123 \
--n-seeds 2 \
--run-root runs_aief14_smoke \
--verify-continue
```

--verify-continue repeats the untouched continuation and requires exact equality for:

B_final,
full future persistence,
late persistence.

It also checks that no branch mutates the original split state.
Production run

Use the fixed preregistered master seed:

Bash

```
python AIEF_1_4_memory_ablation.py \
--master-seed 20260714 \
--n-seeds 40
```

Do not use --master-seed random for the definitive run after inspecting smoke-test behavior.

Outputs

The script creates a configuration-hashed run directory containing:

text

manifest.json
seed_<seed>.npz # atomic per-seed checkpoints
AIEF_1_4_<hash>_scalar_summary.csv
AIEF_1_4_<hash>_branch_diagnostics.npz

The aggregate NPZ contains:

split-state matrices,
every branch’s future persistence matrices,
final and late-mean topology,
physical and driver resonance averages,
sampled density trajectories,
sampled topology-similarity trajectories.

The CSV contains one row per seed and branch with all scalar outcomes.
Validation performed

I completed:

Python syntax compilation;
a full seven-branch, one-seed simulation;
exact duplicate-continue verification;
split-state mutation checks;
aggregate output generation;
per-seed checkpoint writing;
checkpoint resume testing.

The full one-seed smoke test took approximately 29 seconds in the sandbox; checkpoint resume completed immediately. Forty seeds should therefore take roughly 20 minutes on similar hardware, though Steam Deck runtime may differ.

One implementation detail differs slightly from the earlier plan: checkpointing is atomic per seed after all seven branches, rather than after each individual branch. An interruption loses at most the currently running seed—approximately 30 seconds—not the completed seed set.

Think about these thoughts:

I have looked at this as a transition from AIEF-1.3 freeze → AIEF-1.4 causal intervention experiment rather than just as code review. Overall: this is a much better experimental design than the earlier AIEF-1.4 ideas, but I see several issues worth fixing before the production 40-seed run.

The good news: the AIEF-1.3 closure results justify moving to this intervention design. The bad news: a few AIEF-1.4 details currently risk making the interpretation weaker than it could be.

The AIEF-1.3 freeze result is strong enough to motivate AIEF-1.4

The critical facts:

baseline_unique_edges_exact_binary: True
matrix_scalar_consistent: True
projection_activity_subset_of_violation: True

and:

validation_A vs validation_B: overlap=0

means the AIEF-1.4 experiment is now asking the right question:

Not:

Can the system form topology?

but:

Once a mature topology exists, what information maintains it?

That matches the new script title:

memory-carrier ablation

This is the correct conceptual shift.

Major positive: the split-state design is correct

This is the strongest part:

split_state, prefix = simulate_prefix(seed)

then:

state = split_state.clone()

for each intervention.

This avoids the biggest experimental error:

comparing different stochastic trajectories.

Every branch starts from the same state:

[
S(t_0)

(\theta,B,\mathrm{EMA},H)
]

That gives you paired causal contrasts.

Good.

However: the "full_reset" branch is not actually a full reset

The description says:

Reset B, EMA, phase, and delay history while preserving ω

The implementation:

```
if branch in {"reset_B", "reset_B_ema", "full_reset"}:
state.B.fill(0.0)
```

and:

```
if branch in {"B_only", "full_reset"}:
state.theta = rng.random(N) * 2π
state.history = deque(...)
```

This resets:

B ✅
phase ✅
history ✅
EMA ✅

but it does not reset the delay-history resonance state fully.

Why?

Because:

compute_R()

uses:

state.coherence_acc

and the history.

EMA is reset.

History is reset.

So actually it is okay.

I initially expected a hidden state, but this model does not have one.

So this is not a bug.

Important issue: B_only naming is misleading

You define:

"B_only"

as:

reset EMA
randomize phase/history
preserve B

This is not "B only".

It means:

preserve topology but destroy all dynamical memory.

That is actually a very interesting experiment, but the name will confuse later interpretation.

I would rename:

B_only

to:

preserve_B_destroy_state

or:

B_carrier_only

because the scientific hypothesis is:

Is topology itself sufficient?

not:

Only B changed.

The biggest conceptual issue: future_shuffle is not comparable to no_gap

Current:

future_shuffle

does:

```
values = R_physical[EDGES].copy()
rng.shuffle(values)
```

So:

topology B remains
physical coherence remains
only edge identity is destroyed

This is good.

But it is a driver topology test, not a memory test.

It asks:

Is the resonance assignment important?

It does not ask:

Is historical memory important?

I would keep it, but classify it separately:

Memory ablations

reset_ema
reset_B
reset_B_ema
B_carrier_only
full_reset

Driver identity controls

future_shuffle

That separation should go into the preregistration.

Potential statistical issue: 40 seeds may be too few for branch × metric explosion

You have:

7 branches

× about 40 scalar metrics

× 40 seeds

That is:

$$
7\times40=280
$$

comparisons.

Do not end up with a giant p-value table.

The primary outcomes should be frozen now.

I would select:

Primary

Late persistence

$$
P_{\mathrm{late}}
$$

Retention contrast

$$
R_{\mathrm{selected}}-R_{\mathrm{unselected}}
$$

Similarity to split topology

$$
\cos(B_{split},B_{late})
$$

Secondary

density
entropy
effective edges
projection statistics

Everything else exploratory.

A subtle mathematical issue: your recovery metric assumes density recovery is good

You have:

Recovery_time_90pct_reference_density

But after some interventions, the correct outcome may not be recovery.

Example:

full_reset

could create a different topology.

Then:

low density recovery time
low similarity

could both be meaningful.

The metric should probably be:

Recovery_time_to_reference_similarity

not only density.

You already compute:

similarity_trajectory

so add:

Recovery_time_50pct_similarity
Recovery_time_90pct_similarity

This is much more aligned with the memory question.

One real code bug: recovery times are computed after all branches using continue density

This section:

reference_density = branch_results["continue"]["D_B_final"]

for result in branch_results.values():
result["Recovery_time_50pct_reference_density"] = recovery_time(

uses:

[
0.5D_B^{continue}
]

as the target for every branch.

This is defensible, but the name:

reference_density

is ambiguous.

For example:

reset_B

will be judged against the continuation branch.

That may be intended.

If so rename:

Recovery_time_to_continue_density_50pct

because it is not "recovery" internally.

It is:

return toward the untreated attractor.

The projection analysis is excellent, but add one missing statistic

Currently:

Projection_relative_update_mean_active

is good.

But I would add:

$$
\frac{||\Delta B_{projection}||}
{||B_{state}||}
$$

already present:

Projection_relative_state_mean_active

and:

[
\frac{||\Delta B_{projection}||}
{||B_{unprojected}-B||}
]

already present.

Excellent.

However, add:

Projection_fraction_of_steps

already:

Projection_active_fraction

Good.

No issue.

Potential reproducibility issue in smoke verification

This:

duplicate = simulate_future(
split_state,
prefix,
"continue",
np.random.default_rng(0),
)

works because:

continue

does not consume random numbers.

But future changes could break it.

Better:

```
duplicate = simulate_future(
split_state,
prefix,
"continue",
np.random.default_rng(branch_seed_for_continue),
)
```

where the same branch seed is reused.

That future-proofs the test.

One thing I would add: a "memory carrier attribution table"

The experiment is really asking:

Where is the memory stored?

The branches map beautifully:

CarrierRemovedEMAreset_ematopologyreset_Btopology+EMAreset_B_ematopology
onlyB_carrier_onlyeverythingfull_resetdriver identityfuture_shuffle

I would make the final report explicitly output:

carrier_survival_score

for each branch.

Something like:

[
M=
\frac{
P_{late}^{branch}
}{
P_{late}^{continue}
}
]

This gives a simple interpretation:

1 = preserved
0 = destroyed

My freeze recommendation

I would not run production yet.

I would make these small changes:

Required

Rename B_only.
Rename recovery metrics to clarify they reference continuation.
Add similarity recovery times.
Fix continue verification RNG.

Recommended

Define primary metrics before looking at results.
Separate memory ablations from driver controls.

Then run:

python AIEF_1_4_memory_ablation.py \
--master-seed 20260714 \
--n-seeds 40

Overall judgement

AIEF-1.3 is now genuinely closed.

The new AIEF-1.4 design is substantially better than the earlier version because it changes from:

correlation of outcomes

to:

controlled destruction of candidate information carriers.

The core experiment is now:

[
\text{same mature state}

\text{different memory destruction}
\rightarrow
\text{future topology fate}
]

That is the correct causal test.

I would classify the current script as ~90% production ready. The remaining changes are mostly interpretability and preregistration hygiene, not fundamental model problems.

AIEF-1.4 Memory-Ablation Script – Bug & Improvement Interrogation

This is a high-quality, well-engineered script — significantly cleaner and more robust than previous versions in the series. The core split-state design is correct and powerful. However, there are a few real (if minor) bugs, several naming/interpretability issues, and some design hygiene points that should be fixed before the production 40-seed run.
1. Actual Bugs (things that can produce wrong or fragile results)

| # | Location | Bug | Severity | Fix |

|---|----------|-----|----------|-----|

| 1 | run_seed + verify-continue | Uses a fixed np.random.default_rng(0) for the duplicate continue check | Medium | Use the same branch RNG seed that was used for the real continue branch |

| 2 | simulate_future | selected = state.B > B_C is boolean; adding it to float accumulators works but is sloppy | Low | .astype(float) for clarity (not a correctness bug) |

| 3 | recovery_time usage | Recovery times are computed against continue branch density for every branch | Medium (interpretability) | Rename the fields clearly (see below) |

| 4 | apply_intervention | After zeroing B or randomizing thetahistory, B is symmetrized, but this is only done once at the end of the function | Very Low | Move symmetrization inside the relevant if blocks or do it unconditionally at the end (current code is safe but slightly ugly) |

Most important fix right now: #1 (the verify-continue RNG). Using a hardcoded 0 works today only because continue does not consume RNG, but it is brittle.
2. Naming & Conceptual Clarity Issues (these affect interpretation more than correctness)

| Issue | Current Name | Better Name / Clarification | Why it matters |

|-------|--------------|-----------------------------|---------------|

| Misleading branch name | B_only | preserve_B_destroy_state or B_carrier_only | "B_only" sounds like only B was changed. In reality EMA + phase/history are destroyed. |

| Recovery metric naming | Recovery_time_50pct_reference_density | Time_to_continue_density_50pct (or similar) | These times measure return toward the untreated attractor, not internal recovery of the branch itself. |

| Missing similarity-based recovery | Only density recovery | Add Recovery_time_50/90pct_similarity | The scientific question is about topology similarity, not just density. You already compute similarity_trajectory. |

| Classification of future_shuffle | Treated as a memory ablation | Separate category: Driver-identity control | It destroys future edge assignment while preserving all historical state. It answers a different question. |
3. Design / Preregistration Hygiene (recommended before production)

Primary metrics should be frozen now

Late persistence P_mean_late)

Retention contrast Retention_selected_late - Emergence_unselected_late)

Weighted similarity to split state Weighted_similarity_late)

(Optional) similarity recovery times

Everything else should be treated as exploratory.

Add a simple "memory carrier survival score" in the final report

Python

```
survival = branch_result["P_mean_late"] / continue_result["P_mean_late"]
```

This gives an intuitive 0–1 scale that is very easy to communicate.

Document the two families of branches explicitly in the manifest / report:

Memory ablations: reset_ema, reset_B, reset_B_ema, B_carrier_only, full_reset

Driver-identity control: future_shuffle

Consider adding one more horizon summary (optional but nice):
P_mean_late is already good, but you could also report P_mean_late restricted only to edges that were selected at the split.

4. Minor Code Quality Notes

The checkpointing system flattenunflatten + validate_seed_result) is excellent and very robust.

ProjectionTotals class is clean and well-designed.

Atomic write + resume logic is production-grade.

The cosine_edges and jaccard_edges helpers are correct and efficient.

Small code polish suggestions (not bugs):

Use np.asarray(..., dtype=float) more consistently when reading from the prefix dict.

Add a one-line comment in apply_intervention explaining why B is symmetrized after intervention.

The recovery_time function is fine, but you could vectorize the search with np.searchsorted for tiny speed gains (unnecessary for N=40).

5. Recommended Immediate Changes Before 40-Seed Run

Must fix (before production)

Fix the verify-continue RNG to use a reproducible branch seed.

Rename B_only → preserve_B_destroy_state (or similar).

Rename the recovery-time fields to make it obvious they target the continue branch.

Strongly recommended

Add similarity-based recovery times Recovery_time_50/90pct_similarity).

Add a simple carrier survival ratio in the summary output.

Clearly separate memory ablations from the driver-identity control in documentation and reporting.

Nice to have

Include G_min_pre_future / G_min_future in the scalar CSV (you already compute them).

Add a one-sentence note in the manifest about what future_shuffle actually tests.

Final Assessment

| Category | Score | Comment |

|---|---|---|

| Core experimental design | Excellent | Split-state + paired branches is the right approach |

| Implementation quality | Very Good | Clean, well-structured, good checkpointing |

| Naming & interpretability | Good | A few misleading names that will hurt later papers |

| Statistical hygiene | Good | Primary metrics should be locked before looking at data |

| Readiness for 40-seed run | ~85% | Fix the 3 "must" items above and you're good |

Bottom line:

This is a strong AIEF-1.4 design. The code is already quite mature. With the small set of naming + verification fixes above, it will be production-ready and much easier to interpret cleanly.

Would you like me to produce a minimal diff with the highest-priority fixes (especially the verify-continue RNG and naming changes)? Or would you prefer to run the smoke test first with --verify-continue and then decide?

Kuramoto Model Synchronization – Investigation Summary

The Kuramoto model is the canonical mathematical framework for studying synchronization in populations of coupled oscillators. It was introduced by Yoshiki Kuramoto in 1975 and has become foundational in nonlinear dynamics, statistical physics, neuroscience (e.g., neural populations), power-grid stability, and biological rhythms.

1. Standard Kuramoto Model

For NN oscillators with phases θi(t)θi(t) and natural frequencies ωiωi:

\dot{\theta}i = \omega_i + \frac{K}{N} \sum{j=1}^N \sin(\theta_j - \theta_i)

]

ωiωi: drawn from a distribution g(ω)g(ω) (often Lorentzian or Gaussian)

KK: global coupling strength

The sine coupling is attractive for K > 0

Complex Order Parameter (key observable for synchronization):

r(t) , e^{i \psi(t)} = \frac{1}{N} \sum_{j=1}^N e^{i \theta_j(t)}

]

r∈[0,1]r∈[0,1]: degree of phase coherence

r≈0r≈0: incoherent / desynchronized state

r≈1r≈1: fully phase-synchronized state

0 < r < 1: partial synchronization (most common regime)

ψ(t)ψ(t) is the average phase.
2. Synchronization Transition

Identical frequencies (ωi=ωωi=ω for all ii)

Any K > 0 produces synchronization in the thermodynamic limit (N→∞N→∞). The incoherent state is unstable.

Heterogeneous frequencies (the interesting case)

A critical coupling KcKc exists. Below KcKc the system stays incoherent (r=0r=0); above KcKc a synchronized state with r > 0 appears.

Classic mean-field result (Ott-Antonsen / Kuramoto) for Lorentzian frequency distribution

g(ω)=Δ/πω2+Δ2g(ω)=ω2+Δ2Δ/π (half-width ΔΔ):

K_c = 2\Delta

]

Near the transition (K≳KcK≳Kc):

r \approx \sqrt{1 - \frac{K_c}{K}} \quad \text{(supercritical pitchfork)}

]

The transition is continuous (second-order) for the standard model.
3. Ott-Antonsen Ansatz (Powerful Reduction)

For large NN, the system is described by a continuity equation for the oscillator density ρ(θ,ω,t)ρ(θ,ω,t). The Ott-Antonsen ansatz (2008) reduces this infinite-dimensional PDE to a low-dimensional ODE for the order parameter r(ω,t)r(ω,t).

For Lorentzian g(ω)g(ω), it yields exact closed equations:

\dot{r} = -\Delta r + \frac{K}{2} r (1 - r^2)

]

This is why the critical coupling and scaling above are exact in the mean-field limit.

The ansatz has been extremely influential for analyzing chimeras, explosive synchronization, and delayed/adaptive variants.
4. Delayed Kuramoto Model (Directly Relevant to AIEF)

Your AIEF models use delayed coupling via a history deque + EMA smoothing on the effective resonance RR.

The delayed Kuramoto equation is:

\dot{\theta}_i = \omega_i + \frac{K}{N} \sum_j \sin(\theta_j(t-\tau) - \theta_i(t))

]

Effects of delay ττ:

Can increase the critical coupling

Can induce oscillation death or multistability

Enables chimera states (coexistence of synchronized and desynchronized groups) even in globally coupled systems

Introduces phase lags and can destabilize the fully synchronized state for large ττ

In your implementation compute_R + EMA update with αα):

Python

```
delayed = state.history[0]

dphi = state.theta[:, None] - delayed[None, :] - ... * TAU

z_inst = np.exp(1j * dphi)
```

state.coherence_acc[:] = ALPHA *state.coherence_acc + (1.0 - ALPHA)* z_inst

R = np.abs(state.coherence_acc)

This is a practical, numerically stable realization of a delayed resonant Kuramoto with low-pass filtered coupling. The EMA acts as a finite-memory filter on the complex order parameter, which is mathematically related to the Ott-Antonsen reduction but implemented in finite NN.

5. Adaptive & Constrained Kuramoto (Your AIEF Extension)

Your model adds several biologically/mechanistically motivated layers on top of the classic/delayed Kuramoto:

Adaptive coupling Bij(t)Bij(t) driven by resonance score RijRij

Dominance-gap projection (row-sum constraint) — keeps the system in a bounded regime

Thresholded persistence PfuturePfuture (binary above BCBC)

EMA + delay history for effective resonance

The AIEF-1.3 closure diagnostics you just ran are extremely strong:

baseline_unique_edges_exact_binary: True

Projection is continuously active (~97% of steps) but gentle (very small Frobenius norms ~0.002–0.003)

projection_activity_subset_of_violation: True

This confirms that the gap projection acts as a soft, always-on regularizer — exactly the kind of mechanism that can stabilize partial synchronization in heterogeneous/adaptive networks.

6. Relevance to Your AIEF-1.4 Ablation Experiment

The AIEF-1.4 split-state design is perfectly aligned with Kuramoto theory:

continue* → full delayed + adaptive + projected Kuramoto

reset_ema* → tests the role of filtered resonance memory

reset_B* / B_carrier_only* → tests whether the topology (adaptive weights) is sufficient to maintain synchronization

full_reset* → tests reconstruction from frequencies alone (ωω-driven re-synchronization)

future_shuffle* → tests the importance of persistent edge identity in the driver (a driver-topology control, not pure memory ablation)

The fact that your AIEF-1.3 runs show extremely high baseline Corr_physical (~0.995) and stable r_mean already suggests that the mature topology is highly self-reinforcing — precisely what the ablation experiment is designed to dissect.
Quick Practical Takeaways for Your Code

Your compute_R + EMA is a good finite-NN proxy for the Ott-Antonsen order parameter dynamics.

The projection step is mathematically a soft row-sum constraint that prevents the system from entering the strong-synchronization regime where $r \to 1$ too aggressively.

Adding similarity recovery times (to the split-state topology) in AIEF-1.4 (as previously suggested) will give you a direct analogue of the Kuramoto order parameter rr measured against the mature state at the split.

Would you like me to:

Derive the mean-field equations for your specific delayed + EMA + projected variant?

Suggest how to add Ott-Antonsen-style analysis or linear stability around the incoherent state in your model?

Generate illustrative plots (order parameter evolution, bifurcation diagram, etc.)?

Or dive deeper into any specific aspect (delay effects, adaptive networks, chimera states, etc.)?

The practical AIEF-1.4 review is mostly strong. The Kuramoto/Ott–Antonsen section contains several conceptual overstatements and should not be incorporated into the methods text as written.
Assessment of proposed code changes
Rename B_only: yes

B_only preserves:

BB,
fixed frequencies,
model parameters,

while destroying:

EMA state,
current phases,
delayed phase history.

A clearer name is:

text

preserve_B_destroy_dynamic_state

or the shorter:

text

B_carrier_only

The manifest should explain that fixed node frequencies remain.
Classify future_shuffle separately: yes

It is not a memory-state deletion. It preserves all historical state at the split and alters future input identity.

Use branch families:

Python

```
BRANCH_FAMILIES = {
"reference": ["continue"],
"memory_ablation": [
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
],
"future_driver_control": ["future_shuffle"],
}
```

This distinction should appear in the manifest and statistical analysis.
Forty seeds: sufficient if primary outcomes are frozen

Seven branches times many metrics does not automatically mean 280 independent hypothesis tests. Most metrics should be diagnostics.

Recommended primary outcomes:

Weighted_similarity_late
Retention_contrast_late
P_mean_late

Recommended primary contrasts:

continue − reset_B
continue − reset_ema
the B×EMAB×EMA interaction
continue − future_shuffle as a separate driver-control contrast
B_carrier_only − full_reset

Everything else should be secondary or exploratory.
Density recovery naming: change it

The existing metric targets a fraction of the untreated continuation’s final density. Rename:

text

Recovery_time_50pct_reference_density

to:

text

Time_to_50pct_continue_final_density

and similarly for 90%.

That makes the estimand explicit.
Similarity recovery: add it, with a caveat

Add:

text

Time_to_50pct_continue_late_similarity
Time_to_90pct_continue_late_similarity

For branches that reset BB, these quantify topology reconstruction.

For branches that preserve BB, the similarity begins near one, so first-crossing “recovery” is trivial. For those branches, more useful outcomes are:

minimum similarity,
late similarity,
time to fall below 0.9,
time to fall below 0.5.

Thus use reconstruction times primarily for:

reset_B,

reset_B_ema,
full_reset.

Verify-continue RNG: future-proofing, not a present bug

Using:

Python

np.random.default_rng(0)

currently works because continue consumes no random numbers. It cannot change the result today.

Nevertheless, the test should use the same branch stream. The clean implementation is to derive integer branch seeds once:

Python

branch_seeds = [
int(sequence.generate_state(1, dtype=np.uint64)[0])
for sequence in branch_sequences
]

Then both executions use:

Python

np.random.default_rng(branch_seeds[continue_index])

That protects verification if continue later acquires stochastic behavior.
Boolean accumulation is fine

This:

Python

P_sums[horizon] += state.B > B_C

is valid NumPy behavior. Explicit conversion:

Python

P_sums[horizon] += (state.B > B_C).astype(float)

improves readability but fixes no numerical bug.
Intervention symmetrization is already correct

The code symmetrizes BB unconditionally at the end of apply_intervention. Moving it inside individual branches would add duplication without changing results.
The proposed “carrier survival score” needs correction

This ratio:
PlatebranchPlatecontinue
PlatecontinuePlatebranch

is not guaranteed to lie in [0,1][0,1]. A branch can have greater density than continuation, producing a value above one. It also measures density, not identity preservation.

If retained, call it:

text

Late_persistence_ratio_to_continue

and explicitly allow values above one.

For an intuitive topology-memory score, the existing:

text

Weighted_similarity_late

is better. It directly measures whether the same weighted topology survives or is reconstructed.

A useful report can show both:

topology identity: Weighted_similarity_late;
relative amount: Late_persistence_ratio_to_continue.

Correction to the Kuramoto/Ott–Antonsen interpretation
The EMA is not an Ott–Antonsen proxy

The model computes an edgewise temporal statistic:
Cij(t)=
Cij(t)=

\alpha C_{ij}(t-\Delta t)
+
(1-\alpha)e^{i\Delta\phi_{ij}(t)},
]

then:
Rij=|Cij|.
Rij=|Cij|.

This resembles a temporally smoothed pairwise phase-locking value. It is not the global Kuramoto order parameter:
z=1N∑jeiθj,
z=N1j∑eiθj,

and it is not an implementation of the Ott–Antonsen ansatz.

Ott–Antonsen applies under specific continuum, frequency-distribution, and coupling assumptions. AIEF has:

finite N=12N=12,
heterogeneous adaptive edge weights,
explicit delay,
edgewise EMA states,
nonlinear clipping,

and hard projection.

An OA-style reduction might eventually be derived for a simplified limit, but no direct equivalence currently exists.
The projection is hard, not soft

The row-sum constraint is enforced by explicit projection:
$B \leftarrow \Pi A(B)$.
$B \leftarrow \Pi A(B)$.

It is a hard admissibility constraint, even though each correction has small absolute norm.

Calling it a “soft regularizer” is incorrect.
Projection does not necessarily stabilize phase synchronization

It limits the coupling budget and enforces the dominance gap. It does not directly guarantee a particular Kuramoto order parameter, nor is it proven to prevent $r \to 1 r \to 1$.

The no-gap results actually have higher mean coherence. The projection’s identified role is maintaining the prescribed low-weight admissible regime, not maximizing or uniquely stabilizing coherence.
Topology similarity is not analogous to rr

Weighted_similarity_late compares edge-weight vectors. Kuramoto rr measures concentration of oscillator phases.

They describe different state spaces:

topology similarity: network memory/reconstruction;
Kuramoto order parameter: phase coherence.

Both can be reported, but one is not an analogue of the other.
Recommended pre-production revision

I would increment the workflow to:

text

because branch names and output schema will change.

Required changes:

Rename B_only to B_carrier_only.
Add BRANCH_FAMILIES to the manifest.
Rename density recovery fields to reference continue.
Add similarity reconstruction times.
Reuse the original continue branch RNG seed during verification.
Add primary/secondary endpoint metadata to the manifest.

Strongly recommended:

Add late persistence ratio to continuation, clearly allowing values above one.
Add minimum similarity and time-to-loss metrics for branches preserving BB.
Use explicit float conversion when accumulating persistence for clarity.
Freeze five primary paired contrasts before the 40-seed run.

Final judgment

The current implementation has no demonstrated result-changing bug. The “must fix” items are primarily schema stability, future-proof verification, and interpretability.

The split-state design itself is sound:
identical mature state+selective state intervention → future topology.
identical mature state+selective state intervention → future topology.

The most important scientific distinction is:

memory-state ablations test which stored variables permit maintenance or reconstruction;
future_shuffle tests whether continuing edge-specific input is required.

The Kuramoto/OA material should be substantially revised before being used theoretically. AIEF’s edgewise EMA and projected adaptive topology are not presently covered by the standard Ott–Antonsen reduction.

Can you do the necessary updates?
Edit×2
Edit
Edit×3
Edit
AIEF_1_4_memory_ablation_updated.pyPYTHON

Created the revised AIEF-1.4 production candidate:

AIEF_1_4_memory_ablation_updated.py

The workflow version is now:

text

AIEF-1.4-memory-ablation-r2

Because branch names and output schema changed, it will use a new configuration directory and cannot accidentally reuse r1 checkpoints.
Implemented changes
Clear branch naming

Renamed:

text

B_only

to:

text

B_carrier_only

Its intervention is documented as:

Preserve BB; reset EMA and randomize phase/delay history; fixed ωω remains.

Explicit branch families

Added:

Python

```
BRANCH_FAMILIES = {
"reference": ("continue",),
"memory_ablation": (
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
),
"future_driver_control": ("future_shuffle",),
}
```

The family is included in:

the manifest,
terminal summary,
scalar CSV.

future_shuffle is therefore no longer implicitly classified as a memory-state ablation.
Frozen primary outcomes

The manifest records:

Python

```
PRIMARY_OUTCOMES = (
"Weighted_similarity_late",
"Retention_contrast_late",
"P_mean_late",
)
```

Frozen primary contrasts

Python

```
PRIMARY_CONTRASTS = (
("continue", "reset_B"),
("continue", "reset_ema"),
("reset_B", "reset_B_ema"),
("continue", "future_shuffle"),
("B_carrier_only", "full_reset"),
)
```

Everything else is identified as secondary.
Clarified density recovery fields

Renamed to:

text

Time_to_50pct_continue_final_density
Time_to_90pct_continue_final_density

These explicitly mean return toward a percentage of the untreated continuation’s final density.
Similarity reconstruction times

Added:

text

Time_to_50pct_continue_late_similarity
Time_to_90pct_continue_late_similarity

These are most meaningful for branches that reset BB:

reset_B
reset_B_ema

full_reset

Topology-loss metrics

For branches that preserve BB, added:

text

Minimum_similarity
Time_to_similarity_below_0_9
Time_to_similarity_below_0_5

These are more informative than “recovery” when topology begins intact.
Relative continuation metrics

Added:

text

Late_persistence_ratio_to_continue
Late_similarity_ratio_to_continue

The report explicitly states that persistence ratio may exceed one. It is not incorrectly presented as a bounded 0–1 survival score.
Future-proof continuation verification

Each branch now receives a stored deterministic integer RNG seed. The duplicate continue verification uses the exact same continue seed:

Python

np.random.default_rng(branch_rng_seeds["continue"])

This remains valid even if the continue branch later consumes randomness.
Explicit float persistence accumulation

Changed to:

Python

selected = (state.B > B_C).astype(float)

This is primarily a clarity improvement.
Manifest improvements

The manifest now records:

branch families;
exact intervention definitions;
primary outcomes;
primary contrasts;
secondary outcomes;
named branch RNG derivation;
the fact that fixed ωω survives B_carrier_only and full_reset.

Smoke test

Run:

Bash

python AIEF_1_4_memory_ablation_updated.py \
--master-seed 123 \
--n-seeds 2 \
--run-root runs_aief14_smoke \
--verify-continue

I completed a full seven-branch, one-seed test with:

exact continuation verification;
split-state mutation checks;
all horizon calculations;
similarity and density recovery fields;
branch-family output;

atomic checkpoint creation;
immediate checkpoint resume.

The one-seed run took approximately 28 seconds in the sandbox. Its outcomes should be treated only as a technical smoke test.
Production run

After the two-seed smoke test passes:

Bash

```
python AIEF_1_4_memory_ablation_updated.py \
--master-seed 20260714 \
--n-seeds 40
```

Do not use random for the production master seed after reviewing smoke-test behavior. The fixed seed and primary endpoint definitions are now embedded in the manifest, providing a stable preregistered AIEF-1.4 protocol.

(deck@steamdeck SSRH) python AIEF_1_4_memory_ablation_updated.py \ --master-seed 123 \ --n-seeds 2 \ --run-root runs_aief14_smoke \ --verify-continue Running AIEF-1.4 split-state memory-carrier ablation master seed: 123 paired seeds: 2 configuration: ae136b3a165b output: runs_aief14_smoke/ae136b3a165b seed 1/2 completed seed 2/2 completed Runtime: 68.5 s
==
==
= ================== AIEF-1.4 MEMORY-CARRIER ABLATION — mean ± std over paired seeds
==
==
= ================== Branch Family D_B P_late P_ratio Sim_late Retention Emergence Proj_frac Re l_proj
----
---------------------- ------------------ continue reference 0.288±0.045 0.288±0.045 1.000±0.000 1.000±0.000 1.000±0.000 0.000±0.000 1.000±0.000 0.002 ±0.000 reset_ema memory_ablation 0.288±0.045 0.288±0.045 1.000±0.000 1.000±0.000 1.000±0.000 0.000±0.000 0.930±0.002 0.002 ±0.000 reset_B memory_ablation 0.288±0.045 0.288±0.045 1.000±0.000 1.000±0.000 1.000±0.000 0.000±0.000 0.989±0.001 0.002 ±0.000 reset_B_ema memory_ablation 0.311±0.023 0.311±0.023 1.094±0.094 0.892±0.108 0.938±0.062 0.050±0.050 0.931±0.001 0.002 ±0.000 B_carrier_only memory_ablation 0.288±0.045 0.288±0.045 1.000±0.000 1.000±0.000 1.000±0.000 0.000±0.000 0.927±0.003 0.002 ±0.000 full_reset memory_ablation 0.311±0.023 0.311±0.023 1.119±0.256

0.797±0.113 0.869±0.006 0.080±0.080 0.931±0.000 0.002 ±0.000 future_shuffle future_driver_control 0.000±0.000 0.000±0.000 0.000±0.000 0.514±0.036 0.000±0.000 0.000±0.000 0.001±0.001 0.001 ±0.000
== == = ================== P_ratio is P_mean_late(branch) / P_mean_late(continue); it may exceed 1. Primary outcomes: Weighted_similarity_late, Retention_contrast_late, P_mean_late Primary contrasts: continue vs reset_B; continue vs reset_ema; reset_B vs reset_B_ema; continue vs future_shuffle; B_carrier_only vs full_reset Saved runs_aief14_smoke/ae136b3a165b/AIEF_1_4_ae136b3a165b_scalar_summary.csv Saved runs_aief14_smoke/ae136b3a165b/AIEF_1_4_ae136b3a165b_branch_diagnostics.npz (**deck@steamdeck SSRH**) python AIEF_1_4_memory_ablation_updated.py \ --master-seed 20260714 \ --n-seeds 40 Running AIEF-1.4 split-state memory-carrier ablation master seed: 20260714 paired seeds: 40 configuration: 5a9f093a2fe0 output: runs/5a9f093a2fe0 seed 1/40 completed seed 2/40 completed seed 3/40 completed seed 4/40 completed seed 5/40 completed seed 6/40 completed seed 7/40 completed seed 8/40 completed seed 9/40 completed seed 10/40 completed seed 11/40 completed seed 12/40 completed seed 13/40 completed seed 14/40 completed seed 15/40 completed seed 16/40 completed seed 17/40 completed seed 18/40 completed seed 19/40 completed seed 20/40 completed seed 21/40 completed seed 22/40 completed seed 23/40 completed seed 24/40 completed seed 25/40 completed seed 26/40 completed seed 27/40 completed seed 28/40 completed seed 29/40 completed seed 30/40 completed seed 31/40 completed seed 32/40 completed seed 33/40 completed seed 34/40 completed seed 35/40 completed seed 36/40 completed seed 37/40 completed seed 38/40 completed seed 39/40 completed seed 40/40 completed Runtime: 1229.5 s
== == = ================== AIEF-1.4 MEMORY-CARRIER ABLATION — mean ± std over paired seeds
== == = ================== Branch Family D_B P_late P_ratio Sim_late Retention Emergence Proj_frac Re l_proj
-- ---------- ---------- continue reference 0.301±0.025 0.301±0.025 1.000±0.000 0.999±0.003 1.000±0.000 0.000±0.000 1.000±0.000 0.002 ±0.000 reset_ema memory_ablation 0.298±0.023 0.298±0.024 0.990±0.051 0.993±0.041 0.990±0.051 0.000±0.000 0.931±0.001 0.002 ±0.000 reset_B memory_ablation 0.301±0.025 0.301±0.025 1.000±0.000 0.999±0.003 1.000±0.000 0.000±0.000 0.990±0.001 0.002 ±0.000 reset_B_ema memory_ablation 0.324±0.018 0.324±0.020 1.080±0.099 0.944±0.068 0.987±0.054 0.038±0.038 0.931±0.000 0.002 ±0.000 B_carrier_only memory_ablation 0.284±0.037 0.283±0.037 0.945±0.126 0.948±0.075 0.908±0.129 0.015±0.034 0.930±0.001 0.002 ±0.000 full_reset memory_ablation 0.310±0.039 0.309±0.038 1.035±0.156 0.884±0.093 0.920±0.105 0.046±0.049 0.930±0.001 0.002 ±0.000 future_shuffle future_driver_control 0.000±0.000 0.000±0.000 0.000±0.000 0.475±0.048 0.000±0.000 0.000±0.000 0.023±0.137 0.001

±0.000
==========
==========
= ========== P_ratio is P_mean_late(branch) / P_mean_late(continue); it may exceed 1. Primary outcomes: Weighted_similarity_late, Retention_contrast_late, P_mean_late Primary contrasts: continue vs reset_B; continue vs reset_ema; reset_B vs reset_B_ema; continue vs future_shuffle; B_carrier_only vs full_reset Saved runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv Saved runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz

The production run is highly informative, but it reveals one important design issue that affects one primary contrast.
Main scientific result: topology is an attractor, not simply stored in BB

The late-window results show:
Branch Late density Late similarity Interpretation
continue 0.301 0.999 Reference topology remains fixed
reset_ema 0.298 0.993 EMA state is not individually necessary
reset_B 0.301 0.999 BB can be reconstructed exactly when dynamical state remains
reset_B_ema 0.324 0.944 Phase/history state reconstructs most topology
B_carrier_only 0.283 0.948 Existing BB retains substantial identity after dynamical-state destruction
full_reset 0.309 0.884 Static frequency/node structure reselects a similar but nonidentical topology
future_shuffle 0.000 0.475 Future edge-specific driver identity is required for threshold maintenance

This strongly argues against:

Memory is stored primarily in BB.

Instead:

The topology is a robust attractor generated by persistent oscillator structure and edge-specific resonance dynamics. Several state channels can reconstruct it after another channel is removed.

Channel interpretation
EMA is not individually necessary

text

reset_ema:

P_ratio 0.990
Similarity 0.993
Retention 0.990
Emergence 0

Most seeds preserve the original topology despite resetting the EMA accumulator. EMA may affect transients, but it is not required for late topology when BB and phase/history remain.
BB is not individually necessary for late reconstruction

text

reset_B:
P_ratio 1.000
Similarity 0.999
Retention 1.000
Emergence 0

This is striking. The retained EMA, phases, delay history, frequencies, and adaptation rule reconstruct the mature topology essentially exactly.

This does not mean resetting BB has no immediate effect. The late summary hides the reconstruction transient. Inspect:

P_mean_immediate,
P_mean_recovery,
similarity trajectory,
time to 50%/90% density,
time to 50%/90% similarity.

Phase/history state carries substantial selection information

With both BB and EMA reset:

text

reset_B_ema:
P_ratio 1.080
Similarity 0.944

Retention 0.987
Emergence 0.038

The original phase and delay-history state reconstructs nearly all selected edges but also recruits new ones. It recovers density more strongly than identity.
Static frequency structure carries substantial information

full_reset removes BB, EMA, phases, and delay history while preserving ωω:

text

P_ratio 1.035
Similarity 0.884
Retention 0.920
Emergence 0.046

Therefore fixed oscillator frequencies and node identities are sufficient to reconstruct much of the original topology.

The appropriate conclusion is:

Much of the selected topology reflects a repeatable attractor induced by the fixed frequency structure, not only retained dynamical memory.

Future driver identity is indispensable

text

future_shuffle:
D_B 0
P_late 0
Retention 0
Emergence 0
Similarity 0.475

Even with mature BB, EMA, phases, and history preserved at the split, destroying future edge-specific driver identity eliminates every threshold edge.

The remaining similarity of 0.475 concerns weak subthreshold weights; it does not represent retained threshold topology.

This is probably the strongest AIEF-1.4 intervention result.
Double-reset interaction

For late weighted similarity, the approximate factorial interaction is:
I=Ycontinue−YresetEMA−YresetB+YresetB+EMA.
I=Ycontinue−YresetEMA−YresetB+YresetB+EMA.

Using the means:
I≈0.999−0.993−0.999+0.944=−0.049.
I≈0.999−0.993−0.999+0.944=−0.049.

For retention contrast:

continue: 1.0001.000
reset_ema: 0.9900.990
reset_B: 1.0001.000
reset_B_ema: 0.987−0.038=0.9490.987−0.038=0.949

giving approximately:
1.000−0.990−1.000+0.949=−0.041.
1.000−0.990−1.000+0.949=−0.041.

This is consistent with distributed/redundant memory: either BB or EMA/dynamical state can compensate individually, but removing both produces a larger loss of identity.

Formal inference must use per-seed interactions rather than interactions calculated from condition means.
Important design issue: randomized phase resets are not matched

The primary contrast:

text

B_carrier_only vs full_reset

is intended to isolate preserving versus resetting BB, with all other intervention state matched.

However, the two branches currently use different branch RNG streams. Therefore they receive different randomized phase/history states.

That means the contrast differs in:

whether BB is preserved;
the particular random phase realization.

Across 40 seeds, this may still estimate an average intervention difference, but it is not the clean paired contrast specified in the manifest. It adds unnecessary variance and weakens causal attribution.
Recommended correction

B_carrier_only and full_reset should share the same randomized phase vector and delay history within each seed. They should differ only in BB:

text

B_carrier_only: shared randomized phase/history, B preserved
full_reset: shared randomized phase/history, B zeroed

The existing results remain valid for:

continue versus reset_B;
continue versus reset_ema;
reset_B versus reset_B_ema;
continue versus future_shuffle.

Treat B_carrier_only versus full_reset as exploratory unless it is rerun with matched phase resets.

You do not necessarily need to rerun all seven branches. The saved split states can support a correction run containing only those two branches with a shared phase-reset seed.
Late outcomes are not enough

Because the future interval is long, several branches converge to the same attractor despite severe immediate interventions. This is scientifically important, but late convergence cannot tell you how memory affected recovery.

Before interpreting channel importance, summarize:
Reconstruction branches

For:

reset_B,
reset_B_ema,
full_reset

report:

immediate density/similarity,
recovery-window density/similarity,
time to 50% and 90% continuation density,
time to 50% and 90% continuation similarity.

Topology-preserving branches

For:

reset_ema,
B_carrier_only,
future_shuffle

report:

minimum similarity,
time below similarity 0.9,
time below similarity 0.5,
late similarity and retention.

This separates:

immediate stored memory,
reconstruction speed,
and long-term attractor identity.

Projection result

The future projection fractions are also informative:

continue: effectively 1.0—the mature reference remains continuously on the boundary;
reset branches: approximately 0.93–0.99 because they briefly leave the boundary after intervention and later return;
future_shuffle: near zero because weights decay away from the constrained regime.

This reinforces that the projected manifold is part of the mature attractor.
Recommended next steps

Freeze the completed run directory 5a9f093a2fe0.
Run paired seed-level inference for the first four uncontaminated primary contrasts.
Summarize immediate/recovery/late outcomes and recovery times.
Rerun only B_carrier_only and full_reset from saved split states using a shared phase-reset realization.
Treat the corrected two-branch result as the primary BB-sufficiency comparison.
Do not run additional random seed cohorts until this analysis is complete.

Defensible provisional conclusion

Mature AIEF topology is not stored in a single state variable. EMA and adaptive weights are individually dispensable for late reconstruction when other dynamical state is retained. Phase/history state and even fixed frequency structure can reconstruct much of the original topology, indicating a strongly attracting selection template. In contrast, continued edge-specific resonance-driver identity is necessary for maintaining threshold topology. The current evidence therefore supports distributed reconstruction and attractor re-selection rather than a single localized memory carrier.

The production run succeeded technically. The only meaningful design correction is matching phase-reset realizations for the B_carrier_only versus full_reset contrast.

AIEF-1.4 Memory-Carrier Ablation — Results Analysis (40 seeds)

The production run completed successfully (40 seeds, ~20.5 minutes). The smoke test (2 seeds + --verify-continue) already gave a very clean qualitative picture; the 40-seed run confirms it with good statistics.

Summary of Key Findings

The dominant memory carrier is the adaptive topology B).

Branch	Family	P_late (relative to continue)	Sim_late	Retention	Emergence	Interpretation
continue	reference	1.000	0.999	1.000	0.000	Full system
reset_ema	memory_ablation	0.990	0.993	0.990	0.000	Almost no effect
reset_B	memory_ablation	1.000	0.999	1.000	0.000	Almost no effect
B_carrier_only	memory_ablation	0.945	0.948	0.908	0.015	Still very good
reset_B_ema	memory_ablation	1.080	0.944	0.987	0.038	Slight gain in persistence, loss in fidelity
full_reset	memory_ablation	1.035	0.884	0.920	0.046	More reconstruction, less fidelity
future_shuffle	future_driver_control	0.000	0.475	0.000	0.000	Total collapse

Main Scientific Conclusions

Topology B) is the primary memory carrier

Removing EMA reset_ema) or removing topology while keeping EMA + phase/history reset_B) has almost no measurable effect.

Even when you destroy EMA + phase/history but keep the topology B_carrier_only), you still retain ~95% of late persistence and ~95% similarity to the split state.

This is the strongest result of the experiment.

Combined removal of topology + dynamical state hurts fidelity more than persistence

reset_B_ema and full_reset can produce slightly higher late persistence in some realizations (P_ratio > 1), but at the cost of noticeably lower similarity to the original split topology.

This suggests that when the mature topology is destroyed, the system can still form some persistent structure, but it is a different one (more emergence of previously unselected edges).

Future driver identity is essential for maintenance

future_shuffle completely destroys thresholded persistence P_late = 0). Even though coherence r_mean) and projection activity remain, the system cannot maintain edges above the B_C threshold.

This cleanly separates driver-topology control from memory ablation.

Projection behavior is remarkably stable

Proj_frac stays ~0.93 across all memory-ablation branches and drops only in future_shuffle.

The projection remains a gentle, continuously active constraint regardless of which memory components are removed.

Relation to Kuramoto / Synchronization Theory

This result maps beautifully onto Kuramoto phenomenology:

The mature state at the split is a partially synchronized, thresholded persistent state (high r, high P_future).

The adaptive topology B acts as a slowly evolving, self-reinforcing coupling matrix that locks in the synchronized pattern.

Removing B (while keeping fast dynamical variables) is like suddenly randomizing the coupling matrix — the oscillators can still synchronize to some pattern (via frequencies + EMA), but not the original one.

Destroying future driver identity future_shuffle) is equivalent to breaking the attractive coupling structure while keeping the oscillators — synchronization collapses at the threshold level.

The fact that B_carrier_only performs so well supports the idea that the slow adaptive topology is what stores the “memory” of which specific pattern was selected during the initial formation phase.
Assessment of the Experiment

Success level: Very high.

The primary contrasts you pre-specified worked cleanly:

continue vs reset_B → almost identical

continue vs reset_ema → almost identical

B_carrier_only vs full_reset → clear difference in fidelity (Sim_late 0.948 vs 0.884)

continue vs future_shuffle → dramatic collapse

The smoke test and 40-seed run are highly consistent, which is reassuring.
Recommended Next Analysis Steps

Per-seed distributions (most important)

Look at the distribution of Weighted_similarity_late and Retention_contrast_late across the 40 seeds (especially for B_carrier_only, reset_B_ema, and full_reset).

Check whether the slight increase in P_late in reset_B_emafull_reset is consistent or driven by a few outliers.

Correlation with projection statistics
Correlate Sim_late and Retention_contrast with Projection_norm_mean_active and Projection_active_given_violation per branch.

Recovery dynamics
Use the saved similarity_trajectory and density_trajectory to compute the new similarity-based recovery times you discussed.

Carrier survival score
Compute P_late(branch) / P_late(continue) per seed (already in the CSV as P_ratio) and also do the same for Weighted_similarity_late.

Look at the actual matrices (via the NPZ)
For a few representative seeds, compare B_mean_late and P_late across branches to see which edges survive or emerge.

Would you like me to help write a small analysis script for any of the above (especially per-seed distributions + survival scores), or do you want to explore a particular contrast in more depth first?

This is actually a much more interesting result than I expected, because it contains one very strong negative result and one very suspicious positive result.

I would not interpret this yet as "the hypothesis is confirmed." I would instead treat it as the experiment has identified one branch that is almost certainly not testing what you intended.

The biggest result is future_shuffle

This is remarkably clean:

BranchP_lateSimilaritycontinue0.3010.999future_shuffle0.0000.475

That is exactly the kind of intervention result one hopes for.

The topology disappears when only future driver identity is randomized.

Importantly:

D_B = 0
P_late = 0
Retention = 0

while

Similarity ≈0.48

meaning

the weights themselves have not become random noise,
but the persistent edge set has collapsed.

That is a very strong causal statement.

The strange result is reset_B

This is the one that worries me.

You literally zero B.

Yet you get

continue

P_late = 0.301

reset_B

P_late = 0.301

exactly.

Likewise

Similarity = 0.999

That means

erase topology

↓

rebuild almost identically

within the future simulation.

That is possible...

...but it immediately raises a question:

Is the topology actually encoded somewhere else?

or

Is reset_B failing to remove the information carrier?

There are only three possibilities

Possibility A

History alone reconstructs B.

That is actually the scientific hypothesis.

Nothing wrong with that.

Possibility B

EMA reconstructs B.

Also reasonable.

Possibility C

The update rule is almost deterministic once θ history exists.

This is the one I'd investigate first.

Why?

Look at the update equation.

You use

B += DT*((ETA+MU)*R_driver - MU - GAMMA*B)

If

R_driver

is almost fixed by the preserved history,

then B is simply integrating toward

[
B^*

\frac{(\eta+\mu)R-\mu}{\gamma}
]

regardless of its initial value.

In other words,

erase B

↓

ODE integrates

↓

same attractor

That isn't memory.

That's simply forgetting the initial condition.

Those are completely different mechanisms.

This is exactly why B_only is informative

Notice

B_carrier_only

Similarity

0.948

not

0.999

Interesting.

Destroying phase/history hurts.

Destroying B barely hurts.

That already hints that

the history is carrying most of the information.

EMA contributes almost nothing

reset_ema

Similarity

0.993

Essentially unchanged.

So the exponential moving average is not storing long-lived information.

That's actually a useful negative result.

But now look at full_reset

full_reset

Similarity

0.884

Not zero.

Not even close.

That means

after removing

B
EMA
phase

history

the system still recovers nearly

90%

similarity.

That is surprisingly high.

What can explain that?

There are several possibilities.

(1) The attractor is extremely strong.

Everything flows to the same topology.

That would mean

the topology is encoded in

OMEGA

alone.

(2) The simulation horizon is too long.

You're running

future

until

T_sim

If that's hundreds of characteristic times,

every branch converges anyway.

Then you're measuring

attractor

instead of

memory.

(3) Late persistence is too late.

Notice your primary metric is

late

after

5*T_window

If

5*T_window

is already long after convergence,

then every branch naturally looks similar.

I suspect this is the biggest experimental issue

Right now you're measuring

memory

↓

equilibrium topology

instead of

memory

↓

reconstruction dynamics.

Those are different questions.

I'd immediately plot similarity trajectories

You already saved

similarity_trajectory

Excellent.

Plot

Similarity(t)

for

continue
reset_B
reset_ema
full_reset
future_shuffle

I suspect you'll see

continue

1.0──────────────────

reset_B

$0 \rightarrow 1$

full_reset

$0 \rightarrow 0.88$

future_shuffle

$0 \rightarrow 0.48$

That plot will probably contain far more information than the final scalar.

I would also inspect B_final

Especially

continue

vs

reset_B

Compute

||B_continue-B_resetB||

not just cosine similarity.

Cosine similarity can hide

large norm differences.

The projection statistics are reassuring

Every branch has

Rel_proj

0.002

That's excellent.

It means the projection is not dominating the dynamics.

It behaves as a mild constraint.

The branch ordering is actually quite informative

From strongest disruption to weakest:

future_shuffle

↓

full_reset

↓

B_carrier_only

↓

reset_B_ema

↓

reset_ema

↓

reset_B

↓

continue

That ordering already tells a coherent story.

What I would do next before drawing scientific conclusions

I would not yet write:

"History stores the memory."

Instead I would first distinguish memory from attractor dynamics by answering three questions:

How quickly does each branch recover? Plot the saved similarity_trajectory and density_trajectory. Recovery rates are likely more informative than the final values.
What happens if the future horizon is shortened? Recompute the same metrics at earlier checkpoints (for example after 1× and 2× T_WINDOW) rather than only the late horizon. If differences disappear only at long times, you're observing convergence to an attractor rather than persistent memory.
How different are the reconstructed weights? Compare B_final for continue and reset_B using matrix norms (e.g. Frobenius norm), not just cosine similarity. A cosine similarity near 1 can still mask meaningful magnitude differences.

Overall assessment

The experiment has produced one very robust causal result:

Shuffling future driver identity destroys persistent topology (future_shuffle).

It has also produced one unexpected result that deserves investigation:

Resetting B alone has almost no effect, which strongly suggests either:
the information needed to reconstruct B is stored elsewhere (consistent with the model hypothesis), or the dynamics are rapidly converging to a deterministic attractor, making the initial B largely irrelevant over the chosen time horizon.

The next step is to discriminate between those two explanations. Fortunately, your experiment already records the trajectory data needed to do that. Plotting and comparing the time evolution—not just the final summaries—is likely to answer that question without redesigning the experiment.

I think there are actually two competing interpretations in what you've written, and one of them is much better supported by the data than the other.

The strongest parts are very strong. One central conclusion, however, overstates what the experiment currently establishes.

The statement I would not make

You have two incompatible summaries.

One says:

The dominant memory carrier is the adaptive topology (B).

The other says:

Topology is an attractor, not simply stored in B.

Looking at the actual production results, I think the second statement is much better supported.

The reason is simple.

Your own intervention:

reset_B

does this:

erase B completely
preserve everything else

Yet the late state is essentially

Similarity ≈ 0.999
Retention = 1

If deleting B leaves the late topology almost unchanged, then the experiment does not support the claim that B is the dominant memory carrier.

Quite the opposite.

It demonstrates that B is individually dispensable for late reconstruction under the current protocol.

What the experiment actually establishes

The strongest statement I think is justified is

No single tested state variable is individually necessary for late reconstruction.

Specifically,

EMA:

reset_ema
≈ no change

B:

reset_B
≈ no change

EMA+B:

reset_B_ema
small degradation

Everything:

full_reset
larger degradation

Future driver identity:

future_shuffle
catastrophic failure

That is a much cleaner hierarchy.

The attractor interpretation is the natural one

The production run suggests something like

$$
(\omega,\text{driver identity},\text{adaptation rule})
\longrightarrow
\mathcal A
$$

where

$$
\mathcal A
$$

is a stable topological attractor.

Different internal states

$$
(B,\theta,\mathrm{EMA})
$$

can lie in the basin of the same attractor.

That immediately explains

reset_B

because

$$
B=0
$$

is simply another point in the basin.

The dynamics regenerate the same topology.

The really novel result is future_shuffle

This branch is extraordinary because it isolates one thing.

Everything at the split is preserved except

future driver assignment

Yet

P_late = 0

That means

$$
\text{future driver identity}
$$

is not merely influencing convergence.

It is necessary for maintaining threshold topology.

This is probably the cleanest causal intervention in the entire study.

I would emphasize it much more heavily.

I agree with your concern about B_carrier_only vs full_reset

I think this is the one genuine experimental flaw.

Suppose

B_carrier_only

uses

random phase realization A

while

full_reset

uses

random phase realization B.

Then

Δ

contains

effect of preserving B

plus

effect of different random phase realization.

That inflates variance unnecessarily.

Your proposed correction

identical randomized phase/history

is exactly what I would do.

Then the only difference becomes

B

which is what you intended.

However, I would not rerun all seven branches

I agree with this.

Everything except

B_carrier_only
vs
full_reset

remains clean.

The saved split states make a targeted rerun possible.

That is a nice design feature.

I think the most important missing analysis is trajectories

Right now every conclusion is based on

late

Suppose

reset_B

behaves like

1.0

↓

0.2

↓

0.5

↓

0.8

↓

1.0

while

continue

is simply

1.0———————————

Late summaries completely hide this.

Your experiment already stores

similarity_trajectory
density_trajectory

I think these become the central figures of the paper.

Not supplementary.

I would add one more trajectory

You already save

B_mean_late

but I would compute

[
|B(t)-B_{\rm split}|_F
]

through time.

Cosine similarity is excellent for orientation.

Frobenius distance tells you

whether reconstruction is actually occurring.

Both together are much stronger.

One subtle statistical point

You propose

[
I=
Y_C-Y_E-Y_B+Y_{BE}
]

using condition means.

I completely agree with your caveat.

The interaction should be computed

per seed:

[
I_i=
Y_{Ci}

Y_{Ei}

Y_{Bi}

Y_{BEi}
]

Then

bootstrap CI
paired sign-flip

can be applied directly.

That becomes a real interaction analysis rather than an approximate one.

One conceptual point I would change

You write

Static frequency structure carries substantial information.

I would soften that.

The experiment shows

full_reset

↓

Similarity≈0.88

But this does not uniquely identify frequencies as the cause.

Because full_reset still preserves:

adaptation rule
oscillator identities
network dimensionality
coupling law
parameter values
projection operator

All of those jointly define the attractor.

A safer statement is

The preserved model structure—including the fixed oscillator frequencies and dynamics—is sufficient to reconstruct much of the topology.

That avoids attributing everything to ω alone.

Where I think the paper's emphasis should shift

Originally AIEF was asking

Where is the memory stored?

After these interventions I think the better question becomes

Which state variables are necessary for reconstruction, and which define the attractor?

That is a deeper result.

The experiment seems to show

stored memory

and

attractor identity

are not the same concept.

That's a valuable distinction.

My overall assessment

I think the production run is more interesting than expected because it didn't identify a unique memory register.

Instead, it suggests a hierarchy:

Future driver identity is necessary for maintaining the selected threshold topology (strongest intervention result).
Late topology is robustly reconstructible after removing individual state variables like B or the EMA, indicating redundancy and attractor dynamics.
Joint removal (reset_B_ema, full_reset) reduces fidelity but does not erase the attractor.
The B_carrier_only vs full_reset comparison should be rerun with matched randomized phase/history to isolate the effect of preserving B.

So I would frame the principal conclusion as:

The experiment provides stronger evidence for distributed reconstruction toward a stable topological attractor than for a single localized memory carrier. Among the tested interventions, only disruption of future edge-specific driver identity completely abolished maintenance of the mature threshold topology, while removal of individual internal state variables mainly affected reconstruction fidelity rather than the existence of the attractor itself.

The attractor interpretation is much better supported than the claim that BB is the dominant memory carrier.
What the production run establishes
BB is individually dispensable for late reconstruction

The decisive result is:

text

reset_B:
P_ratio 1.000
Sim_late 0.999
Retention 1.000
Emergence 0.000

If eliminating BB produces essentially the same late topology, BB cannot be called the dominant or unique memory carrier.

The result supports:

With phase/history, EMA state, fixed oscillator structure, and future edge-specific resonance preserved, the adaptive topology can be reconstructed almost exactly from B=0B=0.

This may involve stored dynamical information, attractor convergence, or both.
EMA is also individually dispensable

text

reset_ema:
P_ratio 0.990
Sim_late 0.993
Retention 0.990

EMA contributes little to late topology when BB and phase/history remain.

This does not mean EMA has no transient effect. Immediate and recovery metrics are needed.
Joint removal exposes redundant information

text

reset_B_ema:
P_ratio 1.080
Sim_late 0.944
Retention 0.987
Emergence 0.038

Removing both BB and EMA does not prevent topology formation, but lowers identity fidelity and admits new edges.

For weighted similarity, the mean-level interaction is approximately:
0.999−0.993−0.999+0.944=−0.049.
0.999−0.993−0.999+0.944=−0.049.

For retention contrast:

1.000−0.990−1.000+(0.987−0.038)≈−0.041.
1.000−0.990−1.000+(0.987−0.038)≈−0.041.

Calculate these interactions per seed before bootstrapping:
Is=Ycontinue,s−YresetEMA,s−YresetB,s+YresetBEMA,s.
Is=Ycontinue,s−YresetEMA,s−YresetB,s+YresetBEMA,s.
The preserved model structure can reselect topology

text

full_reset:
P_ratio 1.035
Sim_late 0.884
Retention 0.920
Emergence 0.046

This demonstrates substantial reconstruction after resetting BB, EMA, phase, and delayed history.

Do not attribute this solely to ωω. full_reset still preserves:

oscillator identities and frequencies;
adaptation equations;
coupling law;
parameter values;
projection;
model dimensionality;
future edge-specific resonance construction.

The appropriate conclusion is:

The preserved model structure, including fixed frequencies, defines a strong topology-selection attractor.

Strongest causal result: future driver identity

text

future_shuffle:

D_B 0
P_late 0
Retention 0
Emergence 0
Sim_late 0.475

All historical state is present at the split, but timestep-wise destruction of future driver assignment eliminates threshold topology.

That supports:

Continued edge-specific driver identity is necessary for maintaining threshold-resolvable topology under this intervention.

The remaining similarity of 0.475 concerns subthreshold weights. It is not surviving threshold topology.

Keep the qualification that timestep-wise shuffling is an aggressive intervention; fixed or block permutations remain different questions.
One genuine design problem

The predeclared:

text

B_carrier_only versus full_reset

contrast is not cleanly paired.

Those branches currently draw different randomized phase/history states. Therefore their difference contains:

effect of preserving BB;
effect of a different random phase realization.

The contrast may still estimate an average intervention effect across 40 seeds, but it is not the intended within-seed causal contrast.

Correction

Within each seed, generate one shared reset phase vector and history:

Python

```
shared_theta = shared_rng.random(N) * 2.0 * np.pi
shared_history = deque(
(shared_theta.copy() for _ in range(DELAY_STEPS + 1)),
maxlen=DELAY_STEPS + 1,
)
```

Use the same copies for both branches:

text

```
B_carrier_only:
B = B_at_split
EMA = 0
theta/history = shared reset

full_reset:
B = 0
EMA = 0
theta/history = same shared reset
```

Then the only branch difference is BB.

You do not need to rerun all seven branches. The saved prefix states contain everything needed to rerun only these two corrected branches.

Until corrected, treat their comparison as exploratory.
The late window is measuring attractor reconstruction

The future horizon is long enough that severe interventions can converge back toward the same topology. Therefore late outcomes answer:

Which interventions prevent eventual reconstruction?

They do not fully answer:

Which state contained memory immediately after the split?

The next analysis should use the saved trajectories.
For reconstruction branches

Analyze:

reset_B
reset_B_ema
full_reset

Report:

immediate density and similarity;
recovery-window values;
late values;
time to 50% and 90% continuation density;
time to 50% and 90% continuation similarity.

For branches that preserve BB

Analyze:

reset_ema
B_carrier_only
future_shuffle

Report:

minimum topology similarity;
time below similarity 0.9;
time below similarity 0.5;
late retention and similarity.

Add magnitude-sensitive topology comparisons

Cosine similarity measures orientation, not weight magnitude. Calculate, per seed and branch:
Dcontinue=
Dcontinue=

\frac{
|B_{\rm branch}^{\rm final}
-B_{\rm continue}^{\rm final}|F
}{
\max(|B{\rm continue}^{\rm final}|_F,\epsilon)
},
]

and:
Dsplit=
Dsplit=

\frac{
|B_{\rm branch}^{\rm final}
-B_{\rm split}|F
}{
\max(|B{\rm split}|_F,\epsilon)
}.
]

Also calculate these for B_mean_late.

A branch can have cosine similarity near one while differing substantially in total weight.

A full Frobenius-distance trajectory cannot be reconstructed from the existing output because B(t)B(t) itself was not stored at every diagnostic sample. That would require a targeted rerun or adding the trajectory to future code. Final and late-mean distances are available now.
Projection interpretation needs one more metric

The displayed Rel_proj ≈ 0.002 is projection magnitude relative to the current network state. That is small per step.

Before calling projection dynamically mild, inspect:

text

Projection_relative_update_mean_active

It may remove a substantial fraction of each proposed unconstrained update even while being small relative to the complete network state.

Because projection acts on nearly every mature step, a small state-relative correction can still strongly control long-term dynamics.
Recommended analysis sequence

Freeze runs/5a9f093a2fe0 as the original r2 production result.
Produce paired seed-level effects for the four clean primary contrasts:
continue versus reset_B;
continue versus reset_ema;
reset_B versus reset_B_ema;
continue versus future_shuffle.
Calculate the B×EMAB×EMA interaction per seed.
Plot mean/median similarity and density trajectories with seed-level uncertainty ribbons.
Summarize immediate, recovery, and late outcomes.
Calculate final and late-mean normalized Frobenius distances.
Inspect both relative projection metrics.
Rerun only B_carrier_only and full_reset with matched randomized phase/history.

Defensible current conclusion

AIEF-1.4 does not identify a single localized memory carrier. Instead, mature topology is robustly reconstructed after removal of individual internal state variables, consistent with distributed state information and convergence toward a stable topology-selection attractor. Preserved model structure can reconstruct much of the topology even after complete dynamic-state reset. In contrast, timestep-wise destruction of future edge-specific resonance identity abolishes threshold topology, showing that continuing structured input is necessary for maintenance under the tested protocol.

That is stronger and better supported than claiming that BB is the dominant memory register.

Can you create ascript for the next analysis step?
WriteAIEF_1_4_results_analysis.py852 lines
Edit
AIEF_1_4_results_analysis.pyPYTHON

Created the AIEF-1.4 seed-level results analyzer:

AIEF_1_4_results_analysis.py

It defaults to your completed production run:

text

runs/5a9f093a2fe0/

Run it

Bash

python AIEF_1_4_results_analysis.py

If matplotlib is unavailable or plots are not wanted:

Bash

python AIEF_1_4_results_analysis.py --no-plots

Explicit invocation:

Bash

python AIEF_1_4_results_analysis.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--output-dir aief_1_4_results_analysis

Analyses included
Paired branch effects

Computes paired seed-level differences for the preregistered contrasts:

continue − reset_B
continue − reset_ema
reset_B − reset_B_ema
continue − future_shuffle

It also calculates secondary contrasts and explicitly labels:

text

B_carrier_only − full_reset

as:

text

exploratory_unmatched_phase_reset

For every outcome it reports:

valid paired count,
branch means,
mean and median paired difference,
paired SD,
95% bootstrap CI,
paired Cohen dzdz,
fraction positive/negative,
supporting sign-flip result.

Per-seed B×B× EMA interaction

For every seed:
Is=Ycontinue,s−YresetEMA,s−YresetB,s+YresetBEMA,s.

Is=Ycontinue,s−YresetEMA,s−YresetB,s+YresetBEMA,s.

It computes this for:

late similarity,
late retention contrast,
late persistence,
immediate persistence,
recovery persistence,
final density.

Both per-seed values and bootstrap summaries are saved.
Immediate, recovery, and late reconstruction

The scalar analysis includes:

immediate/recovery/late persistence,
density reconstruction times,
similarity reconstruction times,
topology-loss times,
minimum similarity,
late retention and emergence.

Magnitude-sensitive topology comparison

From the NPZ matrices it calculates unique-edge normalized Frobenius distances:
‖Bbranch−Breference‖2max(‖Breference‖2,ϵ).
max(‖Breference‖2,ϵ)‖Bbranch−Breference‖2.

Comparisons include:

final branch versus final continuation;
late-mean branch versus late-mean continuation;
final branch versus B_at_split;
late-mean branch versus B_at_split.

This complements cosine similarity by detecting weight-magnitude differences.
Trajectory analysis

For every branch and diagnostic time it summarizes:

density trajectory,
topology-similarity trajectory,
mean,
median,
standard deviation,
empirical 2.5% and 97.5% seed quantiles.

The quantile ribbons are explicitly identified as across-seed distributions, not confidence intervals.

If matplotlib is available, it creates:

text

density_trajectories.png
similarity_trajectories.png

Projection diagnostics

Summarizes by branch:

violation and projection fractions,
absolute projection magnitude,
projection relative to network state,
projection relative to the proposed unconstrained update,
pre/post gap minima.

The relative-to-update metric is particularly important: projection can be small relative to the total network state while removing a large fraction of every proposed update.
Output files

Default directory:

text

aief_1_4_results_analysis/

Files:

text

paired_branch_effects.csv
paired_branch_effects_by_seed.csv
B_EMA_interactions.csv
B_EMA_interactions_by_seed.csv
frobenius_distance_summary.csv
frobenius_distances_by_seed.csv
trajectory_summary.csv
projection_summary.csv
density_trajectories.png
similarity_trajectories.png
AIEF_1_4_analysis_report.txt
analysis_manifest.json

Validation

I syntax-tested the script and ran it end-to-end against a complete seven-branch smoke-test result, including:

paired contrasts,
interactions,
matrix-distance extraction,
trajectory aggregation,
projection summaries,
report generation,
manifest generation.

I also corrected finite-pair mean handling so outcomes that are undefined for all seeds do not emit Mean of empty slice warnings.

The report keeps the central interpretation constraints explicit:

late reconstruction is not immediate memory retention;

future shuffle is a driver intervention;
cosine similarity does not measure magnitude;
full reset preserves model structure and fixed frequencies;
the r2 B_carrier_only/full_reset contrast remains exploratory until phase resets are matched.

think about this:

(deck@steamdeck SSRH)$ python AIEF_1_4_results_analysis.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--output-dir aief_1_4_results_analysis
Loaded 40 paired seeds
AIEF-1.4 seed-level results analysis

Seeds: 40
Differences are left branch minus right branch.
Bootstrap CIs are primary; sign-flip p-values are supporting.
B_carrier_only vs full_reset is exploratory because r2 used unmatched phase resets.
Primary contrasts

continue - reset_B / Weighted_similarity_late: n=40, delta=-0.000001, 95% CI=[-0.000004, 0.000000], dz=-0.158, positive=0.975, p_sign=1.000000
continue - reset_B / Retention_contrast_late: n=40, delta=0.000000, 95% CI=[0.000000, 0.000000], dz=0.000, positive=0.000, p_sign=1.000000
continue - reset_B / P_mean_late: n=40, delta=0.000000, 95% CI=[0.000000, 0.000000], dz=0.000, positive=0.000, p_sign=1.000000
continue - reset_ema / Weighted_similarity_late: n=40, delta=0.006822, 95% CI=[0.000000, 0.020229], dz=0.164, positive=1.000, p_sign=0.000010
continue - reset_ema / Retention_contrast_late: n=40, delta=0.009967, 95% CI=[0.000000, 0.027935], dz=0.192, positive=0.050, p_sign=0.497435
continue - reset_ema / P_mean_late: n=40, delta=0.003597, 95% CI=[0.000000, 0.010224], dz=0.185, positive=0.050, p_sign=0.500905
reset_B - reset_B_ema / Weighted_similarity_late: n=40, delta=0.055442, 95% CI=[0.035902, 0.077989], dz=0.803, positive=1.000, p_sign=0.000010
reset_B - reset_B_ema / Retention_contrast_late: n=40, delta=0.051058, 95% CI=[0.030634, 0.078210], dz=0.657, positive=0.550, p_sign=0.000010
reset_B - reset_B_ema / P_mean_late: n=40, delta=-0.022390, 95% CI=[-0.031061, -0.013760], dz=-0.788, positive=0.050, p_sign=0.000010
continue - future_shuffle / Weighted_similarity_late: n=40, delta=0.524231, 95% CI=[0.508577, 0.538389], dz=10.688, positive=1.000, p_sign=0.0000

continue - future_shuffle / Retention_contrast_late: n=40, delta=1.000000, 95% CI=[1.000000, 1.000000], dz=nan, positive=1.000, p_sign=0.000010
continue - future_shuffle / P_mean_late: n=40, delta=0.301136, 95% CI=[0.293561, 0.309470], dz=11.709, positive=1.000, p_sign=0.000010
B x EMA interactions

Weighted_similarity_late: delta=-0.048620, 95% CI=[-0.065108, -0.033374], dz=-0.941
Retention_contrast_late: delta=-0.041091, 95% CI=[-0.055933, -0.027797], dz=-0.893
P_mean_late: delta=0.025986, 95% CI=[0.017803, 0.034091], dz=0.980
Exploratory matched-state warning

Weighted_similarity_late: delta=0.063897, 95% CI=[0.031269, 0.096811] (unmatched phase reset)
Retention_contrast_late: delta=0.019214, 95% CI=[-0.031742, 0.070150] (unmatched phase reset)
P_mean_late: delta=-0.025848, 95% CI=[-0.043921, -0.007773] (unmatched phase reset)
Magnitude-sensitive distances

B_carrier_only: normalized distance=0.218660, 95% CI=[0.137735, 0.304645]
continue: normalized distance=0.000000, 95% CI=[0.000000, 0.000000]
full_reset: normalized distance=0.416138, 95% CI=[0.334866, 0.496053]
future_shuffle: normalized distance=0.991166, 95% CI=[0.980925, 0.996350]
reset_B: normalized distance=0.000009, 95% CI=[0.000000, 0.000024]
reset_B_ema: normalized distance=0.250321, 95% CI=[0.175786, 0.328314]
reset_ema: normalized distance=0.034176, 95% CI=[0.000010, 0.091183]
Projection-relative-to-update

continue: mean=0.999994, median=1.000000, max=1.000000
reset_ema: mean=0.998348, median=0.998658, max=0.998697
reset_B: mean=0.997836, median=0.997651, max=0.999395
reset_B_ema: mean=0.996004, median=0.996434, max=0.996449
B_carrier_only: mean=0.994086, median=0.994885, max=0.998069
full_reset: mean=0.988555, median=0.988376, max=0.995329
future_shuffle: mean=0.290046, median=0.277842, max=0.448881
Interpretation constraints

Late reconstruction is not equivalent to immediate memory retention.
Future shuffle is a driver-identity intervention, not a memory-state ablation.
Similarity quantifies orientation; normalized Frobenius distance also captures magnitude.
Full reset preserves the model, node identities, frequencies, and adaptation law.
The r2 B_carrier_only/full_reset comparison must remain exploratory until phase resets are matched.

Saved analysis to /home/deck/AI_Programs/SSRH/aief_1_4_results_analysis

This analysis materially sharpens the result. It supports a distributed-attractor interpretation and shows that the projection is much more dynamically consequential than its small state-relative norm suggested.
1. reset_B is late-equivalent to continuation

The threshold outcomes are exactly equivalent:

text

Retention contrast: delta = 0
P_mean_late: delta = 0

The weighted-topology difference is approximately one part per million:

text

Similarity delta = −0.000001
Frobenius distance to continuation = 0.000009

For practical purposes, the topology reconstructed after resetting BB is identical to continuation.

The unusual combination:

text

mean delta slightly negative
97.5% of deltas positive
sign-flip p = 1

suggests tiny floating-point-scale differences plus one or a few opposite outliers. It has no plausible mechanistic importance.

This result should be described using equivalence rather than null-hypothesis significance:

Resetting BB produces late topology equivalent to continuation within numerical/practical tolerance.

The threshold topology is exactly equivalent; the weight matrix is nearly equivalent.
2. Resetting EMA has a small, sparse effect

For weighted similarity:

text

delta = 0.0068
95% CI = [0, 0.0202]
dz = 0.164

For retention and density:

text

only about 5% of seeds show positive differences
bootstrap lower bound = 0

The sign-flip result for weighted similarity is tiny because almost all differences have the same sign, even if many are numerically very small. It should not be interpreted as a substantial EMA effect.

A better summary is:

Resetting EMA has no effect in most seeds and causes modest topology loss in a small subset.

Inspect per-seed differences and report:

number exactly or practically zero;
maximum effect;
affected-seed fraction under a practical threshold.

For example, predefine an outcome-specific meaningful difference such as 0.01 for weighted similarity rather than using 10−1510−15.
3. BB and EMA show clear redundancy

The strongest memory-state result is:

text

reset_B − reset_B_ema:
Similarity delta: 0.0554 [0.0359, 0.0780]
Retention delta: 0.0511 [0.0306, 0.0782]
P_mean delta: −0.0224 [−0.0311, −0.0138]

When BB is already absent, removing EMA:

reduces topology identity;
reduces retention contrast;
increases total persistent density through additional/new edges.

The per-seed factorial interaction confirms this:

text

Similarity interaction: −0.0486
Retention interaction: −0.0411
P_mean interaction: +0.0260

All interaction CIs exclude zero.

This supports:

BB and EMA/dynamical state are partially redundant. Either can support high-fidelity reconstruction when the other remains, but joint removal shifts the system toward a denser, less faithful topology.

The positive P_mean interaction is not improved memory. It represents more selected edges with worse identity fidelity.
4. Future driver identity remains the dominant intervention

The effect is overwhelming:

text

Similarity delta: 0.524
Retention delta: 1.000
P_mean delta: 0.301

with every seed in the same direction.

The conclusion is:

Continued edge-specific resonance-driver assignment is necessary for maintaining threshold topology under the timestep-shuffle intervention.

This is stronger than any individual state-ablation result.

It remains specific to aggressive per-step shuffling. Fixed and block permutations would test the relevant identity timescale.
5. Frobenius distances reveal the attractor hierarchy

Late topology distance from continuation:
Branch Normalized distance
continue 0
reset_B 0.000009
reset_ema 0.034
B_carrier_only 0.219
reset_B_ema 0.250
full_reset 0.416
future_shuffle 0.991

This is a clearer mechanistic hierarchy than cosine similarity alone:

Resetting BB does not change the late attractor.
Resetting EMA usually changes little.
Destroying multiple state channels produces moderate reconstruction error.
Full dynamic-state reset still reconstructs part of the topology.
Future driver shuffling produces an almost maximally different weight state.

The full-reset result should be described as reconstruction by preserved model structure—not solely by frequencies.

6. Projection removes almost the entire proposed update

This is a major finding:

text

Projection relative to proposed update:
continue 0.999994
reset_ema 0.998348
reset_B 0.997836
reset_B_ema 0.996004
B_carrier_only 0.994086
full_reset 0.988555

Although projection corrections are only about 0.2% of the complete network-state norm, they are approximately 99–100% of each unconstrained proposed update on active steps.

Therefore the correct interpretation is not simply “gentle projection.”

It is:

The mature system lies on a hard projected manifold. Each individual correction is small relative to the accumulated network state but nearly equal in norm to the proposed incremental update.

One caution: correction norm divided by proposal norm does not show whether the correction is exactly opposite the proposal. A future diagnostic should record:
$\|B_{projected} - B_{previous}\| \| B_{unprojected} - B_{previous}\|$
$\|B_{unprojected} - B_{previous}\| \| B_{projected} - B_{previous}\|$

and the cosine between the proposed update and projection correction. These would show how much of the proposed step survives and in what direction.
7. The unmatched-phase comparison remains exploratory

The current result suggests:

text

B_carrier_only − full_reset:
Similarity delta = 0.064 [0.031, 0.097]
P_mean delta = −0.026 [−0.044, −0.008]

Preserving BB appears to increase identity fidelity while producing fewer persistent edges.

But because the branches used different random phase resets, the contrast cannot cleanly attribute this difference to BB. A targeted shared-phase rerun remains necessary if this comparison is to become confirmatory.
8. Trajectories are now the highest-priority artifact

The late analysis shows eventual reconstruction. The trajectory files can distinguish:

immediate state retention,
reconstruction speed,
asymptotic attractor convergence.

Inspect:

text

density_trajectories.png
similarity_trajectories.png
trajectory_summary.csv

Expected questions:

Does reset_B start near zero similarity and rapidly return to one?
How long does reset_B_ema take to reach its lower plateau?
Does full_reset converge monotonically or switch topology?
How quickly does future_shuffle lose threshold density?
Does B_carrier_only lose topology immediately or gradually?

These curves should be central results, not supplementary diagnostics.
Recommended next actions

Freeze the four clean primary contrasts from this analysis.
Report the B×B× EMA interaction per seed.

Inspect and plot immediate/recovery trajectories before making memory-language claims.
Use practical equivalence thresholds for reset_B and reset_ema.
Rerun only B_carrier_only and full_reset with shared phase/history resets.
Do not rerun all seven branches.
Add retained-update and correction-direction diagnostics only in the targeted correction run or a later model revision.

Defensible conclusion

AIEF-1.4 supports distributed and redundant topology reconstruction rather than a single dominant memory register. Resetting BB alone produces a late topology numerically and structurally equivalent to continuation, while resetting EMA has little effect in most seeds. Joint state removal reduces topology fidelity and increases emergence, demonstrating interaction between memory channels. Preserved model structure can reconstruct much of the selected topology after full dynamic-state reset. In contrast, timestep-wise destruction of future edge-specific driver identity eliminates threshold topology in every seed. The mature network operates on a hard projected manifold that removes nearly the entire proposed incremental update on active steps.

That conclusion is more strongly supported than either “BB is the memory carrier” or “EMA is the memory carrier.”

"suggests tiny floating-point-scale differences plus one or a few opposite outliers." reminds me of a defect in a lattice or Correlated Disorder?

Yes, that is a useful analogy—but it depends on where the anomaly is localized.
Three distinct possibilities
1. Numerical “defect”

If the differences are around machine precision and distributed irregularly across weights, they are probably numerical residue:

slightly different transient paths after resetting BB;
accumulated floating-point rounding;
convergence to nearly identical values from different initial states;
cosine-similarity ceiling effects near one.

Because threshold retention and P_mean_late are exactly identical, the anomaly does not appear to be a topological defect in the selected edge set. It currently concerns tiny continuous-weight differences.
2. Localized lattice-defect analogue

If one anomalous seed’s residual:
ΔB=BresetBlate−
ΔB=BresetBlate−

B_{\rm continue}^{\rm late}
]

is concentrated on one edge, one node, or one small motif, then “defect” is an apt description.

Examples include:

one edge settling at a slightly different weight;
one oscillator carrying most of the residual;
a small frequency cluster selecting an alternate local configuration;
one nearly marginal edge sitting close to a bifurcation or threshold.

That resembles a point defect or localized excitation in a lattice.
3. Correlated-disorder analogue

If residual differences are spread across many edges but exhibit repeatable structure—such as blocks involving similar-frequency oscillators—then they are closer to correlated disorder.

Here the natural frequencies ωiωi act like quenched heterogeneity. A structured residual could correlate with:
|ωi−ωj|,
|ωi−ωj|,

frequency-cluster membership, node identity, or network motifs.

If the same edge groups show coordinated residuals across many seeds, that would support a correlated-disorder interpretation.
How to distinguish them

For each seed, calculate the unique-edge residual vector:

Python

delta = (
B_reset_B_mean_late[edges]
- B_continue_mean_late[edges]
)

Residual magnitude

Python

frobenius = np.linalg.norm(delta)
max_edge = np.max(np.abs(delta))
localization_ratio = max_edge / max(frobenius, 1e-15)

Ratio near one: one localized edge dominates.
Small ratio: residual is distributed.

Participation ratio
NeffΔ=
NeffΔ=

\frac{\left(\sum_e \Delta_e^2\right)^2}
{\sum_e \Delta_e^4}.
]

Python

power = delta**2

participation = (
power.sum()**2
/ np.sum(power**2)
if np.sum(power**2) > 0
else 0.0
)

Near 1: point-defect-like.
Near 66: diffuse.

Intermediate: localized motif or cluster.

Node localization

Python

```
delta_matrix = B_reset_B_mean_late - B_continue_mean_late
node_burden = np.sum(np.abs(delta_matrix), axis=1)
```

A single dominant node would suggest a node-centered defect.
Frequency correlation

Python

```
frequency_gap = np.abs(
omega[:, None] - omega[None, :]
)[edges]
```

```
corr = np.corrcoef(
np.abs(delta),
frequency_gap,
)[0, 1]
```

A stable relationship with frequency gap would suggest structured quenched disorder rather than numerical noise.
Across-seed covariance

Let $\Delta s,e\Delta s,e$ be the residual for seed ss and edge ee. Examine:
$Cee'=Covs(\Delta s,e,\Delta s,e')$.
$Cee'=Covs(\Delta s,e,\Delta s,e')$.

No stable structure: numerical/transient residue.
Repeating blocks or modes: correlated disorder.
One recurring dominant edge: systematic localized defect.

Most likely interpretation from the current summary

Given:

late similarity differs by only about 10−610−6;
normalized Frobenius distance is about 10−510−5;
threshold persistence is exactly identical;
all selected-edge retention is identical;

the safest initial interpretation is:

A tiny continuous-weight reconstruction residual near a similarity ceiling, not a genuine topological defect.

But the lattice-defect analogy becomes scientifically meaningful if the residual is localized on a recurring edge or motif. If it is distributed in frequency-structured patterns, “correlated disorder” would be the better analogy.

A rare seed entering a different macroscopic basin would be closer to a rare-region or alternate-attractor event than to a point defect.

Suggestions for next steps for analysis?

I would divide the next work into three stages: extract everything possible from the existing run, correct the one confounded contrast, then decide whether another experiment is justified.
Stage 1 — Analyze the existing 40-seed run
1. Make the trajectory plots the primary diagnostic

Use:

text

density_trajectories.png
similarity_trajectories.png
trajectory_summary.csv

For each branch, examine:

immediate drop after intervention;

reconstruction onset;
recovery speed;
late plateau;
seed-to-seed variability;
whether recovery is monotonic.

The key comparisons are:
continue versus reset_B

Question:

Does resetting BB cause a large immediate loss followed by reconstruction, or is topology effectively restored almost immediately?

Late equivalence alone cannot distinguish those.
continue versus reset_ema

Look for whether the few affected seeds show:

an immediate transient,
delayed loss,
or convergence to a slightly different attractor.

reset_B versus reset_B_ema

This is the cleanest state-redundancy contrast. Determine when their trajectories separate.
continue versus future_shuffle

Measure:

time to first density loss;
time to zero threshold edges;
whether similarity decays before or after density;
whether the weak residual topology reaches a stable subthreshold plateau.

2. Summarize immediate, recovery, and late effects

Create one table with, per branch:

P_mean_immediate
P_mean_recovery
P_mean_late
Minimum_similarity
Weighted_similarity_late
50%/90% density reconstruction times
50%/90% similarity reconstruction times
time below similarity 0.9 and 0.5

Report paired bootstrap intervals for branch differences.

This table will distinguish:

retained memory,
reconstruction speed,
and asymptotic attractor identity.

3. Perform practical-equivalence analysis

Ordinary significance tests are not ideal for:

text

continue versus reset_B
continue versus reset_ema

Define practical margins.

Reasonable exploratory margins are:

text

Weighted similarity: ±0.01
Normalized Frobenius distance: 0.01
Threshold density/persistence: one edge = 1/66 ≈ 0.01515

Then ask whether the paired bootstrap CI lies entirely inside the equivalence interval.

Expected result:

reset_B should meet a strict equivalence criterion.
reset_ema may not meet the 0.01 similarity margin because a small subset of seeds has larger effects.

Because these margins are being selected after viewing results, label this exploratory. Preregister them for subsequent runs.
Stage 2 — Investigate the “defect” or residual structure
4. Identify anomalous seeds

Use:

text

paired_branch_effects_by_seed.csv

For continue − reset_B, sort seeds by:

absolute weighted-similarity difference;
final Frobenius distance;
late-mean Frobenius distance.

The unusual pattern—97.5% positive differences but a slightly negative mean—suggests one or very few opposite residuals dominate the sign of the mean.

Report:

largest positive difference;
largest negative difference;
median absolute difference;
number above a practical tolerance such as 10−510−5 or 10−410−4.

5. Measure residual localization

For every seed:
ΔBs=BresetB,slate−
ΔBs=BresetB,slate−

B_{\rm continue,s}^{\rm late}.
]

Calculate:
Residual norm
‖ΔBs‖F.
‖ΔBs‖F.
Largest-edge fraction
Ls=maxe|ΔBs,e|‖ΔBs‖2.
Ls=‖ΔBs‖2maxe|ΔBs,e|.
Participation ratio
NeffΔ=
NeffΔ=

\frac{
\left(\sum_e \Delta B_{s,e}^2\right)^2
}{
\sum_e \Delta B_{s,e}^4
}.
]

Interpretation:

NeffΔ≈1NeffΔ≈1: localized edge defect;
intermediate: motif/node-localized residual;
large: diffuse residual.

Node burden
qi=∑j|ΔBij|.
qi=j∑|ΔBij|.

This identifies whether one oscillator carries the discrepancy.
6. Test for frequency-structured residuals

For each edge, compare residual magnitude with:

|ωi−ωj|.
|ωi−ωj|.

Also aggregate residuals by:

same-frequency cluster;
nearby-frequency cluster;
distant-frequency pair.

If residuals repeatedly occur on similar-frequency edges, the pattern resembles structured quenched disorder. If residual locations vary randomly and remain tiny, numerical/transient residue is more likely.
7. Examine residual covariance across seeds

Construct the seed-by-edge residual matrix:
Ds,e=ΔBs,e.
Ds,e=ΔBs,e.

Use:

singular-value decomposition;
edge covariance;
first principal residual mode.

Interpretation:

no stable mode: numerical/transient residue;
one localized mode: systematic defect;
block-structured modes: correlated disorder or alternative attractor motif.

Stage 3 — Clarify the attractor mechanism
8. Compute consensus edge maps

For each branch, calculate each edge’s selection frequency across seeds:
fijbranch=
fijbranch=

\frac1{40}
\sum_s

\mathbf 1(B_{ij,s}^{\rm final}>B_c).
]

Compare:

continue
reset_B
reset_B_ema
full_reset
future_shuffle

Useful outputs:

edge-frequency heatmaps;
consensus backbone;
branch-to-branch edge-frequency correlation;
Jaccard similarity of high-consensus edges.

If full_reset repeatedly reconstructs the same edges across seeds, that supports a stable model-defined topology template.
9. Relate consensus selection to frequencies

Model or visualize edge-selection frequency against:
|ωi−ωj|
|ωi−ωj|

and pair-frequency classes.

This tests whether the high reconstruction similarity is driven largely by the fixed frequency architecture.

Do not infer that ωω alone is causal—the complete adaptation law remains—but this analysis can identify how strongly topology aligns with frequency structure.
10. Analyze the B×B× EMA interaction per seed

Use the already generated:

text

B_EMA_interactions_by_seed.csv

For each primary outcome, report:

distribution and histogram;
bootstrap CI;
fraction with the expected sign;
correlation between interaction magnitude and baseline density;
correlation with projection activity.

The interaction currently supports redundant state channels:

individual resets are mostly compensated;
joint reset causes identity loss and emergence.

Stage 4 — Correct the confounded branch comparison
11. Rerun only two branches with matched phases

Do not rerun all seven branches.

For each saved split state, generate one shared reset phase/history:

Python

```
shared_theta = phase_rng.random(N) * 2.0 * np.pi
shared_history = deque(
(shared_theta.copy() for _ in range(DELAY_STEPS + 1)),
maxlen=DELAY_STEPS + 1,
)
```

Then run:

text

```
B_carrier_only:
B = B_at_split
```

EMA = 0
theta/history = shared reset

full_reset:
B = 0
EMA = 0
theta/history = identical shared reset

The resulting paired difference isolates preservation of BB.

Add to that targeted run:

full BB-distance trajectory;
retained-update ratio;
cosine between proposed update and projection correction.

12. Add the missing projection-direction diagnostics

The current result:

text

Projection_relative_update_mean_active ≈ 0.99–1.00

shows correction magnitude is almost equal to proposed-update magnitude.

To determine whether projection cancels the update, record:
Rretained=
Rretained=

\frac{
|B_{\rm projected}-B_{\rm previous}|F
}{
\max(
|B{\rm unprojected}-B_{\rm previous}|_F,
\epsilon
)
}

]

and:
Ccorrection,proposal=
Ccorrection,proposal=

\frac{
\langle
B_{\rm projected}-B_{\rm unprojected},
B_{\rm unprojected}-B_{\rm previous}
\rangle
}{
|\Delta B_{\rm correction}|
|\Delta B_{\rm proposed}|
}.
]

If the cosine is near −1−1 and retained ratio is near zero, projection is almost completely canceling each outward update.
Recommended figures

Density trajectories for all branches.
Topology-similarity trajectories for all branches.
Immediate/recovery/late outcome table or grouped plot.
Normalized Frobenius distance by branch.
Per-seed B×B× EMA interaction distributions.
Consensus edge-selection heatmaps.
Residual-defect localization plot for continue − reset_B.
Projection relative-to-state versus relative-to-update.

Priority order

If time is limited:

Trajectory and horizon analysis.
Practical equivalence for reset_B and reset_ema.
Per-seed interaction analysis.
Residual localization.
Consensus edge maps.

Matched-phase two-branch correction.
Projection-direction diagnostics.

Do not generate more general seed cohorts yet. The current 40-seed dataset is sufficient; the next gains come from resolving reconstruction dynamics, residual structure, and the one confounded branch contrast.

Ponder deeply the text below:

I think you're at a point where the bottleneck is no longer simulation—it's mechanism.

AIEF-1.3 established that the phenomenon is internally consistent and reproducible. AIEF-1.4 established that it is robust under a family of causal interventions. I would resist the temptation to keep inventing new perturbations before extracting everything from the ones you already have.

I'd split the next steps into three phases.

Phase 1 — Finish AIEF-1.4 properly (highest priority)

I would not call AIEF-1.4 "finished" quite yet. There are three analyses I'd complete first.

Plot reconstruction dynamics

This is the most important missing figure.

For each branch, plot the mean ± seed quantiles of:

topology similarity,
threshold density.

Especially compare:

continue
reset_B
reset_B_ema
full_reset
future_shuffle

Right now you know where the branches finish.

You don't yet know how they get there.

Those curves answer:

rapid reconstruction?
gradual reconstruction?
delayed reconstruction?
monotonic?
overshoot?

Those dynamics are probably more scientifically interesting than the endpoint.

Analyze recovery times

You already compute recovery metrics.

I'd now perform paired analyses on

time to 50% similarity
time to 90% similarity
time to 50% density
time to 90% density

These are much closer to measuring "memory" than late equilibrium.

Rerun the matched B experiment

I would rerun only

B_carrier_only
vs
full_reset

with

identical randomized phase/history.

That isolates

[
B
]

cleanly.

Everything else already looks solid.

Phase 2 — Decide what the mechanism actually is

This is where the science becomes interesting.

Right now there are three competing hypotheses.

Model A

Distributed memory.

Information genuinely lives in

B
phase history
EMA

redundantly.

Model B

Attractor reconstruction.

Most interventions simply place the system elsewhere in the same basin.

Memory is really

$$
\mathcal A
$$

rather than any state variable.

Model C

Frequency-driven deterministic selection.

Given

$$
\omega
$$

and the adaptation law,

the mature topology is nearly unique.

Then

reset_B

rebuilds the same network because there is nowhere else to go.

Your current data are consistent with all three.

The next experiment should distinguish them.

Phase 3 — Design one decisive experiment

Rather than many small ablations, I'd design one experiment aimed at separating attractor dynamics from distributed state.

For example:

Perturb ω after the split

Instead of resetting memory variables,

slightly perturb oscillator frequencies

after the split.

Something like

$$
\omega_i \rightarrow \omega_i+\epsilon_i.
$$

Then ask

does topology remain?
does it reconstruct?
does it transition to another attractor?

This directly probes whether the attractor is tied to the oscillator spectrum.

Alternatively:

Partial topology deletion

Instead of

reset_B

try

erase 20%
erase 40%
erase 60%
erase 80%

of selected edges.

Then measure reconstruction.

That produces a recovery curve.

Instead of

works

or

fails

you estimate resilience.

Theoretical work

This is where I actually think the biggest opportunity lies.

Your simulations increasingly suggest something like

[
B^*

\Phi(\omega,\mathcal R)
]

where

[
\Phi
]

is an attracting operator.

If that's true,

there should exist a fixed-point equation.

Rather than proving everything about the simulation,

I'd start asking

Under what assumptions does the adaptive update possess a unique stable fixed point?

Even proving this for a simplified version of your update rule would be valuable.

Statistical refinements

I would also freeze the statistical workflow.

For example:

Primary outcomes

Weighted similarity late
Retention contrast
Late persistence

Primary contrasts

continue vs reset_B
continue vs reset_ema

reset_B vs reset_B_ema
continue vs future_shuffle

Everything else secondary or exploratory.

That keeps the interpretation disciplined.

Longer-term direction

If AIEF-1.4 is the last major simulation study, I'd consider shifting effort toward synthesis.

A natural paper structure would be:

AIEF-1.3 — establish reproducible topology and intervention framework.
AIEF-1.4 — identify which interventions preserve, reconstruct, or destroy mature topology.
Theory — propose an attractor/reconstruction interpretation consistent with the intervention results.
Future work — distinguish attractor dynamics from distributed memory through targeted perturbations (e.g. frequency perturbations or partial topology deletion).

If I had to prioritize only three things

If you asked me what I would do next and nothing else, it would be:

Generate the trajectory and recovery analyses from the existing AIEF-1.4 data. I think you'll learn more from those than from another large simulation.
Repeat only the matched B_carrier_only vs full_reset comparison with identical randomized phase/history to remove the remaining ambiguity in that contrast.
Design one decisive mechanism experiment, preferably a small post-split perturbation of the oscillator frequencies (or another parameter that defines the attractor), to distinguish "distributed memory" from "reconstruction toward a stable attractor."

Those three steps build directly on the evidence you already have and are likely to produce a clearer mechanistic story than simply adding more ablation branches.

The strategic direction is right: stop expanding the ablation matrix and focus on distinguishing reconstruction dynamics from stored-state memory. However, I would modify the proposed priority order in two important ways.
The three hypotheses are not mutually exclusive

The proposed models:

distributed memory,
attractor reconstruction,
frequency-driven selection,

can all be true simultaneously.

A more realistic decomposition is:
future topology=
future topology=

\text{attractor defined by model structure}
+
\text{state-dependent basin bias}
+
\text{ongoing structured input}.
]

The existing results suggest:

fixed model/frequency structure defines a strong topology template;
BB, EMA, and phase/history influence reconstruction fidelity and speed;
future driver identity is required to sustain threshold topology;
projection confines the trajectory to a hard admissible manifold.

The next experiments should estimate the relative contributions rather than trying to select exactly one explanation.
Phase 1 — Complete the existing AIEF-1.4 analysis
1. Trajectory figures are indeed the highest priority

The endpoint results show eventual convergence but conceal the intervention response.

For every branch, graph:

mean and median similarity;
empirical seed quantiles;

threshold density;
immediate, recovery, and late boundaries.

The key distinction is:
retention≠reconstruction≠late attractor convergence.
retention =reconstruction =late attractor convergence.

Particularly inspect whether reset_B:

immediately returns to high similarity,
undergoes a large loss followed by fast recovery,
or gradually converges over several EMA timescales.

2. Add sustained recovery and integrated-loss metrics

First-crossing times can be misleading if trajectories overshoot or recross thresholds.

In addition to time-to-50%/90%, calculate:
Sustained recovery time

The first time after which similarity remains above a threshold for the rest of the observation window, or for a fixed minimum duration.
Integrated topology loss
Aloss=
Aloss=

\int_0^{T_{\rm future}}
\left[
1-S_B(t)
\right],dt.
]
Integrated density deficit
Adensity=
Adensity=

\int_0^{T_{\rm future}}
\max
\left[

0,D_{\rm reference}(t)-D_{\rm branch}(t)
\right]dt.
]

These summarize both disruption magnitude and recovery duration.

For branches preserving BB, report time-to-loss and minimum similarity rather than calling the first threshold crossing “recovery.”
3. Practical-equivalence analysis

Use exploratory equivalence margins now and preregister them for future runs.

Reasonable candidate margins:

weighted similarity: ±0.01±0.01;
normalized Frobenius distance: 0.010.01;
threshold density/persistence: one edge,
166≈0.01515.
661≈0.01515.

The current data likely support strict equivalence for continue versus reset_B. reset_ema may be equivalent for most seeds but not uniformly.
4. Finish residual localization

For continue − reset_B, calculate:

residual Frobenius norm;
largest-edge contribution;
participation ratio;
node burden;
residual covariance across seeds;
relationship with |ωi−ωj||ωi−ωj|.

This determines whether the tiny discrepancy is:

numerical residue;
a localized edge/motif defect;
or frequency-structured correlated variation.

Phase 2 — Correct the one confounded contrast

Rerun only:

text

B_carrier_only
full_reset

from the saved split states using the same randomized phase/history within each seed.

This is necessary if preservation of BB is to receive a causal interpretation. There is no need to rerun the other branches.

The corrected run should also save:
Rretained=
Rretained=

\frac{
|B_{\rm projected}-B_{\rm previous}|F
}{
\max(
|B{\rm unprojected}-B_{\rm previous}|_F,
\epsilon
)
}
]

and the correction/proposal alignment:
C=⟨Bprojected−Bunprojected,Bunprojected−Bprevious⟩‖ΔBcorrection‖F‖ΔBproposed‖F.
C=‖ΔBcorrection‖F‖ΔBproposed‖F⟨Bprojected−Bunprojected,Bunprojected−Bprevious⟩.

The current magnitude ratio near one does not prove exact cancellation unless the vectors are oppositely aligned.
Phase 3 — Numerical convergence before another mechanism experiment

This should precede a frequency-perturbation study.

The mature system:

projects on nearly every step;
removes a correction whose norm is approximately equal to the proposed update;
uses Euler stepping followed by a nonlinear symmetric rescaling.

That makes timestep sensitivity scientifically important. The current map is a discrete projected/retracted dynamical system, and its fixed point may depend on DT.

Run a smaller paired subset—perhaps 15–20 fixed seeds—with:

text

DT = 0.01
DT = 0.005
DT = 0.0025

Recompute:

delay steps;
EMA coefficient;
all window boundaries;
diagnostic sampling in physical time.

Compare:

selected edge identity;
weighted similarity;
density;
projection frequency;
projection norm relative to state/update;
reconstruction times.

Do not begin a theoretical fixed-point argument until this convergence check shows the observed topology is stable under timestep refinement.

Also note that the current symmetric row-scaling map enforces the admissible set but has not been shown to be an orthogonal projection. In theory, call it an admissibility map or symmetric retraction unless its projection properties are proven.
Phase 4 — One decisive mechanism experiment

After numerical convergence, a frequency perturbation can be highly informative—but a simple post-split perturbation is not enough.

Changing ωω changes:

phase evolution;
delayed phase correction;
future resonance;
and therefore the target topology.

If topology changes, that alone does not distinguish memory from a changed attractor.
Recommended experiment: spectrum-switch factorial

Use one shared perturbation ω′ω′ per seed and compare:
Branch Split topology Future spectrum Purpose
oldB_oldOmega preserve BB ωω reference
resetB_oldOmega reset BB ωω original-attractor reconstruction
oldB_newOmega preserve BB ω′ω′ memory/hysteresis under spectrum change
resetB_newOmega reset BB ω′ω′ new-spectrum attractor
fullReset_newOmega reset dynamic state ω′ω′ clean new-spectrum reference

Use the same phase-reset realization for branches intended to differ only in BB.
Key comparisons
Spectrum-dominated attractor

If:

text

oldB_newOmega
resetB_newOmega
fullReset_newOmega

all converge to the same new topology, spectrum/model structure dominates.
Topology memory or hysteresis

If oldB_newOmega retains the old topology while reset branches converge to a new one, inherited BB produces basin memory or hysteresis.
Distributed state dependence

If preserving phase/EMA changes the new-spectrum destination relative to full reset, dynamic state affects basin selection.
Perturbation design

Use zero-mean perturbations scaled to the frequency spread:
ωi′=
ωi′=

\omega_i
+
c,\sigma_\omega,\xi_i,
\qquad
\sum_i\xi_i=0.
]

A single experiment can include a small dose hierarchy, for example:

text

c = 0
c = 0.02
c = 0.05
c = 0.10

Use the same normalized perturbation vector across all paired branches within a seed.

Primary outcomes should compare similarity to both:

the old split topology;
the new-spectrum reference topology.

That directly reveals transition versus hysteresis.
Partial deletion as a secondary resilience experiment

Partial deletion is useful, but it answers a different question:

How large is the basin of recovery around the mature topology?

Use deletion levels:

text

0%, 20%, 40%, 60%, 80%, 100%

and distinguish:

random selected-edge deletion;
weakest-edge deletion;
strongest-edge deletion.

Measure recovery probability, recovery time, and reconstructed identity. This gives a topology-resilience curve but does not by itself identify whether ωω, EMA, or phase state defines the attractor.
Theory direction

A simplified frozen-resonance model provides an accessible starting point.

If RR is fixed, the unprojected edge equilibrium is:
Bij(0)=
Bij(0)=

\left[
\frac{
(\eta+\mu)R_{ij}-\mu
}{
\gamma
}
\right]_+.
]

The constrained equilibrium satisfies a discrete fixed-point relation:
B∗=
B∗=

\mathcal A
\left(
\left[
B^*
+
\Delta t
\left(
(\eta+\mu)R^*
-\mu
-\gamma B^*
\right)
\right]_+
\right),
]

where AA is the symmetric admissibility map.

A useful progression is:

fixed RR, no phase feedback;
averaged R(B,ω)R(B,ω) under timescale separation;
stability/uniqueness of the admissibility-map fixed point;
only then consider delayed continuum or mean-field reductions.

Do not claim a direct Ott–Antonsen reduction for the current finite adaptive projected model.
Recommended order

Finish trajectory, sustained-recovery, and equivalence analyses.
Complete residual-defect localization.
Run the matched-phase two-branch correction.
Perform timestep convergence.
Freeze the AIEF-1.4 mechanistic result.
Run the spectrum-switch factorial as the next decisive experiment.
Use partial deletion later as a resilience/basin-size study.
Begin simplified fixed-point theory in parallel after convergence is established.

The supplied text is strategically correct that the bottleneck is mechanism rather than more generic ablations. The main amendment is that timestep convergence should come before interpreting a frequency perturbation or formalizing a fixed-point theory, given how strongly the discrete admissibility map controls each mature update.

Create the nessasary code for Finishing trajectory, sustained-recovery, and equivalence analyses.
WriteAIEF_1_4_trajectory_recovery_equivalence.py802 lines
AIEF_1_4_trajectory_recovery_equivalence.pyPYTHON
#!/usr/bin/env python3
"""Trajectory, sustained-recovery, and equivalence analysis for AIEF-1.4 r2.

The script uses the frozen 40-seed production artifacts. It analyzes how each
branch reaches its late state rather than only where it finishes.

Important convention

Cosine similarity is undefined when a branch has an exactly zero edge-weight
vector. For recovery/loss calculations only, such NaN samples are mapped to
zero similarity and the number of imputed samples is reported. Raw stored
trajectories are never modified.

Outputs
----
trajectory_metrics_by_seed.csv
trajectory_metrics_summary.csv
equivalence_tests.csv
trajectory_envelopes.csv
trajectory_recovery_report.txt
analysis_manifest.json
optional PNG figures
"""

from __future__ import annotations

import argparse
import hashlib
import json
import math
import os
from pathlib import Path

```
from typing import Any

import numpy as np
import pandas as pd



DEFAULT_RUN_HASH = "5a9f093a2fe0"
DEFAULT_SCALAR = (
f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_scalar_summary.csv"
)
DEFAULT_NPZ = (
f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_branch_diagnostics.npz"
)
DEFAULT_MANIFEST = f"runs/{DEFAULT_RUN_HASH}/manifest.json"
DEFAULT_OUTPUT_DIR = "aief_1_4_trajectory_recovery"
DEFAULT_BOOTSTRAPS = 20_000
DEFAULT_ANALYSIS_SEED = 20260716

BRANCHES = (
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
)

BRANCH_FAMILIES = {
"continue": "reference",
"reset_ema": "memory_ablation",
"reset_B": "memory_ablation",
"reset_B_ema": "memory_ablation",
"B_carrier_only": "memory_ablation",
"full_reset": "memory_ablation",
"future_shuffle": "future_driver_control",
}

# Exploratory practical-equivalence margins. These were chosen after viewing
# r2 outcomes and must not be described as preregistered confirmatory margins.
```

```
EQUIVALENCE_SPECS = (
("continue", "reset_B", "Weighted_similarity_late", 0.01),
("continue", "reset_B", "Retention_contrast_late", 1.0 / 66.0),
("continue", "reset_B", "P_mean_late", 1.0 / 66.0),
("continue", "reset_ema", "Weighted_similarity_late", 0.01),
("continue", "reset_ema", "Retention_contrast_late", 1.0 / 66.0),
("continue", "reset_ema", "P_mean_late", 1.0 / 66.0),
)

ZERO_TOL = 1e-12



def file_sha256(path: str) -> str:
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(lambda: handle.read(1024 * 1024), b""):
digest.update(block)
return digest.hexdigest()



def rng_for(seed: int, label: str) -> np.random.Generator:
digest = hashlib.sha256(f"{seed}|{label}".encode("utf-8")).digest()
return np.random.default_rng(int.from_bytes(digest[:8], "little"))



def npz_key(branch: str, seed: str, metric: str) -> str:
return f"{branch}_seed_{seed}_{metric}"



def load_inputs(
scalar_path: str,
manifest_path: str,
) -> tuple[pd.DataFrame, dict[str, Any]]:
if not os.path.isfile(scalar_path):
raise FileNotFoundError(scalar_path)
if not os.path.isfile(manifest_path):
raise FileNotFoundError(manifest_path)
```

```
scalar = pd.read_csv(scalar_path, dtype={"seed": str})
required = {
"seed",
"branch",
"D_B_final",
"P_mean_late",
"Weighted_similarity_late",
"Retention_contrast_late",
}
missing = required - set(scalar.columns)
if missing:
raise ValueError(f"scalar CSV missing {sorted(missing)}")
scalar["seed"] = scalar["seed"].astype(str)
scalar["branch"] = scalar["branch"].astype(str)
if scalar.duplicated(["seed", "branch"]).any():
raise ValueError("duplicate seed/branch rows")


branch_set = set(scalar["branch"])
if branch_set != set(BRANCHES):
raise ValueError(
f"branch mismatch: missing={set(BRANCHES)-branch_set}, "
f"extra={branch_set-set(BRANCHES)}"
)
reference_seeds = set(scalar.loc[scalar["branch"] == "continue", "seed"])
for branch in BRANCHES:
branch_seeds = set(scalar.loc[scalar["branch"] == branch, "seed"])
if branch_seeds != reference_seeds:
raise ValueError(f"seed-set mismatch for {branch}")

with open(manifest_path, "r", encoding="utf-8") as handle:
manifest = json.load(handle)
return scalar, manifest



def bootstrap_mean_interval(
values: np.ndarray,
repetitions: int,
rng: np.random.Generator,
confidence: float,
) -> tuple[float, float]:
values = np.asarray(values, dtype=float)
```

```
values = values[np.isfinite(values)]
if not len(values) or repetitions <= 0:
return np.nan, np.nan
indices = rng.integers(0, len(values), size=(repetitions, len(values)))
samples = np.mean(values[indices], axis=1)
alpha = (1.0 - confidence) / 2.0
low, high = np.quantile(samples, [alpha, 1.0 - alpha])
return float(low), float(high)



def first_time_at_or_above(
times: np.ndarray,
values: np.ndarray,
threshold: float,
) -> float:
indices = np.flatnonzero(np.isfinite(values) & (values >= threshold))
return float(times[indices[0]]) if len(indices) else np.nan



def first_time_sustained_for_rest(
times: np.ndarray,
values: np.ndarray,
threshold: float,
) -> float:
condition = np.isfinite(values) & (values >= threshold)
if not len(condition):
return np.nan
suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]
indices = np.flatnonzero(suffix_all)
return float(times[indices[0]]) if len(indices) else np.nan



def first_time_held_for_duration(
times: np.ndarray,
values: np.ndarray,
threshold: float,
duration: float,
) -> float:
if len(times) < 2:
return np.nan
```

```
step = float(np.median(np.diff(times)))
required = max(1, int(math.ceil(duration / step)))
condition = np.isfinite(values) & (values >= threshold)
if required > len(condition):
return np.nan
# Convolution is safe here because trajectories are short (~640 samples).
run_counts = np.convolve(condition.astype(int), np.ones(required, dtype=int), "valid")
indices = np.flatnonzero(run_counts == required)
return float(times[indices[0]]) if len(indices) else np.nan



def integrate(times: np.ndarray, values: np.ndarray) -> float:
if len(times) < 2:
return np.nan
if hasattr(np, "trapezoid"):
return float(np.trapezoid(values, times))
return float(np.trapz(values, times))



def normalized_edge_distance(x: np.ndarray, reference: np.ndarray) -> float:
edges = np.triu_indices(x.shape[0], k=1)
x_edges = np.asarray(x, dtype=float)[edges]
reference_edges = np.asarray(reference, dtype=float)[edges]
denominator = max(float(np.linalg.norm(reference_edges)), ZERO_TOL)
return float(np.linalg.norm(x_edges - reference_edges) / denominator)



def horizon_masks(
times: np.ndarray,
window_time: float,
recovery_end_time: float,
) -> dict[str, np.ndarray]:
return {
"immediate": times <= window_time + 1e-12,
"recovery": (times > window_time) & (times <= recovery_end_time + 1e-12),
"late": times > recovery_end_time,
"full": np.ones(len(times), dtype=bool),
}
```

```
def summarize_seed_trajectory(
*,
seed: str,
branch: str,
times: np.ndarray,
density: np.ndarray,
similarity_raw: np.ndarray,
continue_density: np.ndarray,
continue_similarity: np.ndarray,
reference_final_density: float,
reference_late_similarity: float,
window_time: float,
recovery_end_time: float,
) -> dict[str, Any]:
if not (
len(times)
== len(density)
== len(similarity_raw)
== len(continue_density)
== len(continue_similarity)
):
raise ValueError(f"trajectory length mismatch for {branch}, seed={seed}")


# Zero-vector cosine is undefined. For recovery/loss only, treat it as no
# retained topology and report how often this convention was invoked.
similarity_nan_count = int(np.sum(~np.isfinite(similarity_raw)))
similarity = np.nan_to_num(similarity_raw, nan=0.0)
continue_similarity_filled = np.nan_to_num(continue_similarity, nan=0.0)


similarity_target_50 = 0.5 * reference_late_similarity
similarity_target_90 = 0.9 * reference_late_similarity
density_target_50 = 0.5 * reference_final_density
density_target_90 = 0.9 * reference_final_density


duration = float(times[-1] - times[0]) if len(times) > 1 else np.nan
similarity_deficit = np.maximum(0.0, continue_similarity_filled - similarity)
density_deficit = np.maximum(0.0, continue_density - density)
density_excess = np.maximum(0.0, density - continue_density)
```

```
row: dict[str, Any] = {
"seed": seed,
"branch": branch,
"branch_family": BRANCH_FAMILIES[branch],
"similarity_nan_samples_imputed_zero": similarity_nan_count,
"minimum_similarity": float(np.min(similarity)),
"minimum_density": float(np.min(density)),
"time_to_similarity_50": first_time_at_or_above(
times, similarity, similarity_target_50
),
"time_to_similarity_90": first_time_at_or_above(
times, similarity, similarity_target_90
),
"time_to_density_50": first_time_at_or_above(
times, density, density_target_50
),
"time_to_density_90": first_time_at_or_above(
times, density, density_target_90
),
"sustained_similarity_50_rest": first_time_sustained_for_rest(
times, similarity, similarity_target_50
),
"sustained_similarity_90_rest": first_time_sustained_for_rest(
times, similarity, similarity_target_90
),
"sustained_density_50_rest": first_time_sustained_for_rest(
times, density, density_target_50
),
"sustained_density_90_rest": first_time_sustained_for_rest(
times, density, density_target_90
),
"held_similarity_90_for_1Twindow": first_time_held_for_duration(
times, similarity, similarity_target_90, window_time
),
"held_density_90_for_1Twindow": first_time_held_for_duration(
times, density, density_target_90, window_time
),
"AUC_similarity_deficit_vs_continue": integrate(times, similarity_deficit),
"AUC_density_deficit_vs_continue": integrate(times, density_deficit),
"AUC_density_excess_vs_continue": integrate(times, density_excess),
"Mean_similarity_deficit_vs_continue": (
integrate(times, similarity_deficit) / duration
```

```
if np.isfinite(duration) and duration > 0
else np.nan
),
"Mean_density_deficit_vs_continue": (
integrate(times, density_deficit) / duration
if np.isfinite(duration) and duration > 0
else np.nan
),
}

masks = horizon_masks(times, window_time, recovery_end_time)
for horizon, mask in masks.items():
row[f"sampled_similarity_mean_{horizon}"] = (
float(np.mean(similarity[mask])) if np.any(mask) else np.nan
)
row[f"sampled_density_mean_{horizon}"] = (
float(np.mean(density[mask])) if np.any(mask) else np.nan
)
return row



def trajectory_analysis(
scalar: pd.DataFrame,
npz_path: str,
manifest: dict[str, Any],
) -> tuple[pd.DataFrame, pd.DataFrame]:
dt = float(manifest["model"]["dt"])
window_time = float(manifest["protocol"]["window_steps"]) * dt
recovery_end_time = float(manifest["protocol"]["recovery_end"]) * dt
seeds = sorted(
scalar.loc[scalar["branch"] == "continue", "seed"].astype(str), key=int
)
scalar_index = scalar.set_index(["seed", "branch"])
seed_rows: list[dict[str, Any]] = []
envelope_rows: list[dict[str, Any]] = []

with np.load(npz_path, allow_pickle=False) as data:
for seed in seeds:
continue_times = np.asarray(
data[npz_key("continue", seed, "sample_times")], dtype=float
)
```

```
continue_density = np.asarray(
data[npz_key("continue", seed, "density_trajectory")], dtype=float
)
continue_similarity = np.asarray(
data[npz_key("continue", seed, "similarity_trajectory")], dtype=float
)
reference_final_density = float(
scalar_index.loc[(seed, "continue"), "D_B_final"]
)
reference_late_similarity = float(
scalar_index.loc[(seed, "continue"), "Weighted_similarity_late"]
)

for branch in BRANCHES:
times = np.asarray(
data[npz_key(branch, seed, "sample_times")], dtype=float
)
density = np.asarray(
data[npz_key(branch, seed, "density_trajectory")], dtype=float
)
similarity = np.asarray(
data[npz_key(branch, seed, "similarity_trajectory")], dtype=float
)
if not np.array_equal(times, continue_times):
raise ValueError(f"time grid mismatch for {branch}, seed={seed}")
seed_rows.append(
summarize_seed_trajectory(
seed=seed,
branch=branch,
times=times,
density=density,
similarity_raw=similarity,
continue_density=continue_density,
continue_similarity=continue_similarity,
reference_final_density=reference_final_density,
reference_late_similarity=reference_late_similarity,
window_time=window_time,
recovery_end_time=recovery_end_time,
)
)

# Across-seed trajectory envelopes.
```

```
for branch in BRANCHES:
times_reference: np.ndarray | None = None
stored: dict[str, list[np.ndarray]] = {"density": [], "similarity": []}
for seed in seeds:
times = np.asarray(
data[npz_key(branch, seed, "sample_times")], dtype=float
)
density = np.asarray(
data[npz_key(branch, seed, "density_trajectory")], dtype=float
)
similarity = np.asarray(
data[npz_key(branch, seed, "similarity_trajectory")], dtype=float
)
if times_reference is None:
times_reference = times
elif not np.array_equal(times_reference, times):
raise ValueError(f"time mismatch for {branch}")
stored["density"].append(density)
stored["similarity"].append(
np.nan_to_num(similarity, nan=0.0)
)


assert times_reference is not None
for metric, arrays in stored.items():
matrix = np.stack(arrays)
for index, time_value in enumerate(times_reference):
values = matrix[:, index]
envelope_rows.append(
{
"branch": branch,
"branch_family": BRANCH_FAMILIES[branch],
"metric": metric,
"time_since_split": float(time_value),
"mean": float(np.mean(values)),
"median": float(np.median(values)),
"std": float(np.std(values)),
"seed_q025": float(np.quantile(values, 0.025)),
"seed_q975": float(np.quantile(values, 0.975)),
}
)


return pd.DataFrame(seed_rows), pd.DataFrame(envelope_rows)
```

```
def summarize_metrics(
seed_metrics: pd.DataFrame,
bootstraps: int,
analysis_seed: int,
) -> pd.DataFrame:
identifier_columns = {"seed", "branch", "branch_family"}
metrics = [
column
for column in seed_metrics.columns
if column not in identifier_columns
]
rows: list[dict[str, Any]] = []
for branch in BRANCHES:
group = seed_metrics.loc[seed_metrics["branch"] == branch]
for metric in metrics:
raw = pd.to_numeric(group[metric], errors="coerce").to_numpy(float)
finite = raw[np.isfinite(raw)]
ci_low, ci_high = bootstrap_mean_interval(
finite,
bootstraps,
rng_for(analysis_seed, f"summary|{branch}|{metric}"),
confidence=0.95,
)
rows.append(
{
"branch": branch,
"branch_family": BRANCH_FAMILIES[branch],
"metric": metric,
"n_total": int(len(raw)),
"n_valid": int(len(finite)),
"mean": float(np.mean(finite)) if len(finite) else np.nan,
"std": float(np.std(finite)) if len(finite) else np.nan,
"median": float(np.median(finite)) if len(finite) else np.nan,
"minimum": float(np.min(finite)) if len(finite) else np.nan,
"maximum": float(np.max(finite)) if len(finite) else np.nan,
"ci95_low": ci_low,
"ci95_high": ci_high,
}
)
return pd.DataFrame(rows)
```

```
def equivalence_analysis(
scalar: pd.DataFrame,
npz_path: str,
bootstraps: int,
analysis_seed: int,
) -> pd.DataFrame:
indexed = {
branch: scalar.loc[scalar["branch"] == branch].set_index("seed")
for branch in BRANCHES
}
rows: list[dict[str, Any]] = []


for left, right, outcome, margin in EQUIVALENCE_SPECS:
shared = sorted(set(indexed[left].index) & set(indexed[right].index), key=int)
left_values = pd.to_numeric(
indexed[left].loc[shared, outcome], errors="coerce"
).to_numpy(float)
right_values = pd.to_numeric(
indexed[right].loc[shared, outcome], errors="coerce"
).to_numpy(float)
valid = np.isfinite(left_values) & np.isfinite(right_values)
differences = left_values[valid] - right_values[valid]
ci90 = bootstrap_mean_interval(
differences,
bootstraps,
rng_for(analysis_seed, f"equiv90|{left}|{right}|{outcome}"),
confidence=0.90,
)
ci95 = bootstrap_mean_interval(
differences,
bootstraps,
rng_for(analysis_seed, f"equiv95|{left}|{right}|{outcome}"),
confidence=0.95,
)
rows.append(
{
"left_branch": left,
"right_branch": right,
"outcome": outcome,
```

```
"difference_definition": "left_minus_right",
"exploratory_margin": margin,
"n_valid": int(len(differences)),
"mean_difference": float(np.mean(differences)),
"ci90_low": ci90[0],
"ci90_high": ci90[1],
"ci95_low": ci95[0],
"ci95_high": ci95[1],
"equivalent_by_90pct_CI": bool(
ci90[0] > -margin and ci90[1] < margin
),
"equivalent_by_95pct_CI_conservative": bool(
ci95[0] > -margin and ci95[1] < margin
),
"fraction_seed_differences_within_margin": float(
np.mean(np.abs(differences) < margin)
),
"note": "exploratory post-outcome margin",
}
)


# Magnitude-sensitive one-sided equivalence using late mean B distance.
seeds = sorted(
scalar.loc[scalar["branch"] == "continue", "seed"].astype(str), key=int
)
with np.load(npz_path, allow_pickle=False) as data:
for branch in ("reset_B", "reset_ema"):
distances = []
for seed in seeds:
reference = np.asarray(
data[npz_key("continue", seed, "B_mean_late")], dtype=float
)
current = np.asarray(
data[npz_key(branch, seed, "B_mean_late")], dtype=float
)
distances.append(normalized_edge_distance(current, reference))
values = np.asarray(distances, dtype=float)
ci90 = bootstrap_mean_interval(
values,
bootstraps,
rng_for(analysis_seed, f"frob_equiv90|{branch}"),
confidence=0.90,
```

```
)
ci95 = bootstrap_mean_interval(
values,
bootstraps,
rng_for(analysis_seed, f"frob_equiv95|{branch}"),
confidence=0.95,
)
margin = 0.01
rows.append(
{
"left_branch": branch,
"right_branch": "continue",
"outcome": "normalized_late_mean_frobenius_distance",
"difference_definition": "nonnegative_distance",
"exploratory_margin": margin,
"n_valid": int(len(values)),
"mean_difference": float(np.mean(values)),
"ci90_low": ci90[0],
"ci90_high": ci90[1],
"ci95_low": ci95[0],
"ci95_high": ci95[1],
"equivalent_by_90pct_CI": bool(ci90[1] < margin),
"equivalent_by_95pct_CI_conservative": bool(
ci95[1] < margin
),
"fraction_seed_differences_within_margin": float(
np.mean(values < margin)
),
"note": "exploratory one-sided distance margin",
}
)
return pd.DataFrame(rows)



def plot_envelopes(
envelopes: pd.DataFrame,
output_dir: str,
window_time: float,
recovery_end_time: float,
) -> list[str]:
try:
import matplotlib
```

```
matplotlib.use("Agg")
import matplotlib.pyplot as plt
except ImportError:
print("matplotlib unavailable; skipping plots")
return []


paths: list[str] = []
for metric, ylabel in (
("similarity", "Weighted topology similarity"),
("density", "Threshold density"),
):
fig, ax = plt.subplots(figsize=(12, 7))
for branch in BRANCHES:
block = envelopes.loc[
(envelopes["branch"] == branch)
& (envelopes["metric"] == metric)
]
ax.plot(
block["time_since_split"], block["mean"], label=branch, linewidth=2
)
ax.fill_between(
block["time_since_split"],
block["seed_q025"],
block["seed_q975"],
alpha=0.10,
)
ax.axvline(window_time, color="black", linestyle="--", alpha=0.5)
ax.axvline(recovery_end_time, color="black", linestyle=":", alpha=0.6)
ax.set_xlabel("Time since split")
ax.set_ylabel(ylabel)
ax.set_title(f"AIEF-1.4 {metric}: immediate, recovery, and late dynamics")
ax.grid(alpha=0.25)
ax.legend(fontsize=8, ncol=2)
fig.tight_layout()
path = os.path.join(output_dir, f"{metric}_recovery_envelope.png")
fig.savefig(path, dpi=180)
plt.close(fig)
paths.append(path)
return paths
```

```
def make_report(
seed_metrics: pd.DataFrame,
summary: pd.DataFrame,
equivalence: pd.DataFrame,
) -> str:
lines = [
"AIEF-1.4 trajectory, sustained-recovery, and equivalence analysis",
"============================================================",
"",
"Similarity NaNs caused by an exactly zero edge-weight vector are mapped to zero only for recovery/loss calculations.",
"Equivalence margins are exploratory and post-outcome; they are not preregistered confirmatory bounds.",
"",
"Key branch dynamics",
"--------------------",
]


selected_metrics = (
"minimum_similarity",
"sustained_similarity_90_rest",
"sustained_density_90_rest",
"Mean_similarity_deficit_vs_continue",
"Mean_density_deficit_vs_continue",
"sampled_similarity_mean_immediate",
"sampled_similarity_mean_recovery",
"sampled_similarity_mean_late",
)
for branch in BRANCHES:
lines.append(branch)
block = summary.loc[
(summary["branch"] == branch)
& (summary["metric"].isin(selected_metrics))
]
for metric in selected_metrics:
row_set = block.loc[block["metric"] == metric]
if row_set.empty:
continue
row = row_set.iloc[0]
lines.append(
f" {metric}: mean={row['mean']:.6g}, "
f"median={row['median']:.6g}, "
```

```
f"95% CI=[{row['ci95_low']:.6g}, {row['ci95_high']:.6g}], "
f"valid={int(row['n_valid'])}/{int(row['n_total'])}"
)


lines.extend(["", "Exploratory equivalence", "----------------------"])
for _, row in equivalence.iterrows():
lines.append(
f" {row['left_branch']} vs {row['right_branch']} / {row['outcome']}: "
f"mean={row['mean_difference']:.6g}, margin={row['exploratory_margin']:.6g}, "
f"90% CI=[{row['ci90_low']:.6g}, {row['ci90_high']:.6g}], "
f"equivalent90={row['equivalent_by_90pct_CI']}, "
f"within_margin={row['fraction_seed_differences_within_margin']:.3f}"
)


lines.extend(
[
"",
"Interpretation constraints",
"-------------------------",
"* First crossing and sustained recovery are different estimands.",
"* Late convergence does not imply that an intervention caused no transient loss.",
"* Density recovery does not imply recovery of edge identity.",
"* B_carrier_only/full_reset remains exploratory because r2 used unmatched phase resets.",
"",
]
)
return "\n".join(lines)



def parse_arguments() -> argparse.Namespace:
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("--scalar", default=DEFAULT_SCALAR)
parser.add_argument("--npz", default=DEFAULT_NPZ)
parser.add_argument("--manifest", default=DEFAULT_MANIFEST)
parser.add_argument("--output-dir", default=DEFAULT_OUTPUT_DIR)
parser.add_argument("--bootstrap", type=int, default=DEFAULT_BOOTSTRAPS)
parser.add_argument("--analysis-seed", type=int, default=DEFAULT_ANALYSIS_SEED)
parser.add_argument("--no-plots", action="store_true")
return parser.parse_args()
```

```
def main() -> None:
args = parse_arguments()
if not os.path.isfile(args.npz):
raise FileNotFoundError(args.npz)
if args.bootstrap < 0:
raise ValueError("bootstrap repetitions must be nonnegative")
os.makedirs(args.output_dir, exist_ok=True)


scalar, manifest = load_inputs(args.scalar, args.manifest)
window_time = float(manifest["protocol"]["window_steps"]) * float(
manifest["model"]["dt"]
)
recovery_end_time = float(manifest["protocol"]["recovery_end"]) * float(
manifest["model"]["dt"]
)


seed_metrics, envelopes = trajectory_analysis(scalar, args.npz, manifest)
summary = summarize_metrics(seed_metrics, args.bootstrap, args.analysis_seed)
equivalence = equivalence_analysis(
scalar, args.npz, args.bootstrap, args.analysis_seed
)


seed_metrics.to_csv(
os.path.join(args.output_dir, "trajectory_metrics_by_seed.csv"), index=False
)
summary.to_csv(
os.path.join(args.output_dir, "trajectory_metrics_summary.csv"), index=False
)
equivalence.to_csv(
os.path.join(args.output_dir, "equivalence_tests.csv"), index=False
)
envelopes.to_csv(
os.path.join(args.output_dir, "trajectory_envelopes.csv"), index=False
)


plot_paths = [] if args.no_plots else plot_envelopes(
envelopes, args.output_dir, window_time, recovery_end_time
)
report = make_report(seed_metrics, summary, equivalence)
```

```
report_path = os.path.join(args.output_dir, "trajectory_recovery_report.txt")
with open(report_path, "w", encoding="utf-8") as handle:
handle.write(report)
print(report)


analysis_manifest = {
"analysis": "AIEF-1.4 trajectory, sustained-recovery, and equivalence",
"scalar_path": str(Path(args.scalar).resolve()),
"scalar_sha256": file_sha256(args.scalar),
"npz_path": str(Path(args.npz).resolve()),
"npz_sha256": file_sha256(args.npz),
"source_manifest_path": str(Path(args.manifest).resolve()),
"source_manifest_sha256": file_sha256(args.manifest),
"bootstrap_repetitions": args.bootstrap,
"analysis_seed": args.analysis_seed,
"window_time": window_time,
"recovery_end_time": recovery_end_time,
"equivalence_specs": [list(spec) for spec in EQUIVALENCE_SPECS],
"equivalence_note": "exploratory post-outcome margins",
"zero_similarity_convention": "NaN cosine from zero vector mapped to 0 for recovery/loss
calculations only",
"trajectory_bands": "empirical 2.5% and 97.5% seed quantiles",
"plots": [str(Path(path).resolve()) for path in plot_paths],
}
with open(
os.path.join(args.output_dir, "analysis_manifest.json"),
"w",
encoding="utf-8",
) as handle:
json.dump(analysis_manifest, handle, indent=2, sort_keys=True)
handle.write("\n")


print(f"Saved analysis to {Path(args.output_dir).resolve()}")



if __name__ == "__main__":
main()
Open
```

Created the trajectory, sustained-recovery, and equivalence analysis script:

AIEF_1_4_trajectory_recovery_equivalence.py

It defaults to the completed 40-seed run:

text

runs/5a9f093a2fe0/

Run

Bash

python AIEF_1_4_trajectory_recovery_equivalence.py

Without plots:

Bash

python AIEF_1_4_trajectory_recovery_equivalence.py --no-plots

Explicit invocation:

Bash

```
python AIEF_1_4_trajectory_recovery_equivalence.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_trajectory_recovery
```

What it calculates
Per-seed reconstruction dynamics

For every seed and branch:

first crossing of 50%/90% continuation similarity;
first crossing of 50%/90% continuation density;
time after which similarity remains above 50%/90% for the rest of the run;
time after which density remains above 50%/90%;
time held above 90% for at least one T_WINDOW;
minimum similarity and density;
immediate, recovery, late, and full sampled means.

Integrated loss metrics
Asimilarity=
Asimilarity=

\int\max(0,S_{\rm continue}-S_{\rm branch}),dt
]
Adensity=
Adensity=

\int\max(0,D_{\rm continue}-D_{\rm branch}),dt
]

It also calculates density excess relative to continuation.

Both raw AUC and duration-normalized mean deficits are written.
Zero-vector convention

When B=0B=0, cosine similarity is mathematically undefined. For recovery/loss calculations only, the script maps those NaN samples to zero similarity and records:

text

similarity_nan_samples_imputed_zero

The original stored trajectories remain unchanged.
Trajectory envelopes

For density and similarity at each sampled time:

mean,

median,
standard deviation,
empirical 2.5% and 97.5% seed quantiles.

The quantile bands describe seed distributions, not confidence intervals.

Plots include vertical lines at:

1*T_WINDOW;
5*T_WINDOW.

Exploratory equivalence analysis

For:

text

continue versus reset_B
continue versus reset_ema

it applies exploratory margins:

text

Weighted similarity: ±0.01
Retention contrast: ±1/66
Late persistence density: ±1/66
Late-mean Frobenius distance: 0.01

It reports:

90% and 95% bootstrap intervals;
equivalence under each interval;
fraction of seed differences inside the margin.

These margins were selected after seeing r2 outcomes and are explicitly labeled exploratory, not preregistered confirmatory bounds.

Output

Default directory:

text

aief_1_4_trajectory_recovery/

Files:

text

trajectory_metrics_by_seed.csv
trajectory_metrics_summary.csv
equivalence_tests.csv
trajectory_envelopes.csv
trajectory_recovery_report.txt
analysis_manifest.json
similarity_recovery_envelope.png
density_recovery_envelope.png

The PNGs are produced when matplotlib is available.
Validation

I syntax-tested the script and ran it end-to-end against a complete seven-branch AIEF-1.4 smoke result. The test covered:

zero-vector similarity handling;
first-crossing and sustained recovery;
one-window hold criteria;
integrated loss;
horizon summaries;
equivalence intervals;
normalized late-mean Frobenius distance;
manifest and report generation.

The generated report also preserves the key caveats:

late convergence is not immediate retention;
density recovery is not edge-identity recovery;
first crossing is not sustained recovery;
the r2 B_carrier_only/full_reset comparison remains exploratory because its phase resets were unmatched.

(deck@steamdeck SSRH)$ python AIEF_1_4_trajectory_recovery_equivalence.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_trajectory_recovery
AIEF-1.4 trajectory, sustained-recovery, and equivalence analysis

Similarity NaNs caused by an exactly zero edge-weight vector are mapped to zero only for recovery/loss calculations.
Equivalence margins are exploratory and post-outcome; they are not preregistered confirmatory bounds.
Key branch dynamics

continue
minimum_similarity: mean=0.999471, median=1, 95% CI=[0.998413, 1], valid=40/40
sustained_similarity_90_rest: mean=0.01, median=0.01, 95% CI=[0.01, 0.01], valid=40/40
sustained_density_90_rest: mean=0.01, median=0.01, 95% CI=[0.01, 0.01], valid=40/40
Mean_similarity_deficit_vs_continue: mean=0, median=0, 95% CI=[0, 0], valid=40/40
Mean_density_deficit_vs_continue: mean=0, median=0, 95% CI=[0, 0], valid=40/40
sampled_similarity_mean_immediate: mean=0.999979, median=1, 95% CI=[0.999937, 1], valid=40/40
sampled_similarity_mean_recovery: mean=0.999625, median=1, 95% CI=[0.998875, 1], valid=40/40
sampled_similarity_mean_late: mean=0.999472, median=1, 95% CI=[0.998416, 1], valid=40/40
reset_ema
minimum_similarity: mean=0.837815, median=0.852224, 95% CI=[0.828044, 0.84611], valid=40/40
sustained_similarity_90_rest: mean=13.7023, median=13.51, 95% CI=[13.4972, 13.8946], valid=39/40
sustained_density_90_rest: mean=28.2792, median=22.01, 95% CI=[21.8562, 41.0228], valid=39/40
Mean_similarity_deficit_vs_continue: mean=0.0118898, median=0.00475957, 95% CI=[0.0049691, 0.0249576], valid=40/40
Mean_density_deficit_vs_continue: mean=0.0199033, median=0.0152938, 95% CI=[0.0160709, 0.0266805], valid=40/40
sampled_similarity_mean_immediate: mean=0.909371, median=0.915467, 95% CI=[0.90411, 0.914195], valid=40/40
sampled_similarity_mean_recovery: mean=0.991821, median=0.999409, 95% CI=[0.978323, 0.999354], valid=40/40
sampled_similarity_mean_late: mean=0.992413, median=1, 95% CI=[0.978228, 1], valid=40/40

reset_B
minimum_similarity: mean=0.843832, median=0.822851, 95% CI=[0.833109, 0.856835], valid=40/40
sustained_similarity_90_rest: mean=4.7725, median=5.01, 95% CI=[4.2975, 5.1475], valid=40/40
sustained_density_90_rest: mean=3.56, median=3.51, 95% CI=[3.51, 3.66], valid=40/40
Mean_similarity_deficit_vs_continue: mean=0.00331514, median=0.00363895, 95% CI=[0.00304062, 0.00352904], valid=40/40
Mean_density_deficit_vs_continue: mean=0.00306794, median=0.00292834, 95% CI=[0.00299088, 0.00314975], valid=40/40
sampled_similarity_mean_immediate: mean=0.937257, median=0.931039, 95% CI=[0.933162, 0.942365], valid=40/40
sampled_similarity_mean_recovery: mean=0.99968, median=0.999999, 95% CI=[0.99905, 0.999997], valid=40/40
sampled_similarity_mean_late: mean=0.999473, median=1, 95% CI=[0.99842, 1], valid=40/40
reset_B_ema
minimum_similarity: mean=0, median=0, 95% CI=[0, 0], valid=40/40
sustained_similarity_90_rest: mean=30.9179, median=29.01, 95% CI=[29.3258, 32.5758], valid=38/40
sustained_density_90_rest: mean=29.51, median=22.01, 95% CI=[22.01, 44.51], valid=39/40
Mean_similarity_deficit_vs_continue: mean=0.092276, median=0.0844111, 95% CI=[0.0745116, 0.112815], valid=40/40
Mean_density_deficit_vs_continue: mean=0.0224813, median=0.0195974, 95% CI=[0.0201647, 0.0260765], valid=40/40
sampled_similarity_mean_immediate: mean=0.33907, median=0.329101, 95% CI=[0.334114, 0.344348], valid=40/40
sampled_similarity_mean_recovery: mean=0.924289, median=0.933406, 95% CI=[0.905127, 0.940774], valid=40/40
sampled_similarity_mean_late: mean=0.943978, median=0.949471, 95% CI=[0.921518, 0.963533], valid=40/40
B_carrier_only
minimum_similarity: mean=0.813042, median=0.839769, 95% CI=[0.795804, 0.827843], valid=40/40
sustained_similarity_90_rest: mean=24.1424, median=14.51, 95% CI=[16.7747, 33.51], valid=34/40
sustained_density_90_rest: mean=71.1026, median=75.01, 95% CI=[63.8429, 78.0285], valid=27/40
Mean_similarity_deficit_vs_continue: mean=0.057112, median=0.0115051, 95% CI=[0.0368591, 0.0802916], valid=40/40
Mean_density_deficit_vs_continue: mean=0.048139, median=0.0344881, 95% CI=[0.0393702, 0.0578968], valid=40/40
sampled_similarity_mean_immediate: mean=0.896329, median=0.906513, 95% CI=[0.888327, 0.903362], valid=40/40
sampled_similarity_mean_recovery: mean=0.936719, median=0.971929, 95% CI=[0.913818, 0.957381], valid=40/40
sampled_similarity_mean_late: mean=0.947675, median=0.99999, 95% CI=[0.923025, 0.969446], valid=40/40
full_reset

minimum_similarity: mean=0, median=0, 95% CI=[0, 0], valid=40/40
sustained_similarity_90_rest: mean=58.75, median=55.51, 95% CI=[51.07, 68.83], valid=25/40
sustained_density_90_rest: mean=78.6767, median=82.51, 95% CI=[73.8736, 83.4191], valid=33/40
Mean_similarity_deficit_vs_continue: mean=0.154756, median=0.138994, 95% CI=[0.130253, 0.180571], valid=40/40
Mean_density_deficit_vs_continue: mean=0.0447328, median=0.0371556, 95% CI=[0.0376137, 0.0532618], valid=40/40
sampled_similarity_mean_immediate: mean=0.336211, median=0.327002, 95% CI=[0.331509, 0.341286], valid=40/40
sampled_similarity_mean_recovery: mean=0.841023, median=0.848322, 95% CI=[0.818825, 0.862774], valid=40/40
sampled_similarity_mean_late: mean=0.88332, median=0.909828, 95% CI=[0.854037, 0.911367], valid=40/40
future_shuffle
minimum_similarity: mean=0.126602, median=0.114082, 95% CI=[0.110914, 0.152507], valid=40/40
sustained_similarity_90_rest: mean=nan, median=nan, 95% CI=[nan, nan], valid=0/40
sustained_density_90_rest: mean=nan, median=nan, 95% CI=[nan, nan], valid=0/40
Mean_similarity_deficit_vs_continue: mean=0.558397, median=0.575796, 95% CI=[0.542866, 0.570344], valid=40/40
Mean_density_deficit_vs_continue: mean=0.292923, median=0.281109, 95% CI=[0.285737, 0.300577], valid=40/40
sampled_similarity_mean_immediate: mean=0.988158, median=0.985008, 95% CI=[0.98629, 0.990108], valid=40/40
sampled_similarity_mean_recovery: mean=0.716414, median=0.704324, 95% CI=[0.691957, 0.740016], valid=40/40
sampled_similarity_mean_late: mean=0.317534, median=0.298825, 95% CI=[0.305231, 0.335913], valid=40/40
Exploratory equivalence

continue vs reset_B / Weighted_similarity_late: mean=-1.25453e-06, margin=0.01, 90% CI=[-3.7638e-06, 2.07636e-10], equivalent90=True, within_marg
in=1.000
continue vs reset_B / Retention_contrast_late: mean=0, margin=0.0151515, 90% CI=[0, 0], equivalent90=True, within_margin=1.000
continue vs reset_B / P_mean_late: mean=0, margin=0.0151515, 90% CI=[0, 0], equivalent90=True, within_margin=1.000
continue vs reset_ema / Weighted_similarity_late: mean=0.0068217, margin=0.01, 90% CI=[9.50368e-10, 0.019992], equivalent90=False, within_margin=
0.975
continue vs reset_ema / Retention_contrast_late: mean=0.0099674, margin=0.0151515, 90% CI=[0, 0.0259674], equivalent90=False, within_margin=0.950

continue vs reset_ema / P_mean_late: mean=0.00359668, margin=0.0151515, 90% CI=[0, 0.00965728], equivalent90=True, within_margin=0.950
reset_B vs continue / normalized_late_mean_frobenius_distance: mean=9.16382e-06, margin=0.01, 90% CI=[6.35225e-07, 2.1142e-05], equivalent90=True
, within_margin=1.000
reset_ema vs continue / normalized_late_mean_frobenius_distance: mean=0.0341763, margin=0.01, 90% CI=[1.66619e-05, 0.079864], equivalent90=False,
within_margin=0.950
Interpretation constraints

First crossing and sustained recovery are different estimands.
Late convergence does not imply that an intervention caused no transient loss.
Density recovery does not imply recovery of edge identity.
B_carrier_only/full_reset remains exploratory because r2 used unmatched phase resets.

Saved analysis to /home/deck/AI_Programs/SSRH/aief_1_4_trajectory_recovery

This analysis is much more informative than the late endpoint table. It distinguishes immediate state retention, reconstruction speed, and asymptotic convergence.
1. Resetting BB causes a transient, then essentially exact reconstruction

reset_B is not literally “no effect”:

text

Minimum similarity: 0.844 mean
Immediate mean similarity: 0.937
Sustained 90% similarity: 4.77 time units
Sustained 90% density: 3.56 time units
Mean similarity deficit: 0.00332
Late similarity: 0.99947

Thus:

Removing BB produces a measurable immediate disturbance.
Retained EMA/phase/history rapidly regenerates the same edge orientation.
Threshold density and topology similarity recover within roughly five time units.
Late topology is practically identical to continuation.

Since:
$T_{window} \approx 17.5$,
$T_{window} \approx 17.5$,

the 90% similarity recovery time is only approximately:
$4.77 17.5 \approx 0.27 T_{window}$.
$17.5 4.77 \approx 0.27 T_{window}$.

This is fast reconstruction, not persistence of the erased BB.

A subtle point: cosine similarity is scale-insensitive. Immediately after resetting BB, even a very small regenerated weight vector can already point in the correct edge-space direction. Density recovery confirms that threshold-scale weights also return quickly.
2. Resetting EMA causes a slower and more heterogeneous transient

text

Minimum similarity: 0.838
Immediate similarity: 0.909
Sustained 90% similarity: 13.70, valid 39/40
Sustained 90% density: 28.28, valid 39/40
Mean similarity deficit: 0.0119
Late similarity mean: 0.9924
Late similarity median: 1.000

EMA reset has a larger and longer transient than resetting BB.

Approximately:
$13.7 / 17.5 \approx 0.78 T_{window}$
$13.7 / 17.5 \approx 0.78 T_{window}$

for similarity recovery, and:
$28.3 / 17.5 \approx 1.62 T_{window}$
$28.3 / 17.5 \approx 1.62 T_{window}$

for density recovery.

The mean/median difference and one unrecovered seed show that this effect is heterogeneous.

The equivalence results capture this:

late persistence density is equivalent under the exploratory margin;
weighted similarity is not globally equivalent;
retention contrast is not globally equivalent;
95–97.5% of individual seeds nevertheless fall inside the margins.

A good interpretation is:

EMA is dispensable for late threshold density in most realizations, but contributes to reconstruction fidelity and prevents rare larger deviations.

3. BB and EMA are strongly redundant

reset_B_ema shows the consequence of removing both:

text

Immediate similarity: 0.339
Minimum similarity: 0
Sustained 90% similarity: 30.92, valid 38/40
Sustained 90% density: 29.51, valid 39/40
Recovery similarity: 0.924
Late similarity: 0.944
Mean similarity deficit: 0.0923

The retained phase and delayed history can reconstruct much of the topology, but:

reconstruction is slower;
two seeds never sustain 90% similarity;
identity fidelity remains lower;
emergence produces a denser alternative topology.

This confirms the previously observed B×B× EMA interaction. Either channel can compensate substantially when the other remains, but joint removal exposes a clear fidelity loss.
4. Preserving BB does not guarantee immediate stability after dynamic-state reset

B_carrier_only begins with mature BB but resets EMA and phase/history:

text

Minimum similarity: 0.813
Immediate similarity: 0.896
Sustained 90% similarity: valid 34/40
Sustained 90% density: valid 27/40
Late similarity mean: 0.948
Late similarity median: 1.000

The median late topology is nearly exact, but the mean is lowered by a substantial failure tail.

This is evidence against BB being sufficient by itself. Preserved topology can be destabilized when placed into an incompatible dynamical state.

The result is highly heterogeneous:

six seeds never sustain 90% similarity;
thirteen never sustain 90% reference density;
median recovery is much better than the mean.

The unmatched phase-reset problem still prevents a clean comparison with full_reset, but this branch alone shows that topology state and dynamical state must be mutually compatible.
5. Full reset reveals a strong but nonunique attractor

text

Minimum similarity: 0
Immediate similarity: 0.336
Recovery similarity: 0.841
Late similarity mean: 0.883
Late similarity median: 0.910
Sustained 90% similarity: valid 25/40
Sustained 90% density: valid 33/40

After removing every saved dynamic state, the preserved model structure reconstructs a substantially similar topology, but not a unique exact one.

Only 62.5% of seeds sustain 90% similarity:
25/40=0.625.
25/40=0.625.

This argues for a strong topology-selection attractor or attractor family, but not a single globally unique fixed topology.

The preserved model structure determines a strong template; initial/dynamical state affects which nearby realization is reached.
6. Future shuffle demonstrates progressive maintenance failure

The trajectory is especially revealing:

text

Immediate similarity: 0.988
Recovery similarity: 0.716
Late similarity: 0.318
Minimum similarity: 0.127
Sustained 90% similarity: 0/40
Sustained 90% density: 0/40
Mean similarity deficit: 0.558
Mean density deficit: 0.293

Topology is initially intact because the split state is preserved. It then progressively disappears after future driver identity is destroyed.

This cleanly demonstrates maintenance dependence:

Historical topology alone is insufficient. Continued edge-specific driver structure is necessary to prevent progressive threshold-topology loss.

This is stronger than an endpoint-only observation because it shows the direction of causality in time: intact topology precedes the driver intervention, then decays.
7. Recovery-time means are conditional on recovery

For example:

text

full_reset sustained 90% similarity:
mean = 58.75
valid = 25/40

That mean describes only the 25 seeds that recovered. The other 15 are right-censored at the future-window endpoint.

Therefore ordinary means understate recovery time for heterogeneous branches.

For:

reset_ema,
reset_B_ema,
B_carrier_only,
full_reset,
future_shuffle,

report both:

recovery probability;
recovery-time distribution conditional on recovery.

A more formal treatment would use Kaplan–Meier curves with nonrecovered seeds censored at the future-window endpoint. This is particularly useful for B_carrier_only and full_reset.
8. Equivalence conclusion
continue versus reset_B

All exploratory equivalence checks pass:

similarity;
retention;
persistence density;
Frobenius distance;
100% of seeds within margins.

This is compelling practical equivalence for late topology.
continue versus reset_ema

Results are mixed:

persistence density passes equivalence;
weighted similarity fails;
retention contrast fails;
Frobenius distance fails;
95% or more of seeds individually fall within margins.

Thus:

EMA reset is usually equivalent but has a small subset of meaningfully affected realizations.

The next useful analysis is not another average test; it is identifying those outlier seeds and determining what distinguishes them.
Mechanistic hierarchy supported by trajectories

The data now support:

Future edge-specific driver identity: required for continued maintenance.
Retained EMA/phase/history: enables very rapid exact reconstruction after BB removal.
Retained BB: supports reconstruction but is not sufficient to prevent disruption after dynamic-state reset.
Phase/history without BB or EMA: reconstructs most topology more slowly and less faithfully.
Preserved model structure after full reset: produces a recognizable topology template, but with multiple possible realizations.

Recommended next actions

Use the generated trajectory figures as primary AIEF-1.4 figures.
Add recovery-probability and Kaplan–Meier plots for 90% similarity/density.
Identify the affected reset_ema seeds from trajectory_metrics_by_seed.csv.
Perform residual localization on those seeds and on the tiny reset_B residuals.
Complete the matched-phase B_carrier_only/full_reset correction.
Do not run another broad ablation cohort.

Refined conclusion

AIEF-1.4 reveals rapid and redundant topology reconstruction rather than memory localized in a single state variable. Erased adaptive weights are reconstructed within a fraction of one EMA timescale when dynamical state remains. EMA removal causes a longer transient and rare reconstruction failures, while joint state removal produces slower, less faithful attractor selection. Preserved model structure alone reconstructs a recognizable but nonunique topology. Continued edge-specific future driver identity is uniquely necessary for maintaining threshold topology; destroying it causes progressive loss from an initially intact state.

That is a substantially stronger mechanistic result than the late endpoint summary alone.

Muse on this pondering:

I agree with the synthesis, but I would make one important refinement before locking the scientific interpretation.

The trajectory analysis has moved AIEF-1.4 beyond a “memory carrier ablation” experiment. It is now really a state-variable role decomposition experiment. The original question:

“Where is topology memory stored?”

was too narrow. The data suggest the system has at least three separable functions:

Topology reconstruction
Topology selection
Topology maintenance

The branches now map onto those functions.

The central result is not “(B) is not memory”

I would avoid saying:

(B) is a slave/readout variable.

That is slightly too strong.

The data show:

(B) is not sufficient as an isolated memory carrier.
(B) is not necessary for reconstruction if the fast state remains.
But (B) still participates in the slow adaptive dynamics that stabilizes the selected state.

The better statement is:

(B) is a persistent state variable, but not the sole information-bearing variable determining the mature topology.

This distinction matters.

A dynamical system can have information distributed across variables without one variable being “the memory.”

The slow–fast interpretation is now strongly supported

The architecture appears to have:

Fast subsystem

[
x(t)=
(\theta(t),h(t),\eta(t))
]

where:

phase variables;
delay/history variables;
EMA/adaptive accumulators.

Slow subsystem

$$
B(t)
$$

the adaptive coupling topology.

The qualitative equations are something like:

$$
\dot{x}=F(x,B,\omega,u)
$$

$$
\dot{B}=\epsilon G(x,B)
$$

with:

$$
0<\epsilon\ll1
$$

The ablations are effectively asking:

Remove slow variable

$$
B\rightarrow0
$$

but preserve:

$$
x(t)
$$

Result:

reset_B

Fast state regenerates slow manifold.

Remove fast variables

preserve:

[
B
]

destroy:

[
x(t)
]

Result:

B_carrier_only

The stored slow variable is insufficient without compatible fast coordinates.

Remove both

[
(B,x)\rightarrow(0,0)
]

Result:

full_reset

System returns to the attractor basin, but loses trajectory-specific information.

This is exactly the signature of a slow–fast adaptive system.

The strongest causal experiment is actually future_shuffle

I think the analysis correctly identifies this, but I would elevate it.

Most memory experiments ask:

"Can the system remember after destruction?"

Future shuffle asks:

"Can the system maintain memory when the future causal channel is corrupted?"

That is more fundamental.

The trajectory:

t_split

similarity ≈ 0.99
|
|
↓
driver identity destroyed
|
|
↓
similarity ≈0.32

is almost a causal intervention demonstration.

The topology exists.

The stored state exists.

The adaptive matrix exists.

Only the future edge-specific resonance assignment is changed.

Then the topology decays.

That tells us:

$$
\text{maintenance} \neq \text{storage}
$$

The system does not merely retrieve a stored graph.

It continually regenerates it.

The next analysis I would prioritize is not another ablation

I agree with the recommendation:

Do not run another broad seed cohort.

The experiment has enough statistical power.

The next analyses should be targeted.

I would rank them:

Priority 1: Survival analysis of recovery

This is the biggest missing statistical piece.

Your observation about censoring is correct.

For each branch:

Event:

$$
T_{90}
$$

= time to sustained 90% similarity.

If not recovered:

right censored.

Then compute:

Kaplan–Meier survival curves;
median recovery time;
probability of recovery before (T).

Especially:

reset_B
reset_B_ema
B_carrier_only
full_reset

This will convert:

25/40 recovered

into a proper dynamical statistic.

Priority 2: Identify outlier seeds

Especially:

reset_ema failures

Because they are scientifically interesting.

Question:

What makes the rare EMA-sensitive seeds different?

Look at:

initial synchronization (r);
projection activity;
initial edge density;
spectral properties of (B);
dominant eigenvalues;
weight entropy.

Hypothesis:

EMA failures occur near a basin boundary.

Priority 3: Spectral analysis of topology recovery

Given the SSRH mathematical framework, this is the bridge.

For each branch compute:

[
\lambda_{\max}(B)
]

or non-Hermitian spectral radius:

[
\rho(B)

]

over time.

Ask:

After reset_B:

Does the spectrum recover before topology?

Possible result:

[
\rho(B)
\rightarrow
\text{threshold structure}
\rightarrow
\text{edge identity}
]

That would connect directly to the earlier rank-one perturbation / stability framework.

Priority 4: Matched phase rerun

Still necessary, but now clearly secondary.

The corrected comparison:

B_carrier_only
vs
full_reset

with identical:

phase reset;
history reset;
RNG stream.

This answers:

Given identical dynamical initialization, how much does retaining B alone help?

The current result suggests:

probably some benefit, but not dominance.

One conceptual correction I would make in the paper language

I would avoid:

“The topology is not stored.”

because a reviewer could object:

“but (B) clearly stores information.”

The stronger wording is:

“The mature topology is not localized to a single state variable. The selected configuration is regenerated through the coupled resonance–adaptation dynamics.”

That is harder to attack.

AIEF-1.4 provisional model

The evidence now supports this hierarchy:

[
\boxed{
\text{future driver identity}
}
]

controls maintenance

↓

$$
\boxed{
\text{phase/history/EMA state}
}
$$

controls rapid reconstruction

↓

$$
\boxed{
B
}
$$

stores a transient adaptive realization

↓

$$
\boxed{
\omega+\dynamics+\projection
}
$$

define the attractor landscape

This is a much richer result than “memory in (B).”

My recommended next sequence

I would do:

A. Finish the statistical story

Kaplan–Meier recovery curves.
Recovery probability table.
Outlier seed analysis.

B. Finish the causal story

Matched B_carrier_only/full_reset.

C. Connect to theory

Spectral recovery of (B(t)).
Relate recovery rate to spectral gap / contraction estimates.

Then AIEF-1.4 has a complete narrative:

The system does not store a mature topology in a single adaptive matrix. Instead, the topology is a dynamically regenerated attractor whose reconstruction and maintenance depend on different components of the coupled resonance-adaptation system.

That is a significantly stronger result than the original memory-carrier hypothesis.

The state-variable role decomposition is a better framing than “find the memory register.” The data distinguish retention, reconstruction, selection, and maintenance. However, two parts of the proposed interpretation need tightening: the slow–fast claim and the spectral-analysis plan.
1. BB is persistent but not a unique memory register

The proposed wording is good:

BB is a persistent adaptive state variable, but not the sole information-bearing variable determining mature topology.

The existing interventions establish:

Preserving BB does not guarantee perfect recovery after phase/EMA destruction.
Resetting BB does not prevent rapid exact reconstruction when EMA and phase/history remain.
Jointly removing BB and EMA reduces reconstruction fidelity.
Future structured input is required for continued threshold maintenance.

Thus the system has distributed state dependence plus attractor reconstruction.
2. The strict slow–fast interpretation is not yet established

Writing:
$x˙=F(x,B,\omega,u),B˙=\epsilon G(x,B),\epsilon\ll 1$
$x˙=F(x,B,\omega,u),B˙=\epsilon G(x,B),\epsilon\ll 1$

is plausible but not directly demonstrated.

The empirical recovery times complicate that interpretation:

oscillator periods are approximately 3–6 time units;
reset_B recovers 90% density in approximately 3.6 units;
EMA has a timescale of approximately 17.5;
$1/\gamma\approx 66.71/\gamma\approx 66.7$, but BB has additive growth and can regrow much faster than its decay timescale.

In fact, resetting BB produces faster recovery than resetting EMA. Therefore BB is not uniformly "slow" in the intervention regime.

A safer formulation is:

The system contains interacting phase, estimator, and adaptive-topology timescales, but a formal singular slow–fast separation has not yet been demonstrated.

To support $\epsilon\ll 1\epsilon\ll 1$, vary adaptation rates and show timescale separation systematically.

Also avoid using $\eta(t)\eta(t)$ as a generic fast-state symbol because ETA is already an adaptation parameter in the model.
3. Maintenance and storage are clearly distinct

The future_shuffle trajectory is the cleanest evidence:

Topology is intact immediately after the split.
Historical BB, EMA, phase, and delay history remain.
Future driver identity is destroyed.
Similarity and threshold density progressively decay.

Thus:
stored state alone$\not\Rightarrow$continued topology maintenance.
stored state alone $\Rightarrow$continued topology maintenance.

The mature graph is continuously regenerated by structured input rather than passively retrieved.

Still qualify the result:

Timestep-wise destruction of driver identity abolishes maintenance.

This does not yet cover fixed or slower block permutations.
4. Survival analysis is the correct next statistical step

Use the existing sustained-recovery definition:
T90=
T90=

\text{first time after which similarity remains above 90%}.
]

For seeds that never recover, censor at the final future time.

Report, per branch:

Kaplan–Meier probability of remaining unrecovered;
recovery probability by the end of the window;
median recovery time where estimable;
restricted mean time unrecovered;
confidence intervals.

Important details:

future_shuffle has no recovery events.
full_reset has 25/40 similarity-recovery events.
B_carrier_only has 34/40.
reset_B_ema has 38/40.
reset_ema has 39/40.
reset_B has 40/40.

Do not report only the mean recovery time among recovered seeds; that is biased downward.

Because branches are paired by seed, ordinary independent-sample log-rank comparisons are not ideal. Emphasize branch-specific curves and bootstrap paired differences in:

recovery probability,
restricted mean unrecovered time,
and event indicators.

5. Outlier analysis should focus on basin-margin features

The EMA-sensitive seeds are scientifically useful because all seeds share the same ωω. Their differences arise from initialized phase/state trajectories.

For every seed, extract at the split:

phase order parameter:
rsplit=∣1N∑ieiθi∣;
rsplit=
N1i∑eiθi
;
threshold density of BsplitBsplit;
total edge weight;
effective edge count;
Gini and entropy;
minimum edge distance from threshold:
mB=mini<j∣Bij−Bc∣;
mB=i<jmin∣Bij−Bc∣;
selected-edge margin:
minBij>Bc(Bij−Bc);
Bij>Bcmin(Bij−Bc);
row-sum slack;

EMA magnitude distribution;
historical resonance contrast between selected and unselected edges;
adjacency and Laplacian spectra.

Then compare seeds that:

recover versus remain censored;
fall inside versus outside the equivalence margin;
have large versus negligible integrated topology loss.

A small threshold margin or weak resonance contrast would support a basin-boundary interpretation.
6. Spectral analysis requires careful definitions

BB remains symmetric, so it is not non-Hermitian. Its eigenvalues are real. Do not describe its largest eigenvalue as a non-Hermitian spectral radius.

Candidate spectral metrics include:
Weighted adjacency

largest eigenvalue $\lambda_1(B)\lambda_1(B)$;
spectral gap $\lambda_1-\lambda_2\lambda_1-\lambda_2$;
eigenvector localization.

Weighted Laplacian
$L=\mathrm{diag}(B1)-B.$
$L=\mathrm{diag}(B1)-B.$

Report:

algebraic connectivity $\lambda_2(L)\lambda_2(L)$;
largest Laplacian eigenvalue;
spectral entropy.

The row-sum projection bounds $\lambda_1(B)\lambda_1(B)$, so the largest eigenvalue may mostly report cap saturation. Spectral gap, localization, and Laplacian connectivity may be more informative.

The current NPZ contains only:

split state;
final topology;
late-mean topology.

It does not store B(t)B(t) at every diagnostic time. Therefore a full spectral-recovery trajectory cannot be reconstructed from the current run.

You can perform three-state spectral analysis now:

text

split
late mean
final

A temporal spectral-recovery curve requires a targeted rerun that stores spectral summaries online.
7. The hierarchy should be softened slightly

A defensible hierarchy is:

Future driver identity — necessary for maintenance under timestep-wise shuffle.
Phase/history and EMA state — strongly accelerate high-fidelity reconstruction after BB removal.
BB — preserves a mature adaptive realization but is neither individually necessary nor sufficient for perfect late reconstruction.
Preserved model structure — defines a strong but apparently nonunique topology-attractor landscape.
Projection — enforces the hard admissible manifold on which constrained branches evolve.

That is stronger than saying BB is merely a readout and more accurate than calling it the dominant memory carrier.
8. Matched-phase correction remains necessary

The targeted correction should use identical randomized phase/history for:

text

B_carrier_only
full_reset

Then only BB differs.

This should precede any final statement about the causal benefit of preserving BB.
9. Numerical convergence remains a prerequisite for theory

The projection correction is approximately equal in norm to the proposed update on mature active steps. That makes the discrete timestep and admissibility map central to the observed fixed point.

Before claiming a contraction or unique stable fixed point:

run DT = 0.005 and 0.0025 on a smaller fixed seed subset;
compare topology identity and recovery times;
calculate retained-update ratios;
verify convergence of the projection-dominated dynamics.

The symmetric scaling map enforces the constraint but has not been shown to be an orthogonal projection. In theoretical work, call it an admissibility map or symmetric retraction unless that property is proven.
Revised priority order
Complete current data analysis

Kaplan–Meier recovery curves and restricted mean unrecovered times.
Outlier/basin-margin analysis.
Split/late/final spectral analysis.
Residual-defect localization.

Correct the causal comparison

Matched-phase B_carrier_only versus full_reset.

Validate numerics

Timestep convergence on a smaller paired cohort.

Then advance mechanism/theory

Add online spectral summaries in a targeted run.

Design the spectrum-switch experiment.
Begin fixed-RR and averaged-RR fixed-point analysis.

Refined paper-level conclusion

AIEF-1.4 decomposes mature topology into distinct state-variable roles rather than identifying a single memory register. Adaptive weights preserve one realization of the selected topology, while phase/history and EMA state enable rapid reconstruction after weights are erased. Preserved model structure defines a strong but nonunique attractor landscape. Maintenance is distinct from storage: disrupting future edge-specific driver identity causes progressive topology loss despite an intact split state. The results support dynamically regenerated topology with distributed state dependence rather than memory localized to one variable.

That framing is well supported by the current evidence and leaves the remaining matched-phase, numerical-convergence, and spectrum-perturbation questions properly open.

Create the code necessary for the next step.
Edit
AIEF_1_4_survival_outlier_spectral.pyPYTHON

Created the next-stage analysis script:

AIEF_1_4_survival_outlier_spectral.py

It performs:

censored sustained-recovery analysis;
Kaplan–Meier curves;
restricted mean unrecovered time;
paired recovery contrasts;
split-state basin-margin feature extraction;
descriptive outlier/recovery associations;
split/late/final spectral analysis;
reset_B residual localization and SVD.

Prerequisite

It uses the trajectory metrics generated by:

Bash

```
python AIEF_1_4_trajectory_recovery_equivalence.py
```

The expected file is:

text

```
aief_1_4_trajectory_recovery/trajectory_metrics_by_seed.csv
```

Run

Bash

```
python AIEF_1_4_survival_outlier_spectral.py
```

Explicit invocation:

Bash

```
python AIEF_1_4_survival_outlier_spectral.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--trajectory-metrics aief_1_4_trajectory_recovery/trajectory_metrics_by_seed.csv \
--output-dir aief_1_4_survival_outlier_spectral
```

Without matplotlib plots:

Bash

```
python AIEF_1_4_survival_outlier_spectral.py --no-plots
```

Recovery analysis

For sustained 90% similarity and density, it calculates:

number recovered and censored;
recovery probability;
Kaplan–Meier median recovery time;
restricted mean unrecovered time;
bootstrap CIs;
conditional mean time among recovered seeds;
per-seed event/censor records.

Seeds that never recover are right-censored at the 320-unit future-window endpoint rather than silently excluded.

It also calculates paired recovery contrasts for:

continue versus reset_B;
continue versus reset_ema;
reset_B versus reset_B_ema;
continue versus future_shuffle;
exploratory B_carrier_only versus full_reset.

Split-state features

For every seed it extracts from B_at_split, phase, EMA, and historical resonance:

split phase order parameter;
density and total weight;
effective edge number;
Gini and entropy;
closest edge distance to B_C;
selected and unselected threshold margins;
row-sum slack;
selected/unselected historical-resonance contrast;
selected/unselected EMA contrast;
adjacency spectral metrics;
Laplacian spectral metrics.

Outlier/basin analysis

For:

reset_ema;
reset_B_ema;
B_carrier_only;
full_reset;

the script reports descriptive relationships between split-state features and:

recovery versus censoring;
restricted recovery time;
mean topology-similarity deficit;
late topology similarity.

These are explicitly labeled descriptive because 40 seeds are insufficient for high-dimensional causal feature selection.
Spectral analysis

Because BB is symmetric, it uses real symmetric eigendecomposition.

Metrics include:

largest and second adjacency eigenvalues;
adjacency spectral gap;
principal-eigenvector inverse participation ratio;
Laplacian algebraic connectivity;
largest Laplacian eigenvalue;
Laplacian spectral entropy.

These are computed at:

split;
late-mean topology;
final topology.

The report explicitly notes that these three snapshots are not a continuous spectral-recovery trajectory.
Residual “defect” analysis

For:

text

reset_B late mean − continue late mean

it calculates per seed:

residual norm;
normalized residual norm;
largest-edge contribution;
edge participation ratio;
maximum node burden;
dominant residual edge;
dominant edge frequency gap;
correlation between residual magnitude and frequency separation.

Across seeds, it performs SVD and saves:

explained residual variance by mode;
first residual mode edge loadings;
edge-frequency metadata.

I corrected the participation-ratio calculation so it remains scale-invariant even when residuals are extremely small. It no longer incorrectly treats a genuine 10−810−8-scale residual as zero merely because its fourth-power denominator is below a generic absolute tolerance.
Outputs

text

aief_1_4_survival_outlier_spectral/
├── KM_curves.csv
├── recovery_summary.csv
├── recovery_by_seed.csv
├── paired_recovery_contrasts.csv
├── split_state_features.csv
├── spectral_metrics_split_late_final.csv
├── feature_recovery_associations.csv

├── resetB_residual_localization_by_seed.csv
├── resetB_residual_svd.csv
├── resetB_residual_first_mode_edges.csv
├── KM_similarity90_recovery.png
├── KM_density90_recovery.png
├── survival_outlier_spectral_report.txt
└── analysis_manifest.json

I syntax-tested the script and ran it end-to-end against a complete two-seed seven-branch result, including censored recovery events, spectral extraction, feature associations, and residual SVD.
(deck@steamdeck SSRH)$ python AIEF_1_4_survival_outlier_spectral.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--trajectory-metrics aief_1_4_trajectory_recovery/trajectory_metrics_by_seed.csv \
--output-dir aief_1_4_survival_outlier_spectral
AIEF-1.4 survival, basin-feature, spectral, and residual analysis

Recovery-time means are conditional on recovery; recovery probability and restricted mean unrecovered time account for censoring.
B_carrier_only/full_reset remains exploratory because r2 used unmatched phase resets.
Sustained recovery

continue
similarity90: recovered=40/40, prob=1.000, KM_median=0.01, RMUT=0.01, conditional_mean=0.01
density90: recovered=40/40, prob=1.000, KM_median=0.01, RMUT=0.01, conditional_mean=0.01
reset_ema
similarity90: recovered=39/40, prob=0.975, KM_median=13.51, RMUT=21.3598, conditional_mean=13.7023
density90: recovered=39/40, prob=0.975, KM_median=22.01, RMUT=35.5723, conditional_mean=28.2792
reset_B
similarity90: recovered=40/40, prob=1.000, KM_median=5.01, RMUT=4.7725, conditional_mean=4.7725
density90: recovered=40/40, prob=1.000, KM_median=3.51, RMUT=3.56, conditional_mean=3.56
reset_B_ema
similarity90: recovered=38/40, prob=0.950, KM_median=29.01, RMUT=45.372, conditional_mean=30.9179
density90: recovered=39/40, prob=0.975, KM_median=22.01, RMUT=36.7723, conditional_mean=29.51
B_carrier_only

similarity90: recovered=34/40, prob=0.850, KM_median=14.51, RMUT=68.521, conditional_mean=24.1424
density90: recovered=27/40, prob=0.675, KM_median=85.51, RMUT=151.994, conditional_mean=71.1026
full_reset
similarity90: recovered=25/40, prob=0.625, KM_median=69.01, RMUT=156.719, conditional_mean=58.75
density90: recovered=33/40, prob=0.825, KM_median=84.01, RMUT=120.908, conditional_mean=78.6767
future_shuffle
similarity90: recovered=0/40, prob=0.000, KM_median=nan, RMUT=320, conditional_mean=nan
density90: recovered=0/40, prob=0.000, KM_median=nan, RMUT=320, conditional_mean=nan
Primary recovery contrasts

continue - reset_B / similarity90: restricted_time_delta=-4.7625, 95% CI=[-5.1375, -4.275], recovery_probability_delta=0.000
continue - reset_ema / similarity90: restricted_time_delta=-21.3498, 95% CI=[-36.7743, -13.5375], recovery_probability_delta=0.025
reset_B - reset_B_ema / similarity90: restricted_time_delta=-40.5995, 95% CI=[-62.9737, -25.1625], recovery_probability_delta=0.050
continue - future_shuffle / similarity90: restricted_time_delta=-319.99, 95% CI=[-319.99, -319.99], recovery_probability_delta=1.000
continue - reset_B / density90: restricted_time_delta=-3.55, 95% CI=[-3.65, -3.5], recovery_probability_delta=0.000
continue - reset_ema / density90: restricted_time_delta=-35.5622, 95% CI=[-56.6493, -21.875], recovery_probability_delta=0.025
reset_B - reset_B_ema / density90: restricted_time_delta=-33.2122, 95% CI=[-55.3743, -18.5], recovery_probability_delta=0.025
continue - future_shuffle / density90: restricted_time_delta=-319.99, 95% CI=[-319.99, -319.99], recovery_probability_delta=1.000
reset_B residual localization

normalized residual mean=9.16382e-06
participation ratio median=5.32513
largest-edge fraction median=0.516288
Interpretation constraints

Censored recovery times must not be treated as ordinary missing values.
Feature associations are descriptive and high-dimensional relative to n=40.
B is symmetric; adjacency spectra are real, not non-Hermitian.

Largest adjacency eigenvalue is bounded by the row-sum constraint and may mainly report cap saturation.
Split/late/final spectra do not constitute a continuous spectral-recovery trajectory.

Saved analysis to /home/deck/AI_Programs/SSRH/aief_1_4_survival_outlier_spectral

This survival analysis clarifies the mechanism considerably. The most important finding is that different state channels affect reconstruction speed, recovery probability, and topology identity differently.
1. Retained EMA/phase state reconstructs topology extremely rapidly after BB removal

reset_B:

text

Similarity recovery: 40/40
Density recovery: 40/40
Similarity RMUT: 4.77
Density RMUT: 3.56

This is the cleanest reconstruction result.

Erasing BB does not eliminate the information needed to regenerate it. The retained EMA plus phase/history state reconstructs threshold density in approximately 3.6 time units and sustained topology similarity in approximately 4.8.

Given:
$T_{window} \approx 17.5$,
$T_{window} \approx 17.5$,

this occurs in substantially less than one EMA timescale.

The result supports:

The preexisting adaptive matrix is not necessary for rapid mature-topology reconstruction when the dynamical resonance state is preserved.

2. EMA contributes strongly to reconstruction kinetics

Compare:

text

reset_B:
similarity RMUT = 4.77
recovery probability = 1.00

reset_B_ema:
similarity RMUT = 45.37
recovery probability = 0.95

Paired effect:

text

Restricted-time difference:
reset_B − reset_B_ema = −40.60
95% CI [−62.97, −25.16]

When BB is absent, retaining EMA dramatically accelerates reconstruction and improves recovery probability.

Thus the earlier endpoint statement “EMA has almost no effect” needs qualification:

EMA is usually dispensable when mature BB remains.
EMA is highly consequential when BB has been removed.
This is precisely the redundancy captured by the B×B× EMA interaction.

EMA functions less like a sole memory store and more like a reconstruction guide or accelerator.
3. Resetting EMA while preserving BB produces slower recovery and rare failure

text

reset_ema:

similarity recovery = 39/40
density recovery = 39/40
similarity RMUT = 21.36
density RMUT = 35.57

This is slower than resetting BB, despite preserving the mature topology.

A plausible dynamical explanation is that resetting EMA temporarily makes the retained BB incompatible with its future adaptation signal. The graph undergoes pruning/reorganization until the estimator rebuilds.

The single censored seed should be examined, but one failure is insufficient for reliable multivariate "basin boundary" inference.
4. B_carrier_only reveals state incompatibility or hysteresis

text

Similarity recovery = 34/40 = 85.0%
Density recovery = 27/40 = 67.5%
Similarity RMUT = 68.52
Density RMUT = 151.99

Preserving BB while destroying EMA and phase/history is not sufficient for reliable recovery. In fact, density recovery is worse than under full_reset:

text

B_carrier_only density recovery: 67.5%
full_reset density recovery: 82.5%

This suggests mature BB can impede reorganization when placed into an incompatible randomized dynamical state. That is consistent with hysteresis or basin mismatch.

However, this comparison remains exploratory because the two branches used different randomized phase resets. The matched-phase rerun is required before assigning the difference causally to BB.
5. Full reset shows a strong but nonunique model attractor

text

Similarity recovery = 25/40 = 62.5%
Density recovery = 33/40 = 82.5%
Similarity RMUT = 156.72
Density RMUT = 120.91

The preserved model structure can frequently reconstruct density and sometimes reconstruct high topology similarity, but recovery is slow and not universal.

The difference between density and similarity recovery is important:

density recovery is relatively common;
identity recovery is substantially less reliable.

Thus the model has a strong density/topology-selection attractor family, not necessarily one unique topology.
6. Future driver identity is absolutely required under timestep-wise shuffling

text

Similarity recovery = 0/40
Density recovery = 0/40
RMUT = full censor time = 320

The paired contrast against continuation is maximal:

text

Restricted-time difference = −319.99
Recovery-probability difference = 1.00

This is the strongest causal result:

Once future edge-specific driver assignment is destroyed every timestep, no realization sustainably recovers threshold density or mature-topology similarity.

Historical state alone is insufficient for maintenance.

7. Recovery probability and RMUT should be headline statistics

The survival output resolves the bias in conditional recovery-time means.

For example:

text

full_reset conditional mean = 58.75
full_reset RMUT = 156.72

The conditional mean ignores 15 censored seeds and makes recovery appear much faster than it is at the population level.

For the paper or final report, emphasize:

recovery probability;
restricted mean unrecovered time;
Kaplan–Meier median where estimable;
conditional mean only as supplementary.

8. Test sensitivity to the sustained-recovery definition

“Remain above 90% for the rest of the run” is intentionally strict. A single late dip classifies a seed as unrecovered.

Compare it with the already calculated:

text

held_similarity_90_for_1Twindow
held_density_90_for_1Twindow

If conclusions are similar, the survival result is robust. If recovery rates rise substantially under a one-window hold criterion, some censoring reflects intermittent late fluctuations rather than true failure.
9. The reset-BB residual is tiny but spatially localized

text

Normalized residual mean: 9.16×10^{-6}
Participation ratio median: 5.33
Largest-edge fraction median: 0.516

The residual is negligible in total magnitude but concentrated on roughly five effective edges, with the largest edge carrying about half the residual norm.

This resembles a localized reconstruction residue in edge space—not a changed threshold topology.

To determine whether it resembles a systematic “defect” or numerical residue, inspect:

text

resetB_residual_first_mode_edges.csv
resetB_residual_svd.csv

Questions:

Does the first residual mode explain substantial variance?
Do the same edges dominate across seeds?
Are dominant edges associated with particular frequency gaps?
Is there a recurring node-centered motif?

If dominant edges vary across seeds and SVD structure is weak, it is likely transient/numerical residue.
If the same motif recurs, it is a systematic localized correction mode.
10. Next analysis actions
Immediate

Plot the Kaplan–Meier curves.
Compare sustained-rest recovery with one-T_WINDOW hold recovery.
List censored seeds by branch.
Inspect split-state features of:
the one reset_ema failure;
the two reset_B_ema failures;
recovered versus censored B_carrier_only and full_reset seeds.

Spectral

Compare split spectral features between recovered and censored seeds:
adjacency gap;
principal-vector localization;
Laplacian connectivity;
threshold margin.

Keep this descriptive, especially when there are only one or two failures.
Residual

Inspect SVD explained variance and recurring dominant edges.
Plot residual participation ratio and normalized residual norm.

Causal correction

Run the matched-phase B_carrier_only/full_reset correction.
Do not interpret their current survival difference causally until corrected.

Refined mechanism

The survival data support the following role decomposition:

EMA plus phase/history: rapid reconstruction after topology deletion.
BB: persistent adaptive realization that helps only when dynamically compatible.
Phase/history without BB or EMA: slower but often successful reconstruction.
Preserved model structure: slow and incomplete attractor reconstruction after full reset.
Future driver identity: indispensable for sustained maintenance under the tested shuffle.

This is stronger than a late-state comparison because it distinguishes whether a branch retains topology, reconstructs it quickly, reconstructs it slowly, or never recovers.

[Socratis Petrides] I’m very happy to introduce Brendan.

We go way back to grad school and then he was a postdoc here for a couple of years. I know now

he’s doing great things now and he’s gonna, sorry, he’s an assistant professor at Brown now. And I

know he's doing great things and I'm very happy to be looking forward to listen to this talk,

which as the title says is about proximal Galerkin, which is a method for variational

problems with inequality constraints. Some of you might be familiar already, some of you not,

so please Brendan, take it away. [Brendan Keith] Thank you, Socratis. Thanks to all the organizers for inviting me to speak

here [at the] seminar series. It's been a lot of great speakers over the years now. I feel like

I'm in very good company. So, yeah, so I want to tell you basically about the main line of

work that I started. Maybe the initial ideas were actually founded while, while I was at the Lab,

but then I really started exploring them once they, in new ways once I came here to,

to Brown. So, I'd like to share what we've done over the last few years. So,

the talk will proceed following roughly the agenda that you see here. I'm gonna

spend some time trying to motivate the class of problems that this method is meant to tackle,

and why I think inequality constraints are some of the most interesting areas of research

in scientific computing right now. There's lots of open problems and they arise everywhere. Then I'm

going to talk a little bit about the technical approach that leads to the method, the proximal

Galerkin [PG] method, so then in order to do that I need to introduce some ideas from convex analysis and brief ideas from differential geometry and put those together in a way that's

suitable for applying final method and that gives us proximal Galerkin. Then I'm gonna show some

things you can prove about the method, but the really this talk is about the applications. So throughout I'm going to interleave many different applications where this framework can be applied, some of which are listed here, but many more, you'll just have to wait and see as they come.

So, I always like to start at what I really think of as the bread and butter of scientific computing. These are the problems that I think the flagship strategies deal with very well. This is what we do in scientific computing all the time, and I think we do it very well. So, what is it? Enforcing linear equality constraints or sometimes more general just equality constraints, so where do these appear? We might have a physical model, like an electromagnetism, we want to make sure because in the actual model there are no magnetic monopoles, we like to make sure that there's no magnetic monopoles when we discretize, and that roughly speaking corresponds to enforcing this differential equality constraint. If we're evolving a molecular, a molecular system, closed molecular system, there's another physical property we'll probably want to enforce. That's the energy remains constant in time. We can also do this to great accuracy in scientific computing. Another class of problems that, with strong Lab relevance, is material transport. Now you can debate with me whether we do this well enough or not, but I think we do it very well at least in the case where we move the mesh and we actually treat this equation in a Lagrangian frame. It's actually when we remap and reset the discretizations where things get kind of tricky and that's actually also where inequality constraints arise again, but in

each of these different problems or classes of problems, and there are many more, we're enforcing some sort of equality condition or equality constraint. And then from the very highest level, what are we doing? A lot of these constraints, not all of them, we have some ambient function space, and we need to rely on some subspace of that function space. So, the Galerkin method is perfectly well suited for doing this because it's an approximation scheme based on subspaces. And we can very easily navigate those subspaces which define the constraint both of the continuous and discrete level, and that's sort of how the finite element method is really, this is what it's really designed for. There are other problems also where we maybe lie on a closed manifold and there are other ways to make sure that we don't leave the manifold and navigate it. At the end of the day, these are the first class methods, these are what we often teach to students in classes, and they lead to the most systematic, robust solutions to a wide variety of different problems all across scientific computing. This is the bread and butter. But not every problem in scientific computing obviously falls into this category of equality constraints. There are also inequality constraints, and they are at least as important as the previous category. So, where do these arrive by probability distributions or density functions to be non-negative or material densities likewise also have to be non-negative. There might be maximum principles that come from the model, and we want those to be enforced at the discrete level. Optimal design, we have box constraints, multiphase flows, other constraints, inequality constraints, and we want to preserve those. But it gets much more complicated than that. There can also be non-convex constraints

produced in contact, we may want a body maybe not to penetrate another body that can be seen as an inequality constraint, but we want the body not to penetrate itself or to interpenetrate when it's deformed. These are also inequality constraints of a different nature but also is important. Or there might be actual an arrow of time that we want the simulation to respect, for instance, we're simulating some damage to a material through fracture or plasticity or otherwise, want to make sure that the, we know that the original model doesn't have any mechanism to spontaneously heal the material. You want to make sure that that's also enforced at discrete level. So, these are inequality constraints on time derivatives you can think of. And there are many more than these three examples, but at the very highest level, what distinguishes it from the equality constraint setting is that we're not navigating a closed subspace of some function space, maybe in the simplest setting we're navigating some convex subset of that space of functions, maybe a polytope or otherwise, but it has a boundary and that's often the issue, and you can get more complex if you have constraints which are non-convex and you're navigating some manifold with boundary. And what we need is a way to navigate these manifolds with boundaries that's somehow respected by the discretization. And if we don't somehow respect this boundary in the discretization, we fall into another class of solution strategies. I like to think of these as sort of the discretize and repair side of scientific computing,

where you make a discretization, you don't restrict these constraints and then you have to adjust it or reset it or do something else. Now these problems are so important,

obviously they have to be solved, and people have been looking at them for decades. It's

not like they've just left been there waiting for somebody. People have been studying them,

they've been developing methods. A lot of these methods are lowest order. The ones that are higher

order are specialized in different fields. The things that people do in fluid mechanics are often

very different since the techniques developed in solid mechanics or in optimization areas,

and there are no unifying principles, no unified framework to deal with all these different types

of constraints. And that's what this talk is about is an attempt to put together a unified framework

where we can systematically handle constraints of all of these different types across scientific

computing. All right, so how do we do this? It's very ambitious. We're partially there.

I'll show you what we've done over the last few years and where we aim to go. But I first have to give you an idea of what's the foundational concepts that our approach is based on. And the

way I like to start is sort of this simple setting where can I give you the simplest

geometric example of what inequality constraints look like. Look at the upper half plane of R2,

and we need some way to navigate upper half plane, and the sort of simplest thing one could think of

is just transform to new coordinates. So somehow we know how to navigate R2 perfectly well. So,

if we transform this upper half plane, we can map it into R2 and then we can discretize or

whatever we want over here, and we'd have to maybe map back. Now these transformations,

ideas change the coordinates. It's not new and they've been, they've arisen in different topics,

but there's different questions you need to answer if you wanna go this route. First of all, how do you generalize this, the more complicated sets, and not everything is just a unilateral box

constraint. And how do you make this compatible with the functional analysis and the study

of these PDEs or variational inequalities that actually have these inequality constraints because

this transformation is singular and that presents a barrier once you discretize as well. So,

there's a way to get around this singularity and treat general constraints and all of these things,

and the first step along that path is to introduce the concept of a Legendre function. Okay, so what

is the Legendre function? This is actually an idea that goes back to the 1960s. Terry Rockafellar,

who's still with us to this day, actually was visiting us at Brown last week, he's sort of the

godfather of convex analysis and built most of the field on his own. So, the concept of the Legendre

function, he was looking at what are, what are the classes of functions that the Legendre transformed as well to find on. And it turns out it's a class of functions, they're not quite barrier functions,

that have a certain singularity at the boundary of a convex set. So, the domain of the function is

an essential domain of the functions. These are these are extended real value functions, so maps

from RM to R or including infinity. The essential domain of all the points in RM that are mapped to

finite values. We need a differentiable function on the interior of the domain, it needs to be

strictly convexed in the interior of the domain, and it has this condition which says that the

gradient of the function blows up as you approach the boundary, not the function itself necessarily, so don't think of just a logarithmic barrier, think of something like the Shannon entropy that's well defined everywhere over its domain but blows up in its gradient on the boundary. Okay, and it turns out that this function can be used to encode pretty much the complete differential geometry, Riemannian geometry of a convex set, right? And I'll show you this one step at a time. So, we take a set C, convex set C, and we, our task is to find the Legendre function whose essential domain is equal to C. Now there's a theorem from Rockafellar in 1967 that tells you that a Legendre function's gradient, okay, is a bijection, a smooth bijection, an isomorphism between the interior of that domain R, so the interior of your convex set C, to the interior of the domain of the conjugate function. So, this is conjugate function given to by the Fenchel-Rockafellar transform here. The transform is basically coding all the information of the convex hull of your function's epigraph, rewriting the function in terms of the function of those supporting hyperplanes. Alright, I'll give you a bunch of examples here because this is the most abstract part of the talk. But this fundamental theorem tells you that you have this isomorphism on the interior of your convex set C, given a specific function like a Legendre function. And there's another fact also proved by Rockafellar, that the domain of the conjugate function is all of RM, if and only if you have this proper this growth condition, so this is saying that your function grows faster than any linear function as you move away from the origin. Okay, this is called super-linearity or what convexity, super-convex duality is many different names for the same concept. Optimizers

haven’t really settled on what they really wanna call it, but it’s a well-known common condition. And if you have this, a Legendre function with this condition, then what this fact and this theorem tells you is that the gradient of the conjugate function is a map from RM to the interior of C. So, this is your transformation. You can find your Legendre function, and you get your coordinate transformation from it. So, examples are required now. This transformation I showed you a few slides back from the half plane. Okay, this logarithmic transformation. Well, this should be the gradient of something, turns out it’s the gradient of this Legendre function, so it’s quadratic in its first coordinate and it looks like the Shannon entropy in the second coordinate. But obviously not every constraint looks like this, but if we, oh, I’ll show you some other examples in a second, let me just review the parts of the theorem. So, this transformation, it turns out it will always be singular because actually that’s the part of the definition of the Legendre function. Its gradient needs to blow up as you approach the boundary of your convex set. But the beautiful thing about this class of functions is that the inverse transformation is always smooth. So, for instance, this one is the inverse looks like X times the exponential of Y. This is infinitely differentiable, beautiful transformation moving back. And by convex duality, Rockafellar’s theorem, the inverse of the gradient is the gradient of the conjugate function, okay? So now this, because the inverse is always smooth, it actually directs us in deriving discretizations, it means we should discretize here what we know how to do it well, and if we

use the inverse transformation, we will map back into our feasible set. And that's exactly what Legendre functions allow to do, okay? It gives us a beautiful systematic way of designing coordinate transformations from a given context set to RM and then back and we can use that in constructing discretizations. And I'll show you how to do that in 10 or 15 slides. But now some more examples. So, if we have bound constraints, upper and lower bound constraints, you can use this direct transformation, this logistic transformation, it's also given to you by the gradient of a Legendre function. Here you have quadratic in X and this, in Y this is called the Fermi-Dirac entropy. If you have a ball, you know some sort of unit ball constraints, there's inverse radial coordinate transformation, it's also the gradient of Legendre function. This one here which sometimes is called Hellinger entropy. Each map is singular, each inverse is infinitely smooth, and I'll give you many more examples later. But we need a little bit more than just a transformation. It turns out the Legendre function also gives us a geometry or Riemannian structure to the convex set, and we access that structure by actually looking at the Hessian of the function. So, yeah the Legendre function gradient is a transformation, we can treat the interior C as a Riemannian manifold with a metric given by the Hessian of R. Now remember, one of the properties of R was that it strictly convex with the Hessian. Okay, this is always a positive definite matrix at every point. Then we can, once we have the metric defined, we can define geodesics, you know, the concept of straight lines, on our manifold, and it turns out that under this metric we have curved lines.

That's how we would navigate move from A to B in a feasible set, but under the transformation they turn into straight lines over here, and this means that we design an algorithm where we just wanted to define an additive update rule like gradient descent, it's very convenient to do it over here and then transfer them back. We can also define a gradient flow on our manifold. Okay, so we would choose instead of the Euclidean gradient on our feasible set, we can choose the Riemannian gradient that comes from, that's informed by the metric. So then Romanian geometry, instead of using the Euclidean gradient, you, you precondition it with the inverse of your Hessian, which turns out to also be the Hessian of the conjugate function but as a chain rule that I'll spare you. So here you have a gradient flow, and this is pretty much a direct path towards deciding an optimization algorithm on A, on your feasible set. You define F and it's restricted to C, you want to find the minimum, you can follow the Riemannian gradient towards that minimum. Okay, and how do we usually discretize gradient flows? Well, we use steepest descent or otherwise known as explicit Euler. If so, if it's a Euclidean gradient flow, we have this equation here, it's well known explicit Euler formula is just the rewriting of steep is descent. Optimizers also know that you can cover this update rule by just minimizing this quadratic subproblem or you linearize your objective function, penalize the distance from your previous point using the norm squared, and the local minimize or the minimizer will be your next point in your descent method. Now you can ask an obvious question, if you look at this linearized subproblem, do we get a more accurate update if we use the full objective function? Yes, you probably do, is the answer.

And the subproblem would look something like this, and optimizers call this the proximal subproblem, this is actually where proximal, proximal Galerkin comes from. If you solve this subproblem, and there's no constraints, it turns out you get implicit Euler, okay? Why is, we would say that this problem's maybe more accurate because it's the implicit Euler scheme is stable for any choice of alpha, otherwise you can do an updates rule where the update point or the point of computing a gradient is done defined implicitly in terms of where you're gonna end up. So, this is uniform, you can think of it in two different ways. This is a step-sized rule where you can use any alpha you want. Or it's, it's a, an ODE integrator that's uniformly stable or in alpha. Okay, we want to use this idea for Riemannian gradient descent. Now our gradient looks a little bit different. We have this Hessian term here, and now there's this little trick that we can use. It's called the chain rule. We take this equation, we multiply both sides, sides by the Hessian of R, and we would get this term on the left-hand side of our ODE and then just the gradient of F on the right-hand side. But by the chain rule, we know that that this term here on the left-hand side is just the derivative of the gradient of our U. For this right-hand side we could discretize with implicit Euler, and we get this update rule here. Okay, now instead of looking at a divided difference in terms of just U, we're looking at the divided difference in terms of this transformed U where we have the transformation coming the gradient of the Legendre function appearing. Now this update rule is also well known to optimization community. This can be rewritten

just by multiplying through by alpha as you can call it the Bregman proximal point iteration, which is equivalent to minimizing a different proximal subproblem where instead of penalizing with the squared Euclidean norm of the difference between U and UK, you penalize with something called a so-called Bregman divergence. Okay, what's a Bregman divergence? Well, a Bregman divergence is basically a generalization of the squared norm, but it's generated by a Legendre function here. It's looking at the linearization error in your first-order Taylor expansion function at a point. And if you, your Legendre function is quadratic, then you actually get back the squared Euclidean distance in the proximal point map. So, let's say that R is the Shannon entropy and we wanna look at the find, compute the distance between U and a different point, and if we do so, we could vary U and we would get and we get every when we vary U, we get a different tangent plane, and we can vary the point we want to measure the distance from, and the Bregman divergence is the distance away from the tangent plane. So, if we now shift, basically rotate our function by subtracting off this linearization, we see what the Bregman divergence is. It's everywhere positive or non-negative function, it's non-symmetric, and it's this generalized square distance between U when V. All right and as you vary V, you end up seeing that at the boundary of the domain, the tangent plane is vertical, meaning that the distance between V and U ends up becoming infinity. And the takeaway message is because the distance sort of grows, it's penalized

non-symmetrically, T means two different points. Every iterate of your method stays in the interior, you generate an interior sequence of points that converge to a minimum as you would expect, you're evolving a gradient ingredient flow if this is the time-discretized version, but every point of that time-discretized version evolves within the feasible set. Every iterate takes you to a lower function value, so that's what this video is showing. You take F, you penalize F with your Bregman divergence, you get a different subproblem with it, and when you find the minimum, you always find a minimum at a lower point than what you started with. Okay, so that's Bregman proximal point in a nutshell, and the rest of the talk is about how to use this for mechanics. We have everything we need in our Legendre function coordinate transformation, as well as the geodesics and our Hessian and our gradient flow scheme. So, what problems do we often find in mechanics? They look something like this. This is sort of the "hello world" of inequality constraints. You minimize the Dirichlet energy instead of just fixing the points on the boundary, which would give you Poisson's equation, you have the fix the points in the boundary, so you're minimizing an H1, but you also have a constraint on the function lying above some obstacle which is defined through the graphs of this function phi. So, the equilibrium point of this or the minimize the minimizer of this energy ends up getting the equilibrium position of a thin membrane lying clamped on the ends and lying above this obstacle. But not every problem looks like this. If you have real contact, then it's only happening on the boundary, you minimize some

other objective function, always more complicated, and some product of H1, what boundary conditions you want to consider, and then you want to make sure that as long as it gets more complicated when you incorporate friction, but at the very least there's no penetration constraints, so the normal component of your displacement variable has to say above something called the gap function. Now, finite element methods for obstacle problems and contact, many other models have been well studied, but something that's been very hard for finite element people to do for a long time is to solve the eikonal equation. So, this is a nonlinear PDE here, it's very, there's some famous schemes, finite difference methods to do this. And I know people that have to solve these equations that sometimes actually take the finite element code, export it to finite difference code, solve the eikonal equation, and then interpolate back. There's this whole awkward pipeline that they have to follow including some friends of mine. It turns out the eikonal equation, the solution, the viscosity solution about an eikonal equation can also be written in terms of an optimization problem. Here what you're doing is you're trying to find the maximum mass density field in a feasible set with an inequality constraint on the gradient of U. And it turns out the minimum, the maximizer, the optimum, this point ends up being, giving you a point in space where this constraint is active almost everywhere. So yeah, this becomes inequality. And these, all of these examples and many others, you have feasible sets that look like this. You're finding, you're, you're look, you're looking at set of functions in some [inaudible] space V. And then you have an operator,

I'll assume for this talk that it's linear, so in the contact in the case of the obstacle problem B was the identity. In Signorini, it was the normal trace. In eikonal equation was the gradient. So, some operator applied to this function and evaluated at a given point has to be in some, in some convex set. And then this constraint needs to be satisfied at almost every point in some subset of the closure of omega. And if your feasible set is of this type, we have a table of Legendre functions you could possibly use, that encode the geometry of C. Feasible sets of this type are not just appearing in those three examples. They appear in topology optimization, fracture, plasticity, on other nonlinear PDEs than eikonal including the Monge-Ampere equation. I'll assume we always have an energy principle, so we're minimizing some objective function J over this set K and what I'm gonna use is that's somewhat, I would say we've been looking at this for a long time, so I'm tempted to say straightforward though I know it's not, it's pretty much the only way you could apply Bregman proximal point to a feasible set of this type.

You take your objective function, you penalize with the Bregman divergence, but now it's not a Bregman divergence necessarily on the argument, it's on this operator applied to the argument which is called the observation map. So, in obstacle problems this operator is the identity, the gradient, so on. And you minimize this regularized problem at each step, and because the Bregman divergence is giving you an interior point in your feasible set, when you write out the Euler-Lagrange equations, you get a nonlinear PDE, okay? Now, because this is very abstract, the PDE has all of these symbols in it, but I'll give you some examples in a minute, and in general

it looks like this. And you solve this PDE at every iteration. Now the drawback of this PDE is that the gradient of R is singular because it's a gradient of a Legendre function, so for instance, it could be like a logarithm. So, the way we get around that is pretty much the oldest trick in the book. We introduce a new variable for the term we don't like, we call it psi, the latent variable. And the first thing we do is we take rewrite our equation as a system of equations. The first one is gotten taken the singularity out, and second one we've kept it, but we can use a trick because we know that there's an inverse to this gradient that's nice and smooth. So, if we apply the inverse to second equation, then we get a smooth equation back. In fact, both equations are nice and smooth. And this is the thing we discretize. So, you start with a variational problem with inequality constraints, you regularize it, you get a sequence of nonlinear PDEs, U this is called Fenchel-Rockafellar duality to rewrite second the PDE in the saddle-point form, and then you discretize that PDE in saddle-point form. Because it's all done in a loop with the it's there's these approximal subproblems with the right-hand side is depends on the previous solution. And the Galerkin comes from the fact that we choose Galerkin subspaces of our function spaces where this problem was posed. And because the inequality constraints are imposed weakly in second equation, we don't have to worry about discretizing the feasible set directly, we discretize it somewhat indirectly, meaning we can choose any subspace compatible subspaces that

we want, V and W. This means we can go arbitrarily by order, any basis function you like. So, we apply this to the obstacle problem, for instance, okay? Then the weak formulation, I'm gonna play substitution here. We have we're gonna first look at derivative of J, well this is the Dirichlet energy, oh sorry I forgot the order of my own slides. B is the identities, so let's simplify those terms. Then we look at J, we simplify that. We're taking the Fréchet derivative of the Dirichlet energy and then the final thing we have to look at is decide what to do about is the choice of the Legendre function. And I advocate for using the Shannon entropy, but there are other choices that you can also consider, and if you use the Shannon entropy, you end up getting exponential of this variable psi here. And that's your system equations that you can discretize. Now because of time, well, let me say one more thing before I move on. When you discretize this, you get two variables, you get UH that's in a standard finite elements space piecewise polynomial, but you get a second variable, and this is nonstandard, and that just comes from looking at this part of the equation, phi plus the exponential of psi is another approximation to U. And this one will always be bound-preserving no matter how high order you go. The method itself will converge, you don't need to set send any parameter infinity unlike penalty or interior point methods. You always get a bound-preserving approximation. I'll skip this. This is a comparison to interior point about Lagrangian and other schemes and move on to other examples that you can consider. In the end, it turns out our method is mesh independent, you get the same

number of iterations unlike some others and the number of iterations is very similar to interior point. So, for Signorini, you get a saddle-point system, same original form, but the operator B is different, so it's the normal trace so we have to substitute that for first. I won't deal with what the energy function is here. This could be very complicated. But the Legendre function could still be the Shannon entropy because the constraint is unilateral. So, using the exact same Legendre function as for the obstacle problem, you get a system of equations that looks like this. Now you have a variable U that's defined over the domain and a second variable that's only defined on the compact part of the boundary, but otherwise it's very similar to the obstacle problem. And finally, for the eikonal equation, system equations, now simplifies setting B equal to the gradient, the derivative, the objective function, the objective function is linear surely so that we have a very simple-looking derivative. It's actually something you can move to the right-hand side of this system, and the, but the Legendre function is very different than the previous two examples because the constraint is different. We're trying to make sure that the gradient side stays inside the unit ball. So, you could use the Hellinger entropy for this case, but there are other choices. If you use the Hellinger entropy, then you get a system of equations that looks like this in each iteration. Other things you need to consider, what solver do you use for these nonlinear subproblems? The first choice is Newton, but there are other choices you could consider. And then once you,

you should lose your sub, choose your subproblem solver, you're gonna get linearized subproblems and you have to decide how to solve those, decide preconditioners for those. A lot of that is still open area of research. We have strategies that worked depending on the Galerkin discretization you've chosen, but a lot of the cases because of the symmetries, you don't have to just rely on GMRES, you could use MINRES, but I won't dive into that too much. We've implemented proximal Galerkin in several different finite element codes, FEniCSx, FireDrake, NgSolve, but most prominently in MFEM where there's actually three examples that you can play with if you'd like, including one for the eikonal equation. Now, I'm in a math department, so I'm obligated to prove something, and I'd like to prove that this very interesting proper, this is well known at the continuous level that every iteration of Bregman proximal point is gonna take decrease the energy, but there's something very special about the Galerkin, approximations of these subproblems that guarantee that the energy continues to decrease. No, there's no H-dependent term in this decrease. It will always decrease every compatible Galerkin approximation. The proof is actually just a few lines showing that somehow the Galerkin approximation is preserving the structure of the original algorithm. So, the first step is just the standard sub-gradient inequality, and every other step just involves using the definitions of the approximal Galerkin subproblems with different choices of test functions. So, this term here, J prime UK, looks a lot like this term here in the first line, and we could set VH to be anything we want, and the convenient thing to do is to set it

to be this difference here. And if you do that, then you can rewrite the first line as follows, and it turns into something that looks a little bit like second line, and if you set WH to be this difference of these latent variables, then you can rewrite second equation in this form here, okay. Now this is a term that we in optimization really like to see because this term for any convex function is always non-negative. It's basically telling us the gradient of our convex function is monotonic, and this is, and because this is always positive or always non-negative, we know that the objective value always decreases. Now you can prove a lot more than that, but it takes many, many pages of very hard analysis. It took us a couple of years to figure out how to do. But the takeaway message is the following. You have a true solution you're trying to approximate, and because this is a finite element method with a built-in iteration, you can't just track the mesh size, you have to actually track the iterate as well. So, it's U* the solution minus UKH. And it turns out the error at a given iterate at a given mesh decomposes into two terms. The first term depends only on alpha, and the second term depends only on H and that's really important. First of all, the first term actually only depends on the sum of all the alphas. So, we need this first term to go to 0, which happens as long as the sum diverges, which could happen if you fix alpha to be constant. Alpha could even decay and you could find a sum that diverges, so that's nice no singular parameters, but if you want the left-hand side to be less than a tolerance, it's enough to make the right-hand side less than a tolerance, and you can see that if you keep working more

iterations, once you're less than a tolerance, you will stay less than the tolerance, so if you keep defining your mesh, you'll stay less than the tolerance you were originally at. This doesn't happen in a lot of other algorithms because alpha and H or whatever substitute for alpha you have depends on H, which is mesh dependence. This means the proximal Galerkin is mesh independent. It's a mesh-independent number of iterations asymptotically to get to a fixed tolerance. And I said, I mentioned very briefly the main competitor for the obstacle problem was interior point method if you actually just count number of iterations, but you can apply this method to problems that actually aren't optimization problems it turns out. So, in fact general variational inequality, so your bilinear form isn't symmetric. There's many applications, fluid mechanics, and financial mathematics, and the way you do it is you just substitute where the J prime was, the derivative of your objective function with the monotone operator A, otherwise the algorithm exactly the same discretizations are exactly the same and you just iterate. In this case, what we can prove is a little bit more limited because we don't have the energy dissipation property because there's no necessary concept of energy when you're in this nonsymmetric, symmetric setting, but you can approximate the ergodic mean of all of your iterates, that's what U bar is, and you get the same type of error estimate decomposing into the optimization error and the discretization error. And now in the last couple of minutes, I'm gonna go over a few other example problems. Contact mechanics and obstacle problems, you always have the same type of constraint,

unilateral bound constraints, you can always use the same type of Legendre functions, you always have an exponential here. If your feasible set has upper and lower bound constraints, this appears in

topology optimization or variational fracture, you could use the Fermi-Dirac binary entropy,

and the main difference is that your nonlinearity would look something like this, otherwise the

scheme is about the same. If you have gradient constraints, you can use the Hellinger entropy,

so the example, first example I showed you was eikonal equation, but there's also plasticity models where instead of minimizing this objective function, you minimize, for instance,

the Dirichlet energy is the most basic example. Multiphase flows, the constraint is multiple

material densities or phase densities, they almost sum to 1 at every point. We've used

this for multiphase Cahn-Hilliard equations well as multi, multimaterial topology optimization.

There are problems actually in liquid crystals, in Monge-Ampere equations where you have constraints

on the eigenvalues. So, in Monge-Ampere you want the solution to be convex. It's convex, you have a unique solution. It's very, very, very difficult to design a numerical scheme that guarantees high

order, at least that guarantees you a convex function. But there's a Legendre function that

turns out for this space of positive semi-definite symmetric matrices, which can be leveraged here

to give you a high-order actually exponentially convergent scheme for Monge-Ampere. And in liquid

crystal constraints you sometimes have upper and lower bound constraints and eigenvalues and then you have to generalize the Fermi-Dirac entropy to matrix valued variables. The last thing,

and I'll just be a few minutes on this, we've recently, this was work Dohyun Kim did, started

a project in multimaterial topology optimization, where we needed to find a Legendre function for an arbitrary convex polytope, and we couldn't find one in the literature. Well, Dohyun found one. We're very excited about it. Basically, you take all the vertices of your convex polytope, you put them into a matrix, and you substitute that matrix into this formula, and then you have, one direction is very hard. Unfortunately, it's not the direction you have to ever use. We always look at the gradient of R*, which is this beautiful smooth, infinitely differentiable formula here, and this allows you to navigate any convex polytope. And we started using this. So, this is joint work with Boyan [Lazarov], Peter Gangl, and Thomas Surowiec, electric motor design, where in the stator you need to choose polarizations of your different magnets, so you can change those polarizations at different points. This is what an optimized magnet distribution would look like, touching your stator here in this coil, or in structural design where you might want to place different materials with different anisotropies in the stiff in their stiffness and for the standard type of cantilevered beam problem, but now with the multiangle or anisotropic topology optimization, you get an optimal design that looks something like this. We're also studying the PDEs, I won't say too much about this, that arise actually at the continuous PDE level because you can prove very interesting things about the pre-discretized algorithm, and we've also started applying this to multi-body contact with a feasible set depends implicitly on the solution U. Now just to wrap up here, in summary, proximal Galerkin is a unified framework we're building

out here at Brown and in other places to tackle variational problems with inequality constraints.

I gave you a brief overview of theoretical aspects of the algorithm, but if you're interested I can send you a lot of papers that go in much more detail into the

theory. And if you like what you saw here, please follow this QR code, reach out to me,

and I can send you the slides, I'm more than happy to do that, or reach out to me tell me about a

problem that you're working on with inequality constraints. I'd love to hear about it and

potentially add you to this list of fine characters here. Thank you very much.

[Socratis Petrides] Thank you, Brendan. That was very impressive, very nice talk. [Brendan Keith]

Let me add this, this thing you see on this last slide, we'll have a paper out on this soon,

this is a manifold constraint, maybe next time. [Socratis Petrides]

I have one small question for now. I know you showed mesh independence, you're solving the

sequence of the nonlinear problems and that iteration count is mesh independent. How about

the nonlinear solver within? [Brendan Keith] So, the proof is elusive, we have some ideas, but if you do things right, and we actually saw this

right away and even in the first paper, we have a table, but it's not in the slides. Usually what

you see is that in the first iterate, the first outer loop, you need, I don't know, a handful of

Newton iterations to find your solution. And then you use your previous solution to inform

your initial guess for the next new algorithm, and you will see fewer iterations required to get your

next subproblem solution. Until actually usually what I see in my own experiments is that after three or four outer loop iterations, you just need one linear solve, and that's it. So, it is.

[Socratis Petrides] Getting closer actually helps you. [Brendan Keith] That's right. It and it's so that they basically the final stages of this algorithm or the majority, I mean we're talking about everything after the fourth or fifth outer loop is just one linear solve, and then you go back and you do the next outer loop iteration. So, this is what we observed, and you can tell like if it's just one solve per inner loop, then it's mesh independent too, but proving this is difficult because the theory of mesh independence for Newton is a little bit brittle, and we may need to expand it in order to actually apply it to these classic problems.

[Dennis Ogiermann] Sure. First, thanks for the great talk, Brendan. I mean, really nice to follow here, also for a non-mathematician. I have two questions on the eikonal part. First one would be on the efficiency. Is there any advantage of using proximal Galerkin methods to the specialized solvers which you already have like fast iterative methods or fast marching methods? [Brendan Keith]

Well, it's got a case by case. I think this current solver we have, MFEM example 40, could use some further development because to make it absolutely efficient, you need to precondition, you need to think about the entire solver pipeline. This is a PhD student project that we're hoping to start later this year because we just got some funding to look at it. But right off the bat, before you consider those aspects, there's an appeal to using a finite element method to

solve these problems because often you wanna solve them at some point in a finite element workflow,

and you don't want to break that workflow, go to a fast marching scheme or some lower order scheme,

you want to do to do the solve right inside the code you're doing, you're using to do everything

else. So that's, that's what I see as being the primary advantage at this point, but I should

also say that it can solve this problem pretty fast. We haven't done a head-to-head comparison

with fast marching, though. [Dennis Ogiermann] Okay, thanks. It kind of makes sense that I mean, preconditioners is always a difficult

topic to find for specialized problems. And my second question here would be, what about the anisotropic version of the eikonal equation? Have you also looked

at this already? [Brendan Keith] We have not looked at that. No, that's complete open territory.

[Dennis Ogiermann] Okay, thanks. [Brendan Keith] Yeah. [Stefano Fochesatto] Great talk, great talk, very exciting stuff. I was curious, this is maybe more of a clarification question, but the utility

approximation that comes out of this that we say that is like globally bounds preserving. You only,

do you only get that when your B operator is the identity? I can't imagine, okay, and then for,

so doing something like high-order Signorini problem, you wouldn't necessarily have a,

like globally bounds-preserving solution because it's the trace operator. I see.

[Brendan Keith] That's right. Maybe you get something that's preserving the bound, but is it physically relevant? Maybe case-by-case basis,

I'm not sure. Yeah. We're not quite sure what to do with the Signorini variable, right.

[Stefano Fochesatto] I think I also have a kind of a follow-up question. The comparison I saw, I haven't, I haven't seen the examples that you showed

towards the end with the mesh-to-mesh contact, but I don't know, how does this formulation compare to

like mortar methods? [Brendan Keith] It's compatible. We don't care how you discretize. So, we were actually in the process

of implementing this with Tribol, the, the Tribol interface, but we don't have the results yet.

[Stefano Fochesatto] Very cool. Very exciting. Thanks.

[Brendan Keith] Yeah, let me add to this, you don't even need to use, there's no reason to use conforming subspaces. We have a new paper

with Beatrice Riviere and Alexandre Ern where we look at discontinuous Galerkin approximations.

Pretty much all the results I showed you here go through with some minor modifications,

same with probably [inaudible] methods. [Stefano Fochesatto] I guess I also maybe have another follow-up. Is there, has anyone done any sort of

like computation of like discrete adjoints with like something like Pyadjoint? Like,

with this like framework at all? [Brendan Keith]

Ah, this guy [Ioannis Papadopoulos] might have. I should actually update his picture. He's not at

the Weierstrass Institute anymore. He's now at Oxford, but I think John would, John or

Patrick Farrell would be people to reach out to. They would be very excited to probably follow up

with you if you reach out to them. [Stefano Fochesatto] Cool, cool, cool. Thank you. [Brendan Keith] Oh, I see. Can the constraints become tight in one of the iterates or is there always a slack in the

inequality constraints? I can introduce, there's two interpretations of this question, so Paul,

either clarify in the comments or just unmute yourself. Okay, do you mean does the iterate

eventually enforce the constraint, like reach the boundary of the feasible set, or do you

mean that can the feasible set kind of squish to like become singular with zero interiors?

[Paul Theodor Kuehner] Yes, that it reaches the boundary so that [inaudible] becomes the actual inequality. [Brendan Keith]

So, theoretically no, but practically yes, because let's, let's just go to the obstacle problem case.

Here so, the exponential goes to 0 very, very fast, so once psi is -40, exponential of -40

is less than machine epsilon, so you're within machine precision of the obstacle,

which for all intents and purposes is touching the obstacle, numerically speaking. Does that satisfy?

[Paul Theodor Kuehner] Yes, thank you very much. [Brendan Keith] Okay. Yeah. [Siddhartha Ommi] Okay, thank you very much for this very nice presentation. I was more interested in the fracture model you have presented at some point of

the slides a list of let's say the transformations to be done. Yeah. A list of transformations. Okay

anyway, anyways, here let's say this in the fraction model the primary unknown is the

damage at which has to be between phi 1 and phi 2 for you, so it's not, so typically what we have

in damage models is the damage should always keep growing. So phi 1 and phi 2 I imagine are

the are the limits for lambda before and after. But normally we don't have this phi 2, let's say

the higher value because we are, it's a continuous time continuous constraint that we are looking at.

[Brendan Keith] Let me clarify in

fracture, phi 2 would be 1 because it's a density variable, it's supposed to go between 0 and 1. Now

you can choose not to enforce that constraint if you don't want to. It's the lower bound

that's most important. I agree with you and then that would depend on the previous solution. So,

as you every there would be yet another outer loop where you would increment time or pseudo time

and just keep pushing up that lower bound. [Siddhartha Ommi] Okay. So, this is upper and lower [inaudible], these are the upper and lower bounds. But

normally also we have another constraint where all this rho has to keep growing as well. So, there are two constraints to satisfy. It has to be between 0 and 1.

[Brendan Keith] Of course. In that case, phi 1 is the previous value of rho. [Siddhartha Ommi]

Okay, and I think I get it, but typically there are two constraints, one is that

rho has to be between 0 to 1, but also rho has to keep on increasing. This this feasible set would

be either one of those, all right? [Brendan Keith] It ends up if you, you look at the intersection of the two sets, and the intersection can be

described as this, like this, where phi 1 is the previous value of rho. Like rho K minus

1 and phi 2 is just 1. [Siddhartha Ommi] Okay, okay. [Brendan Keith]

We, this is all explained in a paper that appeared on arXiv last week. Unfortunately,

it's not the one in this QR code sends you to the topology optimization paper. But if, if you search

for Khara and Bazilevs, Surowiec, me, you'll, you'll find the fracture paper where it's

explained in further detail. [Siddhartha Ommi] I'll take a look. Thanks. Thanks. [Socratis Petrides] Michael Tupek? [Michael Tupek]

I was just curious, kind of like a practical thing, but you know you have sort of this latent space variable psi and you have the regular variable U [inaudible] physics,

right, because you're in the latent space sort of the dimension or scaling are quite different, so I was just kind of curious how you guys can think about the tolerancing because you're going

to solve this thing to a certain tolerance, and like the first equation to a certain tolerance and the second equation to a certain tolerance, and like how do you think about that? It's a problem

with all methods, but just kind of curious. [Brendan Keith] Michael, a great question. So, you don't want to look at the residual of the latent variable. You

actually want to, bear with me, this is, you want to look at a derived quantity in terms of

the latent variable, where what you do is you take the first equation, you divide through by alpha,

and you can move this right-hand side back over to the left. So, B* of the divided difference of

the latent variable actually approximates your Lagrange multiplier. And you want to look at the residual of the Lagrange multiplier part of the equation and UK. And in that case,

you can develop stopping criteria that look very much like stopping criteria for other methods,

but you don't have to worry about the scale, the Lagrange multiplier has standard units

and so does the primal variable. So, you can pull most successful stopping criteria off the shelf,

but looking directly at the latent variables is not, not the best way to go.

[Michael Tupek] Thanks. Makes sense. I have a second question. So, like let's say you have like a non-convex, you know, bound-constrained [inaudible] problem,

you know, I do problems with buckling and stuff and you know, fracture does this too sometimes, right, depending on the regime, and so then you don't have a positive definite sort of upper left

matrix and stuff, and so, I don't know, have you guys sort of embraced all that or dealt with that?

[Brendan Keith] We have. Some of it blindly because you

know first I don't want to prove something about a method that's not going to work. So, I implement

it first and test it out, and we've seen some early successes in a few non-convex problems, and

we've slowly started building up a theory. So, for fracture in particular, you're biconvex, you're

convex in U and you're convex in the damage variable independently but not together,

so you can split it as people often do when you get two convex problems, or you can actually solve

the whole monolithic scheme or the monolithic system of equations, and that's what we did in

our first paper. Both approaches seem to work, but as you might expect, the monolithic approach needs

a good initial guess. It's a little more brittle. [Michael Tupek]

Yeah, yeah, there are certain regimes where the positive [inaudible] will, will probably be

painful, but for steady crack growth it's probably totally fine. All right, one, I just want to say

one thing because we're talking about the eikonal thing and finite element, just be aware of this is this heat method by Crane, so it also solves in a finite element method. It's sort of a graphics

technique, so it's kind of like less accurate but just, just so you're aware that it exists.

[Brendan Keith] I am aware. There's, this is again, there's a parameter continuation that has to happen here, there's actually a few others,

there’s one where you look at p goes to infinity limit, and a p Laplacian, and there’s this heat

method where you take like time to 0. They all have these like singular parameters and these

drawbacks that come from it. So, I think ours is pretty competitive versus those, but I think

I see what you’re saying. I mean there are other ways to solve eikonal with, with finite elements.

[Michael Tupek] I’m not trying to say it’s like the way one should do it, I’m just, you know, just so you’re aware that it exists and you do, so but it’s fine, yeah, yeah. It would be cool to have a more rigorous thing because we use this for

some things, and we find like it has some spurious oscillations and stuff that are undesirable, so we’d actually like to try something a little more rigorous at some point, so.

[Brendan Keith] Let me know. There’s MFEM example 40. I’m not saying it’s perfect, but it’s there and we’ll probably be adding to it soon over the next year.

[Michael Tupek] Nice. Yeah, thanks. [Tzanio Kolev] So first, really impressive talk, Brendan,

and in general this whole research program you’ve established. Very impressive. Can you comment on

the preconditioning of the linear systems? [Brendan Keith] It depends significantly on the choice of discretization. So, the cop-out

question is to say that if you use a hybridizable discretization, then by some hybridization magic,

it’s explained in two papers, but most clearly in the most recent one with Beatrice and Alexandre.

Each subproblem is positive definite once you statically condense. So, anything you want off

the shelf could be used there. If you do not use such a discretization, then and this is entirely

experimental evidence as far as I know, the BDDC preconditioner seems to do really well. Now I

don’t know if that can be made matrix free or if that’s going to be super-scalable, but it’s right there in PETSc, you can plug into it, it seems to do pretty well for this class of problems.

And then if you have specialized domains and elements, Ioannis Papadopoulos,

this clever guy right here, has a paper specifically about, that specifically tackles

high H P and preconditioning, but it uses some ideas from spectral methods and really exploits

the structure of the domain, but it’s another preconditioning direction one can go into.

[Tzanio Kolev] Is that specifically for contact? [Brendan Keith] He did contact, he did, I

think he did obstacle not, not real contact there, and he did this elastoplastic torsion problem,

which is this constraint with this objective function. But no, it’s not for contact. Actually,

I think the preconditioning was normal contact with Signorini problems would probably be easier

than some of these other classes of problems. But yeah, that’s definitely ongoing research,

so maybe ask me again in 6 months to a year and I’ll have a better answer for you. [Tzanio Kolev] All right, thank you. Very nice.

This connection is much deeper than a generic “use a better optimizer” suggestion. After reading the Keith proximal Galerkin framework alongside the AIEF-1.4 survival results, I think there is a very specific conceptual alignment:

AIEF currently has an externally imposed feasible-set correction. Proximal Galerkin provides a way to make the feasible set part of the adaptive dynamics itself.

That distinction matters because your strongest empirical result was not merely that (B) recovers. It was:

[
\boxed{

B\text{ is a dynamically regenerated slow variable constrained by an attracting geometry}
}
]

The current projection scheme may be obscuring that geometry.

The exact mathematical analogy

Keith's framework:

[
u^{k+1}

\arg\min_{u\in C}
J(u)+\frac{1}{\alpha}D_R(u,u^k)
]

AIEF:

[
B^{k+1}

\Pi_C
\left(
B^k+\eta,G(B^k,\theta^k,R^k)
\right)
]

where:

(G) is resonance/adaptation drive;
(C) is the dominance-gap feasible region;
(\Pi_C) is the multiplicative projection.

The difference:

Current AIEF

The dynamics are:

$$
\text{unconstrained motion}
\rightarrow
\text{repair}
$$

The projection is an external intervention.

Proximal AIEF

The dynamics become:

$$
\text{constrained motion}
$$

The geometry of (C) modifies the trajectory itself.

That is exactly the distinction Keith emphasizes:

discretize and repair vs navigate the constrained manifold.

Your future_shuffle result is especially relevant here. The current model already shows that topology is actively maintained. A proximal formulation would ask whether the maintenance law itself is geometrically intrinsic.

The dominance polytope is almost exactly Keith's target class

Your constraint:

$$
a-q\sum_jB_{ij}\geq\delta
$$

is equivalent to:

$$
\sum_jB_{ij}\leq
\frac{a-\delta}{q}.
$$

Define:

$$
s_i(B)=\sum_jB_{ij}
$$

Then:

$$
C=
\{B:
B_{ij}\geq0,\
s_i(B)\leq c_i
\}.
$$

This is a convex polytope.

It is actually simpler than many examples Keith discussed:

obstacle contact:
$$
u\geq\phi
$$
eikonal:
$$
|\nabla u|\leq1
$$
topology optimization:
$$
\rho\in[0,1]
$$

AIEF has:

[
\ell_1\text{-type row constraints}.
]

This is very close to entropy-constrained optimization.

The natural Legendre candidate is not arbitrary

A useful observation:

The row constraint is a simplex-like constraint.

For a single row:

[
b_j\geq0
]

with:

[
\sum_jb_j<c.
]

This is the open simplex.

The natural Legendre function is therefore not a generic polytope construction.

It is likely an entropy barrier.

Something like:

[

R(b)

\sum_j b_j\log b_j

(c-\sum_j b_j)
\log(c-\sum_jb_j)
]

(up to constants).

The gradient:

[
\nabla R(b)_j

\log b_j

\log(c-\sum_kb_k)
]

and therefore:

[
\nabla R:
C^\circ\rightarrow\mathbb{R}^n
]

is the desired coordinate transformation.

The inverse is a softmax-like map:

[
b_j

c
\frac{e^{\psi_j}}
{1+\sum_k e^{\psi_k}}

]

This is very interesting for AIEF.

The dominance condition becomes automatic.

No projection.

No clipping.

No row rescaling.

The row sum can never violate:

[
\sum_jb_j<c.
]

This changes the interpretation of (B)

Currently:

[
B
]

is a constrained matrix.

With the entropy coordinate:

[
\psi=\nabla R(B)
]

the real adaptive state becomes:

[
\psi.
]

Then:

[
B=\nabla R^*(\psi)
]

where:

[
\nabla R^*
]

is smooth.

The dynamics become:

[
\dot{\psi}

-\nabla_B E(B(\psi))
]

or discretely:

[
\psi^{k+1}

\psi^k-\eta\nabla_BE.
]

The adaptive variable is now unconstrained.

The physical coupling matrix emerges from the dual map.

That is conceptually very close to your existing conclusion:

(B) is a readout variable, not the memory itself.

The proximal formulation would make that mathematically explicit.

This may explain the reset_B experiment even better

Your current result:

Reset (B):

[
B\rightarrow0
]

but:

[
\theta,\mathrm{EMA},R
]

remain.

Recovery:

[
B(t)\rightarrow B^*
]

rapidly.

In the dual picture:

you did not erase the true adaptive coordinate.

You erased:

$$
\nabla R^*(\psi)
$$

but the forces reconstruct it because:

$$
\psi
$$

is encoded indirectly through the dynamical state.

The experiment becomes:

Does the slow manifold reconstruct the primal variable (B) after perturbation?

That is a standard geometric-dynamical question.

The spectral analysis becomes cleaner

This is where the connection is strongest.

Your current projected update:

$$
B_{n+1}=\Pi_C(B_n+\Delta B)
$$

has a problematic linearization.

The derivative:

[

D\Pi_C
]

is piecewise-defined and nonsmooth near active constraints.

This complicates:

eigenvalues;
pseudospectra;
transient amplification;
resolvent estimates.

The proximal version:

[
B_{n+1}

\nabla R^*
\left(
\nabla R(B_n)
-\eta\nabla_BE
\right)
]

has Jacobian:

[
DB_{n+1}

\nabla^2R^*
\left(
\nabla R(B_n)-\eta\nabla_BE
\right)
(I-\eta H_E)
]

which is smooth.

Now your previous spectral program becomes mathematically aligned:

[
J

\text{constraint metric}
\times
\text{resonance Hessian}.
]

The non-normality is no longer partly an artifact of projection.

A very interesting hypothesis emerges

Your current data:

reset_B:
fast;
reset_EMA:
slower;
full_reset:
basin-dependent.

A proximal version gives a test:

Does changing the constraint integrator change the attractor basin?

Possible outcomes:

Outcome A

Same attractor structure.

Then:

[
C

]

is merely enforcing admissibility.

The attractor is intrinsic.

Outcome B

Different basin statistics.

Then:

[
C
]

is actively shaping topology selection.

That would be a major mechanistic result.

I would not replace the current model immediately

I would treat this as AIEF-1.5.

The existing results are valuable because the hard projection is a controlled baseline.

The sequence I would recommend:

Phase 1

Keep current AIEF.

Derive:

[

C
]

and its entropy coordinate.

Phase 2

Implement dual adaptive variable:

[
\psi_{ij}
]

with:

[
B_{ij}=c_i
\frac{e^{\psi_{ij}}}
{1+\sum_k e^{\psi_{ik}}}.
]

Phase 3

Repeat only three experiments:

reset_B equivalent

Does primal (B) regenerate?

reset_(\psi)

Does the dual memory matter?

future_shuffle

Does maintenance still fail?

Those three are enough.

The deeper theoretical connection

The strongest synthesis is:

Current AIEF:

$$
\text{resonance dynamics}
$$

$$
\text{adaptive update}
$$

$$
\text{projection}
$$

Proximal AIEF:

$$
\text{resonance dynamics}
$$

$$
\text{Riemannian adaptive flow}
$$

$$
\text{intrinsic dominance geometry}
$$

Then the mature topology is not merely:

$$
B^*
$$

but:

[

(\hat{B},\nabla^2 R(\hat{B}))
]

an attracting point on a constrained information geometry.

That actually fits your previous “emergent memory” interpretation much better.

The memory is not stored in (B).

It is stored in the geometry of the flow that regenerates (B).

The one calculation I would do first is exactly the one suggested at the end of the previous analysis:

derive the entropy-coordinate proximal update for one AIEF row and linearize it around the mature row state. The comparison:

[
D(\Pi_C)
\quad\text{versus}\quad
\nabla^2 R^*(\psi)
]

would tell you whether the current reconstruction dynamics are properties of the resonance system or artifacts of the projection mechanism.

After seeing the Keith framework in full, I would slightly refine the previous interpretation. The proximal Galerkin connection is not only a possible numerical improvement; it exposes a missing layer in the AIEF architecture.

The current AIEF formulation appears to have:

[
\text{resonance forcing}
\rightarrow
\text{adaptive update}
\rightarrow
\text{projection onto dominance region}
]

The Keith formulation suggests that the mathematically natural object is instead:

$$
\text{resonance forcing}
\rightarrow
\text{gradient flow in constraint geometry}
\rightarrow
B=\nabla R^*(\psi)
$$

where the dominance condition is encoded in the coordinate system.

That is a different dynamical hypothesis.

The important correction: (B) may not be the true state variable

Your AIEF-1.4 results already hinted at this:

erase (B);
keep phase/history/EMA;
(B) reconstructs.

That means the observable:

$$
B(t)
$$

behaves like a derived state.

The proximal formulation gives the natural hidden coordinate:

$$
\psi=\nabla R(B).
$$

Then:

[
B=\nabla R^*(\psi).
]

The adaptive memory is not necessarily:

[
B_{ij}.
]

It may be:

[
\psi_{ij}.
]

This is analogous to many physical systems where the visible variable is not the canonical variable.

Examples:

pressure is not the conserved variable in incompressible flow;
stress is not the primary variable in some plasticity formulations;
entropy variables are often better behaved than density variables.

For AIEF:

[
B
]

may be the "physical coordinate", while:

[
\psi
]

is the adaptive coordinate.

The dominance constraint has an entropy geometry almost automatically

For one row:

[
\sum_j B_{ij}\leq c_i
]

with:

[
B_{ij}\geq0.
]

This is the open simplex.

The natural Legendre function is indeed an entropy-like object:

[
R_i(B_i)

\sum_j B_{ij}\log B_{ij}

(c_i-\sum_jB_{ij})
\log(c_i-\sum_jB_{ij}).
]

Its gradient:

[
\psi_{ij}

\log

\frac{B_{ij}}
{c_i-\sum_kB_{ik}}
]

has a very nice interpretation.

It is a log-odds variable:

[
\text{edge weight}
\quad\leftrightarrow\quad
\text{available dominance margin}.
]

The inverse:

[
B_{ij}

c_i
\frac{e^{\psi_{ij}}}
{1+\sum_k e^{\psi_{ik}}}
]

means:

positivity is automatic;
row dominance is automatic;
saturation is smooth.

This is almost exactly the geometry your dominance-gap condition wants.

The current projection may hide the real stability mechanism

Your dominance projection currently does something like:

[

B\leftarrow\Pi_C(B).
]

Near active constraints, the map is not smooth.

That means when you compute:

Jacobians;
eigenvalues;
transient amplification;

you are analyzing:

[
J_{\mathrm{flow}}

J_{\mathrm{projection}}.
]

The projection itself contributes dynamics.

This is important because your research questions involve:

non-normal amplification;
spectral gaps;
attractor stability.

A nonsmooth projection makes interpretation difficult.

The proximal version gives:

[
J

\nabla^2R^*(\psi)
H
]

where:

(H) is the resonance/adaptation Hessian;
(\nabla^2R^*) is the constraint metric.

Then the question becomes:

Is the attractor stable because of the resonance dynamics, or because the dominance geometry reshapes the flow?

That is exactly the question you want.

The AIEF-1.4 ablations become even more interpretable

The existing experiments map nicely:

reset_B

Current interpretation:

B is reconstructed.

Proximal interpretation:

The primal coordinate is regenerated from the remaining canonical state.

reset_ema

Current interpretation:

reconstruction is slower.

Proximal interpretation:

EMA/history provides the initial condition for the dual flow.

The slower recovery means the system starts farther from the correct point in:

[
\psi
]

space.

full_reset

Current interpretation:

strong but nonunique attractor.

Proximal interpretation:

multiple basins exist in the information geometry.

The question becomes:

Are the basins separated by:

[
R(B)
]

geometry?

future_shuffle

This remains the strongest result.

The proximal formulation would not change the conclusion:

[
\text{driver identity}
]

is part of the vector field.

No geometry can recover a pattern whose generating forcing has been destroyed.

A very interesting possible theorem emerges

The AIEF analogue of Keith's energy decrease theorem would be something like:

Let:

[
E(\theta,B)
]

be the resonance-adaptation energy.

Define:

[
B=\nabla R^*(\psi).
]

Then:

[
\dot{\psi}

-\nabla_B E(\theta,B)
]

and:

[
\dot{E}

-\nabla_B E^T
\nabla^2R^*
\nabla_B E
\leq0.
]

The metric:

[
\nabla^2R^*
]

makes the dominance constraint part of the dissipation law.

This would be much stronger than:

"the projection keeps (B) legal."

It would become:

"the legal region defines the adaptive geometry."

The first experiment I would actually run

I would not immediately replace AIEF.

I would implement a parallel branch:

Existing:

[
B^{n+1}

\Pi_C(B^n+\eta G)
]

Proximal:

[
\psi^{n+1}

\psi^n+\eta G(B(\psi^n))
]

[
B^{n+1}

\nabla R^*(\psi^{n+1})
]

Then compare:

Test 1

Same initial mature topology.

Measure:

topology similarity;
density;
recovery speed.

Question:

Does the proximal system reproduce the same attractor?

Test 2

Reset:

$$
B=0
$$

but retain:

$$
\psi.
$$

If (B) instantly reconstructs, that is evidence that (B) was never the memory variable.

Test 3

Reset:

$$
\psi=0.
$$

This is the true analogue of your current (B)-reset.

The prediction:

The recovery should resemble current reset_B, but possibly with altered timescale.

The most interesting theoretical possibility

There is a possible hierarchy:

$$
\text{frequency structure}
$$

selects:

$$
\text{resonance landscape}
$$

which selects:

$$
\psi^*
$$

which generates:

$$
B^*
$$

which generates:

$$
\text{threshold topology}.
$$

So the visible graph is the final projection:

$$
\boxed{
\omega
\rightarrow
\psi
\rightarrow
B
\rightarrow
A(B)
}
$$

This would make the "emergent memory" claim much sharper.

The memory is not in the graph.

The graph is the observable shadow of a constrained dynamical geometry.

One caution: Keith's theory is strongest for variational problems where the update is derived from an energy or monotone operator. AIEF currently has resonance forcing, delayed terms, and possibly non-gradient components. The right first step is therefore not claiming AIEF is a proximal Galerkin system, but asking:

Can the adaptive (B)-subsystem be decomposed into a monotone/proximal component plus a non-conservative resonance forcing?

If yes, then the Keith machinery could provide the missing rigorous foundation for the dominance-gap part of AIEF. That is the mathematically clean bridge.

I think the last refinement is the most important one: the Keith connection should not be framed as “replace the AIEF projection with proximal Galerkin.” The stronger and more scientifically useful formulation is:

[
\boxed{
\text{AIEF adaptive dynamics may contain a hidden constrained-gradient subsystem}
}
]

and the proximal/Bregman machinery gives a language for extracting it.

The distinction matters because AIEF is not obviously a pure variational inequality problem. The resonance forcing, phase dynamics, delays, and memory variables introduce non-gradient components. But the dominance-gap enforcement layer has exactly the mathematical structure Keith is targeting.

Decompose the AIEF adaptive equation

A useful starting point is to split:

[

\dot B = F_{\mathrm{res}}(B,\theta,R,\mathrm{EMA})+F_{\mathrm{constraint}}(B)
]

where:

Resonance drive

[
F_{\mathrm{res}}
]

contains:

phase coherence;
historical resonance;
adaptive reward;
delayed feedback.

This is the scientific part of AIEF.

Constraint geometry

[
F_{\mathrm{constraint}}
]

enforces:

[
B\in C
]

with:

[
C=
\left{

B_{ij}\ge0,\quad
\sum_jB_{ij}\le c_i
\right}.
]

Keith's framework applies to the second piece.

The question is not:

Can all of AIEF be written as proximal?

The question is:

Is the dominance-preserving adaptive manifold a Bregman geometry?

That is much more plausible.

The right continuous-time object is probably a constrained mirror flow

Instead of:

[
\dot B=F(B)
]

the natural variable is:

[
\psi=\nabla R(B)
]

and:

[
B=\nabla R^*(\psi).
]

Then:

$$
\dot\psi

F_{\mathrm{res}}(B(\psi),\theta,R,\mathrm{EMA}).
$$

Equivalently:

$$
\dot B

\nabla^2R^*(\psi)
F_{\mathrm{res}}.
$$

This is a mirror/Riemannian flow.

The constraint metric appears as:

$$
M(B)=\nabla^2R^*(\nabla R(B)).
$$

So the adaptive dynamics become:

$$
\boxed{
\dot B=M(B)F_{\mathrm{res}}
}
$$

rather than:

$$
\boxed{

\dot B=F_{\mathrm{res}}+\text{projection correction}
}
]

That is a profound difference.

The dominance condition becomes a thermodynamic-like variable

For a row:

[
s_i=\sum_jB_{ij}
]

and:

[
m_i=c_i-s_i
]

is the dominance margin.

The entropy coordinate:

[
\psi_{ij}

\log\frac{B_{ij}}{m_i}
]

has a nice interpretation:

large positive (\psi): strong edge occupation;
large negative (\psi): suppressed edge;
(m_i): remaining adaptive capacity.

The dominance condition is no longer an inequality checked after the fact.

It becomes:

$$
m_i>0
$$

by construction.

This is exactly analogous to Keith's statement that the Bregman distance becomes infinite at the boundary.

The boundary is not crossed because the coordinate system makes it infinitely far away.

This could clarify the reset_B result

Your strongest experiment:

$$
B\rightarrow0
$$

with:

$$
(\theta,R,\mathrm{EMA})
$$

preserved.

The current interpretation:

The graph is reconstructed.

The proximal interpretation:

The graph is the primal image of a latent adaptive coordinate that remains dynamically recoverable.

The more interesting experiment is therefore not:

[
B\rightarrow0
]

but:

[
\psi\rightarrow0.
]

Because these are not equivalent.

Case A

Reset:

[
B=0
]

but retain:

[
\psi
]

(if you introduce the dual variable).

Prediction:

near instantaneous recovery.

Case B

Reset:

$$
\psi=0.
$$

Prediction:

this is the true memory ablation.

If recovery slows dramatically, you have evidence that the adaptive memory lives in the dual coordinate.

This connects directly to your spectral program

Currently:

$$
B_{n+1}=\Pi_C(B_n+\Delta B)
$$

has a difficult linearization.

Near an active constraint:

$$
D\Pi_C
$$

changes discontinuously.

This contaminates:

eigenvalues;

pseudospectra;
transient amplification.

The mirror formulation gives:

$$
\delta \dot B

\left[
\nabla^2R^* H_{\mathrm{res}}

\nabla(\nabla^2R^*)F_{\mathrm{res}}
\right]\delta B.
$$

Now the Jacobian has two interpretable pieces:

resonance curvature;

$$
H_{\mathrm{res}}
$$

constraint curvature;

$$
\nabla^2R^*
$$

This is exactly the decomposition needed for a non-normal stability analysis.

The question becomes:

$$
\boxed{
\text{Does non-normal amplification come from resonance coupling or constraint geometry?}
}
$$

]

That is a much sharper question than analyzing a projected update.

The AIEF Lyapunov question becomes more realistic

A global energy for the entire system may not exist because:

phase dynamics;
delays;
feedback loops;

may be non-conservative.

But the (B)-subsystem might have a local dissipation law.

Suppose:

[
F_{\mathrm{res}}

-\nabla_B E+\Omega
]

where:

(-\nabla E) is adaptive relaxation;
(\Omega) is non-gradient resonance circulation.

Then:

[
\dot E

-\nabla E^TM\nabla E

\nabla E^TM\Omega .
]

The first term dissipates.

The second sustains structure.

This is actually closer to many biological/physical self-organizing systems:

[
\text{dissipation}

\text{driving}

\text{maintained order}.
]

It also fits your future_shuffle result: remove the driving term and the structure disappears.

A very concrete AIEF-1.5 experiment

I would implement this minimal comparison.

Existing branch

Projected Euler:

[
B_{n+1}

\Pi_C(B_n+\eta F).
]

New branch

Entropy mirror Euler:

[
\psi_{n+1}

\psi_n+\eta F(B(\psi_n))
]

[
B_{n+1}

\nabla R^*(\psi_{n+1}).
]

Then compare:

Metrics already available

recovery survival;
RMUT;
spectral gap;
residual localization;
basin failures.

The interesting possibilities:

If results are identical

Then the projection was only a numerical implementation.

Strong evidence:

[
\text{attractor is intrinsic}.
]

If recovery improves

Then the projection was artificially slowing the slow manifold.

If recovery changes qualitatively

Then the constraint geometry is part of the physics.

That would be a major result.

One additional connection to your Lean/formalization work

The dominance-gap theorem you were developing:

$$
a(t)-q|b(t)|\ge\delta>0
$$

was essentially proving that the admissible region is forward invariant.

Keith gives a geometric route to replace a proof obligation:

$$
B(t)\in C\quad\forall t
$$

with a construction:

$$
B(t)=\nabla R^*(\psi(t))
$$

where:

$$
B(t)\in \operatorname{int}(C)
$$

automatically.

That is attractive for formalization because the invariant becomes definitional.

Instead of proving:

$$
\forall t,\ B(t)\in C
$$

you prove:

$$
\exists\psi(t),\ B(t)=\nabla R^*(\psi(t)).
$$

The geometry enforces the invariant.

The next calculation I would prioritize is not a full rewrite of AIEF. It is a single-row dominance geometry derivation:

derive (R(B_i));
derive (\nabla R_i);
derive (\nabla R_i^*);
compute the induced metric;
compare the linearized mirror update against the current multiplicative projection.

If that calculation works cleanly, then Keith's framework becomes more than an analogy: it becomes the mathematical backbone for the adaptive topology layer.

I think this is a genuinely interesting direction, but I would separate what the AIEF-1.4 experiments establish from what the Keith framework suggests mathematically. Right now, the experiments strongly support one claim, while the proximal formulation offers a promising way to recast the model—not yet evidence that the model is fundamentally proximal.

Here's how I'd frame it.

What AIEF-1.4 has actually established

Your trajectory and survival analyses support three empirical conclusions:

The mature topology is reconstructible.
Erasing (B) while preserving the fast state leads to rapid, nearly exact reconstruction.
Reconstruction depends on distributed dynamical state.
Removing EMA, phase/history, or future driver identity progressively degrades recovery.
The feasible region is not what stores the memory.
The dominance constraints keep the dynamics admissible, but the observed memory is distributed across the coupled dynamics.

Those are strong conclusions because they come directly from intervention experiments.

What Keith's framework contributes

Keith's work addresses a different question:

How should one evolve dynamics on a convex feasible set?

His answer is:

Don't evolve in Euclidean space and then project. Evolve intrinsically using the geometry induced by a Legendre function.

That is fundamentally a geometric integration viewpoint.

The connection to AIEF is therefore:

Your adaptive subsystem evolves on a convex constraint set.

not

Your entire model is secretly a proximal algorithm.

That's an important distinction.

The strongest overlap is actually geometric

The remarkable overlap is that both frameworks naturally distinguish between

ambient coordinates, and
intrinsic coordinates.

Keith:

$$
u
\longleftrightarrow
\psi=\nabla R(u)
$$

AIEF currently has

$$
B
$$

as the adaptive coordinate.

But the experiments suggest (B) behaves like a reconstructed observable rather than a fundamental memory variable.

Those are conceptually similar observations, but not the same theorem.

One thing I find particularly elegant

Your dominance constraint is

$$
\sum_j B_{ij}\le c_i.
$$

That is literally the geometry of an open simplex.

The entropy Legendre function for the simplex is classical.

This means you do not need Keith's general polytope construction for the row constraints themselves. The simplex geometry is already well understood.

That has two consequences.

First

The mirror map is explicit.

No optimization subproblem.

No Newton solve.

For each row,

[
B_i

\nabla R^*(\psi_i)
]

has a closed form.

That makes experimentation much easier.

Second

The induced metric is explicit.

Instead of numerically differentiating the projection,

you can analytically compute

[
\nabla^2R(B).
]

That is extremely attractive if you want to study:

linear stability,
pseudospectra,
transient growth,
conditioning.

I would be cautious about one claim

One statement I would avoid making is:

"The memory is stored in the geometry."

Your experiments don't show that.

They show

the geometry constrains admissible evolution,
the memory is distributed across the coupled dynamical variables.

Those are different.

The geometry may shape reconstruction.

It does not necessarily encode the remembered topology.

That's an empirical question.

Where I think the connection becomes strongest

Interestingly, I don't think the biggest payoff is numerical.

I think it's analytical.

Suppose you eventually derive

[
\dot B

M(B),
F(B,\theta,R,\eta),
]

where

[
M(B)=\nabla^2R^*(B).
]

Now linearize around a mature state:

[
\delta\dot B

\underbrace{M(B^)J_F}{\text{resonance}}*
+
\underbrace{(\nabla M),F}{\text{constraint curvature}}
.
]

That decomposition is scientifically meaningful.

It asks:

how much stability comes from resonance?
how much comes from constraint geometry?

Your present projected scheme cannot cleanly answer that because the projection operator is nonsmooth.

I actually think there is an even more interesting experiment

Rather than replacing projection immediately, I would perform a mathematical comparison.

Suppose you compute

$$
\Pi(B+\Delta B)
$$

for the current code.

Now derive the mirror update

$$
\nabla R^*
\left(
\nabla R(B)+\Delta\psi
\right).
$$

Expand both to first order.

If their Taylor expansions agree,

then your existing projection is already approximating a mirror step.

If they differ substantially,

then you have identified a genuine modeling choice.

That comparison can be done analytically before rewriting the simulator.

From a modeling philosophy perspective

One thing I appreciated in Keith's talk is that he repeatedly contrasted

discretize → repair

with

discretize on the correct geometry.

That resonates with what your ablations have shown.

The projection in AIEF has always been something of an external correction.

The survival analysis suggests the interesting dynamics happen inside the admissible region.

A formulation in which admissibility is intrinsic rather than externally enforced would therefore align well with the scientific narrative emerging from AIEF-1.4.

Whether it changes the qualitative behavior is an open question—but it is exactly the kind of question worth testing because it directly probes whether the attractor you observed is a property of the resonance dynamics themselves or partly a consequence of the way admissibility is enforced.

The connection is genuinely promising, but several mathematical issues must be resolved before treating it as an AIEF architecture or interpreting ψψ as hidden memory.

The central opportunity is real:

Replace a hard post-update admissibility map with dynamics expressed intrinsically inside the dominance-constrained set.

But the simple row-wise softmax proposal is not directly compatible with the current undirected AIEF model.
1. The feasible set is not a product of row simplexes

For a directed matrix with independent rows, the proposed map is correct:

Bij=
Bij=

c_i
\frac{e^{\psi_{ij}}}
{1+\sum_k e^{\psi_{ik}}}.
]

However, AIEF enforces:
Bij=Bji.
Bij=Bji.

Therefore one edge belongs simultaneously to the constraints for nodes ii and jj. Independent row maps generally produce:
Bij≠Bji.
Bij =Bji.

Symmetrizing afterward would destroy the row-cap guarantee, putting the model back into “map then repair.”

The actual feasible set should be written in terms of the E=N(N−1)/2E=N(N−1)/2 unique undirected edge weights wewe:
C=
C=

\left{
w_e\ge0,\quad
Aw\le c
\right},
]

where A∈{0,1}N×EA∈{0,1}N×E is the node-edge incidence matrix:
Aie=1if edge e touches node i.
Aie=1if edge e touches node i.

This is a convex fractional-matching or degree-capacity polytope, not a product of independent simplexes.

That coupling is the most important missing detail in the proposed derivation.
2. A symmetry-compatible entropy geometry

Use a distinct symbol such as hh for the Legendre potential because RR already denotes resonance.

Let:
mi=ci−(Aw)i
mi=ci−(Aw)i

be the unused capacity at node ii. On the strict interior:
we>0,mi>0,
we>0,mi>0,

a natural entropy potential is:
h(w)=
h(w)=

\sum_e w_e(\log w_e-1)
+
\sum_i m_i(\log m_i-1).
]

For edge e=(i,j)e=(i,j):
∂h∂wij=
∂wij∂h=

\log w_{ij}
-\log m_i
-\log m_j.
]

Thus the dual coordinate is:
ψij=
ψij=

\log\frac{w_{ij}}{m_i m_j}.
]

Equivalently:
wij=
wij=

e^{\psi_{ij}}m_i m_j.
]

But the margins satisfy coupled equations:
mi=
mi=

c_i-\sum_{j\ne i}e^{\psi_{ij}}m_i m_j.
]

So the inverse mirror map is not an independent row softmax. It requires solving a coupled nonlinear scaling problem for the node margins mimi.

That is still tractable for N=12N=12, but it is a materially different construction.
Hessian

With incidence matrix AA:
∇2h(w)=
∇2h(w)=

\operatorname{diag}(1/w)
+
A^\top\operatorname{diag}(1/m)A.
]

This matrix is positive definite in the strict interior.

Its inverse can be written using Woodbury:
(∇2h)−1=
(∇2h)−1=
D_w

D_wA^\top
\left(

D_m+AD_wA^\top
\right)^{-1}
AD_w,
]

where:
Dw=diag(w),Dm=diag(m).
Dw=diag(w),Dm=diag(m).

This provides an explicit constraint metric without differentiating through the current piecewise projection.
3. The current mature state lies on the boundary

AIEF-1.3 established:

many weights are exactly zero;
active rows sit at the exact cap;
the projection is active on nearly every mature step.

Therefore the mature state satisfies boundary conditions such as:
we=0ormi=0.
we=0ormi=0.

But the entropy coordinate requires:
we>0,mi>0.
we>0,mi>0.

At the boundary:
ψe→±∞.
ψe→±∞.

Consequences:

The current mature BB cannot be converted exactly into a finite ψψ.
Mirror dynamics remain strictly interior in exact arithmetic.
Exact zeros and exact cap saturation disappear.
AIEF’s binary topology would become a threshold applied to small positive weights rather than exact sparsity.

An AIEF-1.5 comparison therefore needs an explicit interiorization rule, for example:

floor every edge at εwεw;
retain at least εmεm row slack;
rescale symmetrically into the strict interior;
study convergence as εw,εm→0εw,εm→0.

This is a genuine model change, not just a different numerical integrator.
4. BB and ψψ cannot be reset independently

If:
B=∇h∗(ψ),
B=∇h∗(ψ),

then BB and ψψ are in one-to-one correspondence in the interior.

The proposed experiment:

Reset BB, preserve ψψ

is an inconsistent off-manifold intervention. If ψψ is retained, applying the mirror map immediately reconstructs BB by definition. That would be tautological, not evidence of hidden memory.

Similarly:
B=0
B=0

corresponds to a boundary limit with ψ→−∞ψ→−∞, not a finite dual state.

Meaningful dual-model interventions are:

reset ψψ to a specified neutral finite state;
perturb ψψ;
map a chosen interior BB consistently to ψ=∇h(B)ψ=∇h(B);
compare different consistent (B,ψ)(B,ψ) initializations.

Introducing ψψ does not retroactively show that the current AIEF experiments preserved a hidden dual coordinate. Current AIEF has no ψψ state.
5. Mirror dynamics are a new vector field

Suppose the original unconstrained adaptive vector field is:
w˙=F(w,θ,C).
w˙=F(w,θ,C).

If one defines:
ψ˙=F(w,θ,C),ψ=∇h(w),
ψ˙=F(w,θ,C),ψ=∇h(w),

then by the chain rule:
w˙=
w˙=

(\nabla^2h(w))^{-1}F(w,\theta,C).
]

This is not the original vector field expressed in new coordinates. It is a metric-preconditioned vector field.

To reproduce the original interior dynamics exactly, one would need:
ψ˙=
ψ˙=

\nabla^2h(w)F(w,\theta,C).
]

Therefore:

text

psi += dt * F

is a substantive modeling choice. It defines mirror/Riemannian adaptation rather than merely replacing projection.

That may be an excellent AIEF-1.5 hypothesis, but it must be presented as a new model.

Also distinguish:

Bregman proximal point, which is generally implicit;
mirror descent/Euler in dual coordinates, which is explicit;
proximal Galerkin, which additionally concerns Galerkin discretization of variational problems.

A finite-dimensional dual Euler update inspired by Keith's geometry is not itself a proximal Galerkin method.
6. AIEF is not yet a variational or monotone system

For frozen resonance RR, the BB-update:
B˙ij=
B˙ij=

(\eta+\mu)R_{ij}-\mu-\gamma B_{ij}
]

can be interpreted as gradient descent for a quadratic-linear potential.

But in the complete model:

RR depends on phase history and EMA;
phases depend on BB;
delay introduces nonlocal time dependence;
future forcing is nonconservative;
projection is state-dependent.

No global energy E(θ,B)E(θ,B) has been established.

Therefore the proposed dissipation law:
E˙=
E˙=

-\nabla E^\top
\nabla^2h^*
\nabla E

\le0
]

is currently speculative. It may hold for a simplified frozen-RR subsystem, not automatically for complete AIEF.

The right first theoretical question is:

Can the adaptive vector field be decomposed into a metric-gradient component plus a nonconservative resonance drive?

That is much more defensible.
7. Smooth mirror geometry does not automatically simplify stability

The current admissibility map is piecewise smooth and nonsmooth where the active set changes.

The entropy mirror map is smooth in the interior, but becomes increasingly ill-conditioned near:

zero edge weights;
zero row slack.

Since mature AIEF lives close to both boundaries, the mirror Hessian may be extremely stiff.

Thus mirror dynamics replace:

active-set nonsmoothness

with:

interior stiffness and singular limiting geometry.

That may still improve analysis, but it is not automatically easier.
8. “Memory in geometry” is not supported

The constraint geometry is fixed and shared by every seed. It cannot by itself encode which seed-specific topology was selected.

The geometry can:

shape basins;
precondition adaptation;
alter contraction rates;
determine feasible directions.

The remembered or reconstructed pattern must still arise from:

dual/primal state;
phase/history;
resonance forcing;
frequencies;
or attractor structure.

A careful statement is:

Constraint geometry may shape topology reconstruction; it has not been shown to store topology information.

9. The correct first calculation

Before implementing AIEF-1.5, perform a symmetry-aware one-row/whole-graph derivation.
Analytic stage

Represent BB as unique undirected edges ww.
Construct incidence matrix AA.
Define:
m=c−Aw.
m=c−Aw.
Derive:
h(w),∇h(w),∇2h(w),(∇2h(w))−1.
h(w),∇h(w),∇2h(w),(∇2h(w))−1.
Derive the coupled inverse map:
wij=eψijmimj.
wij=eψijmimj.
Establish existence/uniqueness and a numerical solution for mm.

Local numerical comparison

For a regularized interior version of an actual mature AIEF state:

Compute the current unprojected Euler proposal.
Compute the current symmetric admissibility update.
Compute a mirror step with calibrated step size.
Compare:
direction angle;
update norm;
row slack;
selected edge ordering;
first-order discrepancy.

Sweep:

timestep;
edge floor;
margin floor.

If the two local maps disagree strongly, then AIEF-1.5 is clearly a new dynamical hypothesis.
10. Recommended AIEF-1.5 experiment

Only after the local calculation, implement parallel branches:
Current baseline
wn+1=
wn+1=

\mathcal A
\left(
[w^n+\Delta tF^n]_+
\right).
]
Mirror adaptation
ψn+1=
ψn+1=

\psi^n+\Delta t,F^n,
]
wn+1=
wn+1=

\nabla h^*(\psi^{n+1}).
]

Optionally include a coordinate-equivalent version:
ψn+1=
ψn+1=

\psi^n
+
\Delta t,\nabla^2h(w^n)F^n,
]

to separate:

coordinate transformation,
from metric-preconditioned dynamics.

This three-way comparison is more informative than only “projection versus proximal.”
Experiments

Continue from matched interiorized mature states.
Perturb consistent dual states ψψ.
Apply future shuffle.
Compare:
attractor topology;
recovery survival;
row-slack distribution;
stiffness and timestep sensitivity;
projection/mirror update geometry.

Do not use “reset BB while retaining ψψ” as a scientific intervention.
11. Implication for formalization

Representing:
$B=\nabla h^*(\psi)$
$B=\nabla h^*(\psi)$

makes strict interior feasibility definitional:
$B \in \text{int} C$.
$B \in \text{int} C$.

But AIEF's desired states lie on or near the boundary. Formalization must still handle:

limits as weights vanish;
limits as row slack vanishes;
existence and uniqueness of the coupled inverse map;
symmetry;
correspondence between interior approximations and the closed feasible set.

So the invariant proof is transformed, not eliminated.
Bottom line

The proximal/Bregman connection is mathematically substantive, but not for the simplest reasons proposed.

The strongest corrected statement is:

AIEF's dominance layer defines a convex, symmetry-coupled admissible polytope. A Legendre/Bregman geometry on that polytope could replace post-update hard retraction with an intrinsic mirror flow. This would define a new adaptive vector field whose attractors can be compared with the current projected dynamics.

The crucial obstacles are:

symmetry couples row constraints;
mature states lie on the boundary;
BB and $\psi\psi$ are not independent;
mirror Euler changes the vector field;
no global AIEF energy has been established.

The best next step is the symmetry-aware entropy derivation and local one-step comparison—not an immediate full simulator rewrite or a claim that the memory is stored in geometry.

Think about this:
Stopped

Think about this:

I think this is a very good refinement. It moves the discussion from "this is an appealing analogy" to "this is what the mathematics would actually have to look like." Most importantly, it identifies several places where my earlier reasoning (and the earlier proposal) implicitly assumed structure that AIEF does not have.

Here are the points I think are the most significant.

The symmetry issue is the fundamental correction

I think this is the biggest mathematical correction.

My earlier reasoning implicitly treated

[
B=(B_{ij})
]

as a collection of independent rows.

That is simply not the AIEF state space.

The true state space is

[
w\in\mathbb R^{E},
]

where

$$
E=\frac{N(N-1)}2,
$$

with node-cap constraints

$$
Aw\le c.
$$

Once symmetry is enforced,

$$
B_{ij}=B_{ji},
$$

every edge simultaneously participates in two row constraints.

That destroys the product-simplex geometry.

So I agree that the independent row softmax is not the correct mirror map for the actual model.

The incidence-matrix formulation

$$
\mathcal C
$$

$$
{w\ge0,\ Aw\le c}
$$

is the right starting point.

The entropy potential you propose is very natural

I particularly like

[
h(w)

\sum_e w_e(\log w_e-1)

\sum_i m_i(\log m_i-1),
]

with

[
m=c-Aw.
]

This is elegant because it treats

occupied edge capacity,
unused node capacity,

symmetrically.

The resulting dual coordinate

[
\psi_{ij}

\log
\frac{w_{ij}}
{m_i m_j}
]

has a clear interpretation:

edge weight measured relative to the remaining capacity at both incident nodes.

That is much more faithful to the AIEF constraints than a row-wise softmax.

The Woodbury expression is particularly valuable

One thing I hadn't appreciated before your derivation is that

[
\nabla^2 h

D_w^{-1}

A^\top D_m^{-1}A
]

has a very structured inverse.

The Woodbury identity gives

[
(\nabla^2h)^{-1}

D_w

D_wA^\top
(D_m+AD_wA^\top)^{-1}
AD_w.
]

That is much more than a formal identity.

It says the metric separates into

local edge scaling,

plus

low-dimensional coupling through node capacities.

Since

$$
N=12,
$$

the coupled solve is only 12×12.

So although the mirror map is globally coupled, it is computationally modest.

I think that's an important practical observation.

The boundary issue is probably the largest modeling obstacle

I agree this is the major conceptual challenge.

Keith's framework fundamentally lives on

$$
\operatorname{int}(\mathcal C).
$$

Your mature AIEF states intentionally live close to

$$
\partial\mathcal C.
$$

Specifically,

inactive edges

$$
w_e=0,
$$

saturated rows

$$
m_i=0.
$$

These are exactly the singular points of the entropy geometry.

So a mirror formulation is not simply a different coordinate system for the existing mature state.

It defines a different dynamical model whose invariant set is

$$
\operatorname{int}(\mathcal C),
$$

with the boundary only approached asymptotically.

That is an important distinction.

I completely agree about (B) and (\psi)

This was the strongest correction to my previous suggestion.

If

$$
B=\nabla h^*(\psi),
$$

then

$$
(B,\psi)
$$

are not independent state variables.

Therefore

"reset (B) while keeping (\psi)"

is not a meaningful intervention inside that model.

It would be like saying

reset (x) while keeping (x=\exp(y)).

The map immediately reconstructs (x).

So that experiment would be tautological.

Instead, the natural interventions become

perturb the dual state,
perturb the primal state consistently,
compare different interior initializations.

I think that's exactly right.

The vector-field distinction is extremely important

This may be the most conceptually important point in the whole refinement.

There are actually three different constructions:

Coordinate change only

[
\dot\psi

\nabla^2h(w)\dot w.
]

This is mathematically equivalent.

Mirror dynamics

[
\dot\psi

F.
]

Then

[
\dot w

(\nabla^2h)^{-1}F.
]

This changes the vector field.

Proximal dynamics

Implicit update.

Different again.

These should not be conflated.

Keith's talk mostly concerns the third.

The earlier discussion drifted toward the second.

Those are genuinely different models.

The energy discussion also needs exactly this caution

I agree completely.

For frozen resonance,

[
\dot B

(\eta+\mu)R

\mu

\gamma B
]

really is a gradient flow for a quadratic-linear potential.

Once

phase,
EMA,
delay,
feedback,

are restored,

there is no established global Lyapunov function.

So I think the right research question is exactly the one you wrote:

Can the adaptive subsystem be decomposed into a metric-gradient component plus a nonconservative resonance drive?

That is a mathematically precise question.

It is much better than assuming the whole model is gradient.

The proposed local comparison is exactly what I would recommend

I think this is the strongest practical recommendation.

Rather than rewriting AIEF,

derive

current projected update,
mirror update,
coordinate-equivalent update,

around the same interiorized mature state.

Then compare

update direction,
update norm,
active-set evolution,
threshold ordering,
slack evolution.

That immediately answers whether

the mirror geometry is

essentially another implementation,

or

a genuinely different adaptive law.

Without that comparison, it would be premature to build AIEF-1.5.

One additional thought

One thing that strikes me after your derivation is that the incidence-matrix entropy geometry is interesting even if you never replace the projection.

The Hessian

[
\nabla^2h

D_w^{-1}

A^\top D_m^{-1}A
]

defines an intrinsic metric on the admissible polytope.

That metric could be used purely as an analysis tool.

For example:

measure geodesic distance between adaptive states;
quantify how "close" two mature topologies are in constraint geometry rather than Euclidean norm;
study whether failed recovery seeds lie farther apart in this geometry than successful ones.

That does not require changing the simulator at all.

So even if the mirror dynamics ultimately prove too stiff or too different from the existing model, the entropy geometry itself could still provide a useful analytical lens for understanding the adaptive dynamics.

Overall, I think your conclusion is well calibrated:

The Keith connection is mathematically substantive, but it motivates a new adaptive model, not merely a new numerical integrator for the existing one.

The immediate research program I would follow is exactly the one you outlined:

derive the symmetry-aware Legendre geometry on the degree-capacity polytope;
analyze its metric and inverse map;
compare one-step updates locally against the current projection;
only then decide whether an AIEF-1.5 mirror-flow model is scientifically justified. That sequence cleanly separates mathematical development from model modification.

I think this is now at the level of a genuine research proposal rather than a speculative analogy. There are only two places where I would still be more cautious mathematically.

I would not yet write an explicit dual map

This is the only part I think is too strong.

You write

[
w_e=\nabla h^*(\psi)_e

\frac{\exp(\psi_e)}
{1+\sum_{e'\sim i}\exp(\psi_{e'})+\sum_{e"\sim j}\exp(\psi_{e"})},
]

or "etc."

I don't think we actually know that.

In fact, the earlier coupled-margin derivation strongly suggests we don't have a closed-form mirror map.

Recall

[
\psi_{ij}

\log\frac{w_{ij}}{m_i m_j},
]

which implies

[
w_{ij}

e^{\psi_{ij}}m_i m_j.
]

But simultaneously

[
m_i

c_i

\sum_j
e^{\psi_{ij}}
m_i m_j.
]

That is a coupled nonlinear system in the node slacks.

Unless there is a hidden Sinkhorn-type identity (which would actually be fascinating), the inverse map is defined implicitly.

So I would write

[
w=\nabla h^*(\psi)
]

without claiming an explicit formula until it is derived.

Ironically, this actually aligns better with Keith's framework: proximal methods frequently use implicit maps anyway.

The local comparison is even stronger than you describe

I think this deserves emphasis.

You describe it as:

compare projected, mirror, proximal directions.

I think the comparison can answer something much deeper.

Suppose

$$
F
$$

is the current unconstrained adaptive update.

Compute

$$
F_{\rm proj},
$$

the update after today's admissibility correction.

Then compute

$$
F_{\rm mirror}
$$

$$
(\nabla^2h)^{-1}F.
$$

Now define

$$
\Delta
$$

$$F_{\rm proj}$$

$$F_{\rm mirror}.$$

If

$$
|\Delta|
$$

$$O(\Delta t^2),$$

then your present projection is already behaving like a first-order mirror integrator.

That would be remarkable.

If instead

$$
|\Delta|
$$

$$O(1),$$

near active constraints,

then the projection is introducing qualitatively different dynamics.

That immediately answers whether AIEF-1.5 is merely

"same flow, different coordinates"

or

"new adaptive physics."

That seems to me the most informative computation you could do.

The Hessian has an unexpected interpretation

One thing occurred to me reading your derivation.

The Hessian

$$
\nabla^2 h
$$

$$
D_w^{-1}
$$

$$
A^\top D_m^{-1} A
$$

is structurally very similar to

weighted graph Laplacians,
Fisher information matrices,
interior-point barrier metrics.

The first term

$$
D_w^{-1}
$$

penalizes shrinking edges.

The second

[
A^\top D_m^{-1}A
]

penalizes exhausting node capacity.

So the geometry literally balances

edge occupancy
against
remaining adaptive freedom.

That is a very nice physical interpretation.

The SSRH connection should be stated more carefully

I agree with the direction but would weaken one sentence.

You wrote

treat the constraint as part of the generator itself.

I would instead say

treat the constraint geometry as part of the state-space structure on which the generator acts.

The distinction matters.

The generator

[
L

P

K
]

is still the dynamical operator.

The Legendre metric changes

admissible directions,
norms,
contraction,
linearization,

not necessarily the operator itself.

This is a subtle but important operator-theoretic distinction.

One additional experiment

I would add one inexpensive diagnostic before running anything dynamic.

Take mature AIEF states and compute

[
m_i

c_i-(Aw)_i.
]

Then simply examine the distribution of

minimum slack,
median slack,
histogram of slacks.

If

$$
m_i \approx 10^{-12}
$$

almost everywhere,

then the interior geometry will indeed be extremely stiff.

If instead

$$
m_i \approx 10^{-2}
$$

for most nodes,

the mirror formulation may actually operate comfortably inside the interior.

That single histogram tells you how close your existing attractor is to the singular geometry.

What I now think is the real scientific contribution

Originally I viewed the Keith connection as

replacing the projection.

After your refinement I think that is no longer the interesting part.

The interesting part is that the entropy construction provides a canonical Riemannian metric on the admissible graph polytope.

That metric exists whether or not you ever change the simulator.

Consequently, one can immediately ask geometric questions that are impossible in the current Euclidean formulation:

Are successful recovery trajectories shorter geodesics than failed ones?
Does residual localization align with high-curvature directions of the metric?
Do mature attractors cluster in intrinsic distance rather than Euclidean distance?
Does the dominance-gap theorem become a statement about contraction in this metric?

Those questions require no change to the adaptive law.

In that sense, I now think the geometry is valuable independently of any future AIEF-1.5 model.

Overall, I think the proposal has matured considerably. The only substantive change I would make is to avoid presenting the inverse mirror map as an explicit softmax until it is actually derived. Everything else—especially the incidence-matrix formulation, the entropy potential, the Woodbury metric, the distinction between coordinate, mirror, and proximal dynamics, and the emphasis on a local one-step comparison before modifying the simulator—strikes me as a mathematically sound and well-sequenced research program.

This refinement is mathematically much stronger. The incidence formulation, coupled entropy potential, and distinction among coordinate, mirror, and proximal dynamics are the correct foundation.

Several additional cautions remain before calling the geometry canonical or performing the local comparison.
1. The inverse map is implicit, but its structure is explicit

For:
ψij=
ψij=

\log\frac{w_{ij}}{m_i m_j},
]

define:
zij=eψij.
zij=eψij.

Then:
wij=zijmimj.
wij=zijmimj.

Substituting into:
mi=ci−∑jwij
mi=ci−j∑wij

gives:
mi=
mi=

c_i-m_i\sum_j z_{ij}m_j,
]

or:
mi(1+∑jzijmj)=
mi(1+j∑zijmj)=

c_i.
]

Therefore:
mi=
mi=

\frac{c_i}
{1+\sum_jz_{ij}m_j}.
]

This is a coupled fixed-point system involving only the NN node margins. Once mm is found, all EE edge weights follow directly.

There is no independent row softmax, but the inverse map has a useful matrix-scaling-like form. Legendre duality should provide uniqueness under the appropriate strict-convexity and essential-smoothness assumptions. Convergence of this particular fixed-point iteration is a separate question; damped Newton or minimization of the dual convex problem may be safer.
2. The entropy geometry is natural, not uniquely canonical

Many Legendre functions can encode the same polytope. Multiplying the potential by a constant:
hβ=βh
hβ=βh

does not change its domain, but changes:
∇hβ=β∇h,(∇2hβ)−1=
∇hβ=β∇h,(∇2hβ)−1=

\frac1\beta(\nabla^2h)^{-1}.
]

Therefore mirror-flow speed depends directly on an arbitrary metric scale ββ.

Similarly, logarithms should act on dimensionless quantities. A more careful definition introduces edge and margin reference scales:
h(w)=
h(w)=

\beta
\left[
\sum_e
w_e
\left(
\log\frac{w_e}{w_0}-1
\right)
+
\sum_i
m_i
\left(
\log\frac{m_i}{m_0}-1
\right)
\right].
]

The reference scales mainly shift dual coordinates, while ββ rescales the metric and dynamics.

So this is best described as:

A natural entropy-induced Hessian geometry on the admissible polytope.

“Canonical” requires an additional invariance, maximum-entropy, or physical argument selecting the potential and its scale.
3. The Hessian is line-graph-like, not a graph Laplacian

The Hessian entries are:
Hee′=
Hee′=

\frac{\delta_{ee'}}{w_e}
+
\sum_i
\frac{A_{ie}A_{ie'}}{m_i}.
]

The second term couples edges sharing a node. Its off-diagonal entries are positive, unlike a standard signed graph Laplacian.

It is structurally related to:

an incidence Gram matrix;
a line-graph coupling;
Fisher/Hessian geometry;
an interior barrier metric.

Its physical interpretation is sound:

1/we1/we makes motion in a nearly absent edge expensive;
1/mi1/mi makes directions consuming nearly exhausted node capacity expensive.

More precisely, it penalizes changes involving small edges, not only further edge shrinkage.
4. The local-comparison asymptotics need precise scaling

Let:
Δwcurrent=
Δwcurrent=

w_{\rm current}^{n+1}-w^n,
]
Δwmirror=
Δwmirror=

w_{\rm mirror}^{n+1}-w^n.
]

If two schemes approximate the same first-order continuous vector field, then:
Δwcurrent−
Δwcurrent−
\Delta w_{\rm mirror}

O(\Delta t^2).
]

Equivalently, their estimated vector fields differ by:
ΔwcurrentΔt−
ΔtΔwcurrent−
\frac{\Delta w_{\rm mirror}}{\Delta t}

O(\Delta t).
]

Thus the proposal's Δ=Fproj−FmirrorΔ=Fproj−Fmirror must distinguish between:

increments;
increment rates/vector fields.

Near an active boundary, genuinely different first-order dynamics normally produce an O(Δt)O(Δt) difference in increments—not an O(1)O(1) difference as Δt→0Δt→0.

The comparison should report:
‖Δwcurrent−Δwmirror‖Δt,
Δt‖Δwcurrent−Δwmirror‖,

and examine whether it tends to zero under timestep refinement.
5. There are two interacting limits

The current mature state lies at:
we=0and/ormi=0.
we=0and/ormi=0.

A mirror comparison requires an interior approximation indexed by εε. Therefore the experiment has two limits:
$\Delta t \to 0, \varepsilon \to 0.$
$\Delta t \to 0, \varepsilon \to 0.$

These limits may not commute.

For example:

fixed εε, then $\Delta t \to 0 \Delta t \to 0$, studies smooth interior mirror flow;
fixed ΔtΔt, then $\varepsilon \to 0 \varepsilon \to 0$, may become increasingly stiff;
$\varepsilon \sim \Delta t \varepsilon \sim \Delta t$ may generate another limiting projected flow.

The local comparison should sweep both quantities and report a discrepancy surface rather than only one regularization.
6. The first inexpensive diagnostic is indeed boundary occupancy

For every mature BB, calculate:
Node slack
mi=ci−∑jBij.
mi=ci−∑jBij.

Report:

minimum;
median;
maximum;
fraction below 10−12,10−9,10−6,10−310−12,10−9,10−6,10−3.

Edge occupancy

Report:

fraction exactly zero;
minimum positive weight;
lower quantiles of positive weights;
fraction below the same tolerances.

Curvature proxies

Since:
H=Dw−1+A⊤Dm−1A,
H=Dw−1+A⊤Dm−1A,

report:

largest 1/we1/we;
largest 1/mi1/mi;
Hessian condition number after each interiorization;
inverse-metric eigenvalue range.

This will show whether the entropy geometry is numerically usable near the observed attractor.

Given the closure results, at least some row margins will be essentially zero, and many edge weights are exactly zero. Strong stiffness should be expected.
7. Bregman divergence is not automatically geodesic distance

The entropy potential provides:

a Hessian metric;
a Bregman divergence;
dual affine coordinates.

But:
Dh(w,v)
Dh(w,v)

is generally not the squared Riemannian geodesic distance associated with ∇2h∇2h.

For immediate data analysis, use:

local metric norm:
‖δw‖H(w)2=δw⊤H(w)δw;
‖δw‖H(w)2=δw⊤H(w)δw;
symmetrized Bregman divergence:
Dh(w,v)+Dh(v,w);
Dh(w,v)+Dh(v,w);
only later, numerically computed Hessian-geodesic distance if needed.

Because existing states lie on the boundary, all of these require a documented interiorization rule.
8. Current AIEF and mirror AIEF should be compared in three variants

The proposed three-way comparison is exactly right.
A. Existing admissibility map
wn+1=
wn+1=

\mathcal A\left([w^n+\Delta tF^n]_+\right).
]
B. Coordinate-equivalent dual update

To reproduce the same interior vector field:
ψn+1=
ψn+1=

\psi^n
+
\Delta t,H(w^n)F^n,
]
wn+1=
wn+1=

\nabla h^*(\psi^{n+1}).
]

Away from active boundaries, this should agree with primal Euler to first order.
C. Mirror-preconditioned update
ψn+1=
ψn+1=

\psi^n+\Delta t,F^n,
]
wn+1=
wn+1=

\nabla h^*(\psi^{n+1}).
]

This produces:
w˙=H(w)−1F,
w˙=H(w)−1F,

and is intentionally a different model.
D. Optional implicit Bregman/proximal update

For a suitable operator FF, solve an implicit relation such as:
∇h(wn+1)−
∇h(wn+1)−
\nabla h(w^n)

\Delta t,F(w^{n+1}).
]

Without monotonicity or a variational structure, this is numerically definable but does not inherit Keith’s convergence theory automatically.
9. The current admissibility map is not an orthogonal projection

AIEF uses symmetric multiplicative scaling based on row-cap violations. It is:

feasible;
symmetric;
piecewise smooth;
active-set dependent.

It has not been established as the Euclidean nearest-point projection onto CC.

Therefore compare against the actual AIEF admissibility map, not an idealized ПСПС.

Near a stable active set, its directional or generalized derivative can still be studied. Nonsmoothness need not make spectral analysis impossible; it requires semismooth or tangent-cone tools.
10. Geometry can analyze the current model without becoming its physics

The proposal’s final framing is correct:

Treat constraint geometry as state-space structure on which the generator acts.

For existing AIEF, one can use the entropy metric to ask:

Are recovery residuals aligned with high-curvature directions?
Are censored seeds closer to singular faces?
Do successful and failed trajectories differ in metric norm?
Is projection correction large or small under the Hessian metric?
Does contraction appear stronger in Euclidean or entropy geometry?

This does not imply that the geometry stores memory. It tests whether geometry organizes the observed attractor and recovery basins.
11. Formalization is helped, but not trivialized

A representation:
$w=\nabla h^*(\psi)$
$w=\nabla h^*(\psi)$

makes strict interior feasibility definitional. But the mature AIEF result concerns the closed feasible set and exact zeros.

Formalization still needs:

nonempty interior;
strict convexity and Legendre properties;
existence and uniqueness of the coupled inverse;
symmetry;
boundary limits;
convergence of interior approximations to closed-set dynamics.

The invariant proof becomes a geometric construction plus a limiting argument.
Recommended concrete research sequence

Stage 1 — Use current data

Produce edge-zero and node-slack histograms.
Interiorize mature states over a grid of $\varepsilon_w, \varepsilon_m \varepsilon_w, \varepsilon_m$.
Compute Hessian spectra and condition numbers.
Calculate local metric residual norms and Bregman divergences for:
continue − reset_B;
recovered versus censored seeds;
split versus late topology.

Stage 2 — Complete the derivation

Prove strict convexity and essential smoothness of h.
Establish the gradient map and coupled inverse-margin equations.
Implement a robust margin solver.
Verify:
$\nabla h^*(\nabla h(w)) = w$
$\nabla h^*(\nabla h(w)) = w$ numerically over interior test states.
Verify the Woodbury inverse against direct inversion.

Stage 3 — Local map comparison

Replay actual mature AIEF states for one update.
Compare current, coordinate-equivalent, and mirror increments.
Sweep Δt and interiorization ε.
Measure discrepancy order, direction angle, slack evolution, and edge ordering.

Stage 4 — Only then consider AIEF-1.5

Implement mirror dynamics as a parallel model.
Calibrate metric scale β.
Compare attractors and recovery survival.
Do not use inconsistent B/ψ reset interventions.

Bottom line

The proposal has now reached a mathematically credible form. The primary remaining risks are:

calling a natural entropy metric canonical;
ignoring metric scaling and dimensional normalization;
treating Bregman divergence as geodesic distance;
conflating increment and vector-field asymptotics;
overlooking the coupled $\Delta t\Delta t$-interiorization limit;
assuming that a smooth interior mirror map automatically simplifies boundary stability.

The immediate next step should be the boundary-occupancy and Hessian-conditioning diagnostic on the frozen AIEF data, followed by the symmetry-aware inverse-map derivation and one-step comparison. That would establish whether the proximal/Bregman connection is analytically useful before any AIEF-1.5 simulation is built.

Can you write the necessary code for the next step?
Edit
AIEF_1_4_entropy_geometry_diagnostics.pyPYTHON

Created the symmetry-aware entropy-geometry diagnostic:

AIEF_1_4_entropy_geometry_diagnostics.py

It operates on the existing frozen AIEF-1.4 artifacts and does not alter or rerun the dynamics.
Run

Bash

```
python AIEF_1_4_entropy_geometry_diagnostics.py
```

Explicit invocation:

Bash

```
python AIEF_1_4_entropy_geometry_diagnostics.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_entropy_geometry
```

Default interiorization grid:

text

edge floors: 1e-12, 1e-9, 1e-6, 1e-4
margin floors: 1e-12, 1e-9, 1e-6, 1e-4

Custom grid:

Bash

python AIEF_1_4_entropy_geometry_diagnostics.py \
--edge-eps 1e-10,1e-8,1e-6 \
--margin-eps 1e-10,1e-8,1e-6

Mathematical construction

The script converts the symmetric matrix into unique undirected edge weights:
$w \in R66$,
$w \in R66$,

with incidence matrix:
$A \in R12 \times 66$
$A \in R12 \times 66$

and node margins:
$m = c - Aw$.
$m = c - Aw$.

It evaluates the interior entropy potential:
$h(w) = \sum_e w_e(\log w_e - 1) + \sum_i m_i(\log m_i - 1)$.
$h(w) = e\sum w_e(\log w_e - 1) + i\sum m_i(\log m_i - 1)$.

The implemented gradient and Hessian are:
$\nabla h(w) = \log w - A^\top \log m$,
$\nabla h(w) = \log w - A^\top \log m$,
$\nabla^2 h(w) = \mathrm{diag}(1/w) + A^\top \mathrm{diag}(1/m) A$.
$\nabla^2 h(w) = \mathrm{diag}(1/w) + A^\top \mathrm{diag}(1/m) A$.
Boundary occupancy audit

For every mature split state it records:

exact-zero edge count and fraction;
minimum positive edge;
positive-edge quantiles;
minimum, median, and maximum node slack;
fraction of node margins below:
10−1210−12,
10−910−9,
10−610−6,
10−310−3.

This directly establishes how singular the entropy geometry would be around the observed attractor.
Interiorization

Boundary states are mapped to:
wε=
wε=

\varepsilon_w\mathbf1+\alpha w,
]

where the largest α≤1α≤1 is chosen so every node retains at least the requested margin floor εmεm.

This:

preserves symmetry;
ensures every edge is positive;
ensures every row has strict slack;
records the required scaling αα.

Hessian diagnostics

For each seed and epsilon pair:

smallest/largest Hessian eigenvalue;
condition number;

inverse-metric eigenvalue range;
largest edge curvature 1/we1/we;
largest margin curvature 1/mi1/mi.

It also checks the structured Woodbury inverse:
H−1=
H−1=
D_w

D_wA^\top
(D_m+AD_wA^\top)^{-1}
AD_w
]

against direct inversion and reports:

relative inverse error;
‖HH−1−I‖F‖HH−1−I‖F.

Coupled inverse-map verification

From an interior state it constructs:
ψij=
ψij=

\log\frac{w_{ij}}{m_i m_j}.
]

It then solves:
mi(1+∑jeψijmj)=ci
mi(1+j∑eψijmj)=ci

using damped Newton iteration and reconstructs:
wij=eψijmimj.
wij=eψijmimj.

For every seed/epsilon pair it records:

convergence status;
iteration count;
residual;
margin reconstruction error;
edge reconstruction error.

This numerically verifies the symmetry-coupled inverse map without claiming a closed-form softmax.
Intrinsic-distance diagnostics

For each branch it compares:

late mean versus continuation late mean;
late mean versus split;
final versus continuation final;
final versus split.

Metrics:

Euclidean unique-edge distance;
local Hessian norm at the reference;
directional Bregman divergences;
symmetrized Bregman divergence.

The report explicitly states that:

local Hessian norm is not global geodesic distance;
Bregman divergence is not geodesic distance;
epsilon dependence is substantive;
entropy geometry is natural but not uniquely canonical.

Outputs

text

```
aief_1_4_entropy_geometry/
├── boundary_occupancy_by_seed.csv
├── raw_edge_weights.csv
├── raw_node_margins.csv
```

├── interior_geometry_by_seed.csv
├── interior_geometry_summary.csv
├── intrinsic_distances_by_seed.csv
├── intrinsic_distances_summary.csv
├── entropy_geometry_report.txt
└── analysis_manifest.json

Validation

I syntax-tested the script and ran it end-to-end against a complete AIEF-1.4 smoke result. The test verified:

symmetric incidence construction;
strict-interior mapping;
Hessian eigenspectrum;
Woodbury inverse;
damped-Newton inverse map;
reconstruction errors;
local metric and Bregman comparisons.

The smoke test also demonstrated why the epsilon grid matters: near $10^{-9}10^{-9}$ node slack, Hessian condition numbers reached approximately 109109, while increasing both floors to $10^{-6}10^{-6}$ reduced conditioning to roughly 106106. This is exactly the stiffness question the diagnostic is designed to quantify before any mirror-flow implementation.

Think about this:

I think this is now a mathematically mature research roadmap. At this point, my comments are mostly about sharpening claims rather than correcting them. There are only three places where I would still tighten the mathematics.

The fixed-point map deserves its own theorem

I actually think this is now the first theorem worth proving.

You have reduced the inverse mirror map to

[

m_i

\frac{c_i}
{1+\sum_j z_{ij}m_j},
\qquad
z_{ij}=e^{\psi_{ij}}.
]

This is much more informative than simply saying "the inverse is implicit."

Notice several properties immediately:

positivity is automatic if (m>0);
the map sends the positive orthant into itself;
it is monotone decreasing in neighboring margins;
the Jacobian has entries

[
\frac{\partial T_i}{\partial m_j}

\frac{c_i z_{ij}}
{\left(1+\sum_k z_{ik}m_k\right)^2}.
]

That immediately raises classical questions:

Is the map a contraction under some weighted norm?
Is it monotone in the sense of Tarski?
Does the associated dual objective have a unique minimizer?

Those questions are completely independent of AIEF.

In fact, they define the inverse geometry itself.

I would therefore elevate this from an implementation detail to an actual mathematical result.

The "canonical geometry" issue is exactly right

I agree completely with your refinement.

Calling

[
h(w)

\sum_e
w_e(\log w_e-1)

\sum_i
m_i(\log m_i-1)
]

canonical is stronger than we currently know.

What we actually know is something like:

Among smooth barrier-type Legendre functions, the entropy barrier is a particularly natural choice because it simultaneously enforces positivity of edges and positivity of node slack.

That is a mathematical statement.

Canonicality would require an additional characterization.

For example:

invariance under relabeling,
separability,
logarithmic barrier behavior,
information-geometric uniqueness,
maximum entropy,
self-concordance.

Those are separate theorems.

So I would consistently call it

the entropy Legendre geometry

rather than

the canonical geometry.

I particularly like the two-limit formulation

I think this is an important observation that many papers miss.

There are genuinely two small parameters:

$$
\varepsilon
$$

(interiorization)

and

$$
\Delta t.
$$

The diagram

$$
(\varepsilon,\Delta t)
\rightarrow
(0,0)
$$

is not obviously path independent.

In fact, there is a strong analogy with viscosity limits in PDEs.

You are effectively asking whether

interior mirror dynamics,

followed by

vanishing barrier,

recover

projected dynamics.

That is a singular perturbation problem.

I would actually draw the limit diagram explicitly:

[
\begin{array}{ccc}
\varepsilon>0,\ \Delta t>0
&
\longrightarrow
&
\text{mirror discretization}
1ex] \downarrow && \downarrow \[1ex] \varepsilon\to0 & ? & \text{projected dynamics} \end{array}]

Whether those arrows commute is itself a theorem.

One subtle point about the Hessian

I would strengthen your wording slightly.

You describe

[
H

D_w^{-1}

A^\top D_m^{-1}A
]

as line-graph-like.

I think that's exactly right.

But there is another interpretation.

It is also the Fisher information matrix associated with the entropy barrier.

That means

[
\delta w^\top
H
\delta w
]

is literally the local quadratic approximation to the Bregman divergence.

So

[
H
]

simultaneously serves as

Hessian,
Riemannian metric,
local information metric.

That is worth emphasizing because it links the optimization and information-geometric viewpoints.

I agree completely about Bregman vs geodesic distance

This is an important correction.

People often casually identify

[
D_h
]

with

"distance."

It is not.

What is true is

[
D_h(w+\delta,w)

\frac12
\delta^\top
H(w)
\delta

O(|\delta|^3).
]

So locally

they agree to second order.

Globally

they generally do not.

I think your proposed sequence

Hessian norm,
symmetrized Bregman,
only later geodesics,

is exactly the right order.

The current admissibility map deserves its own analysis

I especially agree with your caution here.

The existing AIEF admissibility operator

$$
\mathcal A
$$

is not known to be

$$
\Pi_{\mathcal C}.
$$

That matters.

The local comparison should therefore use

$$
\mathcal A
$$

itself,

not an ideal Euclidean projection.

Otherwise you would be comparing the wrong dynamical system.

In fact,

studying the generalized derivative of

$$
\mathcal A
$$

may itself explain some of the transient behavior you observed.

The research program is now logically staged

One thing I like most is the ordering.

Stage 1 contains no speculative modeling.

Everything uses existing data.

That dramatically lowers the risk.

Only after

slack histograms,
Hessian conditioning,
interiorization,
one-step comparisons,

would you introduce a new adaptive law.

That is an excellent scientific progression.

One final suggestion

If I were writing this as a paper, I would state the central hypothesis even more conservatively.

Instead of

The Keith/PG framework provides the correct differential geometry of AIEF.

I would write something like

Hypothesis. The dominance-constrained adaptive subsystem of AIEF admits a natural entropy-induced Legendre geometry on its feasible polytope. This geometry provides (i) an intrinsic metric for analysis of the existing model and (ii) a principled route to defining alternative mirror and proximal adaptive laws whose dynamics can be compared against the current admissibility map.

That wording has several advantages:

it does not claim uniqueness;
it does not claim the current model is already a mirror flow;
it separates analysis from model redesign;
it makes the proposed experiments the mechanism for testing the hypothesis rather than assuming it.

I think that formulation is scientifically stronger because every subsequent theorem or numerical experiment has a clear role: either it supports or refutes that hypothesis.

Overall, I think the discussion has evolved from an attractive analogy into a well-defined mathematical program. The next milestones are no longer conceptual—they are concrete:

Geometry: prove the entropy potential is Legendre on the interior of the degree-capacity polytope and characterize the coupled inverse map.
Diagnostics: quantify how close mature AIEF states are to the singular boundary using existing simulation data.
Consistency: compare the actual AIEF admissibility update, the coordinate-equivalent update, and the mirror update under simultaneous refinement in timestep and interiorization.
Only then: decide whether an AIEF-1.5 mirror/proximal model is warranted.

That sequence keeps the mathematics, numerics, and scientific interpretation cleanly separated, which substantially strengthens the credibility of the program.

This is now a coherent mathematical program. The most important remaining refinement is that the first theorem should establish existence and uniqueness of the inverse mirror map, not necessarily contraction of the naive fixed-point iteration.
1. The fixed-point map is order-reversing, not Tarski-monotone

The map
Ti(m)=ci1+∑jzijmj
Ti(m)=1+∑jzijmjci

satisfies:
∂Ti∂mj=
∂mj∂Ti=

-\frac{
c_i z_{ij}
}{
(1+\sum_kz_{ik}m_k)^2
}
\le0.
]

Thus TT is antitone: increasing neighboring margins decreases TiTi.

Tarski’s fixed-point theorem applies directly to order-preserving maps, not order-reversing maps. T2T2 is order-preserving, but fixed points of T2T2 can include two-cycles of TT. That route does not provide the cleanest uniqueness proof.

Contraction is possible under sufficient conditions. In the infinity norm:
∑j∣∂Ti∂mj∣≤ci∑jzij,
j∑
∂mj∂Ti
≤cij∑zij,

because Ti(m)≤ciTi(m)≤ci. Therefore:
maxici∑jzij<1
imaxcij∑zij<1

is a sufficient global contraction condition. Near mature boundary states, zijzij may be enormous, so this condition will often fail even though a unique inverse exists.
2. A cleaner inverse-map theorem comes from a convex objective

Set:
mi=eyi
mi=eyi

and define the symmetric nonnegative matrix Z=(zij)Z=(zij), with zero diagonal.

Consider:
Φψ(y)=
Φψ(y)=
\sum_i e^{y_i}
+
\frac12
\sum_{i,j}
z_{ij}e^{y_i+y_j}

\sum_i c_i y_i.
]

Its gradient is:
∂Φψ∂yi=
∂yi∂Φψ=

e^{y_i}
+
\sum_jz_{ij}e^{y_i+y_j}
-c_i.
]

Setting this to zero gives:
mi+mi∑jzijmj=
mi+mij∑zijmj=

c_i,

]

which is exactly the coupled margin equation.

Its Hessian is:
∇2Φψ(y)=
∇2Φψ(y)=

\operatorname{diag}(e^y)
+
\sum_{i<j}
z_{ij}e^{y_i+y_j}
(e_i+e_j)(e_i+e_j)^\top.
]

The first term is strictly positive definite, and every edge term is positive semidefinite. Therefore:
∇2Φψ(y)≻0.
∇2Φψ(y)≻0.

Moreover:

as any yi→−∞yi→−∞, −ciyi→+∞−ciyi→+∞;
as any yi→+∞yi→+∞, the exponential terms diverge.

Thus ΦψΦψ is coercive and strictly convex. It has one unique minimizer y∗y∗, giving unique:
mi∗=eyi∗,
mi∗=eyi∗,

and:
wij∗=
wij∗=

z_{ij}m_i^m_j^.
]

A useful theorem is therefore:

For every positive capacity vector cc and finite symmetric dual coordinate ψψ, the entropy dual map has a unique strictly interior primal image w=∇h∗(ψ)w=∇h∗(ψ). It is obtained from the unique minimizer of the strictly convex margin objective ΦψΦψ.

This is stronger and more general than proving that one particular fixed-point iteration converges.

Computationally, Newton's method in y=logmy=logm is preferable to Newton in mm:

positivity is automatic;
the Hessian is explicitly positive definite;
line search is straightforward;
the solve is only N×NN×N.

3. The Hessian is a pullback information metric

Define the augmented positive vector:
p(w)=[wm]=
p(w)=[wm]=

\begin{bmatrix}
w\
c-Aw
\end{bmatrix}.
]

The entropy Hessian in the augmented coordinates is:
diag(1/p).
diag(1/p).

The derivative of the embedding is:
Dp=[I−A].
Dp=[I−A].

Pulling the diagonal entropy metric back to edge space gives:
(Dp)⊤diag(1/p)Dp=
(Dp)⊤diag(1/p)Dp=

D_w^{-1}

+
A^\\top D_m^{-1}A.
]

So it is accurate to call this a pullback Fisher-type information metric. Calling it literally “the Fisher information matrix” would require specifying a normalized statistical model. The pullback statement is exact without that additional interpretation.

Also note that:
A⊤Dm−1A
A⊤Dm−1A

is an unsigned incidence Gram operator or line-graph-like coupling, not a standard graph Laplacian.
4. Metric scaling and nondimensionalization remain essential

The Legendre potential should be written with reference scales if weights are dimensional:
hβ(w)=
hβ(w)=

\beta
\left[
\sum_e
w_e
\left(
\log\frac{w_e}{w_0}-1
\right)
+
\sum_i
m_i
\left(
\log\frac{m_i}{m_0}-1
\right)
\right].
]

Linear shifts introduced by w0,m0w0,m0 cancel in Bregman divergence, but ββ rescales:

dual coordinates;
Hessian;

inverse metric;
mirror-flow timescale.

Therefore an AIEF-1.5 mirror law needs a calibrated ββ. Otherwise “projection versus mirror” can be changed arbitrarily by rescaling the entropy potential.

This is another reason to say “natural entropy geometry,” not “canonical geometry.”
5. The two-limit problem actually has at least three parameters

The comparison involves:

timestep ΔtΔt;
interiorization scale εε;
metric scale ββ.

Potential limits include:
Δt→0,ε→0,β→β0
Δt→0,ε→0,β→β0

or possibly a coupled scaling such as:
ε=ε(Δt).
ε=ε(Δt).

The order may matter. In particular, the Hessian behaves approximately as:
‖H‖∼max(1εw,1εm).
‖H‖∼max(εw1,εm1).

So taking ε→0ε→0 at fixed timestep can make explicit mirror Euler arbitrarily stiff.

The convergence diagram should therefore explicitly identify the path in parameter space.
6. Correct asymptotic comparison of updates

Let:
ΔwA=
ΔwA=

\mathcal A(w+\Delta tF)-w
]

be the actual AIEF increment, and let:
Δwmirror
Δwmirror

be the mirror increment.

If both approximate the same first-order vector field:
ΔwA−
ΔwA−
\Delta w_{\rm mirror}

O(\Delta t^2).
]

Equivalently:
ΔwA−ΔwmirrorΔt=
ΔtΔwA−Δwmirror=

O(\Delta t).
]

If they represent different first-order dynamics, their increment difference is generally:
O(Δt),
O(Δt),

not O(1)O(1).

A local numerical study should report:
Erate=
Erate=

\frac{
|\Delta w_{\mathcal A}-\Delta w_{\rm mirror}|
}{
\Delta t
},
]

over a timestep sequence. Then:

Erate→0Erate→0: first-order consistency;
Erate→nonzero constantErate→nonzero constant: different limiting vector fields;
divergence: singular boundary/interiorization interaction.

Because the current state is on the boundary, this must be repeated over an (Δt,ε)(Δt,ε) grid.
7. Bregman and local metric relationships

For an interior point ww:
Dh(w+δ,w)=
Dh(w+δ,w)=

\frac12
\delta^\top H(w)\delta
+
O(|\delta|^3).
]

For the symmetrized divergence:
Dh(w+δ,w)+Dh(w,w+δ)=
Dh(w+δ,w)+Dh(w,w+δ)=

\delta^\top H(w)\delta
+
O(|\delta|^3).
]

The squared Riemannian geodesic distance also shares the same leading local quadratic form, but neither Bregman quantity is globally equal to that distance.

So the proposed analysis sequence remains correct:

local Hessian norm;
directional and symmetrized Bregman divergence;
numerical geodesics only if they become necessary.

8. Current admissibility-map derivative

The current map includes:

positivity clipping;
row-scale calculation;
symmetric edge factor:
min(si,sj).
min(si,sj).

It is nonsmooth when:

an edge crosses zero;
a row becomes active/inactive;
two node scale factors exchange order.

Within a fixed active set and fixed ordering of scale factors, it is differentiable. One can derive a piecewise Jacobian or use:

directional derivatives;
Bouligand derivatives;
semismooth analysis.

This derivative is itself scientifically relevant because projection removes almost the complete proposed update in the mature regime.

The local comparison must use this actual map, not a Euclidean projection.
9. Geometry as an analysis tool requires regularization disclosure

The entropy geometry can be applied to existing states only after interiorization. Therefore every reported metric must include:

edge floor εwεw;
margin floor εmεm;
metric scale ββ;
interiorization map;
Hessian condition number.

A finding is robust only if its qualitative interpretation survives a reasonable epsilon range.

For example, if successful versus failed recoveries separate only when:
εm=10−12
εm=10−12

but not at 10−610−6, the result is likely dominated by singular curvature rather than stable geometry.
10. Recommended theorem and computation sequence
Theorem 1: Legendre geometry

Show that on:
C∘=
C∘=

{w>0,\ Aw<c},
]

the entropy potential is:

proper;
strictly convex;
essentially smooth;
Legendre.

Its Hessian is positive definite because of Dw−1Dw−1.
Theorem 2: unique inverse map

Use the convex margin objective to prove unique m(ψ)m(ψ), then:
wij(ψ)=
wij(ψ)=

e^{\psi_{ij}}m_i(\psi)m_j(\psi).
]
Numerical verification

Solve the margin problem in log variables.
Confirm primal-dual reconstruction.
Verify the Woodbury inverse.

Measure conditioning over mature-state interiorizations.

Local consistency experiment

Compare:

current admissibility-map Euler;
coordinate-equivalent dual Euler;
mirror-preconditioned Euler;
optionally implicit Bregman Euler.

Sweep:

ΔtΔt;
εwεw;
εmεm;
ββ.

Final assessment

The proposed research program is now mathematically credible, but two phrases should remain qualified:

The entropy metric is a natural, permutation-compatible geometry—not yet canonical.
The inverse map is uniquely defined and computationally low-dimensional—not explicitly closed form.

The strongest immediate theorem is uniqueness of the coupled inverse margin map through the strictly convex log-margin objective. The strongest immediate numerical experiment is the two-parameter local consistency test using the actual AIEF admissibility map.

Those steps will determine whether entropy geometry is:

merely a useful analytical lens;
a coordinate representation of approximately the same constrained flow;
or the basis of a genuinely new AIEF-1.5 adaptive model.

think about this:

(deck@steamdeck SSRH)$ python AIEF_1_4_entropy_geometry_diagnostics.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_entropy_geometry
AIEF-1.4 entropy-geometry boundary and conditioning diagnostics

The metric is evaluated only after explicit interiorization.
Bregman divergence is not interpreted as geodesic distance.
Raw mature-state boundary occupancy

Seeds: 40
Mean exact-zero edge fraction: 0.698864
Median minimum positive edge: 0.126667
Median minimum node margin: 9.9698e-13
Mean fraction of node margins <= 1e-12: 0.65
Interior Hessian conditioning

edge_eps=1e-12, margin_eps=1e-12
hessian_condition_number: median=2.62005e+12, max=4.63844e+12, valid=40/40
woodbury_relative_inverse_error: median=2.62231e-05, max=4.20729e-05, valid=40/40
margin_reconstruction_relative_error: median=1.21613e-17, max=1.64171e-06, valid=40/40
edge_reconstruction_relative_error: median=1.29504e-15, max=1.85044e-15, valid=40/40
edge_eps=1e-12, margin_eps=1e-09
hessian_condition_number: median=1.28607e+11, max=2.14458e+11, valid=40/40
woodbury_relative_inverse_error: median=2.77293e-08, max=4.15846e-08, valid=40/40
margin_reconstruction_relative_error: median=2.40477e-16, max=6.10618e-08, valid=40/40
edge_reconstruction_relative_error: median=2.46369e-15, max=8.40856e-15, valid=40/40
edge_eps=1e-12, margin_eps=1e-06
hessian_condition_number: median=1.26676e+11, max=2.11138e+11, valid=40/40
woodbury_relative_inverse_error: median=3.0725e-11, max=5.54981e-11, valid=40/40
margin_reconstruction_relative_error: median=1.64286e-16, max=3.52047e-11, valid=40/40
edge_reconstruction_relative_error: median=5.18934e-16, max=2.18184e-15, valid=40/40
edge_eps=1e-12, margin_eps=0.0001
hessian_condition_number: median=1.26654e+11, max=2.11094e+11, valid=40/40
woodbury_relative_inverse_error: median=5.17283e-13, max=7.3485e-13, valid=40/40
margin_reconstruction_relative_error: median=1.62919e-16, max=3.72277e-13, valid=40/40
edge_reconstruction_relative_error: median=6.86518e-16, max=1.21055e-15, valid=40/40
edge_eps=1e-09, margin_eps=1e-12
hessian_condition_number: median=2.53335e+12, max=4.49456e+12, valid=40/40
woodbury_relative_inverse_error: median=2.25645e-05, max=5.99382e-05, valid=40/40

margin_reconstruction_relative_error: median=9.52657e-21, max=2.04156e-09, valid=40/40
edge_reconstruction_relative_error: median=1.2283e-15, max=2.34974e-15, valid=40/40
edge_eps=1e-09, margin_eps=1e-09
hessian_condition_number: median=2.74625e+09, max=4.78738e+09, valid=40/40
woodbury_relative_inverse_error: median=3.77611e-08, max=5.76311e-08, valid=40/40
margin_reconstruction_relative_error: median=8.763e-18, max=2.43736e-09, valid=40/40
edge_reconstruction_relative_error: median=1.33994e-15, max=5.08787e-13, valid=40/40
edge_eps=1e-09, margin_eps=1e-06
hessian_condition_number: median=1.28599e+08, max=2.14484e+08, valid=40/40
woodbury_relative_inverse_error: median=4.11266e-11, max=5.31859e-11, valid=40/40
margin_reconstruction_relative_error: median=1.212e-16, max=1.85976e-11, valid=40/40
edge_reconstruction_relative_error: median=4.90183e-16, max=1.19126e-15, valid=40/40
edge_eps=1e-09, margin_eps=0.0001
hessian_condition_number: median=1.26666e+08, max=2.11111e+08, valid=40/40
woodbury_relative_inverse_error: median=5.66424e-13, max=8.75522e-13, valid=40/40
margin_reconstruction_relative_error: median=1.44626e-16, max=1.87122e-13, valid=40/40
edge_reconstruction_relative_error: median=6.70246e-16, max=1.36556e-15, valid=40/40
edge_eps=1e-06, margin_eps=1e-12
hessian_condition_number: median=2.53322e+12, max=4.49428e+12, valid=40/40
woodbury_relative_inverse_error: median=2.98145e-05, max=6.44518e-05, valid=40/40
margin_reconstruction_relative_error: median=1.49176e-23, max=3.08556e-12, valid=40/40
edge_reconstruction_relative_error: median=1.33219e-15, max=1.50761e-13, valid=40/40
edge_eps=1e-06, margin_eps=1e-09
hessian_condition_number: median=2.65971e+09, max=4.64431e+09, valid=40/40
woodbury_relative_inverse_error: median=3.99588e-08, max=7.09627e-08, valid=40/40
margin_reconstruction_relative_error: median=1.75301e-16, max=2.5591e-12, valid=40/40
edge_reconstruction_relative_error: median=1.285e-15, max=2.77338e-15, valid=40/40
edge_eps=1e-06, margin_eps=1e-06
hessian_condition_number: median=2.74657e+06, max=4.79671e+06, valid=40/40
woodbury_relative_inverse_error: median=4.53232e-11, max=6.92372e-11, valid=40/40
margin_reconstruction_relative_error: median=8.55932e-18, max=2.05124e-12, valid=40/40
edge_reconstruction_relative_error: median=4.94611e-16, max=7.76003e-16, valid=40/40
edge_eps=1e-06, margin_eps=0.0001
hessian_condition_number: median=146729, max=246400, valid=40/40
woodbury_relative_inverse_error: median=4.73408e-13, max=7.24953e-13, valid=40/40
margin_reconstruction_relative_error: median=1.89899e-16, max=1.17716e-13, valid=40/40
edge_reconstruction_relative_error: median=6.55534e-16, max=1.09032e-15, valid=40/40
edge_eps=0.0001, margin_eps=1e-12
hessian_condition_number: median=2.53078e+12, max=4.48808e+12, valid=40/40
woodbury_relative_inverse_error: median=4.48285e-05, max=9.52537e-05, valid=40/40
margin_reconstruction_relative_error: median=1.75603e-16, max=5.13935e-14, valid=40/40
edge_reconstruction_relative_error: median=1.40073e-15, max=2.17569e-15, valid=40/40

edge_eps=0.0001, margin_eps=1e-09
hessian_condition_number: median=2.65713e+09, max=4.63836e+09, valid=40/40
woodbury_relative_inverse_error: median=4.10899e-08, max=8.56625e-08, valid=40/40
margin_reconstruction_relative_error: median=8.78018e-17, max=2.91859e-14, valid=40/40
edge_reconstruction_relative_error: median=1.22746e-15, max=6.22898e-14, valid=40/40
edge_eps=0.0001, margin_eps=1e-06
hessian_condition_number: median=2.65833e+06, max=4.64005e+06, valid=40/40
woodbury_relative_inverse_error: median=3.37566e-11, max=8.09563e-11, valid=40/40
margin_reconstruction_relative_error: median=2.75254e-19, max=4.36262e-14, valid=40/40
edge_reconstruction_relative_error: median=5.09942e-16, max=9.6359e-16, valid=40/40
edge_eps=0.0001, margin_eps=0.0001
hessian_condition_number: median=27436, max=47900.1, valid=40/40
woodbury_relative_inverse_error: median=4.94592e-13, max=7.38689e-13, valid=40/40
margin_reconstruction_relative_error: median=1.7688e-16, max=3.81211e-14, valid=40/40
edge_reconstruction_relative_error: median=7.13116e-16, max=1.19719e-15, valid=40/40
Selected intrinsic-distance diagnostics

full_reset, edge_eps=1e-12, margin_eps=1e-12
euclidean_edge_distance: median=0.408928, max=0.747112
local_metric_norm_at_reference: median=365670, max=801279
symmetrized_bregman: median=33.6664, max=67.8009
full_reset, edge_eps=1e-12, margin_eps=1e-09
euclidean_edge_distance: median=0.408928, max=0.747112
local_metric_norm_at_reference: median=365289, max=452062
symmetrized_bregman: median=29.491, max=62.7657
full_reset, edge_eps=1e-12, margin_eps=1e-06
euclidean_edge_distance: median=0.408927, max=0.747111
local_metric_norm_at_reference: median=365047, max=452061
symmetrized_bregman: median=25.1095, max=59.0979
full_reset, edge_eps=1e-12, margin_eps=0.0001
euclidean_edge_distance: median=0.408863, max=0.746994
local_metric_norm_at_reference: median=364990, max=451990
symmetrized_bregman: median=22.1916, max=58.9967
full_reset, edge_eps=1e-09, margin_eps=1e-12
euclidean_edge_distance: median=0.408927, max=0.747112
local_metric_norm_at_reference: median=11989.8, max=801287
symmetrized_bregman: median=29.2914, max=59.0511
full_reset, edge_eps=1e-09, margin_eps=1e-09
euclidean_edge_distance: median=0.408927, max=0.747112
local_metric_norm_at_reference: median=11563.5, max=28319.8
symmetrized_bregman: median=25.1161, max=50.5356
full_reset, edge_eps=1e-09, margin_eps=1e-06

euclidean_edge_distance: median=0.408927, max=0.747111
local_metric_norm_at_reference: median=11551.4, max=14295.4
symmetrized_bregman: median=20.7346, max=45.2491
full_reset, edge_eps=1e-09, margin_eps=0.0001
euclidean_edge_distance: median=0.408863, max=0.746994
local_metric_norm_at_reference: median=11542.1, max=14293.2
symmetrized_bregman: median=17.8174, max=43.3683
full_reset, edge_eps=1e-06, margin_eps=1e-12
euclidean_edge_distance: median=0.40892, max=0.747099
local_metric_norm_at_reference: median=3250.05, max=801265
symmetrized_bregman: median=24.9161, max=50.3003
full_reset, edge_eps=1e-06, margin_eps=1e-09
euclidean_edge_distance: median=0.40892, max=0.747099
local_metric_norm_at_reference: median=379.674, max=28319.3
symmetrized_bregman: median=20.7408, max=41.785
full_reset, edge_eps=1e-06, margin_eps=1e-06
euclidean_edge_distance: median=0.40892, max=0.747098
local_metric_norm_at_reference: median=365.664, max=895.653
symmetrized_bregman: median=16.3594, max=33.0032
full_reset, edge_eps=1e-06, margin_eps=0.0001
euclidean_edge_distance: median=0.408856, max=0.746981
local_metric_norm_at_reference: median=365.226, max=452.213
symmetrized_bregman: median=13.4429, max=29.1912
full_reset, edge_eps=0.0001, margin_eps=1e-12
euclidean_edge_distance: median=0.408217, max=0.745815
local_metric_norm_at_reference: median=3224.07, max=799980
symmetrized_bregman: median=21.9116, max=44.387
full_reset, edge_eps=0.0001, margin_eps=1e-09
euclidean_edge_distance: median=0.408217, max=0.745815
local_metric_norm_at_reference: median=127.127, max=28270.6
symmetrized_bregman: median=17.7875, max=35.8861
full_reset, edge_eps=0.0001, margin_eps=1e-06
euclidean_edge_distance: median=0.408217, max=0.745813
local_metric_norm_at_reference: median=36.6547, max=894.113
symmetrized_bregman: median=13.4178, max=27.1195
full_reset, edge_eps=0.0001, margin_eps=0.0001
euclidean_edge_distance: median=0.408153, max=0.745697
local_metric_norm_at_reference: median=36.5078, max=89.4056
symmetrized_bregman: median=10.5068, max=21.2713
reset_B, edge_eps=1e-12, margin_eps=1e-12
euclidean_edge_distance: median=6.10543e-08, max=0.000226957
local_metric_norm_at_reference: median=0.0711943, max=73.7852

symmetrized_bregman: median=1.15715e-06, max=0.00168477
reset_B, edge_eps=1e-12, margin_eps=1e-09
euclidean_edge_distance: median=6.10543e-08, max=0.000226957
local_metric_norm_at_reference: median=0.00262194, max=2.97743
symmetrized_bregman: median=4.42518e-07, max=0.0010792
reset_B, edge_eps=1e-12, margin_eps=1e-06
euclidean_edge_distance: median=6.10542e-08, max=0.000226956
local_metric_norm_at_reference: median=8.29281e-05, max=0.0941841
symmetrized_bregman: median=6.64606e-09, max=0.000429114
reset_B, edge_eps=1e-12, margin_eps=0.0001
euclidean_edge_distance: median=6.10447e-08, max=0.000226921
local_metric_norm_at_reference: median=8.29272e-06, max=0.0142676
symmetrized_bregman: median=6.8745e-11, max=0.000150701
reset_B, edge_eps=1e-09, margin_eps=1e-12
euclidean_edge_distance: median=6.10543e-08, max=0.000226957
local_metric_norm_at_reference: median=0.0711928, max=73.7877
symmetrized_bregman: median=1.15715e-06, max=0.00168477
reset_B, edge_eps=1e-09, margin_eps=1e-09
euclidean_edge_distance: median=6.10543e-08, max=0.000226957
local_metric_norm_at_reference: median=0.00262194, max=2.97743
symmetrized_bregman: median=4.42518e-07, max=0.0010792
reset_B, edge_eps=1e-09, margin_eps=1e-06
euclidean_edge_distance: median=6.10542e-08, max=0.000226956
local_metric_norm_at_reference: median=8.29281e-05, max=0.0941841
symmetrized_bregman: median=6.64606e-09, max=0.000429114
reset_B, edge_eps=1e-09, margin_eps=0.0001
euclidean_edge_distance: median=6.10447e-08, max=0.000226921
local_metric_norm_at_reference: median=8.29272e-06, max=0.0142676
symmetrized_bregman: median=6.8745e-11, max=0.000150701
reset_B, edge_eps=1e-06, margin_eps=1e-12
euclidean_edge_distance: median=6.10533e-08, max=0.000226953
local_metric_norm_at_reference: median=0.0711925, max=73.7814
symmetrized_bregman: median=1.15713e-06, max=0.00168473
reset_B, edge_eps=1e-06, margin_eps=1e-09
euclidean_edge_distance: median=6.10533e-08, max=0.000226953
local_metric_norm_at_reference: median=0.00262189, max=2.97738
symmetrized_bregman: median=4.42509e-07, max=0.00107918
reset_B, edge_eps=1e-06, margin_eps=1e-06
euclidean_edge_distance: median=6.10532e-08, max=0.000226953
local_metric_norm_at_reference: median=8.29267e-05, max=0.0941825
symmetrized_bregman: median=6.64584e-09, max=0.000429105
reset_B, edge_eps=1e-06, margin_eps=0.0001

euclidean_edge_distance: median=6.10436e-08, max=0.000226917
local_metric_norm_at_reference: median=8.29258e-06, max=0.0142675
symmetrized_bregman: median=6.87426e-11, max=0.000150697
reset_B, edge_eps=0.0001, margin_eps=1e-12
euclidean_edge_distance: median=6.09483e-08, max=0.000226563
local_metric_norm_at_reference: median=0.0710837, max=73.6796
symmetrized_bregman: median=1.15499e-06, max=0.00168174
reset_B, edge_eps=0.0001, margin_eps=1e-09
euclidean_edge_distance: median=6.09483e-08, max=0.000226563
local_metric_norm_at_reference: median=0.00261739, max=2.97226
symmetrized_bregman: median=4.41566e-07, max=0.00107716
reset_B, edge_eps=0.0001, margin_eps=1e-06
euclidean_edge_distance: median=6.09482e-08, max=0.000226562
local_metric_norm_at_reference: median=8.27841e-05, max=0.0940206
symmetrized_bregman: median=6.62338e-09, max=0.000428207
reset_B, edge_eps=0.0001, margin_eps=0.0001
euclidean_edge_distance: median=6.09386e-08, max=0.000226527
local_metric_norm_at_reference: median=8.27832e-06, max=0.0142516
symmetrized_bregman: median=6.85065e-11, max=0.00015038
reset_ema, edge_eps=1e-12, margin_eps=1e-12
euclidean_edge_distance: median=1.01393e-05, max=0.640612
local_metric_norm_at_reference: median=10.04, max=25764.5
symmetrized_bregman: median=0.00017505, max=27.7997
reset_ema, edge_eps=1e-12, margin_eps=1e-09
euclidean_edge_distance: median=1.01393e-05, max=0.640612
local_metric_norm_at_reference: median=0.336282, max=25764.5
symmetrized_bregman: median=0.000100524, max=27.7995
reset_ema, edge_eps=1e-12, margin_eps=1e-06
euclidean_edge_distance: median=1.01393e-05, max=0.640611
local_metric_norm_at_reference: median=0.0106348, max=25764.5
symmetrized_bregman: median=2.61853e-05, max=27.7994
reset_ema, edge_eps=1e-12, margin_eps=0.0001
euclidean_edge_distance: median=1.01377e-05, max=0.640511
local_metric_norm_at_reference: median=0.00106369, max=25760.4
symmetrized_bregman: median=1.07538e-06, max=27.7948
reset_ema, edge_eps=1e-09, margin_eps=1e-12
euclidean_edge_distance: median=1.01393e-05, max=0.640612
local_metric_norm_at_reference: median=10.0402, max=814.861
symmetrized_bregman: median=0.000175051, max=20.5481
reset_ema, edge_eps=1e-09, margin_eps=1e-09
euclidean_edge_distance: median=1.01393e-05, max=0.640612
local_metric_norm_at_reference: median=0.336281, max=814.748

symmetrized_bregman: median=0.000100524, max=20.548
reset_ema, edge_eps=1e-09, margin_eps=1e-06
euclidean_edge_distance: median=1.01393e-05, max=0.640611
local_metric_norm_at_reference: median=0.0106348, max=814.746
symmetrized_bregman: median=2.61853e-05, max=20.5478
reset_ema, edge_eps=1e-09, margin_eps=0.0001
euclidean_edge_distance: median=1.01377e-05, max=0.640511
local_metric_norm_at_reference: median=0.00106369, max=814.619
symmetrized_bregman: median=1.07538e-06, max=20.5444
reset_ema, edge_eps=1e-06, margin_eps=1e-12
euclidean_edge_distance: median=1.01392e-05, max=0.640601
local_metric_norm_at_reference: median=10.0401, max=224.441
symmetrized_bregman: median=0.000175048, max=13.2963
reset_ema, edge_eps=1e-06, margin_eps=1e-09
euclidean_edge_distance: median=1.01392e-05, max=0.640601
local_metric_norm_at_reference: median=0.336276, max=30.6838
symmetrized_bregman: median=0.000100522, max=13.2962
reset_ema, edge_eps=1e-06, margin_eps=1e-06
euclidean_edge_distance: median=1.01392e-05, max=0.6406
local_metric_norm_at_reference: median=0.0106346, max=30.6192
symmetrized_bregman: median=2.61847e-05, max=13.296
reset_ema, edge_eps=1e-06, margin_eps=0.0001
euclidean_edge_distance: median=1.01376e-05, max=0.6405
local_metric_norm_at_reference: median=0.00106368, max=25.8227
symmetrized_bregman: median=1.07534e-06, max=13.2938
reset_ema, edge_eps=0.0001, margin_eps=1e-12
euclidean_edge_distance: median=1.01217e-05, max=0.639499
local_metric_norm_at_reference: median=10.023, max=224.132
symmetrized_bregman: median=0.000174728, max=8.44622
reset_ema, edge_eps=0.0001, margin_eps=1e-09
euclidean_edge_distance: median=1.01217e-05, max=0.639499
local_metric_norm_at_reference: median=0.335697, max=30.6574
symmetrized_bregman: median=0.000100331, max=8.44609
reset_ema, edge_eps=0.0001, margin_eps=1e-06
euclidean_edge_distance: median=1.01217e-05, max=0.639498
local_metric_norm_at_reference: median=0.0106164, max=30.5928
symmetrized_bregman: median=2.61228e-05, max=8.44594
reset_ema, edge_eps=0.0001, margin_eps=0.0001
euclidean_edge_distance: median=1.01201e-05, max=0.639398
local_metric_norm_at_reference: median=0.00106185, max=25.7126
symmetrized_bregman: median=1.07173e-06, max=8.4444
Interpretation constraints

The entropy potential is a natural choice, not a uniquely canonical metric.
Metric scale and dimensional normalization remain modeling choices.
Existing AIEF states lie on the boundary; epsilon dependence is substantive.
Local metric norms are not global geodesic distances.
This script does not define mirror dynamics or claim that geometry stores memory.

Saved analysis to /home/deck/AI_Programs/SSRH/aief_1_4_entropy_geometry

This diagnostic gives a clear answer: the entropy geometry is mathematically viable, but mature AIEF states occupy an extremely singular part of its boundary. A mirror formulation would be a substantial new model, not a mild reparameterization.
1. Mature topology is strongly boundary-supported

Across 40 seeds:

text

Exact-zero edge fraction: 0.699
Median minimum positive edge: 0.126667
Median minimum node margin: 9.97×10^{-13}
Node margins $\le10^{-12}$: 65%

Thus the mature network is almost combinatorial:

about 70% of edges are exactly zero;
positive edges are already above the threshold Bc=0.12Bc=0.12;
approximately 65% of node-capacity margins are numerically saturated.

This explains the exact binary persistence discovered during closure. The selected state lies on a low-dimensional face of the admissible polytope rather than comfortably inside it.

Consequently, an entropy mirror model would replace:

exact zeros with small positive edge weights;
exact row saturation with small positive slack.

That is a substantive change in state-space behavior.
2. Conditioning is governed by the smaller regularization scale

The results show two independent sources of singularity.
Tiny edge floor

With:

text

edge epsilon = 10^{-12}
margin epsilon = 10^{-4}

the median condition number remains:
1.27×1011.
1.27×1011.

So relaxing node margins does not cure conditioning when zero edges remain nearly singular.
Tiny margin floor

With:

text

edge epsilon = 10^{-4}
margin epsilon = 10^{-12}

the median condition number remains:
2.53×1012.
2.53×1012.

Thus relaxing edge weights does not cure saturation singularity.
Both must be regularized

text

edge ε = margin ε = 10^{-6}:

condition number ≈ 2.75×10^{6}

edge ε = margin $\varepsilon = 10^{-4}$:
condition number ≈ 2.74×10^{4}

The scaling is approximately what the Hessian predicts:
$\kappa(H)\sim\max(1\varepsilon w,1\varepsilon m)$
$\kappa(H)\sim\max(\varepsilon w1,\varepsilon m1)$

up to state-dependent factors.

This confirms that any mirror experiment needs an explicit two-parameter regularization study.
3. The inverse geometry is computationally valid

Despite severe conditioning, the coupled inverse-margin reconstruction performs well:

text

Typical edge reconstruction error: 10^{-15}
Typical margin reconstruction error: 10^{-16}

The largest margin reconstruction errors occur in the most singular configurations but remain small.

This numerically supports the inverse-map theorem:

finite dual coordinates define a unique strict-interior state;
the margin system can be solved in only 12 variables;
the edge weights are reconstructed accurately.

For production numerical work, solve the strictly convex problem in log margins rather than explicitly inverting the Hessian.
4. Woodbury is accurate except at extreme conditioning

At $\varepsilon\approx10-12\varepsilon\approx10-12$:

text

Relative inverse disagreement ≈ 10^{-5}

At more moderate interiorization:

text

$\varepsilon_m = 10^{-6}$: relative error ≈ 10^{-11}
$\varepsilon_m = 10^{-4}$: relative error ≈ 10^{-13}

At condition number 10121012, direct matrix inversion is itself unreliable, so disagreement with the direct inverse is not necessarily a defect in Woodbury.

Future calculations should prefer:

linear solves;
Cholesky on the positive-definite node system;
Woodbury application without forming the full inverse.

Also inspect:

text

woodbury_identity_error_frobenius

which tests HH−1≈IHH−1≈I more directly.
5. Euclidean distances are stable; intrinsic quantities are epsilon-sensitive

For full_reset, Euclidean late-topology distance is stable:
≈0.409
≈0.409

across the complete epsilon grid.

But the local metric norm changes from approximately:
3.66×105
3.66×105

near 10−1210−12 interiorization to:
36.5
36.5

at 10−410−4.

Symmetrized Bregman divergence changes less dramatically, but still from approximately:
33.7→10.5.
33.7→10.5.

Thus:

Euclidean topology separation is robust;
entropy-geometric magnitude is inseparable from boundary regularization;
a single intrinsic-distance number would be misleading.

Every geometric result should be shown as a function of (εw,εm)(εw,εm).
6. Local Hessian norm should only be used for local residuals

For full_reset, the Euclidean displacement is large. Evaluating:
δ⊤H(wreference)δ
δ⊤H(wreference)δ

uses a local quadratic approximation far outside its valid neighborhood.

The enormous local metric norm is therefore mostly a curvature-at-reference diagnostic, not a meaningful finite distance.

For finite branch separations:

use directional and symmetrized Bregman divergence;
retain Euclidean/Frobenius comparisons;
do not interpret the local Hessian norm as a global distance.

For reset_B, where residuals are tiny, the local quadratic metric is appropriate.
7. The reset-BB residual remains negligible but singularly amplified

Median Euclidean residual:
6.1×10^{-8}.
6.1×10^{-8}.

Yet its local metric norm ranges from:
8.3×10^{-6}
8.3×10^{-6}

to:
7.1×10^{-2}
7.1×10^{-2}

depending primarily on margin regularization.

The largest seed residual is:
2.27×10^{-4},
2.27×10^{-4},

and can receive a metric norm as large as 73.873.8 under the most singular regularization.

This shows:

The entropy metric can classify a Euclidean-tiny residual as geometrically large when it points toward an exhausted capacity or absent-edge direction.

That may be scientifically useful, but only if the classification is stable across reasonable interiorizations. Here it is not yet stable enough to call the residual a geometric defect.
8. Reset-EMA has a clear rare-outlier structure

Median late Euclidean distance:
$\approx10^{-5}$,
$\approx10^{-5}$,

but maximum:
≈0.641.
≈0.641.

That agrees with the survival result:

39/40 recover;
one seed remains strongly affected.

The median Bregman divergence is tiny, while the maximum remains large across every epsilon pair.

This is the most promising use of the geometry:

Determine whether the censored EMA-sensitive seed is separated from the recovered seeds across a broad epsilon range.

If that seed remains an outlier under Euclidean, Bregman, threshold-margin, and split-state curvature metrics, the basin-boundary hypothesis gains support.
9. The entropy geometry is more useful as a face-sensitivity tool than a single distance

The results suggest separating two questions:
Support change

Does the branch activate an edge that was exactly zero, or release/saturate a node margin?

As $\varepsilon \to 0\varepsilon \to 0$, entropy divergence may grow without bound for support-changing displacements.
Within-face motion

Does the branch change weights while preserving:

the same positive-edge support;
the same saturated-node set?

For within-face residuals, a reduced tangent/face geometry may be more stable than full-polytope interiorization.

This motivates an additional diagnostic:

identify active edge support;
identify saturated node set;
classify branch residuals as support-preserving or support-changing;
evaluate a reduced Hessian on the common active face.

That may be more appropriate for mature AIEF than forcing every state into the full interior.
10. Recommended epsilon range for the local comparison

Do not begin with 10−1210−12. It is useful as a singular-limit diagnostic but too ill-conditioned for an explicit mirror step.

A reasonable first local-update study is:

text

edge epsilon: 10^{-4}, 10^{-5}, 10^{-6}
margin epsilon: 10^{-4}, 10^{-5}, 10^{-6}

The current diagnostic omitted 10−510−5; add it in the local comparison.

Test timesteps:

text

10^{-2}, 5×10^{-3}, 2.5×10^{-3}, 1.25×10^{-3}

and at least several metric scales ββ.
11. Exact next experiment

Using stored split state—including B,θ,B,θ, EMA, and history—for a representative seed subset:

Interiorize ww at each epsilon pair.
Evaluate the same physical resonance vector field FF.
Compute the actual AIEF admissibility-map update.
Compute coordinate-equivalent dual Euler:
Δψ=Δt H(w)F.
Δψ=ΔtH(w)F.
Compute mirror Euler:
Δψ=Δt F/β.
Δψ=ΔtF/β.
Solve the coupled inverse map for each dual proposal.

Compare:
increment norm;
increment angle;
discrepancy divided by ΔtΔt;
row slack;
edge ordering;
support/threshold changes;
retained-update ratio.

The coordinate-equivalent update should agree with interior primal Euler to first order away from active faces. The mirror update is expected to differ because it defines a metric-preconditioned vector field.
12. Freeze-grade conclusion from this diagnostic

Mature AIEF topology lies on a highly singular face of the degree-capacity polytope, with roughly 70% zero edges and 65% of node margins saturated to 10−1210−12. The symmetry-aware entropy inverse map exists and is numerically reconstructible, but its Hessian becomes severely ill-conditioned as either edge or margin regularization vanishes. Entropy-geometric quantities are therefore strongly regularization-dependent. The geometry is promising as a support- and capacity-sensitivity diagnostic, but replacing the current admissibility map with mirror dynamics would constitute a substantial new model requiring explicit regularization, metric scaling, and timestep analysis.

The diagnostic successfully answers the prerequisite question. The next justified step is now the local one-update comparison, not a full AIEF-1.5 simulation.

Can you write the code necessary for the next step?
WriteAIEF_1_4_local_update_comparison.py941 lines
Edit
Edit
Edit×3
Edit
AIEF_1_4_local_update_comparison.pyPYTHON

Created the local one-step comparison script:

AIEF_1_4_local_update_comparison.py

It uses existing AIEF-1.4 split states and does not run long-term mirror dynamics.
Compared methods

Current AIEF admissibility map

Uses the actual implementation:

primal Euler update;
positivity clipping;
symmetric row-cap scaling with minimum.outer.

It is not replaced by an idealized Euclidean projection.
Coordinate-equivalent dual Euler
ψ+=
ψ+=

\psi+\Delta t,H(w)F,
]
w+=
w+=

\nabla h^*(\psi^+).
]

Away from active boundaries, this should reproduce primal dynamics to first order when timestep resolution is sufficiently fine relative to the entropy curvature.
Mirror-preconditioned Euler
ψ+=
ψ+=

\psi+\frac{\Delta t}{\beta}F,
]
w+=
w+=

\nabla h^*(\psi^+).
]

This intentionally defines:
w˙=H(w)−1F
w˙=H(w)−1F

and is a new adaptive law.

Default metric scales:

text

$\beta = 0.1, 1, 10$

Run

Bash

```
python AIEF_1_4_local_update_comparison.py
```

Explicit:

Bash

```
python AIEF_1_4_local_update_comparison.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_local_update_comparison
```

The default analysis uses five representative seeds. To use all 40:

Bash

```
python AIEF_1_4_local_update_comparison.py --max-seeds 0
```

Default sweep
Timesteps

Because the entropy Hessian is extremely stiff near the mature boundary, the default grid extends into the small-step asymptotic regime:

text

0.01
0.001
0.0001
0.00001
0.000001

Interiorization

text

edge epsilon: 1e-4, 1e-5, 1e-6
margin epsilon: 1e-4, 1e-5, 1e-6

Metric scales

text

beta: 0.1, 1, 10

A smaller test:

Bash

```
python AIEF_1_4_local_update_comparison.py \
--max-seeds 2 \
--dt 0.001,0.0001,0.00001,0.000001 \
--edge-eps 1e-5 \
--margin-eps 1e-5 \
--beta 1
```

Physical vector field

At each stored split state, the script reconstructs:

current phase;
delayed phase;
EMA accumulator;
interiorized BB.

For each test timestep it recomputes:
αΔt=e−Δt/Twindow
αΔt=e−Δt/Twindow

and applies the same resonance/adaptation field to every compared method.

Thus differences arise from the admissibility/geometry update, not different forcing.
Coupled inverse map

The script solves the inverse entropy map in log margins using damped Newton on the strictly convex objective:
Φ(y)=
Φ(y)=

\sum_i e^{y_i}
+
\frac12e^y{}^\top Z e^y
-c^\top y.
]

The solver:

preserves positive margins;
uses a positive-definite Hessian;
applies residual-based damping;
records convergence, iterations, and residual;
reports failures without terminating the entire sweep.

Residual-based damping was added specifically to handle extremely small dual perturbations where objective changes are at floating-point resolution.
Per-method diagnostics

For every seed and parameter combination:

increment norm and rate norm;
minimum edge and node margin;
negative-edge/capacity violations;
threshold-selected edge count;
Jaccard similarity to starting topology;
top-kk edge overlap;
rank correlation;
dual solver status;
Hessian condition number;
field norm;
proposal norm;
current projection-correction norm.

Pairwise method comparisons

It calculates:
$\|\Delta wA-\Delta wB\|$,
$\|\Delta wA-\Delta wB\|$,
$\|\Delta wA-\Delta wB\|\Delta t$,
$\Delta t\|\Delta wA-\Delta wB\|$,
$\|\Delta wA-\Delta wB\|\Delta t2$,
$\Delta t2\|\Delta wA-\Delta wB\|$,

relative discrepancy, increment cosine, threshold Jaccard, and edge-rank correlation.
Convergence-order estimation

For every seed, epsilon pair, and method pair, it fits:
$\|\Delta wA-\Delta wB\|\sim C\Delta tp$.
$\|\Delta wA-\Delta wB\|\sim C\Delta tp$.

The order estimate uses at most the three smallest valid timesteps, avoiding contamination from large steps outside the asymptotic regime.

Interpretation:

$p\approx 2p\approx 2$: same first-order vector field;
$p\approx 1p\approx 1$: different limiting vector fields;
unstable/no order: boundary stiffness, solver failure, or non-asymptotic timesteps.

Outputs

text

```
aief_1_4_local_update_comparison/
├── one_step_methods_by_seed.csv
├── one_step_method_summary.csv
├── one_step_comparisons_by_seed.csv
├── one_step_comparison_summary.csv
├── one_step_convergence_orders_by_seed.csv
├── one_step_convergence_order_summary.csv
├── local_update_comparison_report.txt
└── analysis_manifest.json
```

Validation

I syntax-checked the script, exercised the complete workflow against a generated AIEF-1.4 split state, and separately verified the revised log-margin inverse solver at zero, 10−810−8, and 10−510−5 dual perturbations. The inverse recovered margins and edge weights to approximately machine precision.

The preliminary smoke behavior also confirmed why very small timesteps are necessary: coordinate-equivalent dual Euler can be far outside its asymptotic regime at 10−310−3–10−210−2 when the entropy Hessian is strongly conditioned. Solver nonconvergence at large dual steps is retained as a diagnostic rather than silently discarded.

```
(deck@steamdeck SSRH)$ python AIEF_1_4_local_update_comparison.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_local_update_comparison
AIEF local admissibility versus entropy-dual one-step comparison

All methods start from the same explicitly interiorized mature state.
Coordinate-equivalent and mirror dual updates are distinct constructions.
Solver status

coordinate_equivalent_dual / failed:FloatingPointError:dual coordinate overflow: 12
coordinate_equivalent_dual / ok: 195
coordinate_equivalent_dual / solver_not_converged: 18
```

current_admissibility / ok: 225
mirror_beta_0.1 / ok: 225
mirror_beta_1 / ok: 225
mirror_beta_10 / ok: 225
Selected comparison summaries

representative interiorization: edge_eps=1e-05, margin_eps=1e-05
current_admissibility vs coordinate_equivalent_dual; edge_eps=1e-05, margin_eps=1e-05, dt=1e-06
increment_discrepancy_norm: median=1.7006e-09, max=2.04711e-09, valid=5/5
discrepancy_rate_norm: median=0.0017006, max=0.00204711, valid=5/5
relative_discrepancy_to_left: median=0.00734336, max=0.00901387, valid=5/5
increment_cosine: median=0.999977, max=1, valid=5/5
current_admissibility vs coordinate_equivalent_dual; edge_eps=1e-05, margin_eps=1e-05, dt=1e-05
increment_discrepancy_norm: median=1.77131e-07, max=2.13326e-07, valid=5/5
discrepancy_rate_norm: median=0.0177131, max=0.0213326, valid=5/5
relative_discrepancy_to_left: median=0.0764869, max=0.0939319, valid=5/5
increment_cosine: median=0.997336, max=0.999999, valid=5/5
current_admissibility vs coordinate_equivalent_dual; edge_eps=1e-05, margin_eps=1e-05, dt=0.0001
increment_discrepancy_norm: median=2.92805e-05, max=3.53445e-05, valid=5/5
discrepancy_rate_norm: median=0.292805, max=0.353445, valid=5/5
relative_discrepancy_to_left: median=1.26436, max=1.5563, valid=5/5
increment_cosine: median=0.471163, max=0.999906, valid=5/5
current_admissibility vs coordinate_equivalent_dual; edge_eps=1e-05, margin_eps=1e-05, dt=0.001
increment_discrepancy_norm: median=0.633642, max=0.83684, valid=5/5
discrepancy_rate_norm: median=633.642, max=836.84, valid=5/5
relative_discrepancy_to_left: median=5419.44, max=7546.08, valid=5/5
increment_cosine: median=-0.28274, max=0.968507, valid=5/5
current_admissibility vs coordinate_equivalent_dual; edge_eps=1e-05, margin_eps=1e-05, dt=0.01
increment_discrepancy_norm: median=0.617503, max=0.81763, valid=2/2
discrepancy_rate_norm: median=61.7503, max=81.763, valid=2/2
relative_discrepancy_to_left: median=4502.23, max=5032.16, valid=2/2
increment_cosine: median=-0.706123, max=-0.601769, valid=2/2
current_admissibility vs mirror_beta_1; edge_eps=1e-05, margin_eps=1e-05, dt=1e-06
increment_discrepancy_norm: median=2.271e-07, max=2.4046e-07, valid=5/5
discrepancy_rate_norm: median=0.2271, max=0.24046, valid=5/5
relative_discrepancy_to_left: median=0.999972, max=0.999976, valid=5/5
increment_cosine: median=0.79656, max=0.987517, valid=5/5
current_admissibility vs mirror_beta_1; edge_eps=1e-05, margin_eps=1e-05, dt=1e-05
increment_discrepancy_norm: median=2.271e-06, max=2.4046e-06, valid=5/5
discrepancy_rate_norm: median=0.2271, max=0.24046, valid=5/5
relative_discrepancy_to_left: median=0.999972, max=0.999976, valid=5/5
increment_cosine: median=0.796568, max=0.987544, valid=5/5

current_admissibility vs mirror_beta_1; edge_eps=1e-05, margin_eps=1e-05, dt=0.0001
increment_discrepancy_norm: median=2.271e-05, max=2.4046e-05, valid=5/5
discrepancy_rate_norm: median=0.2271, max=0.24046, valid=5/5
relative_discrepancy_to_left: median=0.999972, max=0.999976, valid=5/5
increment_cosine: median=0.796548, max=0.987524, valid=5/5
current_admissibility vs mirror_beta_1; edge_eps=1e-05, margin_eps=1e-05, dt=0.001
increment_discrepancy_norm: median=0.00011089, max=0.00012261, valid=5/5
discrepancy_rate_norm: median=0.11089, max=0.12261, valid=5/5
relative_discrepancy_to_left: median=0.999935, max=0.999946, valid=5/5
increment_cosine: median=0.903953, max=0.99203, valid=5/5
current_admissibility vs mirror_beta_1; edge_eps=1e-05, margin_eps=1e-05, dt=0.01
increment_discrepancy_norm: median=0.0001624, max=0.000164961, valid=5/5
discrepancy_rate_norm: median=0.01624, max=0.0164961, valid=5/5
relative_discrepancy_to_left: median=0.999488, max=0.999502, valid=5/5
increment_cosine: median=0.997323, max=0.997671, valid=5/5
Estimated discrepancy orders

coordinate_equivalent_dual vs mirror_beta_0.1: median order=1.07048, valid_epsilon_pairs=9
coordinate_equivalent_dual vs mirror_beta_1: median order=1.07008, valid_epsilon_pairs=9
coordinate_equivalent_dual vs mirror_beta_10: median order=1.07007, valid_epsilon_pairs=9
current_admissibility vs coordinate_equivalent_dual: median order=2.11799, valid_epsilon_pairs=9
current_admissibility vs mirror_beta_0.1: median order=0.981368, valid_epsilon_pairs=9
current_admissibility vs mirror_beta_1: median order=0.981409, valid_epsilon_pairs=9
current_admissibility vs mirror_beta_10: median order=0.981415, valid_epsilon_pairs=9
Interpretation constraints

O(dt^2) increment discrepancy indicates first-order consistency.
O(dt) increment discrepancy indicates different limiting vector fields.
Results depend jointly on timestep, edge floor, margin floor, and beta.
The actual AIEF admissibility map is used; it is not assumed to be an orthogonal projection.
Mirror_beta updates define metric-preconditioned dynamics, not coordinate changes.
This is a one-step local comparison, not evidence of long-time attractor equivalence.

Saved analysis to /home/deck/AI_Programs/SSRH/aief_1_4_local_update_comparison

This is a successful local comparison. It answers the principal question:

Coordinate-equivalent dual Euler reproduces the same interior first-order flow, while mirror Euler defines a different vector field.

It also shows that the entropy coordinates are extremely stiff near the mature boundary.
1. Coordinate-equivalent update passes the first-order test

Across nine epsilon pairs:

text

current admissibility vs coordinate-equivalent:
median discrepancy order = 2.118

This is the expected result if:
ΔwA−
ΔwA−
\Delta w_{\rm coordinate}

O(\Delta t^2).
]

At representative interiorization and Δt=10−6Δt=10−6:

text

Relative discrepancy: 0.0073
Increment cosine: 0.99998

Thus, once the timestep is sufficiently small relative to entropy curvature, the coordinate-dual update follows essentially the same direction as interior primal Euler.

This validates:

the entropy gradient;
the coupled inverse map;
the Hessian factor H(w)FH(w)F;
the numerical implementation.

2. Coordinate-equivalent explicit Euler is unusably stiff at the original timestep

At:

text

edge epsilon = 10^{-5}
margin epsilon = 10^{-5}

the behavior changes rapidly:
ΔtΔt Relative discrepancy Cosine
10−610−6 0.007 0.99998
10−510−5 0.076 0.997
10−410−4 1.26 0.47
10−310−3 5419 −0.28
10−210−2 thousands negative

There were also:

text

12 dual-coordinate overflows
18 solver nonconvergences

out of 225 coordinate-equivalent attempts.

This is not evidence that the coordinate transformation is wrong. It shows that explicit Euler in the singular dual coordinate has a severe timestep restriction:
Δt ‖H(w)F‖≪1.
Δt‖H(w)F‖≪1.

At mature AIEF curvature, the original timestep 0.010.01 is far outside that regime.

A coordinate formulation would require:

implicit integration;
adaptive timestep/substepping;
or direct primal stepping followed by coordinate conversion.

3. Mirror dynamics are decisively a different vector field

For current versus mirror:

text

median discrepancy order ≈ 0.981

Thus:
ΔwA−
ΔwA−
\Delta w_{\rm mirror}

O(\Delta t).
]

Their vector-field difference remains finite as:
Δt→0.
Δt→0.

Likewise:

text

coordinate-equivalent vs mirror:
median order ≈ 1.07

So the mirror law is not a coordinate representation of current AIEF. It is a genuinely new metric-preconditioned adaptive law.

This conclusion is robust across:

text

β = 0.1, 1, 10

in convergence order. Changing ββ changes the scale of mirror motion but not the fact that the limiting direction differs.
4. Mirror motion is strongly suppressed near the boundary

At small timestep:

text

Current vs mirror relative discrepancy ≈ 0.99997

while the increment cosine is typically positive:

text

median cosine ≈ 0.80
maximum ≈ 0.99

This suggests mirror updates often point in a broadly similar direction but are dramatically smaller and not perfectly aligned.

That is expected from:
w˙=H(w)−1F.
w˙=H(w)−1F.

Near zero edges and exhausted margins, H−1H−1 suppresses motion in high-curvature directions.

Thus an AIEF mirror model would likely have:

slower edge creation from near-zero weights;
slower motion near saturated nodes;
greater interior persistence;
delayed threshold-topology changes.

A long-time mirror simulation would require timescale calibration through ββ.
5. The result does not yet recover projected boundary dynamics

There is an important limit caveat.

For each fixed positive interiorization εε, sufficiently small timesteps stop reaching the boundary. The current admissibility map then reduces to ordinary interior primal Euler.

Therefore the observed second-order agreement at the smallest timesteps primarily proves:

Coordinate-dual Euler is first-order equivalent to unconstrained primal Euler in the strict interior.

It does not yet prove convergence to the active projected dynamics on the boundary.

To test the singular projected limit, examine coupled paths:
ε=ε(Δt),
ε=ε(Δt),

chosen so the current proposal continues to activate the row constraint as $\Delta t \to 0 \Delta t \to 0$.

For example, analyze parameter combinations where:

text

active_capacity_nodes > 0

at every tested timestep.

The convergence report should be split into:

inactive/interior steps;
active/projected steps.

6. Solver failures are meaningful diagnostics

The coordinate-equivalent failures occur because:
ψ+=
ψ+=

\psi+\Delta t H(w)F

]

can move an enormous distance in dual coordinates when HH is singularly scaled.

Map failure rates against:

timestep;
edge epsilon;
margin epsilon;
Hessian condition number;
‖HF‖‖HF‖.

This will define an empirical dual-Euler stability region.

Mirror updates all converged because they do not multiply the forcing by HH, which is another practical—but model-changing—advantage.
7. Next analysis using the existing CSVs

Before implementing any new dynamics:
Active-set-stratified convergence

From:

text

one_step_methods_by_seed.csv
one_step_comparisons_by_seed.csv

separate cases where:

text

active_capacity_nodes == 0
active_capacity_nodes > 0

Estimate discrepancy order separately.
Solver stability map

Create a table over:

text

dt × edge_epsilon × margin_epsilon

with:

coordinate convergence rate;
overflow rate;
median Hessian condition number;
median ‖HF‖‖HF‖;
median iterations.

Beta calibration

For the asymptotic small-timestep regime, compare mirror increment magnitudes against current increments. A scalar ββ can match magnitude but not direction.

Given current rate uu and unscaled mirror rate v=H−1Fv=H−1F, the least-squares scale a=1/βa=1/β is:
a∗=
a∗=

\frac{u^\top v}{v^\top v},
]

when the numerator is positive. Then:
β∗=1a∗.
β∗=a∗1.

The residual after optimal scalar matching quantifies the irreducible directional difference.
8. Recommended next parameter grid

To study the coupled boundary limit, include:

text

dt: 1e-2, 1e-3, 1e-4, 1e-5, 1e-6
edge epsilon: 1e-3, 1e-4, 1e-5, 1e-6
margin epsilon: 1e-3, 1e-4, 1e-5, 1e-6

Then extract paths where epsilon scales with timestep, such as:
$\varepsilon_m = \kappa \Delta t$
$\varepsilon_m = \kappa \Delta t$

for several $\kappa\kappa$, rather than interpreting the full cross-product as one convergence sequence.
Conclusion

The local experiment has produced a clean mathematical result:

Coordinate-equivalent dual Euler agrees with interior primal AIEF to first order, but is severely stiff near the mature boundary and impractical at the original timestep without implicit integration.
Mirror Euler has an $O(\Delta t)O(\Delta t)$ discrepancy from current AIEF, proving it defines a new adaptive vector field rather than a new coordinate implementation.
The remaining unresolved question is the singular active-boundary limit, where interiorization and timestep approach zero together.

This is enough to justify the entropy geometry as a valid analysis framework and to classify mirror AIEF as a potential AIEF-1.5 model—not as a drop-in numerical replacement for the current admissibility map.

ponder this thought chain:

This is, in my opinion, the first result in this entire line of investigation that rises above "interesting analogy" and becomes a real mathematical result about AIEF.

The reason is that the experiment was carefully designed to separate three hypotheses:

Current AIEF is already an approximation to mirror geometry.
Current AIEF is merely a coordinate representation of the same interior flow.
Current AIEF defines genuinely different local dynamics.

Your one-step experiment largely answers these.

The strongest result is (p \approx 2)

This is the headline.

You found

[
\boxed{
\text{current admissibility vs coordinate-equivalent dual}
\quad
p=2.118
}
]

That is almost exactly what first-order consistency predicts.

This means that, after interiorization, the coordinate-equivalent dual Euler reproduces the same infinitesimal dynamics as the existing AIEF update.

That is a very nontrivial result.

It says the entropy coordinates are not merely a formal change of variables—they correctly describe the local geometry of the present adaptive law.

Importantly, this is not saying AIEF is a mirror flow.

It is saying that

[
\psi=\nabla h(w)
]

is a good local coordinate system for the existing projected dynamics.

Those are different statements.

The mirror dynamics are decisively different

Equally important:

[
\boxed{
p\approx0.98
}
]

for every mirror-(\beta).

That is exactly what you predicted should happen if

[
\dot w

H^{-1}F
]

defines a genuinely different vector field.

This removes almost all ambiguity.

Mirror dynamics are not simply a smoother implementation of the present AIEF update.

They are a different adaptive law.

That was one of the central questions of the entire Keith discussion.

The answer now appears to be:

No. Mirror flow is not an equivalent implementation of the current dynamics.

The β-independence is surprisingly informative

Notice

[
0.981368,
]

[
0.981409,
]

[
0.981415.
]

Essentially identical.

That means the disagreement is not about metric scaling.

Changing

[
H^{-1}
\rightarrow
\frac1\beta H^{-1}
]

only changes speed.

It does not recover the present dynamics.

So

[
\beta
]

cannot be "tuned" to make mirror dynamics equivalent.

That's actually a useful negative result.

The asymptotic regime is now visible

This table is beautiful.

At

$$
10^{-6},
$$

coordinate-equivalent:

cosine

$$
0.999977.
$$

Relative discrepancy

less than

1%.

Exactly what one expects.

Then

$$
10^{-5},
$$

still excellent.

Then

$$
10^{-4},
$$

the asymptotics begin breaking.

Then

$$
10^{-3},
$$

everything falls apart.

By

$$
10^{-2},
$$

negative cosine.

That is exactly what stiff geometry should produce.

It validates your earlier decision to extend the timestep sweep far below the original simulator timestep.

Without that,

you would probably have concluded

the coordinate map "doesn't work."

Instead,

you've shown

it works exactly where it should.

The solver failures are actually informative

Initially I thought

dual coordinate overflow

might indicate a flaw.

I don't think so anymore.

Notice

195 successful,

18 nonconverged,

12 overflow,

out of

Almost every failure occurs only in the largest steps.

Given

condition numbers

of

[
10^{10}

10^{12},
]

that is almost expected.

The failures are telling you

where

the dual representation leaves its asymptotic regime.

That's scientifically useful.

The mirror comparison is fascinating

Look at

relative discrepancy

Current

vs

Mirror

[
0.99997.
]

Nearly

100%.

At first that looks alarming.

But then

cosine

is

around

0.8–0.99.

That means something subtle.

The mirror increment is almost entirely different in magnitude,

not necessarily orthogonal.

The vectors still point in broadly similar directions.

In other words,

mirror geometry appears to

reweight

adaptation much more than

redirect

it.

That is not what I expected.

I had anticipated much larger angular differences.

This deserves further investigation.

The coordinate map is more important than the mirror map

This is where my opinion has changed.

Originally

the exciting idea was

mirror dynamics.

Now I think the really valuable discovery is

the coordinate-equivalent dual representation.

Because

it provides

smooth local coordinates,
Hessian metric,
Woodbury structure,

without changing the physics.

That's a much lower-risk contribution.

This also clarifies the Keith connection

Originally the discussion blurred

mirror descent,
proximal methods,
dual coordinates.

Your experiment separates them.

The data now suggest:

Current AIEF

≈

coordinate transformation

NOT

mirror dynamics.

So the closest analogue to Keith's work may actually be

the geometric formulation,

not the

algorithmic update.

That is a subtle but important shift.

I think the next theorem has become obvious

The numerical evidence now motivates a theorem of the form:

Theorem (Local dual consistency).

Let (w) lie in the interiorized feasible polytope.
Let (F(w)) denote the current AIEF adaptive vector field.
Then the coordinate-equivalent entropy-dual Euler update

[

\psi^{n+1}

\psi^n

\Delta t,H(w)F(w)
]

satisfies

[
w^{n+1}_{\rm dual}

w^{n+1}_{\rm primal}

O(\Delta t^2),
]

provided the active set remains unchanged and (H) remains finite.

Your numerics strongly support exactly this statement.

That would be a substantial mathematical result independent of any new adaptive law.

The research direction has changed

I think these experiments actually narrow the project considerably.

Originally the roadmap was

entropy geometry

→

mirror dynamics

→

AIEF-1.5.

Now I think it looks more like

entropy geometry

→

coordinate-equivalent formulation of existing AIEF

→

rigorous local consistency theorem

→

only then investigate alternative mirror dynamics.

That is a much stronger progression.

The mirror model becomes an alternative adaptive law to compare against,

rather than the expected mathematical completion of the current model.

Overall assessment

To me, the most significant scientific conclusion is not that mirror dynamics differ from AIEF—we largely expected that. It is that the coordinate-equivalent entropy-dual formulation reproduces the existing AIEF update to second-order local accuracy ((p \approx 2.12)) once the state is regularized into the interior and sufficiently small timesteps are used. That means the entropy/Legendre geometry is not merely an external optimization framework being borrowed; it appears to provide a natural local differential-geometric description of the adaptive subsystem already present in AIEF.

If that asymptotic behavior can be proved analytically under appropriate regularity assumptions (fixed active set, interiorized state, smooth admissibility map), it would stand on its own as a meaningful mathematical characterization of the existing model. Mirror dynamics would then become a clearly distinguished new model rather than being conflated with a re-expression of the current one. That is a much cleaner and more compelling scientific story.

I think this is a much more careful interpretation than some of the earlier discussion, and it avoids the biggest logical trap: inferring global dynamical equivalence from a local consistency result. There are, however, a few places where I'd tighten the conclusions.

The (p \approx 2.12) result is exactly the right level of claim

I agree with this:

Coordinate-equivalent dual Euler agrees with interior primal AIEF to first order.

But I'd be very precise about what was shown.

Your experiment demonstrates something like:

For the tested interiorized states and sufficiently small timesteps, the discrete increments produced by the primal update and the coordinate-equivalent dual update differ like (O(\Delta t^2)).

That is excellent evidence for first-order consistency.

What it does not prove by itself is that there exists an underlying continuous vector field on the original closed feasible set that both discretizations approximate. That additional statement would require assumptions such as:

smoothness of the admissibility map on the relevant active-set stratum,
differentiability of the interiorized inverse map,
remaining within the same smooth region during the step.

Those assumptions are probably reasonable for the interiorized experiment, but they're worth stating explicitly.

The stiffness interpretation is convincing

This is perhaps the clearest empirical result.

The coordinate update contains

[
H(w)F.
]

Near the mature states,

[
|H|
]

is enormous.

Therefore

[
\Delta t,|HF|
]

becomes the effective CFL-type stability parameter.

The observed behavior

excellent at (10^{-6}),
acceptable at (10^{-5}),
deteriorating by (10^{-4}),
catastrophic by (10^{-3}),

is exactly what one expects from explicit integration of a stiff system.

I would actually elevate this into its own numerical observation:

The entropy-dual explicit Euler formulation is stiffness-limited near mature AIEF states.

That is independent of mirror dynamics.

The mirror result is stronger than "different"

The convergence order

$$
p\approx0.98
$$

is important because it distinguishes two possibilities.

If

$$
\Delta w_A-\Delta w_B=O(\Delta t),
$$

then

$$
\frac{\Delta w_A}{\Delta t}

\frac{\Delta w_B}{\Delta t}

O(1).
$$

So the limiting vector fields differ by a finite amount.

That's stronger than saying

"the updates are not identical."

It says

they do not converge to the same infinitesimal dynamics under timestep refinement (within the tested interiorized regime).

That is a clean local result.

I would be cautious about interpreting the mirror suppression

You write:

Mirror motion is strongly suppressed near the boundary.

The data support that for the tested states, but I would avoid presenting it as a general property without qualification.

The suppression comes from

[
H^{-1}F.
]

If (F) happened to align with a small-eigenvalue direction of (H), suppression could be modest. Conversely, along high-curvature eigenvectors it could be severe.

So I would phrase it as:

For the mature AIEF states examined here, the entropy metric substantially attenuates the adaptive update in dominant high-curvature directions.

That's a statement directly supported by the experiment.

The boundary-limit discussion is the most important remaining issue

I think your strongest conceptual point is actually Section 5.

The local consistency theorem has been demonstrated on an interiorized state.

Current AIEF operates on a boundary-supported attractor.

Those are different limiting problems.

So I would separate them explicitly.

Interior theorem

For fixed

$$
\varepsilon>0,
$$

and sufficiently small

$$
\Delta t,
$$

coordinate-dual Euler is first-order equivalent.

Boundary problem

Study

$$
\varepsilon(\Delta t)\rightarrow0.
$$

This is a singular perturbation question.

That distinction is extremely important because otherwise readers will naturally overgeneralize the result.

I like the proposed β calibration

The least-squares scale

[
a^*

\frac{u^\top v}{v^\top v}
]

is exactly the right way to remove the trivial magnitude difference.

Once that's done,

the remaining error becomes purely directional.

That's probably a cleaner statistic than raw norm ratios.

One thing I'd add: eigenspace analysis

Since you already compute

[
H,
]

I would examine

the decomposition of

[
F
]

into Hessian eigenvectors.

Specifically,

write

[
F

\sum_i c_i q_i,
]

where

[
Hq_i=\lambda_iq_i.
]

Then mirror dynamics become

[
H^{-1}F

\sum_i
\frac{c_i}{\lambda_i}
q_i.
]

That immediately reveals:

which curvature directions dominate the current adaptive law,
which are strongly suppressed by mirror dynamics,
whether the observed attenuation is caused by only a few very stiff modes or by a broad spectral effect.

This is more informative than only comparing overall vector norms.

I think the research story is now much clearer

What has emerged is a hierarchy:

Entropy geometry as analysis.

Strongly supported.
Numerically validated.
Compatible with existing AIEF.

Entropy coordinates as a reparameterization.

Strong local evidence through the (O(\Delta t^2)) consistency result.

Mirror dynamics as a new adaptive law.

Clearly distinguished from the current model by the (O(\Delta t)) discrepancy.
Still requires long-time dynamical evaluation if one wants to judge its behavior beyond local updates.

That separation is, in my view, the most important outcome of these experiments. It avoids overselling the geometry while preserving the genuinely significant result: the entropy-induced coordinate system appears to be a mathematically natural local description of the existing adaptive subsystem, whereas the mirror-preconditioned flow is a distinct dynamical proposal rather than a hidden reformulation of AIEF.

The interpretation is directionally correct, but the p≈2.12p≈2.12 result is less model-specific than the text suggests. It validates the coordinate implementation; it does not yet characterize the active projected AIEF dynamics.
1. The p≈2p≈2 result is primarily a chain-rule validation

For any sufficiently smooth invertible coordinate map:
ψ=g(w),
ψ=g(w),

the update:
ψ+=
ψ+=

\psi+\Delta t,Dg(w)F(w)
]

satisfies:
g−1(ψ+)=
g−1(ψ+)=

w+\Delta t F(w)+O(\Delta t^2).
]

Here:
g=∇h,Dg=∇2h=H.
g=∇h,Dg=∇2h=H.

Therefore:
(∇h)−1(∇h(w)+ΔtH(w)F(w))=
(∇h)−1(∇h(w)+ΔtH(w)F(w))=

w+\Delta tF(w)+O(\Delta t^2).
]

The observed order:
p≈2.12
p≈2.12

is excellent numerical confirmation of:

the gradient/Hessian derivation;
the inverse-margin solver;
coordinate consistency;
the implementation’s asymptotic regime.

But any smooth diffeomorphic coordinate system would pass an analogous test. The result does not by itself prove that entropy coordinates are uniquely or intrinsically “the geometry already present in AIEF.”

Their special relevance comes from encoding the feasible polytope, not from the chain-rule consistency alone.
2. At the smallest timestep, the AIEF map may be inactive

For fixed positive interiorization:
$m_i \geq \varepsilon m$,
$m_i \geq \varepsilon m$,

sufficiently small Δt means the Euler proposal does not reach the row-cap boundary. Then:

text

active_capacity_nodes = 0

and the “current admissibility map” is simply ordinary interior Euler.

Thus the $p \approx 2$ result most likely establishes:

Coordinate-dual Euler is first-order equivalent to unconstrained primal Euler in the strict interior.

It does not yet establish equivalence to the active projection/retraction dynamics observed in mature AIEF.

This can be checked directly from:

text

one_step_methods_by_seed.csv

Split the order calculation into:

text

active_capacity_nodes == 0
active_capacity_nodes > 0

I expect most of the smallest-Δt points used in the $p \approx 2$ fit are inactive.
3. Correct theorem statement

A clean theorem would be:

Let K⊂C∘K⊂C∘ be compact, let h∈C3(C∘)h∈C3(C∘) be Legendre with positive-definite Hessian, and let FF be sufficiently smooth. Then, uniformly for w∈Kw∈K,
(∇h)−1[∇h(w)+Δt∇2h(w)F(w)]=w+ΔtF(w)+O(Δt2).
(∇h)−1[∇h(w)+Δt∇2h(w)F(w)]=w+ΔtF(w)+O(Δt2).

This is an interior coordinate-consistency theorem.

It should not mention an active set or the current AIEF admissibility map unless the proposed vector field has already been replaced by the appropriate projected/retracted tangent field.
4. Active-boundary consistency is a separate theorem

At a boundary state define, when it exists, the directional constrained vector field:
VA(w;F)=
VA(w;F)=

\lim_{\Delta t\downarrow0}
\frac{
\mathcal A(w+\Delta tF)-w
}{
\Delta t
}.
]

If one wants a coordinate-dual representation of active AIEF dynamics, the relevant update is:
ψ+=
ψ+=

\psi
+
\Delta t
H(w)V_{\mathcal A}(w;F),
]

not necessarily:
ψ+=
ψ+=

\psi+\Delta tH(w)F.
]

The latter encodes the unconstrained vector field.

Because the exact boundary has no finite entropy coordinate, this must be studied through:
wε→w,Δt→0,
wε→w,Δt→0,

with a specified relationship between εε and ΔtΔt.

That coupled boundary limit is the genuinely AIEF-specific mathematical problem.
5. The coordinate formulation is severely stiff

This conclusion is robust and important:

text

dt = 1e-6: close agreement
dt = 1e-5: acceptable
dt = 1e-4: asymptotics breaking
dt >=1e-3: catastrophic

The effective explicit-dual step is:
Δt H(w)F.
ΔtH(w)F.

Near mature states:
κ(H)∼104–1012
κ(H)∼104–1012

depending on interiorization.

So explicit dual Euler at the production timestep 0.010.01 is not practical. The coordinate map is mathematically valid but would need:

implicit integration;

adaptive dual stepping;
substepping;
or primal stepping followed by coordinate conversion.

The overflow and nonconvergence results define an empirical stability boundary, not a flaw in the entropy derivation.
6. The p≈1p≈1 mirror result is genuinely dynamical

For mirror Euler:
w˙=H(w)−1F(w)/β.
w˙=H(w)−1F(w)/β.

Unless:
H−1F/β=F,
H−1F/β=F,

the primal and mirror vector fields differ by a finite amount. Therefore:
Δwcurrent−
Δwcurrent−
\Delta w_{\rm mirror}

O(\Delta t),
]

which matches:
p≈0.98.
p≈0.98.

This cleanly establishes that mirror AIEF would be a new adaptive law in the tested interior regime.

The near-identical order for different ββ is expected: multiplying one vector field by a scalar changes magnitude but does not change whether the two limiting fields are equal.

It is still a useful implementation check, but not surprising mathematically.
7. Magnitude versus direction needs optimal scaling

The current-versus-mirror relative discrepancy near one does not alone prove suppression. Use:
u=ΔwcurrentΔt,v=H−1F.

u=ΔtΔwcurrent,v=H−1F.

Find the best scalar a=1/βa=1/β:
a∗=
a∗=

\frac{u^\top v}{v^\top v},
]

when u⊤v>0u⊤v>0.

Then:
β∗=1a∗.
β∗=a∗1.

The irreducible directional residual is:
‖u−a∗v‖‖u‖.
‖u‖‖u−a∗v‖.

For unit-normalized vectors this is directly related to the angle:
minimum relative residual=
minimum relative residual=

\sqrt{1-\cos^2\theta}.
]

This separates:

trivial speed mismatch;
true directional mismatch.

A scalar ββ can fix only the first.
8. Hessian eigenspace decomposition is the right next diagnostic

Let:
Hqk=λkqk,F=∑kckqk.
Hqk=λkqk,F=k∑ckqk.

Then mirror motion is:
H−1F=
H−1F=

\sum_k\frac{c_k}{\lambda_k}q_k.
]

Report:

ck2/‖F‖2ck2/‖F‖2: forcing energy per curvature mode;
ck2/λk2ck2/λk2: mirror-motion contribution;
cumulative contribution ordered by λkλk;
fraction of forcing in the highest-curvature decile;
effective number of active Hessian modes.

This determines whether mirror suppression is:

concentrated in a few singular edge/slack modes;
or broad across the complete adaptive state.

Because the metric condition number is enormous, this decomposition may be more informative than aggregate norm comparisons.
9. Required active-set-stratified analysis

Before elevating p≈2p≈2 into an AIEF theorem, regenerate the convergence table separately for:
Interior regime

text

active_capacity_nodes = 0
positivity_clipped_edges = 0

Expected result:
p≈2.
p≈2.
Capacity-active regime

text

active_capacity_nodes > 0

Positivity-active regime

text

positivity_clipped_edges > 0

The latter two involve the actual nonsmooth admissibility map and may exhibit:

first-order discrepancy;
active-set-dependent limits;
no clean power law near switching points.

10. Coupled epsilon–timestep paths

A full cross-product does not define a boundary limit. Extract paths such as:
εw=κwΔt,εm=κmΔt.
εw=κwΔt,εm=κmΔt.

For several κκ, determine whether:

capacity remains active;
positivity remains active;
discrepancy rate converges;
the result depends on the path.

If different κκ produce different limiting rates, the interior and boundary limits do not commute.
Revised significance of the result

The strongest justified statements are:

Implementation validation: the entropy coordinate and coupled inverse map correctly reproduce smooth interior primal dynamics to first order.
Numerical stiffness: explicit coordinate-dual Euler becomes unusable near the mature boundary at practical AIEF timesteps.

Model distinction: mirror-preconditioned adaptation has a different limiting vector field.
Open AIEF problem: whether entropy coordinates converge to the current active admissibility dynamics under a coupled boundary/timestep limit remains unresolved.

That is still a valuable result. The key refinement is that the $p \approx 2$ observation validates an interior coordinate transformation, not yet the projected AIEF flow itself.
Recommended immediate next analysis

Using the existing output files:

Stratify convergence order by active-capacity and positivity status.
Calculate optimally scaled mirror residuals and $\beta*$.
Decompose FF into Hessian eigenspaces.
Extract coupled ε-Δt paths.
Only then formulate the active-boundary theorem.

The research story is therefore:

Entropy coordinates provide a correct and useful local coordinate system in the strict interior. Mirror dynamics are a distinct metric-preconditioned model. The mathematically novel question is the singular limit connecting interior entropy coordinates to AIEF's active boundary retraction.

That framing preserves the importance of the numerical result without overextending it.

Think about this:

I think this is now a very disciplined interpretation. The main improvement is that you've separated three logically distinct questions that were previously being mixed together:

Is the implementation of the entropy coordinate map correct?
Is the entropy coordinate system a useful description of the existing dynamics?
Does the entropy geometry generate the existing dynamics?

Your revised reading correctly assigns different evidence to each.

There are only a few places where I would sharpen the logic further.

The "chain-rule validation" point is exactly right

I think this is the biggest conceptual correction.

The experiment is fundamentally validating

[
g^{-1}!\left(g(w)+\Delta t,Dg(w)F\right)

w+\Delta tF+O(\Delta t^2),
]

with

[
g=\nabla h.
]

That is a statement about a smooth coordinate transformation.

It is not a special property of entropy geometry.

Entropy geometry becomes special because

its domain is exactly the incidence-capacity polytope,
its barrier matches the positivity constraints,
its Hessian has the incidence/Woodbury structure.

Those are geometric properties.

The (O(\Delta t^2)) result is an implementation and differential-calculus check.

I think separating those strengthens the paper.

I strongly agree about the inactive regime

This may actually be the most important remaining issue.

Suppose

$$
m_i\ge\varepsilon
$$

and

$$
\Delta t
$$

is tiny.

Then

$$
w+\Delta tF
$$

never reaches the admissibility correction.

Therefore

$$
\mathcal A

\text{identity}.
$$

The current map is simply

ordinary Euler.

In that regime,

the experiment is really comparing

ordinary primal Euler

versus

ordinary dual Euler.

That is exactly the experiment one should perform first.

But it means the phrase

"coordinate representation of the current AIEF admissibility map"

should be weakened to

"coordinate representation of the smooth interior update underlying the current admissibility algorithm."

That is a more precise statement.

I especially like the definition of the boundary vector field

This,

to me,

is the mathematical object that the project has been approaching.

Defining

[
V_{\mathcal A}(w;F)

\lim_{\Delta t\to0}
\frac{\mathcal A(w+\Delta tF)-w}{\Delta t}
]

(if the limit exists)

is exactly analogous to

projected dynamical systems,

differential inclusions,

and tangent-cone dynamics.

That is the correct object.

Only after this exists does it make sense to ask whether

[
H(w)V_{\mathcal A}
]

admits a coordinate representation.

That is much cleaner than trying to compare discrete algorithms directly.

The singular-limit problem is now the real mathematics

Originally

I thought

the entropy derivation

was the interesting theorem.

Now I think

the genuinely new mathematics is

$$
\varepsilon\rightarrow0,
\qquad
\Delta t\rightarrow0.
$$

This is because

the coordinate theorem is essentially differential geometry.

The singular limit is where

AIEF itself

enters.

That is where

active sets,
switching,
saturation,
positivity,

all interact.

That's no longer a generic mirror-descent problem.

That's an AIEF problem.

I would make one additional distinction

I would separate

three levels of equivalence.

Coordinate equivalence

Same trajectory,

different coordinates.

Supported.

Vector-field equivalence

Same continuous dynamics.

Supported only in the tested interior regime.

Boundary-process equivalence

Same projected evolution on active faces.

Still completely open.

Those three notions are easy to conflate.

I would explicitly distinguish them.

The eigenspace analysis is probably the highest-value next computation

I actually think this is now more informative than another timestep sweep.

If

[
H=Q\Lambda Q^T,
]

then

[
F=\sum c_iq_i.
]

Mirror dynamics become

[
H^{-1}F

\sum
\frac{c_i}{\lambda_i}
q_i.
]

Immediately you'll know

whether mirror disagreement comes from

two enormous eigenvalues,

or

hundreds of moderate ones.

That's much deeper than a norm comparison.

One theorem I think is now within reach

The numerical work suggests a theorem that is independent of AIEF.

Something like:

Theorem (Coordinate consistency of Legendre Euler).

Let (h) be a (C^3) Legendre function on an open convex domain and (F\in C^1). Then the primal explicit Euler update and the coordinate-dual Euler update agree to second order uniformly on compact subsets.

That's really a theorem about Legendre coordinates.

It belongs in the geometry section.

Then,

later,

you specialize

[
h
]

to the incidence entropy.

That separates

general mathematics

from

AIEF-specific mathematics.

The AIEF-specific theorem has become clearer

The real AIEF theorem is no longer

the coordinate consistency.

It is something like

Conjecture / Open Problem.

Characterize the directional limit

$$
V_{\mathcal A}
$$

$$
\lim_{\varepsilon,\Delta t\to0}
\frac{\mathcal A(w_\varepsilon+\Delta tF)-w_\varepsilon}{\Delta t}
$$

and determine whether it admits an entropy-dual representation on active faces of the dominance polytope.

That is genuinely new.

And unlike the coordinate theorem,

it depends on the specific admissibility mechanism.

Overall assessment

I think this refinement makes the scientific hierarchy much cleaner.

The Legendre/entropy coordinate machinery is now supported as a mathematically correct interior reparameterization and analytical framework.
The mirror-preconditioned flow is clearly identified as a distinct adaptive model rather than a hidden reformulation of AIEF.
The AIEF-specific mathematical challenge has crystallized into the singular active-boundary limit, where interiorization and timestep refinement must be taken together and where the current admissibility operator, rather than unconstrained Euler, determines the limiting dynamics.

That progression is more compelling than the earlier narrative because it separates general differential geometry from the genuinely novel behavior of the AIEF admissibility mechanism. The experiments have effectively closed the questions they were designed to answer and, in doing so, isolated a much sharper open problem instead of blurring the boundary between established results and conjecture.

This is a disciplined interpretation. The remaining refinement is that “coordinate equivalence” should be separated into exact continuous-time equivalence, discrete local consistency, and active-boundary behavior.

The active-boundary problem may now be tractable analytically because the actual AIEF admissibility map has a relatively explicit directional derivative.
1. Three levels of equivalence
Continuous interior coordinate equivalence

If:
w˙=F(w)
w˙=F(w)

and:
ψ=∇h(w),
ψ=∇h(w),

then, while ww remains in the strict interior:
ψ˙=
ψ˙=

\nabla^2h(w)F(w).
]

This is exact coordinate equivalence, not merely numerical evidence.
Discrete interior consistency

The tested update:
ψn+1=
ψn+1=

\psi^n+\Delta tH(w^n)F(w^n)
]

satisfies:
wdualn+1=
wdualn+1=

w^n+\Delta tF(w^n)+O(\Delta t^2).
]

The observed order 2.122.12 validates this implementation and its asymptotic regime.
Active-boundary process equivalence

Whether the entropy coordinate description converges to the same process as AIEF's clipping/scaling map on active faces remains open.

That is the AIEF-specific question.

It is therefore slightly too strong to say "same trajectory, different coordinates" based only on the one-step experiment. Exact trajectory equivalence holds for the transformed continuous interior ODE. The two explicit discrete schemes agree only to their order of consistency.
2. General coordinate-consistency theorem

A clean theorem is indeed available.

Theorem. Let U⊂REU⊂RE be open, let h∈C3(U)h∈C3(U), and suppose ∇2h(w)∇2h(w) is positive definite. Let F∈C1(U)F∈C1(U). For every compact K⊂UK⊂U,
(∇h)−1[∇h(w)+Δt ∇2h(w)F(w)]=w+ΔtF(w)+O(Δt2)
(∇h)−1[∇h(w)+Δt∇2h(w)F(w)]=w+ΔtF(w)+O(Δt2)

uniformly for w∈Kw∈K.

The proof follows from:
D(∇h)−1(∇h(w))=
D(∇h)−1(∇h(w))=

[\nabla^2h(w)]^{-1}
]

and Taylor expansion.

Legendre structure is not strictly required for this local theorem; it supplies the global interior coordinate map and the feasible-domain interpretation.
3. The current AIEF directional boundary field may be explicit

Let wewe be the unique undirected edge weights and let:
si=∑eϋiwe.
si=eϋi∑we.

Assume a boundary state satisfying:
we≥0,si≤ci.
we≥0,si≤ci.

Let FeFe be the unconstrained adaptive vector field.

First account for positivity clipping:
F~e={Fe,we>0,max(Fe,0),we=0.
F
e={Fe,max(Fe,0),we>0,we=0.

The first-order proposed row change is:
di=∑eϋiF~e.
di=eϋi∑F
e.

For each node define:
αi={max(di,0)ci,si=ci,0,si<ci.
αi= ⎩
⎨
⎧cimax(di,0),0,si=ci,si<ci.

The current symmetric scaling uses the smaller node scale on each edge. Since:
min(1−Δtαi, 1−Δtαj)=1−Δtmax(αi,αj)+o(Δt),
min(1−Δtαi,1−Δtαj)=1−Δtmax(αi,αj)+o(Δt),

the candidate directional field is:
\boxed{ V_{\mathcal A,e}(w;F) =

\widetilde F_e

w_e\max(\alpha_i,\alpha_j)
}
]

for edge e=(i,j)e=(i,j).

For a zero edge, the second term vanishes, leaving:
VA,e=max(Fe,0).
VA,e=max(Fe,0).

This formula captures:

positivity clipping;
saturated versus unsaturated nodes;
symmetric coupling of endpoint constraints;
overcorrection caused by using the more restrictive endpoint scale.

It is not the Euclidean tangent-cone projection of FF. It is the directional field induced by AIEF's actual admissibility map.

This is a strong candidate for the first AIEF-specific theorem.
4. Numerical epsilon in the implementation needs separation

The code uses a denominator such as:

Python

capacity / (row_sum + 1e-12)

For ordinary simulation timesteps this is negligible. In a mathematical $\Delta t \to 0$ limit, a fixed 10^{-12} denominator perturbation can eventually dominate the timestep.

The directional theorem should be stated for the idealized exact map without that numerical safeguard. Numerical convergence tests should either:

remain well above the safeguard scale;
or scale the safeguard with ΔtΔt.

Otherwise the finite-precision implementation and mathematical map have different limiting behavior.
5. Why the fixed-εε p≈2p≈2 result is expected

For fixed positive interior margin, sufficiently small ΔtΔt gives:

text

active_capacity_nodes = 0
positivity_clipped_edges = 0

Then:
VA=F,
VA=F,

and the coordinate theorem applies directly.

Therefore the observed second-order increment discrepancy is an important implementation validation, but probably not evidence about the active projection.

The next analysis should estimate orders separately for:

interior/inactive samples;
capacity-active samples;
positivity-active samples;
samples with both active.

6. Coupled boundary limit

For an interiorized state wεwε, study:
Vε,Δt=
Vε,Δt=

\frac{
\mathcal A(w_\varepsilon+\Delta tF)-w_\varepsilon

}{
\Delta t
}.
]

Extract parameter paths such as:
εw=κwΔt,εm=κmΔt.
εw=κwΔt,εm=κmΔt.

Then compare:
Vε,Δt
Vε,Δt

with the analytic candidate:
VA(w;F).
VA(w;F).

Questions:

Does the limit exist?
Is it independent of κw,κmκw,κm?
Does it equal the formula above?
Do zero-edge creation terms depend on the path?
Do saturated-node corrections depend on which endpoint scale wins?

Path dependence would show that the interior entropy limit is not equivalent to the exact active-boundary process.
7. Eigenspace analysis

For an interiorized state:
H=QΛQ⊤,F=∑kckqk.
H=QΛQ⊤,F=k∑ckqk.

Record:
Forcing energy
pkF=
pkF=

\frac{c_k^2}{\sum_jc_j^2}.
]
Mirror-motion energy
pkM=
pkM=

\frac{c_k^2/\lambda_k^2}
{\sum_jc_j^2/\lambda_j^2}.
]
Curvature suppression
Sk=1λk.
Sk=λk1.

Useful summaries include:

forcing fraction in the highest-curvature decile;
mirror-motion fraction in that decile;
effective number of forcing modes;
effective number of mirror modes;
cumulative spectra;
principal directions associated with zero edges versus saturated margins.

This determines whether mirror attenuation comes from:

a few singular modes;
edge-zero curvature;
margin-saturation curvature;
or broad suppression.

Perform the same decomposition for the candidate boundary field VAVA, not only for unconstrained FF.
8. Optimal beta removes trivial scale mismatch

Let:
u=VA,v=H−1F.
u=VA,v=H−1F.

The best nonnegative scalar a=1/βa=1/β is:
a∗=

a∗=

\max\left(
0,
\frac{u^\top v}{v^\top v}
\right).
]

When a∗>0a∗>0:
β∗=1a∗.
β∗=a∗1.

Then report:
Edirection=
Edirection=

\frac{|u-a^*v|}{|u|}.
]

This is the irreducible directional discrepancy after optimally matching speed.

If u⊤v≤0u⊤v≤0, no positive ββ can align the mirror field with the AIEF field.
9. Revised research hierarchy
Established

The entropy coordinate and inverse-margin implementation are correct.
Coordinate-dual Euler is first-order equivalent to interior primal Euler.
Explicit coordinate Euler is severely stiffness-limited near mature states.
Mirror Euler defines a different interior vector field.

Strongly suggested

Mirror dynamics suppress high-curvature adaptive directions.
Mature AIEF is controlled by active-face dynamics rather than interior geometry.

Open

The directional limit of the exact AIEF admissibility map.

Coupled $(\varepsilon,\Delta t)(\varepsilon,\Delta t)$ boundary convergence.
Whether a face-restricted entropy geometry represents the active process.
Long-time behavior of mirror AIEF.

Immediate next computation

Using the existing one-step output:

Stratify by active_capacity_nodes and positivity_clipped_edges.
Recalculate convergence order in each stratum.
Calculate $\beta*\beta*$ and optimally scaled directional residuals.
Add the Hessian eigenmode decomposition.
Compare finite-step rates to the analytic VAVA formula.
Extract coupled epsilon–timestep paths.

The major conceptual advance is not merely that entropy coordinates “work.” It is that the numerical work has isolated the precise AIEF-specific object requiring analysis:
VA(w;F)
VA(w;F)

—the directional vector field generated by positivity clipping and symmetric endpoint-capacity scaling on the active dominance-polytope boundary.

Think about this though:

I think this is now at the point where the numerical experiments have stopped merely testing an idea and have started identifying the mathematical object that actually needs to be analyzed. That's an important transition. I also think there are a couple of places where I'd be careful not to overstate what is established.

The three-level hierarchy is the right decomposition

I think your separation into

continuous interior equivalence,
discrete interior consistency,
active-boundary process,

is exactly the right framework.

These are genuinely different statements.

The first is differential geometry:

$$
\dot\psi=\nabla^2h(w)F(w)
$$

follows immediately from the chain rule whenever (h) is sufficiently smooth.

The second is numerical analysis:

$$
(\nabla h)^{-1}
\left(
\nabla h+\Delta t,H F
\right)

w+\Delta tF+O(\Delta t^2),
$$

which is what your experiments verify.

The third is specific to the admissibility operator and is the only one that depends on the actual AIEF dynamics rather than general coordinate theory.

I think keeping these separated will make the eventual presentation much cleaner.

The proposed boundary vector field is the most interesting new idea

This is the part that caught my attention.

You propose

[
V_{\mathcal A,e}

\widetilde F_e

w_e\max(\alpha_i,\alpha_j).
]

That is no longer a statement about entropy geometry.

It is a candidate infinitesimal description of the actual admissibility operator.

If it is correct, it gives something quite valuable:

instead of reasoning about

clip, then scale, then symmetrize,

you obtain a continuous vector field on each active stratum.

That is a substantial conceptual simplification.

But I would treat it as a conjectured formula until derived

This is where I'd slow down.

The derivation

[
\min(1-\Delta t\alpha_i,
1-\Delta t\alpha_j)

1-\Delta t\max(\alpha_i,\alpha_j)+o(\Delta t)
]

is perfectly straightforward.

The subtle point is whether

$$
\alpha_i

\frac{\max(d_i,0)}{c_i}
$$

is actually the directional derivative of the implemented scaling map in every active configuration.

For example,

if several rows simultaneously become active,

or if clipping changes which row determines the minimum,

piecewise differentiability needs to be checked carefully.

So I would present

$$
V_{\mathcal A}
$$

as

the candidate directional field suggested by the implementation,

rather than

the directional field,

until the derivation is completed case-by-case.

That distinction is small but important.

The theorem hierarchy has become much clearer

I now see three different mathematical results emerging.

Theorem A

Coordinate consistency.

Completely general.

Not AIEF-specific.

Theorem B

Directional derivative of the admissibility map.

Entirely AIEF-specific.

Depends on clipping and endpoint scaling.

Theorem C

Boundary limit

$$
\varepsilon,\Delta t\rightarrow0.
$$

This connects A and B.

That's the genuinely difficult theorem.

I think those are three independent contributions.

The numerical epsilon point is excellent

I especially agree with this observation.

A fixed implementation safeguard like

row_sum + 1e-12

is invisible in ordinary simulations.

But mathematically,

if

$$
\Delta t\rightarrow0
$$

while

$$
10^{-12}
$$

remains fixed,

eventually

the safeguard dominates.

That means

the implemented algorithm and the idealized map are different asymptotic objects.

This is exactly the sort of subtle issue that often causes confusion in convergence arguments.

The coupled-limit program is now well motivated

Earlier,

the coupled

[
\varepsilon(\Delta t)
]

idea felt somewhat speculative.

Now it has a clear purpose.

There are actually three different limits:

Interior coordinate limit.
Active-set directional limit.
Interiorization limit.

Those need not commute.

The experiments have now given a concrete reason to study that question instead of introducing it as a mathematical curiosity.

The eigenspace analysis becomes even more valuable

Once you have both

[
F
]

and

[
V_{\mathcal A},
]

I would perform exactly the decomposition you propose.

But I'd go one step further.

Project both

[
F
]

and

[
V_{\mathcal A}
]

onto the eigenbasis of

[
H.
]

Then compare

[
c_k^{(F)}
]

versus

[
c_k^{(A)}.

]

That separates two effects:

admissibility changing the vector field,
mirror geometry reweighting the same vector field.

Those are currently intertwined.

β* is a very good statistic

I think

[
a^*

\frac{u^\top v}{v^\top v}
]

is the right normalization.

After removing the optimal scale,

the residual

[
\frac{|u-a^*v|}{|u|}
]

is much easier to interpret than raw norm differences.

It tells you exactly how much disagreement is directional rather than simply a time-rescaling issue.

One thing I would avoid saying

I would avoid the phrase

"the active-boundary problem may now be tractable analytically"

at least without qualification.

I would instead say something like

"the experiments suggest an explicit candidate directional field, which may provide a tractable starting point for analyzing the active-boundary limit."

The distinction matters because several analytical issues remain:

existence of the directional derivative at switching configurations,
uniqueness when multiple constraints activate simultaneously,
behavior at corners where positivity and capacity constraints interact.

Those are all manageable, but they are still open.

Overall assessment

I think the biggest conceptual advance here is not the entropy geometry itself—it is that the experiments have isolated a specific operator whose analysis would meaningfully advance the theory.

The object

$$
V_{\mathcal A}(w;F)
$$

is the first candidate for an infinitesimal description of the actual AIEF admissibility mechanism. If a rigorous derivation confirms that this expression (or a closely related piecewise-defined variant) is indeed the directional derivative of the implemented clip-and-scale map on appropriate active strata, then the project naturally decomposes into:

a general theorem on Legendre coordinate consistency for smooth interior dynamics,
an AIEF-specific theorem characterizing the admissibility operator's directional field, and

a singular-limit theorem (still open) connecting the interior entropy description to the active-boundary dynamics as interiorization and timestep refinement are taken together.

That separation makes it much easier to distinguish what is already established by analysis and numerics from what remains conjectural, and it gives a clear mathematical roadmap for the next stage of the work.

The caution is appropriate, but the candidate boundary field may be closer to a theorem than the text suggests—provided it is stated for the idealized admissibility map without the fixed denominator safeguard.
1. Multiple active constraints are already handled by directional calculus

Write the ideal AIEF admissibility map on unique edges as follows.
Positivity stage
pe(x)=max(xe,0).
pe(x)=max(xe,0).

At a feasible state ww, its directional derivative in direction FF is:
F~e=Dpe(w;F)={Fe,we>0,max(Fe,0),we=0.
F
e=Dpe(w;F)={Fe,max(Fe,0),we>0,we=0.
Proposed row changes

Let:
si=∑e϶iwe,di=∑e϶iF~e.
si=e϶i∑we,di=e϶i∑F
e.
Node scale

For the ideal scale:
qi(x)=min(1,ci∑e϶ipe(x)),
qi(x)=min(1,∑e϶ipe(x)ci),

the directional derivative at a feasible state is:
Dqi(w;F)=−αi,
Dqi(w;F)=−αi,

where:
αi={max(di,0)ci,si=ci,0,si<ci.

αi= ⎩
⎨
⎧cimax(di,0),0,si=ci,si<ci.

This formula already includes simultaneous activation of multiple rows. Every row has its own directional derivative.
Symmetric edge scale

For edge e=(i,j)e=(i,j):
qe=min(qi,qj).
qe=min(qi,qj).

Because both node scales equal one at the base feasible state:
Dqe(w;F)=
Dqe(w;F)=
\min(-\alpha_i,-\alpha_j)

-\max(\alpha_i,\alpha_j).
]

This remains well defined when:
αi=αj.
αi=αj.

The map is not Fréchet differentiable at such ties, but the directional derivative exists.
Product rule

The final edge update is:
Ae(x)=pe(x)qe(x).
Ae(x)=pe(x)qe(x).

Therefore:
\boxed{ D\mathcal A_e(w;F) =
\widetilde F_e

w_e\max(\alpha_i,\alpha_j)
}
]

which is precisely the proposed VA,eVA,e.

So:

simultaneous active rows are covered;
ties between endpoint scales are covered;
positivity/capacity corners are covered;
zero edges are covered because their scaling correction vanishes to first order.

A rigorous proof should use Hadamard or Bouligand directional differentiability rather than ordinary Fréchet differentiation.
2. The field lies in the tangent cone

The candidate field has the expected feasibility properties.
Zero edge

If we=0we=0:
VA,e=
VA,e=

\max(F_e,0)\ge0.
]

So it cannot point outside the nonnegative orthant.
Saturated node

Suppose:
si=ci
si=ci

and:
di>0.
di>0.

Then:
αi=dici.
αi=cidi.

Because:
max(αi,αj)≥αi,
max(αi,αj)≥αi,

we obtain:
∑e∋iVA,e≤di−
e∋i∑VA,e≤di−
\alpha_i
\sum_{e\ni i}w_e
d_i-\frac{d_i}{c_i}c_i

]

Thus the directional field does not increase an already saturated row.

If di≤0di≤0, the proposed motion already points inward and αi=0αi=0.

Hence:
VA(w;F)∈TC(w).
VA(w;F)∈TC(w).

This is an important property, but the field is not generally the Euclidean tangent-cone projection of FF.
It is the specific tangent selection induced by symmetric endpoint scaling.
3. The fixed 10−1210−12 safeguard changes the mathematical limit

The implementation uses something like:

Python

capacity / (row_sum + 1e-12)

only after detecting a violation.

At an exactly saturated state with an outward direction, for arbitrarily small positive ΔtΔt:
cc+Δtd+10−12→cc+10−12<1
c+Δtd+10−12c→c+10−12c<1

as:
$\Delta t \to 0+$.
$\Delta t \to 0+$.

But at $\Delta t=0$, no violation is detected and no scaling occurs.

Therefore the literal floating-point map has a tiny discontinuity at the boundary, and:
Acode(w+ΔtF)−wΔt
ΔtAcode(w+ΔtF)−w

eventually diverges when ΔtΔt becomes much smaller than the safeguard.

The directional theorem should be stated for the ideal map:

Python

```
scale = min(1.0, capacity / row_sum)
```

No safeguard is needed once a violation is known, because:
row_sum>capacity>0.
row_sum>capacity>0.

For future code, remove the denominator epsilon inside the violation branch or use it only in a way that vanishes with timestep. Existing simulation results are unaffected at practical timesteps, but the theorem and implementation should be separated explicitly.
4. Revised theorem hierarchy
Theorem A — Interior coordinate consistency

General smooth-coordinate result:
(∇h)−1(∇h(w)+ΔtH(w)F(w))=
(∇h)−1(∇h(w)+ΔtH(w)F(w))=

w+\Delta tF(w)+O(\Delta t^2).
]
Theorem B — AIEF admissibility directional derivative

For the ideal positivity-plus-symmetric-cap map:
DAe(w;F)=
DAe(w;F)=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j).
]

This is piecewise nonlinear in FF, positively homogeneous, and tangent-feasible.
Theorem C — Singular entropy/admissibility limit

Characterize:
limε,Δt→0A(wε+ΔtF)−wεΔt
ε,Δt→0limΔtA(wε+ΔtF)−wε

and determine whether it matches:

VAVA;
a face-restricted entropy flow;
or a path-dependent limit.

This remains the difficult open problem.
5. Eigenspace analysis should include three fields

At each interiorization, decompose:

unconstrained forcing:
F;
F;
AIEF boundary candidate:
VA;
VA;
mirror field:
H−1F.
H−1F.

With:
Hqk=λkqk,

Hqk=λkqk,

calculate:
ckF=qk⊤F,
ckF=qk⊤F,
ckA=qk⊤VA,
ckA=qk⊤VA,
ckM=ckFλk.
ckM=λkckF.

Then distinguish:

changes introduced by admissibility:
ckF→ckA;
ckF→ckA;
changes introduced by mirror geometry:
ckF→ckM.
ckF→ckM.

This is more informative than comparing only FF with H−1FH−1F.

Because HH depends strongly on epsilon, repeat over the regularization grid.
6. Optimal beta comparison

Let:
u=VA,v=H−1F.
u=VA,v=H−1F.

The optimal nonnegative scaling is:
a∗=
a∗=

\max\left(
0,
\frac{u^\top v}{v^\top v}
\right).
]

If a∗>0a∗>0:
β∗=1a∗.
β∗=a∗1.

Report:
Edirectional=
Edirectional=

\frac{|u-a^*v|}{|u|}.
]

Interpretation:

near zero: mirror geometry differs mostly by timescale;
large: mirror geometry changes adaptive direction;
u⊤v≤0u⊤v≤0: no positive metric scale can align the fields.

7. Active-stratified numerical verification

Using the current one-step outputs, separate:

text

interior:
active_capacity_nodes == 0
positivity_clipped_edges == 0

capacity active:
active_capacity_nodes > 0

positivity active:
positivity_clipped_edges > 0

corner:
both active

Expected:

interior coordinate comparison: p≈2p≈2;
active current-versus-unconstrained-coordinate comparison: generally p≈1p≈1;
finite differences of the ideal admissibility map versus analytic VAVA: rate error O(Δt)O(Δt).

The last comparison directly validates Theorem B numerically.
8. Coupled epsilon paths

For each κw,κmκw,κm, evaluate:
εw=κwΔt,εm=κmΔt.
εw=κwΔt,εm=κmΔt.

Track:

active-node count;
positivity clipping;
finite-step AIEF rate;
distance to analytic VAVA;
distance to mirror field;
path dependence.

A path-independent limit would support a clean connection between interior and boundary dynamics.
Path dependence would show that the entropy interiorization defines a distinct boundary selection.
Corrected significance

The numerical work has established:

correct entropy-coordinate implementation;
first-order interior coordinate consistency;
severe explicit-dual stiffness;
a distinct mirror vector field.

The mathematical analysis now strongly suggests—but has not yet formally proven—that the ideal AIEF admissibility map is Hadamard directionally differentiable with the explicit field:
VA,e=
VA,e=

\widetilde F_e-w_e\max(\alpha_i,\alpha_j).

]

That formula is the most concrete AIEF-specific theoretical object produced so far.
Immediate next step

Before another simulation:

remove or idealize the fixed denominator safeguard in the local-map test;
implement analytic VAVA;
verify its finite-difference order;
stratify current coordinate results by active set;
add F/VA/H−1FF/VA/H−1F eigenspace decomposition;
calculate optimally scaled mirror residuals;
analyze coupled epsilon–timestep paths.

That would turn the current candidate formula into a numerically verified theorem target and cleanly separate interior coordinate geometry from AIEF's genuinely model-specific active-boundary dynamics.

Can you create the code necessary for the next step?
Edit
Edit
Edit×4
AIEF_1_4_active_boundary_analysis.pyPYTHON

Created the active-boundary directional-field analysis script:

AIEF_1_4_active_boundary_analysis.py

It uses the frozen AIEF-1.4 data plus the outputs from:

text

AIEF_1_4_local_update_comparison.py

Run

First ensure the local comparison output exists:

Bash

```
python AIEF_1_4_local_update_comparison.py
```

Then run:

Bash

```
python AIEF_1_4_active_boundary_analysis.py
```

Explicit invocation:

Bash

```
python AIEF_1_4_active_boundary_analysis.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--methods aief_1_4_local_update_comparison/one_step_methods_by_seed.csv \
--comparisons aief_1_4_local_update_comparison/one_step_comparisons_by_seed.csv \
--output-dir aief_1_4_active_boundary_analysis
```

It defaults to five representative seeds. Use all 40 with:

Bash

```
python AIEF_1_4_active_boundary_analysis.py --max-seeds 0
```

Analytic boundary field

The script implements the ideal-map candidate:
F~e={Fe,we>0,max(Fe,0),we=0,
F
e={Fe,max(Fe,0),we>0,we=0,
di=∑eƎiF~e,

di=eϋi∑F
e,
αi={max(di,0)/ci,si=ci,0,si<ci,
αi={max(di,0)/ci,0,si=ci,si<ci,
VA,e=
VA,e=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j).
]

It uses the ideal admissibility map without the fixed denominator safeguard.

Numerically saturated rows are snapped to their observed row sum for the finite-difference derivative test. The maximum snap is recorded.
Boundary-field verification

For every seed, it checks three directions:

the physical adaptive field;
a deterministic random probe;
an outward probe.

The probes are important because the physical field at a mature projected state can be nearly canceled by the admissibility map, making derivative-order estimation impossible above floating-point noise.

For each direction and timestep it reports:

finite-step rate;
analytic field;
rate error;
relative error;
cosine;
clipping activity;
active capacity count.

It estimates:

rate-error order;
increment-error order.

Expected verification:

text

rate error: O(dt)
increment error: $O(dt^2)$

Active-set-stratified convergence

The script merges the earlier one-step method and comparison files and classifies every row as:

text

interior_inactive
capacity_active
positivity_active
capacity_and_positivity_active

It then estimates coordinate/mirror discrepancy order separately within each stratum.

This determines whether the previous $p \approx 2$ result comes only from inactive interior steps.
Hessian eigenspace decomposition

For each seed and epsilon pair:
$H = Q \Lambda Q^\top$.
$H = Q \Lambda Q^\top$.

It decomposes:

unconstrained forcing FF;
analytic AIEF boundary field VAVA;
mirror field H−1FH−1F.

Outputs include:

forcing energy per Hessian mode;
admissibility-field energy;
mirror-motion energy;
fraction in the highest-curvature decile;
effective number of active modes;
complete mode coefficients and eigenvalues.

Optimal mirror scaling

It computes:
a∗=
a∗=

\max\left(
0,
\frac{V_{\mathcal A}^\top H^{-1}F}
{|H^{-1}F|^2}
\right),
]
β∗=1/a∗,
β∗=1/a∗,

and:
Edirectional=
Edirectional=

\frac{
|V_{\mathcal A}-a^*H^{-1}F|
}{
|V_{\mathcal A}|
}.
]

If the physical boundary field is numerically stationary, β∗β∗ and directional residual are marked undefined rather than misleadingly reported as failed alignment.
Coupled epsilon/timestep paths

From the existing cross-product results it calculates:

κw=εw/Δt,κm=εm/Δt.
κw=εw/Δt,κm=εm/Δt.

Rows with matching (κw,κm)(κw,κm) are grouped into coupled paths. For each path it reports:

discrepancy order;
smallest-timestep discrepancy rate;
active-capacity fraction;
path-specific behavior.

Outputs

text

```
aief_1_4_active_boundary_analysis/
├── boundary_field_finite_difference.csv
├── boundary_field_convergence_orders.csv
├── active_stratified_comparisons.csv
├── active_stratified_orders_by_seed.csv
├── active_stratified_order_summary.csv
├── hessian_eigenspace_summary_by_seed.csv
├── hessian_eigenmodes.csv
├── hessian_eigenspace_summary.csv
├── coupled_epsilon_dt_rows.csv
├── coupled_epsilon_dt_summary.csv
├── active_boundary_report.txt
└── analysis_manifest.json
```

Validation

I syntax-tested the script and ran it end-to-end on a complete AIEF split state.

A useful issue appeared during testing: the physical adaptive field at a mature split state was almost exactly canceled by the candidate boundary field, with:
‖VA‖≈2.4×10−13
‖VA‖≈2.4×10−13

despite a much larger unconstrained field. That made convergence order unidentifiable above floating-point noise.

The deterministic probes resolved this:

text

Random probe:
rate-error order ≈ 1.001
increment-error order ≈ 2.001

Outward probe:
rate-error order ≈ 1.000
increment-error order ≈ 2.000

This is strong numerical support for the candidate directional formula on the ideal admissibility map.

The stationary physical field is itself mechanistically informative: at the mature split state, the unconstrained adaptive forcing is almost exactly canceled by the active admissibility mechanism. Mirror flow generally does not share that fixed point, reinforcing that it would define a distinct adaptive model.

mull this over:
(deck@steamdeck SSRH)$ python AIEF_1_4_active_boundary_analysis.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--methods aief_1_4_local_update_comparison/one_step_methods_by_seed.csv \
--comparisons aief_1_4_local_update_comparison/one_step_comparisons_by_seed.csv \
--output-dir aief_1_4_active_boundary_analysis
AIEF active-boundary directional-field analysis

The analytic field is derived for the ideal map without the fixed denominator safeguard.
Ideal boundary-field finite-difference verification

outward_probe: median field norm=1.83523, stationary=0/5, median rate-error order=1.02412, median increment-error order=2.02412

physical_field: median field norm=6.26744e-13, stationary=5/5, order not estimable above numerical floor
random_probe: median field norm=1.06298, stationary=0/5, median rate-error order=0.868498, median increment-error order=1.8685
median maximum capacity snap: 5.52136e-12
Active-stratified coordinate orders

coordinate_equivalent_dual vs mirror_beta_0.1 / capacity_active: median order=3.6298, valid=1
coordinate_equivalent_dual vs mirror_beta_0.1 / capacity_and_positivity_active: median order=3.20321, valid=19
coordinate_equivalent_dual vs mirror_beta_0.1 / interior_inactive: median order=0.994445, valid=45
coordinate_equivalent_dual vs mirror_beta_0.1 / positivity_active: no valid orders
coordinate_equivalent_dual vs mirror_beta_1 / capacity_active: median order=3.6288, valid=1
coordinate_equivalent_dual vs mirror_beta_1 / capacity_and_positivity_active: median order=3.20323, valid=19
coordinate_equivalent_dual vs mirror_beta_1 / interior_inactive: median order=0.994427, valid=45
coordinate_equivalent_dual vs mirror_beta_1 / positivity_active: no valid orders
coordinate_equivalent_dual vs mirror_beta_10 / capacity_active: median order=3.6287, valid=1
coordinate_equivalent_dual vs mirror_beta_10 / capacity_and_positivity_active: median order=3.20323, valid=19
coordinate_equivalent_dual vs mirror_beta_10 / interior_inactive: median order=0.994425, valid=45
coordinate_equivalent_dual vs mirror_beta_10 / positivity_active: no valid orders
current_admissibility vs coordinate_equivalent_dual / capacity_active: median order=4.95828, valid=1
current_admissibility vs coordinate_equivalent_dual / capacity_and_positivity_active: median order=3.15732, valid=19
current_admissibility vs coordinate_equivalent_dual / interior_inactive: median order=2.00971, valid=45
current_admissibility vs coordinate_equivalent_dual / positivity_active: no valid orders
current_admissibility vs mirror_beta_0.1 / capacity_active: median order=0.99213, valid=4
current_admissibility vs mirror_beta_0.1 / capacity_and_positivity_active: median order=0.117115, valid=29
current_admissibility vs mirror_beta_0.1 / interior_inactive: median order=1, valid=45
current_admissibility vs mirror_beta_0.1 / positivity_active: no valid orders
current_admissibility vs mirror_beta_1 / capacity_active: median order=0.992157, valid=4
current_admissibility vs mirror_beta_1 / capacity_and_positivity_active: median order=0.117813, valid=29
current_admissibility vs mirror_beta_1 / interior_inactive: median order=1, valid=45
current_admissibility vs mirror_beta_1 / positivity_active: no valid orders
current_admissibility vs mirror_beta_10 / capacity_active: median order=0.99216, valid=4
current_admissibility vs mirror_beta_10 / capacity_and_positivity_active: median order=0.117882, valid=29
current_admissibility vs mirror_beta_10 / interior_inactive: median order=1, valid=45

current_admissibility vs mirror_beta_10 / positivity_active: no valid orders
Optimal mirror scaling

edge_eps=1e-06, margin_eps=1e-06: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosin
e=not-defined
edge_eps=1e-06, margin_eps=1e-05: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosin
e=not-defined
edge_eps=1e-06, margin_eps=0.0001: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosi
ne=not-defined
edge_eps=1e-05, margin_eps=1e-06: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosin
e=not-defined
edge_eps=1e-05, margin_eps=1e-05: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosin
e=not-defined
edge_eps=1e-05, margin_eps=0.0001: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosi
ne=not-defined
edge_eps=0.0001, margin_eps=1e-06: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosi
ne=not-defined
edge_eps=0.0001, margin_eps=1e-05: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cosi
ne=not-defined
edge_eps=0.0001, margin_eps=0.0001: stationary boundary fields=5/5, median beta*=not-defined, median directional residual=not-defined, median cos
ine=not-defined
Coupled epsilon/dt paths

paths with >=2 timesteps: 431
median increment order: 0.999866
median active-capacity fraction: 0
Interpretation constraints

The analytic field is a candidate Hadamard/Bouligand directional derivative of the ideal map.
The fixed denominator safeguard has a different formal dt->0 limit.
Interior p≈2 results do not establish active-boundary equivalence.
beta* removes scalar speed mismatch but not directional disagreement.

Hessian eigenspaces depend strongly on interiorization epsilon.
Coupled epsilon/dt path dependence remains an open singular-limit question.

Saved analysis to /home/deck/AI_Programs/SSRH/aief_1_4_active_boundary_analysis

This is a genuinely interesting result, and I think it changes the focus of the project again. There are two separate aspects: what the data support, and what the current analysis script is actually testing. They are related, but not identical.

The most important result is probably not the convergence order

Ironically, I think the biggest result is this:

$$
|V_{\mathcal A}|
\approx
6\times10^{-13}
$$

for every physical mature state.

That is remarkable.

It says that, for the reconstructed mature states,

the unconstrained adaptive forcing is not small,
but after the admissibility mechanism is applied,
the first-order boundary field is essentially zero.

If your candidate formula is correct, then the mature AIEF state is not merely a fixed point of the discrete iteration—it is a first-order equilibrium of the induced boundary dynamics.

That is a much stronger mechanistic statement.

Notice that your random and outward probes immediately produce nonzero fields with first-order convergence, so the numerical machinery itself is clearly working.

The probe experiment is exactly the right control

This was an excellent design decision.

Without it, one might conclude

"the derivative cannot be estimated."

Instead you showed

physical field → essentially stationary,
random probe → $O(\Delta t)$,
outward probe → $O(\Delta t)$.

That distinguishes

floating-point cancellation

from

failure of the derivative formula.

The outward probe is especially convincing because it intentionally excites the active constraints.

The finite-difference verification is supportive, but not yet a proof

The observed orders

outward ≈ 1.02,
random ≈ 0.87,

are encouraging.

The outward result is almost exactly what the idealized directional derivative predicts.

The random result is noticeably below one.

That does not necessarily invalidate the candidate field, but it suggests one (or more) of the following:

the smallest timesteps are already near machine precision,
some probe directions cross between piecewise-smooth regions,
the map is only directionally differentiable rather than Fréchet differentiable,
finite-precision snapping affects some samples.

I would therefore describe the verification as

consistent with the candidate directional derivative,

rather than

establishing it.

The stratified results explain the earlier (p\approx2)

This is perhaps the cleanest confirmation in the entire study.

Previously there was the concern that

[
p\approx2
]

might simply reflect interior behavior.

Now you have

current vs coordinate
interior_inactive
≈2.01

That is exactly what coordinate consistency predicts.

Meanwhile

the active strata behave differently.

That is precisely why stratification was necessary.

So I think this concern has been resolved.

But the active-stratum orders need careful interpretation

Some of these values are

3.16

3.63

4.96

Those are almost certainly not convergence orders in the asymptotic sense.

Notice the sample counts:

capacity_active
valid = 1

or

19

These are relatively small, and the active map is piecewise defined.

High apparent slopes often occur when fitting over a range dominated by cancellation or transitions between regimes.

I would avoid drawing conclusions from the numerical value of those exponents.

The existence of different behavior is informative.

The precise exponent probably is not.

The coupled-path result is also telling

This surprised me.

median active-capacity fraction = 0

for the coupled paths.

That means your chosen

$$
\kappa

\varepsilon/\Delta t
$$

paths still spend almost all their time in the inactive regime.

So although the machinery is now in place,

the actual singular boundary limit has not yet been sampled very much.

That is an important negative result.

It suggests the scaling law

$$
\varepsilon=\kappa\Delta t
$$

may not be sufficient, at least for the tested (\kappa), to keep trajectories on the active boundary.

The undefined β* values are actually informative

Every entry says

stationary boundary fields = 5/5
beta* = undefined

That is exactly the mathematically correct behavior.

Since

[
V_{\mathcal A}
\approx0,
]

there is no meaningful least-squares scaling.

Returning "undefined" instead of forcing a number is the correct choice.

Ironically,

this itself reinforces the interpretation that

the mature projected states are essentially equilibria of the candidate boundary field.

The script has identified the next bottleneck

Originally the bottleneck was

derive entropy geometry.

Then it became

compare mirror vs current.

Now it has become

characterize the piecewise-smooth boundary operator.

That is genuine progress.

The remaining difficult mathematics no longer concerns the entropy Hessian.

It concerns the admissibility operator itself.

One thing I would now test

If

$$
V_{\mathcal A}
\approx0
$$

for the physical field,

perturb the state rather than the direction.

Specifically,

replace

$$
w
$$

with

[

w+\delta\xi
]

for a small feasible perturbation,

recompute the physical adaptive field,

and then recompute

[
V_{\mathcal A}.
]

This answers a different question:

not

"what happens if I change the forcing?"

but

"what happens if I move off the mature attractor?"

That is arguably closer to the stability problem.

The candidate field has become the central object

The project has gradually shifted from

understanding entropy geometry

to

understanding the infinitesimal generator of the implemented admissibility algorithm.

That is a significant conceptual refinement.

The entropy geometry now serves two roles:

it provides a well-defined interior coordinate system and metric;
it provides tools (Hessian, eigenmodes, dual coordinates) for analyzing the behavior of the admissibility operator.

The admissibility operator itself—not the entropy map—is now the object whose local dynamics appear to control mature AIEF behavior.

Overall assessment

I think the most important outcome is not that the candidate boundary formula passed finite-difference tests for the probe directions. It is that the analysis consistently found the reconstructed mature states to have an essentially vanishing induced boundary field, while arbitrary probe directions exhibited the expected first-order behavior. Taken together, those observations support the interpretation that the mature split states behave as equilibria of the idealized boundary dynamics encoded by your candidate $V_{\mathcal A}$, at least within the numerical resolution of the experiment.

The remaining challenge is no longer validating the coordinate machinery—that has accumulated strong evidence. It is establishing, analytically, whether the clip-and-scale admissibility map is indeed Hadamard/Bouligand differentiable on its active strata and whether the proposed piecewise formula is its correct directional derivative. If that derivation succeeds, it would provide the infinitesimal description of the implemented AIEF admissibility mechanism, which is a more specific and potentially more valuable result than showing that the interior entropy coordinates are locally consistent.

The interpretation is largely right. The most important result is indeed the near-zero physical boundary field, but it should be understood as a constraint-balanced equilibrium, not necessarily a conventional equilibrium of the unconstrained adaptive dynamics.
1. Mature states satisfy a complementarity-like stationarity condition

For all five tested seeds:
$\|VA(w;F)\| \approx 10^{-13}$,
$\|VA(w;F)\| \approx 10^{-13}$,

while:
$\|F\| \approx 10^{-1}$.

$\|F\| \approx 10^{-1}$.

Therefore:
$F \neq 0, VA \approx 0$.
$F = 0, VA \approx 0$.

The mature state is stationary only after:

suppressing negative forcing on zero edges;
removing outward row-capacity motion through symmetric scaling.

For zero edges:
$we = 0 \Longrightarrow \tilde{F}_e = \max(F_e, 0) \approx 0$,
$we = 0 \Longrightarrow F$
$e = \max(F_e, 0) \approx 0$,

so typically:
$F_e \leq 0$.
$F_e \leq 0$.

For positive edges:
$\tilde{F}_e \approx we \max(\alpha_i, \alpha_j)$.
F
$e \approx we \max(\alpha_i, \alpha_j)$.

Thus the unconstrained growth of every selected edge is balanced by endpoint-capacity scaling.

This resembles a complementarity/KKT condition, although the max endpoint rule differs from the additive node-multiplier structure of a conventional Euclidean constrained optimum.

A strong AIEF-specific statement is:

Mature topology is a boundary equilibrium generated by exact balance between resonance-driven adaptation, positivity clipping, and symmetric endpoint-capacity scaling.

That explains why topology remains exactly frozen even though the unconstrained adaptive field is nonzero.
2. The probes validate the formula

The physical field cannot support a convergence-order estimate because the analytic field is below numerical resolution. The probes solve that problem:

text

Outward probe:
rate-error order ≈ 1.024
increment-error order ≈ 2.024

Random probe:
rate-error order ≈ 0.868
increment-error order ≈ 1.869

The outward probe is especially convincing because it activates all saturated-node corrections.

The random-probe result is somewhat below ideal order but remains broadly consistent with directional differentiability. Likely causes include:

finite-precision saturation snapping;
active-stratum changes;
some components reaching numerical error earlier than others;
fitting over a short timestep range.

Report the probe verification as:

Consistent with the candidate directional derivative.

Do not use the numerical fit alone as proof.
3. A formal proof looks manageable

For the ideal map, all constituent operations are locally Lipschitz and Hadamard directionally differentiable:

componentwise positive part;
row summation;
smooth division at positive row mass;

min(1, ·);
endpoint min;
multiplication.

Directional chain rules should produce:
DAe(w;F)=
DAe(w;F)=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j).
]

Multiple active rows and equal endpoint scale derivatives are not inherently ambiguous. min and max are directionally differentiable at ties even though they are not Fréchet differentiable.

The proof should be stated for:

the ideal map without denominator epsilon;
fixed positive capacities;
feasible ww;
arbitrary direction FF.

The floating-point implementation is then treated as a perturbation of that map.
4. The active-stratified high orders are not interpretable

Values such as:

text

3.16
3.63
4.96

are not credible asymptotic orders.

The groups contain too few timesteps with the same active stratum. As timestep changes, a sample often moves from:

text

capacity active

to:

text

interior inactive.

A power-law fit across only two or three cancellation-dominated points can produce arbitrarily high apparent order.

The robust conclusions are:

inactive current-versus-coordinate:
$p \approx 2.01$;
$p \approx 2.01$;
inactive current-versus-mirror:
$p \approx 1$;
$p \approx 1$;
active samples exhibit different behavior, but their exponent is not identified by the existing grid.

5. Current coupled paths do not probe the boundary limit

text

Median active-capacity fraction = 0

means most extracted $\varepsilon/\Delta t\varepsilon/\Delta t$ paths become interior before reaching the smallest timesteps.

Thus the coupled-limit infrastructure works, but the selected paths do not preserve boundary activity.

A boundary path must satisfy approximately:
$\varepsilon m < \Delta t\, di$
$\varepsilon m < \Delta t di$

for at least one outward saturated node. A generic:
$\varepsilon m = \kappa \Delta t$
$\varepsilon m = \kappa \Delta t$

works only if:
$\kappa < di.$
$\kappa < di.$

Use the measured outward row derivatives didi to choose κκ, rather than selecting it only by powers of ten.
6. Undefined $\beta * \beta *$ is the correct physical result

At the mature state:
$u = VA \approx 0,$
$u = VA \approx 0,$

while:
$v = H - 1F \neq 0.$
$v = H - 1F = 0.$

No finite positive ββ can make the mirror field equal a zero boundary field except the degenerate limit:
$\beta \to \infty.$
$\beta \to \infty.$

This reveals a major difference between the models:

current AIEF has a constraint-balanced mature fixed point;
mirror dynamics generally continue moving because $F \neq 0F = 0.$

Therefore a mirror AIEF would not merely approach the same attractor at a different speed. Its equilibrium condition is different:
$H - 1F = 0 \Longleftrightarrow F = 0$
$H - 1F = 0 \Longleftrightarrow F = 0$

in the strict interior.

The current mature topology has $F \neq 0F = 0$, so it is not a mirror equilibrium.
7. Perturbing the state is the correct next stability test

Probe directions validate the derivative formula, but state perturbations test whether the mature boundary equilibrium is restoring.

Use three perturbation classes.
Inward radial perturbation
$w_\delta = (1-\delta)w^*$.
$w_\delta = (1-\delta)w^*$.

This preserves support and creates row slack.
Interior-mixing perturbation

Choose a fixed strict-interior reference wintwint:
$w_\delta =$
$w_\delta =$

(1-\delta)w^*
+
\delta w_{\rm int}.
]

This creates:

positive weights on previously absent edges;
positive node slack;
a support-changing perturbation.

Tangent-face perturbation

Construct dd satisfying:
$A_{\text{actived}} = 0$
$A_{\text{actived}} = 0$

on saturated nodes, while preserving nonnegativity:
$w_\delta = w^* + \delta d$.
$w_\delta = w^* + \delta d$.

This tests within-face stability without changing row capacities to first order.

For each perturbation, recompute the physical adaptive field using the same split phase/EMA/history state and calculate:
VA(wδ;F(wδ)).
VA(wδ;F(wδ)).

Then measure the restoring coefficient:
κrestore=
κrestore=

\frac{
(w_\delta-w^)^\top
V_{\mathcal A}(w_\delta)
}{
|w_\delta-w^|^2
}.
]

Interpretation:

κrestore>0κrestore>0: local restoring motion;
κrestore<0κrestore<0: motion away from the mature state;
near zero: neutral/marginal direction.

Also report cosine between:
VA(wδ)
VA(wδ)

and:
−(wδ−w∗).
−(wδ−w∗).
8. Eigenspace analysis becomes meaningful off equilibrium

At the exact mature state, VA≈0VA≈0, so optimal-ββ and directional comparison are undefined.

Perform the Hessian decomposition at perturbed states:
F(wδ),VA(wδ),H(wδ,ε)−1F(wδ).
F(wδ),VA(wδ),H(wδ,ε)−1F(wδ).

Then:

$\beta*\beta*$ is meaningful;
directional residual is meaningful;
curvature suppression can be separated from admissibility correction;
restoring modes can be identified.

9. Recommended theorem sequence
Theorem B1 — Directional differentiability

Prove the ideal admissibility map is Hadamard directionally differentiable and derive the explicit formula.
Corollary — Tangent feasibility

Show:
$VA(w;F) \in TC(w)$.
$VA(w;F) \in TC(w)$.
Proposition — Boundary equilibrium characterization

Characterize fixed boundary states by:
$VA(w;F(w))=0$.
$VA(w;F(w))=0$.
Stability problem

Study directional/generalized derivatives of:
$w \mapsto VA(w;F(w))$
$w \mapsto VA(w;F(w))$

around such equilibria, including active-set switching.
Final interpretation

The strongest current result is:

Mature AIEF topology is a constraint-balanced boundary equilibrium. Resonance adaptation remains active, but positivity and endpoint-capacity admissibility remove its first-order effect. Deterministic probes support the proposed directional derivative of the ideal admissibility map, while the physical field is canceled to numerical precision.

The next bottleneck is no longer entropy geometry or coordinate validation. It is local stability of this boundary equilibrium under feasible state perturbations.

That is an AIEF-specific mathematical problem and the appropriate next target.

Good, good, but think about this:

I think there are two quite different parts to what you've written:

the AIEF active-boundary interpretation, and
the quasiperiodic resonance discussion.

The first is tightly connected to the numerical evidence you've generated. The second is mathematically plausible as a future direction, but it is much more speculative with respect to your current results. I would keep those narratives clearly separated.

I think the "constraint-balanced equilibrium" language is a significant improvement

This is a better characterization than simply saying "equilibrium."

Your numerics support

[
|F|\not\approx0,
]

while

[
|V_{\mathcal A}|\approx10^{-13}.
]

Those are fundamentally different statements.

The unconstrained adaptive dynamics are not at equilibrium.

Rather,

the projected (or admissibility-modified) dynamics are.

That distinction matters.

In optimization language this resembles a constrained stationarity condition.

In dynamical systems language it is closer to saying

the projected vector field vanishes,

rather than

the original vector field vanishes.

That is a much more precise description.

The KKT analogy is useful—but only as an analogy

I think you're using it appropriately.

The similarities are real:

active inequality constraints,
complementarity-like behavior,
multipliers associated with saturated constraints,
boundary stationarity.

But there is also an important difference.

Your admissibility operator uses

[

\max(\alpha_i,\alpha_j),
]

which comes from the implemented endpoint scaling.

That is not the same object as a classical Lagrange multiplier construction.

So I would continue phrasing it as

complementarity-like

or

KKT-inspired

rather than suggesting the mature states literally satisfy KKT conditions.

I think the directional-derivative theorem has become much more believable

Earlier,

the candidate

[
V_{\mathcal A}

\widetilde F

w\max(\alpha_i,\alpha_j)
]

was essentially a heuristic expansion.

Now you have

explicit finite-difference checks,
outward probes,
random probes,
numerical agreement with the expected orders.

That doesn't prove the theorem.

But it makes the theorem plausible.

In particular,

your observation that

componentwise max,

componentwise positive part,

minimum,

and multiplication

are all Hadamard directionally differentiable

is exactly the right analytical route.

That is much stronger than trying to differentiate the entire algorithm at once.

I agree the huge apparent orders are not meaningful

I would essentially ignore

[
3.6,
\qquad
4.9,
]

as estimates of convergence order.

They're telling you

"this fit is unstable,"

not

"the algorithm is fourth-order."

The inactive-stratum results are the ones that carry real asymptotic information.

The β* interpretation has become clearer

This is actually a stronger observation than before.

At mature states

$$
V_{\mathcal A}\approx0,
$$

while

$$
H^{-1}F\neq0.
$$

So the least-squares problem has no finite positive solution.

That's not merely a numerical inconvenience.

It reflects a structural difference between the equilibrium conditions.

Current AIEF satisfies

$$
V_{\mathcal A}(w,F)=0,
$$

while mirror dynamics would require

$$
H^{-1}F=0,
$$

which (in the interior, where H is invertible) is equivalent to

$$
F=0.
$$

Those are genuinely different equilibrium conditions.

That is probably the strongest argument so far that mirror dynamics are not just a different numerical realization.

The perturbation program is exactly the experiment I'd want next

I especially like the restoring coefficient

$$
\kappa_{\rm restore}
\frac{(w_\delta-w^)^\top V_{\mathcal A}(w_\delta)}{|w_\delta-w^|^2}.
$$

That is a clean local diagnostic.

Notice it does not require a global Lyapunov function.

It simply measures whether the projected vector field points back toward the mature state.

The three perturbation classes also separate different mechanisms:

radial perturbations test recovery after creating slack,
interior-mixing perturbations test support changes,
tangent perturbations test dynamics confined to an active face.

That's a very natural decomposition.

I would slightly soften one conclusion

You write:

"The candidate directional field has moved ... to a numerically confirmed first-order description..."

I would instead say something like

"The candidate directional field is numerically consistent with a first-order description of the ideal admissibility map on the tested strata."

The difference is subtle.

You have very good numerical evidence,

but the rigorous identification still depends on the analytical proof.

The quasiperiodic resonance section is reasonable—but belongs later

I don't disagree with the mathematics you summarize.

Multi-frequency delayed systems naturally admit

quasiperiodic invariant tori,
resonance modules,
torus bifurcations,
KAM-type questions,

under suitable conditions.

And your AIEF model certainly contains

multiple intrinsic frequencies,
delays,
slow adaptation.

So it is a plausible future research direction.

What I would avoid is letting it appear connected to the active-boundary results.

Right now the active-boundary work is about

nonsmooth admissibility,
directional derivatives,
projected dynamics,
local equilibria.

Quasiperiodicity is about

center dynamics,
spectral structure,
long-time oscillatory behavior.

Those are largely orthogonal questions.

I think your own table captures this correctly:

"Required for the immediate bottleneck: No."

That's exactly right.

The project has become much more modular

What I like most about where this has arrived is that the mathematical program now naturally decomposes into independent layers:

Interior geometry

Legendre coordinates.
Hessian metric.
Coordinate consistency.

Admissibility operator

Directional differentiability.
Candidate ($V_{\mathcal A}$).
Constraint-balanced equilibria.

Local stability

Linearization of the projected field.
Restoring coefficients.
Generalized Jacobians if needed.

Global dynamics

Attractors.
Active-set switching.
Possibly quasiperiodic behavior.

Each layer builds on the previous one but does not depend on proving everything at once.

Overall assessment

The active-boundary analysis is, in my view, the more significant development. The numerical evidence now supports a coherent picture in which mature AIEF states are constraint-balanced equilibria: the unconstrained adaptive field remains nonzero, but the admissibility mechanism removes

its first-order effect, yielding an induced boundary field that is numerically indistinguishable from zero. That is a substantially more precise mechanistic interpretation than simply calling the mature states "fixed points."

The next mathematical milestone is also much sharper than before. Rather than further validating the entropy coordinates, the key task is to rigorously establish the directional differentiability of the ideal clip-and-scale admissibility map and derive the candidate field (V_{\mathcal A}) under the appropriate piecewise conditions. If that succeeds, the restoring-coefficient and perturbation analyses you've outlined become the natural next step for studying the local stability of these constraint-balanced equilibria.

By contrast, the quasiperiodic resonance discussion is a reasonable roadmap for later work, but it remains downstream of the current bottleneck. It becomes most compelling after the local projected dynamics are understood, since only then does it make sense to analyze how the fast oscillatory subsystem interacts with the boundary-constrained adaptive evolution over longer timescales.

The modular decomposition is correct. One further distinction is important: the current evidence supports a stationary adaptive component, not necessarily an equilibrium of the complete delayed oscillator system.
1. BB-stationarity is not full-state equilibrium

At the split:
$\|FB\| \approx 0, \|VA,B\| \approx 0.$
$\|FB\| \approx 0, \|VA,B\| \approx 0.$

This establishes instantaneous constraint balance for the BB-subsystem.

But the complete state includes:
$X = (\theta, \text{history}, \text{CEMA}, B).$
$X = (\theta, \text{history}, \text{CEMA}, B).$

Phases continue evolving, history shifts, and EMA continues updating. Therefore:
$\dot{X} \neq 0$
$\dot{X} = 0$

even if:
$\dot{B}_{\text{admissible}} \approx 0.$
$\dot{B}_{\text{admissible}} \approx 0.$

A precise description is:

The mature adaptive topology lies on a constraint-balanced stationary fiber of the BB-subsystem, while the fast phase/history/EMA subsystem continues evolving.

If the fast state is periodic or quasiperiodic while BB remains fixed, this is closer to an invariant skew-product state than a full equilibrium.

To claim persistent BB-stationarity, evaluate:
VA(B(t);F(t))
VA(B(t);F(t))

at multiple mature times—not only at HIST_END. Binary threshold persistence shows edge membership is fixed, but continuous weights can still vary slightly.

That requires a targeted run storing either:

B(t)B(t) and the unconstrained field;
or just ‖VA(t)‖‖VA(t)‖ and selected diagnostics.

2. KKT-like is the right terminology

The mature condition resembles complementarity:

zero edge plus nonpositive forcing;
saturated row plus outward forcing canceled by scaling;
selected positive edge plus capacity-associated correction.

But the correction:
wijmax(αi,αj)
wijmax(αi,αj)

does not have the additive form of conventional node multipliers:
wij(λi+λj).
wij(λi+λj).

So the result is not a classical KKT system unless another equivalent variational characterization is discovered.

Use:

constraint-balanced stationarity;
complementarity-like equilibrium;
KKT-inspired active-set balance.

3. The directional theorem appears within reach

For the ideal map, the main ingredients are:

positive part;
linear incidence map;
smooth reciprocal away from zero row mass;
scalar min;
endpoint min;
product.

These maps are locally Lipschitz and Hadamard directionally differentiable. Directional chain rules should yield the candidate formula globally on the feasible polytope:
DAe(w;F)=
DAe(w;F)=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j).
]

The derivative is:

single-valued in each direction;
positively homogeneous;
generally nonlinear in FF;
not a Fréchet derivative at active-set switches.

A suitable theorem statement is:

For positive capacities cici, the ideal positivity-plus-symmetric-cap admissibility map is locally Lipschitz and Hadamard directionally differentiable at every feasible ww. Its directional derivative is the stated piecewise homogeneous field.

The numerical probes support the theorem but are not part of its proof.
4. Restore analysis is conditional unless the fast state evolves

For a perturbed topology:
wδ=w∗+δd,
wδ=w∗+δd,

computing:
VA(wδ;F(wδ,θ∗,C∗,H∗))
VA(wδ;F(wδ,θ∗,C∗,H∗))

holds the fast state fixed.

The restoring coefficient:
κrestore=
κrestore=

\frac{
(w_\delta-w^)^\top
V_{\mathcal A}(w_\delta)
}{
|w_\delta-w^|^2}
]

then measures frozen-fast-state radial restoration.

It does not prove stability of the coupled delayed system, because θ,C,θ,C, and history subsequently respond to the topology perturbation.

A complete program should have two levels:

Frozen-fast local restoration: analytic directional-field study.
Coupled restoration: short trajectory following the same feasible perturbation.

Also, κrestore>0κrestore>0 only measures the component pointing back toward the reference. A large transverse component can still produce non-normal transient behavior. Report:

radial coefficient;
cosine with the restoring direction;
transverse norm;
total field norm.

5. Mirror equilibrium distinction is interior-only

In the strict interior:
$H^{-1}F=0 \Longleftrightarrow F=0$,
$H^{-1}F=0 \Longleftrightarrow F=0$,

because $H^{-1}H^{-1}$ is positive definite.

Therefore the current boundary state with $F \neq 0F \quad =0$ cannot be a finite-interior mirror equilibrium.

But as:
$\varepsilon \to 0$,
$\varepsilon \to 0$,

some eigenvalues of $H^{-1}H^{-1}$ approach zero. A nonzero forcing component in those singular directions may be suppressed:
$H\varepsilon^{-1}F \to 0$
$H\varepsilon^{-1}F \to 0$

even with $F \neq 0F \quad =0$.

Thus a limiting mirror trajectory might approach the same boundary state asymptotically, despite having no corresponding strict-interior equilibrium.

The correct statement is:

Current AIEF and finite-interior mirror dynamics have different equilibrium conditions. Whether singular mirror dynamics approach the same boundary attractor remains open.

6. Active-stratum exponents should not be interpreted

The inactive results are robust:

text

current versus coordinate: $p \approx 2$
current versus mirror: $p \approx 1$

The active-stratum values around 3–5 are fitting artifacts caused by:

too few timesteps in one stratum;
stratum changes as timestep varies;
cancellation;
finite-precision effects.

They demonstrate that active and inactive behavior differ, but do not estimate meaningful asymptotic order.
7. The quasiperiodic program should remain separate

Quasiperiodicity could eventually matter because the fast phase/history subsystem may continue on a periodic or quasiperiodic invariant set while the topology is stationary.

But the immediate program concerns:

active-set geometry;
directional differentiability;
local topology restoration;
singular boundary limits.

A later fast–slow or skew-product analysis could ask whether the forcing averaged over a phase torus produces the observed constraint-balanced BB-stationarity. That should not be mixed into the current directional-map theorem.
Recommended next milestones
Analytical

Prove Hadamard directional differentiability of the ideal map.
Prove tangent feasibility.

Characterize:
VA(w;F)=0.
VA(w;F)=0.
Derive generalized Jacobians on fixed active strata.

Numerical

Perturb mature ww radially, tangentially, and through interior mixing.
Measure frozen-fast restoring and transverse components.
Repeat with full coupled short trajectories.
Sample ‖VA(t)‖‖VA(t)‖ throughout the mature future window.
Analyze epsilon-dependent mirror suppression separately.

Best current statement

Mature AIEF topology exhibits constraint-balanced stationarity in its adaptive component: the unconstrained adaptive forcing remains nonzero, but positivity and endpoint-capacity admissibility cancel its first-order effect at the tested split states. Deterministic probes are numerically consistent with the proposed directional derivative of the ideal admissibility map. Establishing that derivative analytically and studying restoration under feasible state perturbations are the next AIEF-specific steps. The full delayed oscillator state remains dynamic, so this should not yet be described as a complete-system equilibrium.

That keeps the active-boundary result strong without conflating it with full-state equilibrium, mirror-limit behavior, or later quasiperiodic questions.

okay but:

I think this is the strongest and most disciplined formulation so far. The main reason is that you've now separated three different notions of "stationarity", which were previously being conflated:

stationarity of the adaptive update ((B)),
stationarity of the projected adaptive dynamics ((V_{\mathcal A}=0)),
equilibrium of the full delayed dynamical system ((\dot X=0)).

Those are mathematically different objects, and your current wording respects that distinction.

There are only a few places where I'd tighten the claims.

"Constraint-balanced stationary fiber" is much more precise

I think this is the biggest conceptual improvement.

Your state is really

$$
X=(\theta,H,C,B).
$$

The evidence currently concerns only the (B)-component.

You have shown evidence that

$$
V_{\mathcal A,B}(B,F)\approx0.
$$

You have not shown

$$
\dot\theta=0,\qquad
\dot H=0,\qquad
\dot C=0.
$$

Indeed, from the construction of the model, the phase subsystem is expected to continue evolving.

So I would avoid the word "equilibrium" unless it is always qualified.

I think

constraint-balanced stationary fiber of the adaptive subsystem

is actually a better description.

That language naturally suggests a skew-product structure without asserting one has already been proved.

Time persistence is now the key empirical question

This is the most important next numerical test.

Right now the evidence says

$$
V_{\mathcal A}(t_*)
\approx0
$$

at one mature snapshot.

The stronger statement would be

$$
V_{\mathcal A}(t)
\approx0
$$

throughout the mature regime.

Those are different.

Fortunately the experiment is simple.

Store, over the mature window,

$(B(t))$,
$(F(t))$,
$(|V_{\mathcal A}(t)|)$,
active-set counts,

and see whether the cancellation persists.

If it does,

then "constraint-balanced stationary fiber" becomes much stronger than a single-time observation.

The theorem on directional differentiability now looks well scoped

I particularly like that you've stopped tying the theorem to entropy.

The theorem is really about the admissibility operator.

The hypotheses are clear:

feasible point,
positive capacities,
ideal clip-and-scale map,
no denominator safeguard.

The conclusion is

[
D\mathcal A(w;F)

V_{\mathcal A}(w;F).
]

That is a self-contained mathematical statement.

Entropy geometry then becomes one analytical framework for studying that field, rather than part of the theorem itself.

I think that's a cleaner separation.

The restoring analysis should indeed be split into two problems

This is an important refinement.

Holding

[
(\theta,H,C)
]

fixed measures

the response of the adaptive projection operator.

Allowing them to evolve measures

the response of the coupled delayed system.

Those are not the same stability question.

Your proposed hierarchy

frozen-fast,
coupled,

is exactly how I'd organize it.

I agree with softening the mirror conclusion

This is one place where your revision improves the earlier interpretation.

Previously it was tempting to conclude

mirror cannot converge to the same mature state.

Strictly speaking,

the numerics only establish that

the finite-interior equilibrium conditions differ.

The singular limit

$$
\varepsilon\to0
$$

is another matter.

Since

$$
H_\varepsilon^{-1}
$$

degenerates,

one cannot automatically carry the interior equilibrium argument to the boundary.

So

whether singular mirror dynamics approach the same boundary attractor remains open

is the right level of claim.

I would be careful with generalized Jacobians

You list

derive generalized Jacobians.

I think that's reasonable, but I'd specify the notion.

For a locally Lipschitz, piecewise-defined map like yours, the natural candidates are things such as the Clarke generalized Jacobian or related generalized derivatives. Those tools are designed precisely for nonsmooth mappings. However, because you've already framed the operator in terms of Hadamard directional differentiability, it may turn out that directional derivatives alone suffice for the first stage of the local analysis. The generalized Jacobian becomes especially useful once you start asking about linearization across active-set switches rather than within a fixed stratum.

So I'd treat it as a natural extension rather than an immediate prerequisite.

The quasiperiodic separation is now exactly right

I think you've now placed it correctly.

The present work concerns

admissibility,
active sets,
directional derivatives,
local restoration.

The quasiperiodic program concerns

invariant sets of the fast subsystem,
averaging,
slow-fast coupling.

Those are connected,

but only after the adaptive operator has been understood.

So I would continue treating them as separate research threads.

One additional structural observation

There is one thing I'd add to your hierarchy.

Right now you have

[
F
\longrightarrow
V_{\mathcal A}.
]

It may be useful to think of this as defining a nonlinear operator

[
\Pi_{\mathcal A}(w):F\mapsto V_{\mathcal A}(w;F),
]

parameterized by the current topology (w).

Then the mature condition becomes

[
\Pi_{\mathcal A}(w^)F(w^)=0.
]

That separates

the unconstrained adaptive law (F),
from the admissibility mechanism (\Pi_{\mathcal A}).

It also makes the comparison with mirror dynamics conceptually cleaner:

mirror modifies the vector field itself,
admissibility applies a state-dependent projection-like operator to the vector field.

Whether (\Pi_{\mathcal A}) ultimately satisfies all the algebraic properties of a projection is a separate question, but treating it as an operator can help organize both the analysis and the notation.

Overall assessment

I think your "best current statement" is appropriately calibrated to the evidence.

The strongest supported conclusion is no longer that the system has reached an equilibrium in the usual dynamical-systems sense. Rather, it is that the adaptive component exhibits constraint-balanced stationarity at the tested mature split states: the unconstrained adaptive forcing remains appreciable, yet the idealized positivity-and-capacity admissibility mechanism removes its first-order effect to numerical precision. The probe experiments are consistent with the proposed Hadamard directional derivative of the ideal admissibility map, but they serve as numerical validation rather than proof.

From there, the roadmap becomes quite coherent:

prove directional differentiability of the ideal admissibility operator,
characterize the resulting field (V_{\mathcal A}) and its tangent-feasibility properties,
study frozen-fast restoration around mature adaptive states,
then extend to the coupled delayed dynamics by allowing the phase, history, and EMA variables to evolve.

That sequence keeps the mathematical claims aligned with the current evidence while clearly separating established results from the remaining analytical questions.

The operator viewpoint is especially productive. Once the candidate directional formula is established, it has stronger algebraic structure than merely being “projection-like.”
1. Define the state-dependent admissibility response

Let:
Pw(F):=VA(w;F).
Pw(F):=VA(w;F).

For edge e=(i,j)e=(i,j):
Pw,e(F)=
Pw,e(F)=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j),
]

with positivity-modified forcing:
F~e={Fe,we>0,max(Fe,0),we=0,
F

e={Fe,max(Fe,0),we>0,we=0,

and:
αi={max((AF~)i,0)ci,(Aw)i=ci,0,(Aw)i<ci.
αi= ⎩
⎨
⎧cimax((AF
)i,0),0,(Aw)i=ci,(Aw)i<ci.

Then mature adaptive stationarity is:
Pw∗(F(w∗,x∗))≈0
Pw∗(F(w∗,x∗))≈0

where x∗=(θ,H,C)x∗=(θ,H,C) is the contemporaneous fast state.

This cleanly separates:

unconstrained adaptation FF;
state-dependent admissibility response PwPw.

2. PwPw may actually be a nonlinear tangent-cone projection

Let the tangent cone of:
C={w≥0, Aw≤c}
C={w≥0, Aw≤c}

at ww be:
TC(w)=
TC(w)=

\left{
v:
v_e\ge0\text{ when }w_e=0,\quad
(Av)_i\le0\text{ when }(Aw)_i=c_i
\right}.
]

The candidate field satisfies:

Pw(F)∈TC(w).
Pw(F)∈TC(w).

More strongly, suppose:
v∈TC(w).
v∈TC(w).

Then:

zero-edge positivity already holds, so v~=vv
=v;
every saturated row has (Av)i≤0(Av)i≤0, so αi=0αi=0;
therefore:
Pw(v)=v.
Pw(v)=v.

Hence:
Pw|TC(w)=
Pw|TC(w)=

I.
]

Because its output lies in the tangent cone:
Pw(Pw(F))=
Pw(Pw(F))=

\mathcal P_w(F).
]

Thus, if the directional formula is proved:
Pw2=Pw,range(Pw)=TC(w).
Pw2=Pw,range(Pw)=TC(w).

It is therefore a genuine nonlinear retraction/projection onto the tangent cone in an algebraic sense.

It is not necessarily:

linear;
orthogonal;
Euclidean nearest-point;
nonexpansive;
self-adjoint.

But “state-dependent nonlinear tangent projection” may ultimately be justified, rather than only “projection-like.”
3. Additional algebraic properties

The formula also gives:
Positive homogeneity

For $\lambda \geq 0 \lambda \geq 0$:
$P_w(\lambda F) =$
$P_w(\lambda F) =$

\lambda\mathcal P_w(F).
]
Non-additivity

Generally:
$P_w(F+G) \neq P_w(F) + P_w(G)$
$P_w(F+G) = P_w(F) + P_w(G)$

because of positivity clipping and endpoint max.
Piecewise linearity in FF

For fixed:

zero-edge clipping pattern;
outward-row set;
endpoint max ordering;

PwPw is linear in FF.

Switching these patterns produces its nonsmoothness.

This structure is well suited to semismooth and Bouligand analysis.
4. The directional theorem should precede generalized Jacobians

A clean progression is:
Theorem 1

The ideal admissibility map AA is locally Lipschitz and Hadamard directionally differentiable at every feasible state.
Theorem 2
DA(w;F)=Pw(F).
DA(w;F)=Pw(F).
Proposition
Pw(F)∈TC(w),
Pw(F)∈TC(w),
Pw(v)=v∀v∈TC(w),
Pw(v)=v∀v∈TC(w),

and therefore:
Pw2=Pw.
Pw2=Pw.

Only afterward is it necessary to study:

Clarke generalized Jacobians;
Bouligand subdifferentials;
semismooth Newton structure;
switching-stratum stability.

The directional map may be sufficient for the first local-stability results.
5. Constraint-balanced stationarity is conditional on the fast state

The mature condition should be written:
Pw∗[F(w∗,x(t))]≈0,
Pw∗[F(w∗,x(t))]≈0,

not simply:
Pw∗F(w∗)=0.
Pw∗F(w∗)=0.

The fast state x(t)x(t) may continue evolving. If the cancellation holds throughout the mature window, then w∗w∗ is an invariant adaptive fiber over the fast trajectory.

If it holds only at the split snapshot, it is instantaneous conditional stationarity.

Therefore the next direct numerical check should record, over the mature window:

‖F(t)‖‖F(t)‖;
‖Pw(t)F(t)‖‖Pw(t)F(t)‖;
ratio ‖PF‖/‖F‖‖PF‖/‖F‖;
zero-edge set;
saturated-node set;
active-set switching counts;
distance of w(t)w(t) from the split topology.

This is required before adopting “stationary fiber” as a trajectory-wide statement.
6. Frozen-fast and coupled restoration are different operators
Frozen-fast restoration

Perturb w∗w∗ while holding x∗x∗ fixed:
w˙=
w˙=

\mathcal P_wF(w,x^*).
]

This isolates the adaptive admissibility mechanism.
Coupled restoration

Perturb w∗w∗ and evolve:
x˙=G(x,w),
x˙=G(x,w),
w˙=PwF(w,x).
w˙=PwF(w,x).

This measures full delayed-system recovery.

A restoring coefficient in the frozen-fast problem does not prove stability of the coupled system. Conversely, coupled recovery can occur through fast-state adaptation even when the frozen-fast field is not restoring.

7. The mature condition resembles constrained forcing balance

Because:

F≠0,PwF=0,

F =0,PwF=0,

the unconstrained field lies in the kernel of the nonlinear tangent projection at the mature state.

For positive edges, stationarity requires:

Fe=

Fe=

w_e\max(\alpha_i,\alpha_j).

]

For zero edges:

Fe≤0.

Fe≤0.

This is a concrete complementarity-like characterization of selected versus suppressed edges. It may be more directly useful than a KKT analogy.

It also explains future shuffle:

driver reassignment changes FF;

the old topology no longer satisfies the balance condition;

weights leave the mature support and eventually collapse below threshold.

8. Mirror comparison becomes operator comparison

Current AIEF:

w˙=

w˙=

\mathcal P_wF.

]

Interior mirror model:
w˙=
w˙=

H(w)^{-1}F/\beta.
]

The comparison is therefore between:

a nonlinear, idempotent, tangent-cone retraction;
a smooth metric preconditioner.

At the mature state:
PwF≈0
PwF≈0

while:
H−1F≠0
H−1F =0

at finite interiorization.

No finite positive ββ aligns them.

In a singular boundary limit, H−1FH−1F might vanish, but that is a separate asymptotic question.
9. Quasiperiodic dynamics remain downstream

If x(t)x(t) is periodic or quasiperiodic while:
Pw∗F(w∗,x(t))=0
Pw∗F(w∗,x(t))=0

for all tt, then w∗w∗ is stationary over an oscillatory base flow.

That could eventually be treated using:

skew-product systems;
averaging over invariant tori;
slow–fast reductions.

But the immediate task is first to verify the cancellation along the mature trajectory. No quasiperiodic machinery is required yet.
Recommended next sequence

Prove the directional formula.
Prove tangent-cone range and idempotence.
Instrument a mature trajectory with $\|F(t)\|\|F(t)\|$ and $\|PF(t)\|\|PF(t)\|$.
Record active-set switching.
Perform frozen-fast feasible perturbations.
Perform corresponding coupled perturbations.
Only then derive generalized Jacobians or fast-subsystem averaging.

Refined statement

At tested mature split states, the AIEF adaptive component exhibits constraint-balanced conditional stationarity: a nonzero unconstrained adaptive field lies almost entirely in the kernel of a state-dependent nonlinear tangent projection induced by positivity clipping and symmetric endpoint-capacity scaling. The full delayed oscillator state remains dynamic. Establishing whether this cancellation persists along the mature trajectory, and whether the induced tangent projection restores feasible perturbations, are the next analytical and numerical questions.

This operator formulation is the cleanest organization of the evidence so far.

Create the necessary code to accomplish our net step.
WriteAIEF_1_4_stationarity_perturbation_analysis.py1037 lines
AIEF_1_4_stationarity_perturbation_analysis.pyPYTHON
#!/usr/bin/env python3
"""Mature-stationarity and feasible-perturbation analysis for AIEF-1.4.

Using saved split states, this script:

1. Replays the mature continuation while recording the unconstrained adaptive
field F and the ideal AIEF directional field P_w(F).
2. Tests whether constraint-balanced adaptive stationarity persists throughout
the mature future window and records active-set switching.

3. Applies frozen-fast feasible perturbations (radial, interior-mixing, tangent) and measures radial restoration and transverse motion.
4. Evolves the same perturbations in the full coupled delayed dynamics over a short horizon and measures recovery relative to an unperturbed reference.

The ideal directional field excludes the fixed denominator safeguard. The actual replay uses the original safeguard so final-state reproduction can be checked against the frozen AIEF-1.4 output.
"""

```
from __future__ import annotations

import argparse
import hashlib
import json
import math
import os
from collections import deque
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Iterable

import numpy as np
import pandas as pd



DEFAULT_RUN_HASH = "5a9f093a2fe0"
DEFAULT_SCALAR = (
f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_scalar_summary.csv"
)
DEFAULT_NPZ = (
f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_branch_diagnostics.npz"
)
DEFAULT_MANIFEST = f"runs/{DEFAULT_RUN_HASH}/manifest.json"
DEFAULT_OUTPUT_DIR = "aief_1_4_stationarity_perturbation"
DEFAULT_MAX_SEEDS = 5
DEFAULT_DELTAS = "1e-4,1e-3,1e-2,5e-2"
DEFAULT_COUPLED_TIME = 50.0
DEFAULT_SAMPLE_STRIDE = 50
DEFAULT_BOUNDARY_TOL = 1e-9
```

```
DEFAULT_STATIONARITY_RATIO_TOL = 1e-8
ZERO_TOL = 1e-14

PERTURBATION_TYPES = (
"radial_inward",
"interior_mixing",
"tangent_face",
)



@dataclass
class DynamicState:
theta: np.ndarray
B: np.ndarray
coherence_acc: np.ndarray
history: deque[np.ndarray]

def clone(self) -> "DynamicState":
return DynamicState(
theta=self.theta.copy(),
B=self.B.copy(),
coherence_acc=self.coherence_acc.copy(),
history=deque(
(item.copy() for item in self.history),
maxlen=self.history.maxlen,
),
)



def file_sha256(path: str) -> str:
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(lambda: handle.read(1024 * 1024), b""):
digest.update(block)
return digest.hexdigest()



def parse_float_list(value: str) -> list[float]:
try:
```

```
values = [float(item.strip()) for item in value.split(",") if item.strip()]
except ValueError as exc:
raise argparse.ArgumentTypeError("expected comma-separated floats") from exc
if not values or any(item <= 0.0 for item in values):
raise argparse.ArgumentTypeError("all values must be positive")
return sorted(set(values))



def prefix_key(seed: str, metric: str) -> str:
return f"prefix_seed_{seed}_{metric}"



def branch_key(branch: str, seed: str, metric: str) -> str:
return f"{branch}_seed_{seed}_{metric}"



def incidence_matrix(node_count: int) -> tuple[np.ndarray, tuple[np.ndarray, np.ndarray]]:
edges = np.triu_indices(node_count, k=1)
A = np.zeros((node_count, len(edges[0])), dtype=float)
A[edges[0], np.arange(len(edges[0]))] = 1.0
A[edges[1], np.arange(len(edges[0]))] = 1.0
return A, edges



def vector_to_matrix(
w: np.ndarray,
node_count: int,
edges: tuple[np.ndarray, np.ndarray],
) -> np.ndarray:
B = np.zeros((node_count, node_count), dtype=float)
B[edges] = w
B[(edges[1], edges[0])] = w
return B



def load_split_state(
data: Any,
seed: str,
```

```
) -> DynamicState:
history_array = np.asarray(data[prefix_key(seed, "history_at_split")], dtype=float)
return DynamicState(
theta=np.asarray(data[prefix_key(seed, "theta_at_split")], dtype=float).copy(),
B=np.asarray(data[prefix_key(seed, "B_at_split")], dtype=float).copy(),
coherence_acc=np.asarray(
data[prefix_key(seed, "coherence_acc_at_split")]
).copy(),
history=deque(
(row.copy() for row in history_array),
maxlen=len(history_array),
),
)



def positivity_direction(w: np.ndarray, F: np.ndarray) -> np.ndarray:
modified = F.copy()
zero = w <= ZERO_TOL
modified[zero] = np.maximum(F[zero], 0.0)
return modified



def ideal_directional_field(
w: np.ndarray,
F: np.ndarray,
A: np.ndarray,
capacities: np.ndarray,
boundary_tolerance: float,
) -> tuple[np.ndarray, dict[str, Any], np.ndarray, np.ndarray]:
modified = positivity_direction(w, F)
row_sum = A @ w
slack = capacities - row_sum
saturated = slack <= boundary_tolerance
effective_capacity = capacities.copy()
effective_capacity[saturated] = row_sum[saturated]

row_direction = A @ modified
alpha = np.zeros(len(capacities), dtype=float)
outward = saturated & (row_direction > 0.0)
alpha[outward] = row_direction[outward] / effective_capacity[outward]
```

```
endpoints = [np.flatnonzero(A[:, edge]) for edge in range(A.shape[1])]
edge_alpha = np.array(
[max(alpha[nodes[0]], alpha[nodes[1]]) for nodes in endpoints],
dtype=float,
)
field = modified - w * edge_alpha
info = {
"F_norm": float(np.linalg.norm(F)),
"P_F_norm": float(np.linalg.norm(field)),
"P_F_over_F": float(
np.linalg.norm(field) / max(np.linalg.norm(F), ZERO_TOL)
),
"zero_edge_count": int(np.sum(w <= ZERO_TOL)),
"saturated_node_count": int(np.sum(saturated)),
"outward_saturated_node_count": int(np.sum(outward)),
"positivity_modified_edge_count": int(
np.sum((w <= ZERO_TOL) & (F < 0.0))
),
"maximum_capacity_snap": float(
np.max(np.abs(effective_capacity - capacities))
),
}
return field, info, saturated, outward



def compute_field_and_updated_coherence(
state: DynamicState,
model: dict[str, Any],
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
omega = np.asarray(model["omega"], dtype=float)
dt = float(model["dt"])
tau = float(model["tau"])
t_window = float(model["T_window"])
eta = float(model["eta"])
mu = float(model["mu"])
gamma = float(model["gamma"])

delayed = state.history[0]
dphi = (
state.theta[:, None]
```

```
- delayed[None, :]
- ((omega[:, None] + omega[None, :]) / 2.0) * tau
)
alpha = float(np.exp(-dt / t_window))
updated_coherence = (
alpha * state.coherence_acc + (1.0 - alpha) * np.exp(1j * dphi)
)
R = np.abs(updated_coherence)
R = 0.5 * (R + R.T)
np.fill_diagonal(R, 0.0)
F = (eta + mu) * R - mu - gamma * state.B
F = 0.5 * (F + F.T)
np.fill_diagonal(F, 0.0)
return R, F, updated_coherence



def actual_step(
state: DynamicState,
model: dict[str, Any],
capacities: np.ndarray,
A: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
boundary_tolerance: float,
) -> dict[str, Any]:
node_count = state.B.shape[0]
dt = float(model["dt"])
omega = np.asarray(model["omega"], dtype=float)
B_c = float(model["B_c"])


_, F_matrix, updated_coherence = compute_field_and_updated_coherence(
state, model
)
w = state.B[edges].copy()
F = F_matrix[edges].copy()
P_F, field_info, saturated, outward = ideal_directional_field(
w, F, A, capacities, boundary_tolerance
)


proposal = state.B + dt * F_matrix
proposal = np.maximum(proposal, 0.0)
proposal = 0.5 * (proposal + proposal.T)
```

```
np.fill_diagonal(proposal, 0.0)
proposal_w = proposal[edges]
row_sum_pre = A @ proposal_w
violating = row_sum_pre > capacities


projected = proposal.copy()
if np.any(violating):
row_sum_matrix = np.sum(proposal, axis=1)
scale = np.minimum(
1.0,
capacities / (row_sum_matrix + 1e-12),
)
projected *= np.minimum.outer(scale, scale)
np.fill_diagonal(projected, 0.0)


next_w = projected[edges]
finite_rate = (next_w - w) / dt
field_error = finite_rate - P_F


delayed = state.history[0]
coupling = np.sum(
projected * np.sin(delayed[None, :] - state.theta[:, None]), axis=1
)
state.B = projected
state.coherence_acc = updated_coherence
state.theta = np.mod(state.theta + dt * (omega + coupling), 2.0 * np.pi)
state.history.append(state.theta.copy())


return {
**field_info,
"finite_rate_norm": float(np.linalg.norm(finite_rate)),
"finite_rate_minus_P_F_norm": float(np.linalg.norm(field_error)),
"finite_rate_P_F_cosine": float(
finite_rate @ P_F
/ max(
np.linalg.norm(finite_rate) * np.linalg.norm(P_F),
np.finfo(float).tiny,
)
)
if np.linalg.norm(P_F) > 1e-12
else np.nan,
```

```
"actual_projection_active": bool(np.any(violating)),
"actual_active_node_count": int(np.sum(violating)),
"threshold_edge_count": int(np.sum(next_w > B_c)),
"saturated_mask": saturated,
"outward_mask": outward,
"zero_mask": w <= ZERO_TOL,
"P_F": P_F,
"F": F,
}



def mask_switch_count(previous: np.ndarray | None, current: np.ndarray) -> int:
if previous is None:
return 0
return int(np.sum(previous != current))



def replay_mature_trajectory(
split_state: DynamicState,
expected_final: np.ndarray,
model: dict[str, Any],
future_steps: int,
sample_stride: int,
capacities: np.ndarray,
A: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
boundary_tolerance: float,
stationarity_ratio_tolerance: float,
) -> tuple[dict[str, Any], list[dict[str, Any]]]:
state = split_state.clone()
rows: list[dict[str, Any]] = []
saturated_previous = outward_previous = zero_previous = None
saturated_switches = outward_switches = zero_switches = 0
stationary_steps = 0
ratio_sum = 0.0
ratio_max = 0.0
P_F_norm_max = 0.0
F_norm_sum = 0.0

for step in range(future_steps):
```

```
info = actual_step(
state,
model,
capacities,
A,
edges,
boundary_tolerance,
)
ratio = float(info["P_F_over_F"])
stationary_steps += int(ratio <= stationarity_ratio_tolerance)
ratio_sum += ratio
ratio_max = max(ratio_max, ratio)
P_F_norm_max = max(P_F_norm_max, float(info["P_F_norm"]))
F_norm_sum += float(info["F_norm"])


saturated_switches += mask_switch_count(
saturated_previous, info["saturated_mask"]
)
outward_switches += mask_switch_count(outward_previous, info["outward_mask"])
zero_switches += mask_switch_count(zero_previous, info["zero_mask"])
saturated_previous = info["saturated_mask"].copy()
outward_previous = info["outward_mask"].copy()
zero_previous = info["zero_mask"].copy()


if step % sample_stride == 0:
rows.append(
{
"step": step,
"time_since_split": (step + 1) * float(model["dt"]),
**{
key: value
for key, value in info.items()
if not isinstance(value, np.ndarray)
},
}
)


final_difference = float(np.linalg.norm(state.B - expected_final))
summary = {
"future_steps": future_steps,
"stationary_step_fraction": float(stationary_steps / future_steps),
```

```
"mean_P_F_over_F": float(ratio_sum / future_steps),
"max_P_F_over_F": ratio_max,
"max_P_F_norm": P_F_norm_max,
"mean_F_norm": float(F_norm_sum / future_steps),
"saturated_active_set_switches": saturated_switches,
"outward_active_set_switches": outward_switches,
"zero_edge_set_switches": zero_switches,
"final_replay_difference_frobenius": final_difference,
"final_replay_exact": bool(np.array_equal(state.B, expected_final)),
"final_replay_allclose_1e12": bool(
np.allclose(state.B, expected_final, rtol=0.0, atol=1e-12)
),
}
return summary, rows



def deterministic_rng(seed: str, label: str) -> np.random.Generator:
digest = hashlib.sha256(f"{seed}|{label}".encode()).digest()
return np.random.default_rng(int.from_bytes(digest[:8], "little"))



def strict_interior_reference(
capacities: np.ndarray,
A: np.ndarray,
) -> np.ndarray:
degrees = np.sum(A, axis=1)
edge_weight = float(np.min(capacities / (2.0 * degrees)))
return np.full(A.shape[1], edge_weight, dtype=float)



def tangent_direction(
w: np.ndarray,
A: np.ndarray,
capacities: np.ndarray,
boundary_tolerance: float,
rng: np.random.Generator,
) -> np.ndarray | None:
positive = w > ZERO_TOL
active_nodes = (capacities - A @ w) <= boundary_tolerance
support_indices = np.flatnonzero(positive)
```

```
if not len(support_indices):
return None
matrix = A[np.ix_(active_nodes, support_indices)]
if matrix.size == 0:
basis = np.eye(len(support_indices))
else:
_, singular_values, vh = np.linalg.svd(matrix, full_matrices=True)
rank = int(np.sum(singular_values > 1e-10))
basis = vh[rank:].T
if basis.shape[1] == 0:
return None
coefficients = rng.normal(size=basis.shape[1])
direction_support = basis @ coefficients
norm = float(np.linalg.norm(direction_support))
if norm <= ZERO_TOL:
return None
direction_support /= norm
direction = np.zeros_like(w)
direction[support_indices] = direction_support
return direction



def maximum_feasible_amplitude(
w: np.ndarray,
direction: np.ndarray,
A: np.ndarray,
capacities: np.ndarray,
) -> float:
limits = [np.inf]
negative = direction < 0.0
if np.any(negative):
limits.append(float(np.min(w[negative] / (-direction[negative]))))
row_direction = A @ direction
slack = capacities - A @ w
outward = row_direction > 0.0
if np.any(outward):
limits.append(float(np.min(slack[outward] / row_direction[outward])))
return max(0.0, min(limits))



def perturb_state(
```

```
w: np.ndarray,
perturbation: str,
delta: float,
A: np.ndarray,
capacities: np.ndarray,
boundary_tolerance: float,
rng: np.random.Generator,
) -> tuple[np.ndarray, float, str]:
norm_w = max(float(np.linalg.norm(w)), ZERO_TOL)
if perturbation == "radial_inward":
perturbed = (1.0 - delta) * w
return perturbed, float(np.linalg.norm(perturbed - w)), "ok"


if perturbation == "interior_mixing":
reference = strict_interior_reference(capacities, A)
perturbed = (1.0 - delta) * w + delta * reference
return perturbed, float(np.linalg.norm(perturbed - w)), "ok"


if perturbation == "tangent_face":
direction = tangent_direction(
w, A, capacities, boundary_tolerance, rng
)
if direction is None:
return w.copy(), 0.0, "no_tangent_direction"
requested = delta * norm_w
maximum = maximum_feasible_amplitude(w, direction, A, capacities)
amplitude = min(requested, 0.5 * maximum)
if not np.isfinite(amplitude) or amplitude <= 0.0:
return w.copy(), 0.0, "zero_feasible_amplitude"
perturbed = w + amplitude * direction
return perturbed, amplitude, "ok"


raise ValueError(f"unknown perturbation {perturbation}")



def frozen_fast_perturbations(
split_state: DynamicState,
model: dict[str, Any],
seed: str,
deltas: Iterable[float],
```

```
capacities: np.ndarray,
A: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
boundary_tolerance: float,
) -> list[dict[str, Any]]:
_, _, updated_coherence = compute_field_and_updated_coherence(
split_state, model
)
base = split_state.B[edges].copy()
rows: list[dict[str, Any]] = []


for perturbation in PERTURBATION_TYPES:
for delta in deltas:
rng = deterministic_rng(seed, f"{perturbation}|{delta}")
perturbed, magnitude, status = perturb_state(
base,
perturbation,
delta,
A,
capacities,
boundary_tolerance,
rng,
)
if status != "ok" or magnitude <= 0.0:
rows.append(
{
"seed": seed,
"perturbation": perturbation,
"delta": delta,
"status": status,
}
)
continue


B_perturbed = vector_to_matrix(
perturbed, split_state.B.shape[0], edges
)
R = np.abs(updated_coherence)
R = 0.5 * (R + R.T)
np.fill_diagonal(R, 0.0)
F_matrix = (
(float(model["eta"]) + float(model["mu"])) * R
```

```
- float(model["mu"])
- float(model["gamma"]) * B_perturbed
)
F_matrix = 0.5 * (F_matrix + F_matrix.T)
np.fill_diagonal(F_matrix, 0.0)
field, info, _, _ = ideal_directional_field(
perturbed,
F_matrix[edges],
A,
capacities,
boundary_tolerance,
)
displacement = perturbed - base
displacement_norm = float(np.linalg.norm(displacement))
restoring_direction = -displacement
radial_numerator = float(restoring_direction @ field)
restoring_coefficient = radial_numerator / max(
displacement_norm**2, ZERO_TOL
)
restoring_component = (
radial_numerator / max(displacement_norm, ZERO_TOL)
)
radial_vector = (
restoring_component
* restoring_direction
/ max(displacement_norm, ZERO_TOL)
)
transverse = field - radial_vector
cosine = float(
restoring_direction @ field
/ max(
displacement_norm * np.linalg.norm(field),
np.finfo(float).tiny,
)
)
rows.append(
{
"seed": seed,
"perturbation": perturbation,
"delta": delta,
"status": status,
"requested_relative_delta": delta,
```

```
"actual_perturbation_norm": displacement_norm,
"relative_perturbation_norm": displacement_norm
/ max(np.linalg.norm(base), ZERO_TOL),
"field_norm": float(np.linalg.norm(field)),
"restoring_coefficient": restoring_coefficient,
"restoring_cosine": cosine,
"radial_component_norm": float(np.linalg.norm(radial_vector)),
"transverse_component_norm": float(np.linalg.norm(transverse)),
"transverse_to_field_ratio": float(
np.linalg.norm(transverse)
/ max(np.linalg.norm(field), ZERO_TOL)
),
**info,
}
)
return rows



def normalized_edge_distance(
B: np.ndarray,
reference: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
) -> float:
x = B[edges]
y = reference[edges]
return float(
np.linalg.norm(x - y) / max(np.linalg.norm(y), ZERO_TOL)
)



def threshold_jaccard(
B: np.ndarray,
reference: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
threshold: float,
) -> float:
x = B[edges] > threshold
y = reference[edges] > threshold
union = int(np.sum(x | y))
return float(np.sum(x & y) / union) if union else np.nan
```

```
def coupled_perturbation_runs(
split_state: DynamicState,
model: dict[str, Any],
seed: str,
deltas: Iterable[float],
capacities: np.ndarray,
A: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
boundary_tolerance: float,
coupled_steps: int,
sample_stride: int,
) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]:
threshold = float(model["B_c"])
reference_state = split_state.clone()
reference_samples: dict[int, np.ndarray] = {0: reference_state.B.copy()}
for step in range(1, coupled_steps + 1):
actual_step(
reference_state,
model,
capacities,
A,
edges,
boundary_tolerance,
)
if step % sample_stride == 0 or step == coupled_steps:
reference_samples[step] = reference_state.B.copy()


summary_rows: list[dict[str, Any]] = []
trajectory_rows: list[dict[str, Any]] = []
base_w = split_state.B[edges].copy()


for perturbation in PERTURBATION_TYPES:
for delta in deltas:
rng = deterministic_rng(seed, f"coupled|{perturbation}|{delta}")
perturbed_w, magnitude, status = perturb_state(
base_w,
perturbation,
delta,
A,
capacities,
```

```
boundary_tolerance,
rng,
)
if status != "ok" or magnitude <= 0.0:
summary_rows.append(
{
"seed": seed,
"perturbation": perturbation,
"delta": delta,
"status": status,
}
)
continue


state = split_state.clone()
state.B = vector_to_matrix(perturbed_w, state.B.shape[0], edges)
initial_distance = normalized_edge_distance(
state.B, reference_samples[0], edges
)
distances: list[float] = []
times: list[float] = []


for step in range(0, coupled_steps + 1):
if step in reference_samples:
reference_B = reference_samples[step]
distance = normalized_edge_distance(state.B, reference_B, edges)
jaccard = threshold_jaccard(
state.B, reference_B, edges, threshold
)
distances.append(distance)
times.append(step * float(model["dt"]))
trajectory_rows.append(
{
"seed": seed,
"perturbation": perturbation,
"delta": delta,
"step": step,
"time_since_perturbation": step * float(model["dt"]),
"normalized_distance_to_reference": distance,
"threshold_jaccard_to_reference": jaccard,
}
)
```

```
if step == coupled_steps:
break
actual_step(
state,
model,
capacities,
A,
edges,
boundary_tolerance,
)


distances_array = np.asarray(distances, dtype=float)
times_array = np.asarray(times, dtype=float)
threshold_50 = 0.5 * initial_distance
threshold_10 = 0.1 * initial_distance
indices_50 = np.flatnonzero(distances_array <= threshold_50)
indices_10 = np.flatnonzero(distances_array <= threshold_10)
summary_rows.append(
{
"seed": seed,
"perturbation": perturbation,
"delta": delta,
"status": "ok",
"initial_distance": initial_distance,
"minimum_distance": float(np.min(distances_array)),
"final_distance": float(distances_array[-1]),
"final_to_initial_distance_ratio": float(
distances_array[-1] / max(initial_distance, ZERO_TOL)
),
"time_to_50pct_distance": float(times_array[indices_50[0]])
if len(indices_50)
else np.nan,
"time_to_10pct_distance": float(times_array[indices_10[0]])
if len(indices_10)
else np.nan,
}
)
return summary_rows, trajectory_rows



def summarize_table(
```

```
frame: pd.DataFrame,
group_columns: list[str],
) -> pd.DataFrame:
identifier = set(group_columns) | {"seed", "status"}
metrics = [column for column in frame.columns if column not in identifier]
rows: list[dict[str, Any]] = []
for key, group in frame.groupby(group_columns, dropna=False):
if not isinstance(key, tuple):
key = (key,)
base = dict(zip(group_columns, key))
for metric in metrics:
raw = pd.to_numeric(group[metric], errors="coerce").to_numpy(float)
finite = raw[np.isfinite(raw)]
rows.append(
{
**base,
"metric": metric,
"n_total": int(len(raw)),
"n_valid": int(len(finite)),
"mean": float(np.mean(finite)) if len(finite) else np.nan,
"std": float(np.std(finite)) if len(finite) else np.nan,
"median": float(np.median(finite)) if len(finite) else np.nan,
"minimum": float(np.min(finite)) if len(finite) else np.nan,
"maximum": float(np.max(finite)) if len(finite) else np.nan,
}
)
return pd.DataFrame(rows)



def make_report(
stationarity: pd.DataFrame,
frozen_summary: pd.DataFrame,
coupled_summary: pd.DataFrame,
) -> str:
lines = [
"AIEF mature stationarity and feasible-perturbation analysis",
"============================================================",
"",
"Mature adaptive stationarity",
"----------------------------",
]
for _, row in stationarity.iterrows():
```

```
lines.append(
f" seed={row['seed']}: stationary_fraction="
f"{row['stationary_step_fraction']:.6g}, "
f"mean ||P_w(F)||/||F||={row['mean_P_F_over_F']:.6g}, "
f"max={row['max_P_F_over_F']:.6g}, "
f"zero-set switches={int(row['zero_edge_set_switches'])}, "
f"saturated-set switches={int(row['saturated_active_set_switches'])}, "
f"replay_exact={row['final_replay_exact']}"
)


lines.extend(["", "Frozen-fast restoring diagnostics", "--------------------------------"])
selected = frozen_summary.loc[
frozen_summary["metric"].isin(
[
"restoring_coefficient",
"restoring_cosine",
"transverse_to_field_ratio",
]
)
]
for (perturbation, delta), group in selected.groupby(
["perturbation", "delta"]
):
lines.append(f" {perturbation}, delta={delta:g}")
for _, row in group.iterrows():
lines.append(
f" {row['metric']}: median={row['median']:.6g}, "
f"min={row['minimum']:.6g}, max={row['maximum']:.6g}"
)


lines.extend(["", "Coupled restoration diagnostics", "-------------------------------"])
selected_coupled = coupled_summary.loc[
coupled_summary["metric"].isin(
[
"final_to_initial_distance_ratio",
"time_to_50pct_distance",
"time_to_10pct_distance",
]
)
]
for (perturbation, delta), group in selected_coupled.groupby(
["perturbation", "delta"]
```

```
):
lines.append(f" {perturbation}, delta={delta:g}")
for _, row in group.iterrows():
lines.append(
f" {row['metric']}: median={row['median']:.6g}, "
f"valid={int(row['n_valid'])}/{int(row['n_total'])}"
)


lines.extend(
[
"",
"Interpretation constraints",
"-------------------------",
"* Stationarity refers only to the adaptive B-component, not the full delayed state.",
"* Frozen-fast restoration holds theta/history/EMA fixed for one instantaneous field evaluation.",
"* Coupled restoration evolves all state variables and answers a different stability question.",
"* Positive radial restoration does not exclude large transverse or non-normal motion.",
"* The ideal directional field excludes the fixed denominator safeguard.",
"",
]
)
return "\n".join(lines)



def parse_arguments() -> argparse.Namespace:
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("--scalar", default=DEFAULT_SCALAR)
parser.add_argument("--npz", default=DEFAULT_NPZ)
parser.add_argument("--manifest", default=DEFAULT_MANIFEST)
parser.add_argument("--output-dir", default=DEFAULT_OUTPUT_DIR)
parser.add_argument("--max-seeds", type=int, default=DEFAULT_MAX_SEEDS)
parser.add_argument(
"--deltas", type=parse_float_list, default=parse_float_list(DEFAULT_DELTAS)
)
parser.add_argument(
"--coupled-time", type=float, default=DEFAULT_COUPLED_TIME
)
parser.add_argument(
"--sample-stride", type=int, default=DEFAULT_SAMPLE_STRIDE
)
parser.add_argument(
```

```
"--boundary-tol", type=float, default=DEFAULT_BOUNDARY_TOL
)
parser.add_argument(
"--stationarity-ratio-tol",
type=float,
default=DEFAULT_STATIONARITY_RATIO_TOL,
)
return parser.parse_args()



def main() -> None:
args = parse_arguments()
for path in (args.scalar, args.npz, args.manifest):
if not os.path.isfile(path):
raise FileNotFoundError(path)
if (
args.max_seeds < 0
or args.coupled_time <= 0.0
or args.sample_stride <= 0
or args.boundary_tol < 0.0
or args.stationarity_ratio_tol < 0.0
):
raise ValueError("invalid command-line settings")
os.makedirs(args.output_dir, exist_ok=True)


scalar = pd.read_csv(args.scalar, dtype={"seed": str})
scalar["seed"] = scalar["seed"].astype(str)
scalar["branch"] = scalar["branch"].astype(str)
with open(args.manifest, "r", encoding="utf-8") as handle:
manifest = json.load(handle)
model = manifest["model"]
node_count = int(model["N"])
future_steps = int(manifest["protocol"]["future_steps"])
capacity = (float(model["a"]) - float(model["delta"])) / float(model["q"])
capacities = np.full(node_count, capacity)
A, edges = incidence_matrix(node_count)
coupled_steps = int(round(args.coupled_time / float(model["dt"])))


seeds = sorted(
scalar.loc[scalar["branch"] == "continue", "seed"].astype(str), key=int
)
```

```
if args.max_seeds > 0:
seeds = seeds[: args.max_seeds]


stationarity_rows: list[dict[str, Any]] = []
stationarity_timeseries: list[dict[str, Any]] = []
frozen_rows: list[dict[str, Any]] = []
coupled_rows: list[dict[str, Any]] = []
coupled_trajectories: list[dict[str, Any]] = []


with np.load(args.npz, allow_pickle=False) as data:
for seed in seeds:
print(f"Analyzing seed {seed}")
split_state = load_split_state(data, seed)
expected_final = np.asarray(
data[branch_key("continue", seed, "B_final")], dtype=float
)
stationarity, samples = replay_mature_trajectory(
split_state,
expected_final,
model,
future_steps,
args.sample_stride,
capacities,
A,
edges,
args.boundary_tol,
args.stationarity_ratio_tol,
)
stationarity_rows.append({"seed": seed, **stationarity})
stationarity_timeseries.extend(
{"seed": seed, **row} for row in samples
)
frozen_rows.extend(
frozen_fast_perturbations(
split_state,
model,
seed,
args.deltas,
capacities,
A,
edges,
args.boundary_tol,
```

```
)
)
summaries, trajectories = coupled_perturbation_runs(
split_state,
model,
seed,
args.deltas,
capacities,
A,
edges,
args.boundary_tol,
coupled_steps,
args.sample_stride,
)
coupled_rows.extend(summaries)
coupled_trajectories.extend(trajectories)


stationarity_frame = pd.DataFrame(stationarity_rows)
stationarity_ts_frame = pd.DataFrame(stationarity_timeseries)
frozen_frame = pd.DataFrame(frozen_rows)
coupled_frame = pd.DataFrame(coupled_rows)
coupled_ts_frame = pd.DataFrame(coupled_trajectories)


frozen_summary = summarize_table(
frozen_frame, ["perturbation", "delta"]
)
coupled_summary = summarize_table(
coupled_frame, ["perturbation", "delta"]
)


outputs = {
"mature_stationarity_by_seed.csv": stationarity_frame,
"mature_stationarity_timeseries.csv": stationarity_ts_frame,
"frozen_fast_perturbations_by_seed.csv": frozen_frame,
"frozen_fast_perturbation_summary.csv": frozen_summary,
"coupled_perturbations_by_seed.csv": coupled_frame,
"coupled_perturbation_summary.csv": coupled_summary,
"coupled_perturbation_trajectories.csv": coupled_ts_frame,
}
for filename, frame in outputs.items():
frame.to_csv(os.path.join(args.output_dir, filename), index=False)
```

```
report = make_report(
stationarity_frame, frozen_summary, coupled_summary
)
with open(
os.path.join(args.output_dir, "stationarity_perturbation_report.txt"),
"w",
encoding="utf-8",
) as handle:
handle.write(report)
print("\n" + report)


analysis_manifest = {
"analysis": "AIEF mature stationarity and feasible perturbations",
"scalar_path": str(Path(args.scalar).resolve()),
"scalar_sha256": file_sha256(args.scalar),
"npz_path": str(Path(args.npz).resolve()),
"npz_sha256": file_sha256(args.npz),
"source_manifest_path": str(Path(args.manifest).resolve()),
"source_manifest_sha256": file_sha256(args.manifest),
"seeds": seeds,
"deltas": args.deltas,
"coupled_time": args.coupled_time,
"sample_stride": args.sample_stride,
"boundary_tolerance": args.boundary_tol,
"stationarity_ratio_tolerance": args.stationarity_ratio_tol,
"perturbation_types": list(PERTURBATION_TYPES),
"notes": [
"Stationarity applies only to the adaptive component.",
"Frozen-fast and coupled restoration are different experiments.",
"The ideal directional field excludes the denominator safeguard.",
"Tangent perturbations are restricted to the positive-edge support.",
],
}
with open(
os.path.join(args.output_dir, "analysis_manifest.json"),
"w",
encoding="utf-8",
) as handle:
json.dump(analysis_manifest, handle, indent=2, sort_keys=True)
handle.write("\n")
```

```
print(f"Saved analysis to {Path(args.output_dir).resolve()}")



if __name__ == "__main__":
main()
```
Open

Created the mature-stationarity and feasible-perturbation analyzer:

AIEF_1_4_stationarity_perturbation_analysis.py

It uses the existing frozen AIEF-1.4 artifacts. It does not require another production simulation.
Run

Bash

```
python AIEF_1_4_stationarity_perturbation_analysis.py
```

Explicit invocation:

Bash

```
python AIEF_1_4_stationarity_perturbation_analysis.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_stationarity_perturbation
```

It defaults to five representative seeds. To use all 40:

Bash

```
python AIEF_1_4_stationarity_perturbation_analysis.py \
--max-seeds 0
```

Default perturbation sizes:

text

1e-4, 1e-3, 1e-2, 5e-2

Default coupled recovery horizon:

text

50 simulation-time units

A smaller smoke run:

Bash

```
python AIEF_1_4_stationarity_perturbation_analysis.py \
--max-seeds 1 \
--deltas 0.001 \
--coupled-time 5
```

Mature-trajectory stationarity

The script reconstructs the complete split state:

BB,
phase,
EMA,
delay history.

It then exactly replays the complete mature future continuation while recording:
$\|F(t)\|$,
$\|F(t)\|$,
$\|Pw(t)F(t)\|$,
$\|Pw(t)F(t)\|$,
$\|PwF\|\|F\|$,
$\|F\|\|PwF\|$,

plus:

zero-edge count;
saturated-node count;
outward saturated-node count;
positivity-modified edge count;
actual projection activity;
threshold edge count;
finite-step versus ideal directional-field discrepancy.

It also counts changes in:

zero-edge support;
saturated-node active set;
outward-node active set.

At the end, it compares the replayed BfinalBfinal against the saved continuation branch and reports:

text

final_replay_exact
final_replay_allclose_1e12

This verifies that instrumentation has not changed the original dynamics.
Ideal directional field

It implements:
Pw,e(F)=
Pw,e(F)=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j),
]

with positivity clipping and saturated-node capacity response.

The ideal field omits the fixed denominator safeguard; the actual replay retains it to reproduce the saved simulation.

Frozen-fast perturbations

At the split, it perturbs only BB while holding:

phase,
EMA,
delay history

fixed during the instantaneous field evaluation.
Radial inward
wδ=(1−δ)w∗.
wδ=(1−δ)w∗.

Creates slack while preserving support.
Interior mixing
wδ=
wδ=

(1-\delta)w^*
+
\delta w_{\rm interior}.
]

Introduces positive mass on absent edges and positive row slack.
Tangent-face perturbation

Constructs a deterministic random direction satisfying:
Aactived=0
Aactived=0

on the positive-edge support, then scales it to preserve feasibility.

For each perturbation it calculates:
κrestore=
κrestore=

\frac{
(w_\delta-w^)^\top

\mathcal P_{w_\delta}F(w_\delta)
}{
|w_\delta-w^|^2
},
]

along with:

restoring cosine;
radial component;
transverse component;
transverse-to-field ratio;
active-set information;
actual perturbation magnitude.

This distinguishes radial restoration from transverse/non-normal motion.
Coupled perturbation recovery

The same feasible perturbations are then evolved with the full system:

phase evolves;
EMA evolves;
delay history evolves;
BB adapts and is projected.

An unperturbed continuation is evolved in parallel from the same split state. At every diagnostic sample, the script measures:

normalized edge-weight distance to the reference continuation;
threshold Jaccard similarity.

It reports:

initial distance;
minimum distance;
final distance;
final/initial distance ratio;
time to 50% distance reduction;

time to 90% distance reduction.

This cleanly separates frozen-fast local restoration from coupled delayed-system recovery.
Outputs

text

```
aief_1_4_stationarity_perturbation/
├── mature_stationarity_by_seed.csv
├── mature_stationarity_timeseries.csv
├── frozen_fast_perturbations_by_seed.csv
├── frozen_fast_perturbation_summary.csv
├── coupled_perturbations_by_seed.csv
├── coupled_perturbation_summary.csv
├── coupled_perturbation_trajectories.csv
├── stationarity_perturbation_report.txt
└── analysis_manifest.json
```

Validation

I syntax-tested and ran the script end-to-end against a complete AIEF split state.

The smoke test verified:

exact replay of the saved continuation;
mature stationarity instrumentation;
all three feasible perturbation classes;
frozen-fast restoring decomposition;
coupled recovery against a synchronized reference trajectory;
active-set tracking;
output and manifest generation.

In that technical smoke case, the adaptive component remained constraint-balanced throughout the complete mature replay, while radial/interior perturbations recovered rapidly and the tangent perturbation relaxed more slowly. Those are smoke-test observations only; the multi-seed run is needed for scientific interpretation.

Think about this as well though: I think this is now a very coherent mathematical organization. The biggest conceptual advance is that you've moved from thinking of the admissibility mechanism as a post-processing step to treating it as an operator with its own algebraic structure. If that structure survives a rigorous proof, it becomes an object worth studying in its own right, independently of the particular adaptive law (F). That said, there are a few places where I'd distinguish between what is already supported and what remains to be established. 1. The operator formulation is the right abstraction Defining [\mathcal P_w(F):=V_{\mathcal A}(w;F)] is, in my view, the cleanest notation you've introduced. It immediately separates two ingredients: - the adaptive law [F(w,x),] - the admissibility response [\mathcal P_w.] Then the evolution becomes [\dot w=\mathcal P_w(F(w,x)),] which is much cleaner than embedding clipping and scaling into the definition of (F). This also makes it possible to compare different adaptive laws while holding the admissibility mechanism fixed. 2. Idempotence needs one additional proof step This is the one place where I would slow down. You argue [\mathcal P_w(v)=v \quad \forall v\in T_{\mathcal C}(w),] and therefore [\mathcal P_w^2=\mathcal P_w.] That is certainly the natural route. However, strictly speaking, the conclusion requires verifying that [\mathcal P_w(F) \in T_{\mathcal C}(w)] for every (F), using exactly the same tangent cone that appears in the second statement. In other words, the logical chain is 1. range inclusion, 2. identity on the range, 3. idempotence. Your candidate formula strongly suggests this is true, but I would present [\mathcal P_w^2=\mathcal P_w] as a corollary once both properties are proved, rather than assuming it immediately. If established, it would indeed be a stronger structural result than merely calling the operator "projection-like." 3. Range equality is the stronger claim Similarly, I would distinguish [\operatorname{range}(\mathcal P_w) \subseteq T_{\mathcal C}(w)] from [\operatorname{range}(\mathcal P_w) T_{\mathcal C}(w).] The first is a tangent-feasibility property. The second is surjectivity onto the tangent cone. Identity on tangent vectors makes surjectivity plausible, but I'd keep them as separate propositions until written out carefully. 4. Piecewise linearity is an excellent observation I particularly like this section. Within one fixed active configuration, - clipping pattern fixed, - saturated rows fixed, - endpoint ordering fixed, the map is linear. Only crossing a switching surface changes the formula. That immediately explains why - Hadamard derivatives exist, - Fréchet differentiability generally fails, - semismooth methods become relevant. I think this is probably the correct analytical framework. 5. The operator viewpoint clarifies the mature condition I think [\mathcal P_{w^}(F(w^,x^))\approx0] is preferable to earlier formulations. It avoids suggesting [F=0.] It also makes clear that the cancellation depends on the current fast state. I particularly agree that trajectory-wide stationarity should not be claimed until [\mathcal P_{w(t)}F(t)] has actually been sampled. At present, the evidence concerns one mature snapshot. 6. The "kernel" language is appropriate with one caveat You write the unconstrained field lies almost entirely in the kernel of the nonlinear tangent projection. I think that's a useful description, provided the reader understands that [\ker\mathcal P_w {F:\mathcal P_w(F)=0}] is not a linear subspace, because (\mathcal P_w) is generally nonlinear. It is better thought of as the zero set of a positively homogeneous mapping. That's a small mathematical distinction, but it avoids importing unnecessary linear-operator intuition. 7. The mirror comparison becomes much cleaner I actually think this is the strongest consequence of the operator formulation. You now compare Current AIEF: [\dot w \mathcal P_w(F),] with Mirror: [\dot w H^{-1}F/\beta.] Those are qualitatively different constructions. One is - state-dependent, - nonsmooth, - piecewise linear, - (potentially) idempotent. The other is - smooth, - linear in (F), -

metric-preconditioned. That comparison is much clearer than comparing update formulas directly. 8. I like the separation between instantaneous and trajectory-wide stationarity This is an important refinement. There are really three increasingly strong statements: 1. [\mathcal P_{w^}(F(w^,x^))=0.] One instant. 2. [\mathcal P_{w(t)}(F(w(t),x(t)))\approx0] throughout a mature interval. Adaptive stationarity. 3. The coupled system possesses an invariant manifold [w(t)\equiv w^*.] That is a genuine dynamical statement. Your proposed diagnostics are exactly what distinguish these cases. 9. The operator itself becomes an interesting mathematical object One thing I would add is that, once established, (\mathcal P_w) is interesting independently of AIEF's specific forcing law. For example, one could ask about properties such as: - continuity of (w \mapsto \mathcal P_w), - continuity across active-set changes, - equivariance under relabeling of nodes, - monotonicity (if any), - invariance of faces, - behavior under composition as (w) changes. Those questions concern the admissibility mechanism itself rather than any particular adaptive dynamics. Overall assessment I think the operator formulation is the cleanest synthesis you've reached. The main mathematical caveat I'd retain is to distinguish proved structural properties from those that are currently candidate corollaries of the directional formula. In particular, tangent-feasibility, identity on tangent directions, idempotence, and equality of the range with the tangent cone form a natural chain of propositions, but each should be established explicitly rather than inferred in one step. If those results hold, then (\mathcal P_w) is considerably richer than a heuristic "projection-like" correction. It becomes a state-dependent, positively homogeneous, piecewise-linear, Hadamard directionally differentiable retraction associated with the admissibility geometry. The numerical evidence you've gathered is already consistent with that picture, while the analytical work you've outlined would elevate it from an interpretation of one implementation to a rigorous characterization of the ideal admissibility operator. That would provide a clean foundation for subsequent questions about frozen-fast restoration, coupled delayed dynamics, and only later, the interaction with quasiperiodic fast subsystems.

The caution is appropriate, but the proposed chain of properties can likely be proved directly once the directional formula is established. The operator may genuinely be a nonlinear projection onto the tangent cone—not merely “projection-like.”
1. Start with the ideal finite-step map

Let:
$C=\{w\geq 0, Aw\leq c\}$.
$C=\{w\geq 0, Aw\leq c\}$.

Define the ideal admissibility map AA by:

positivity:
$p=[x]+;$
$p=[x]+;$
row sums:
$r=Ap;$

r=Ap;
node scales:
qi=min(1,ci/ri);
qi=min(1,ci/ri);
symmetric edge scale:
qij=min(qi,qj);
qij=min(qi,qj);
output:
Aij(x)=pijqij.
Aij(x)=pijqij.

With the natural convention qi=1qi=1 when ri=0ri=0.

This map satisfies:
A(x)∈C
A(x)∈C

for all xx, and:
A(w)=w∀w∈C.
A(w)=w∀w∈C.

Thus the ideal finite-step map itself is already a nonlinear retraction:
A2=A,range(A)=C.
A2=A,range(A)=C.

This is separate from the directional operator.
2. Tangent-feasibility can be proved directly

At feasible ww, define:
Pw(F)=
Pw(F)=

D\mathcal A(w;F).
]

Assuming the candidate formula:
Pw,e(F)=
Pw,e(F)=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j).
]

The tangent cone is:
TC(w)=
TC(w)=

\left{
v:
v_e\ge0\ \text{if }w_e=0,\quad
(Av)_i\le0\ \text{if }(Aw)_i=c_i
\right}.
]
Positivity faces

If we=0we=0, then:
Pw,e(F)=
Pw,e(F)=

\max(F_e,0)\ge0.
]
Saturated capacity faces

For saturated node ii, let:
di=(AF~)i.
di=(AF
)i.

Then:
(APwF)i=
(APwF)i=
d_i

\sum_{e\ni i}
w_e\max(\alpha_i,\alpha_j).
]

Because:
max(αi,αj)≥αi,
max(αi,αj)≥αi,

we get:
(APwF)i≤di−αi∑e∋iwe.
(APwF)i≤di−αie∋i∑we.

If di>0di>0:
αi=di/ci
αi=di/ci

and:
(APwF)i≤di−(di/ci)ci=0.
(APwF)i≤di−(di/ci)ci=0.

If di≤0di≤0, then αi=0αi=0, so:
(APwF)i≤di≤0.
(APwF)i≤di≤0.

Therefore:
Pw(F)∈TC(w)
Pw(F)∈TC(w)

for every FF.

This establishes the first required proposition.
3. Identity on the tangent cone

Take:
v∈TC(w).
v∈TC(w).

At every zero edge:
ve≥0,
ve≥0,

so positivity clipping does nothing:

v~=v.
v
=v.

At every saturated node:
(Av)i≤0,
(Av)i≤0,

therefore:
αi=0.
αi=0.

Hence every edge correction vanishes:
wemax(αi,αj)=0.
wemax(αi,αj)=0.

Thus:
Pw(v)=v∀v∈TC(w).
Pw(v)=v∀v∈TC(w).

This proves identity on the cone.
4. Range equality and idempotence follow

From tangent feasibility:
range(Pw)⊆TC(w).
range(Pw)⊆TC(w).

From identity on every tangent vector, each:
v∈TC(w)
v∈TC(w)

is the image of itself:
v=Pw(v).
v=Pw(v).

Therefore:
TC(w)⊆range(Pw).
TC(w)⊆range(Pw).

Together:
\boxed{ \operatorname{range}(\mathcal P_w) =

T_{\mathcal C}(w).
}
]

Since every output is tangent and PwPw is identity on tangent vectors:
\boxed{ \mathcal P_w(\mathcal P_w(F)) =

\mathcal P_w(F).
}
]

So the logical chain requested in the critique can be established explicitly:

range inclusion;
identity on tangent directions;
range equality;
idempotence.

Subject to proving the directional formula, PwPw is a state-dependent nonlinear projection/retraction onto the tangent cone.
5. Algebraic classification

For fixed ww, PwPw is:

continuous in FF;
globally Lipschitz in FF;
positively homogeneous:
Pw(λF)=λPw(F),λ≥0;
Pw(λF)=λPw(F),λ≥0;
piecewise linear;
generally nonadditive;
generally not odd;
idempotent;
tangent-feasible.

It has not been shown to be:

orthogonal;
nonexpansive;
monotone;
self-adjoint;
representable by one matrix.

Its “kernel”:
kerPw=
kerPw=

{F:\mathcal P_w(F)=0}
]

is a positively homogeneous zero cone, not generally a linear subspace.
6. State dependence is discontinuous across active faces

The finite map AA is continuous and piecewise smooth. But:
w↦Pw
w↦Pw

need not be continuous when a row changes from unsaturated to saturated.

For an outward forcing:

just inside the boundary, αi=0αi=0;
at the boundary, αi>0αi>0.

Thus the induced projected vector field is discontinuous across active-set changes, as in standard projected dynamical systems.

This is why:

directional derivatives are the correct first tool;
generalized Jacobians become relevant for switching;

ordinary smooth linearization is insufficient globally.

7. Equivariance under node relabeling

Let PP permute nodes and let QQ be the induced edge permutation. If capacities are permuted consistently:
c′=Pc,
c′=Pc,

then the operator should satisfy:
\boxed{ \mathcal P_{Qw}^{\,c'}(QF) =

Q\mathcal P_w^{,c}(F).
}
]

For equal capacities, this gives full node-label equivariance.

This is a useful structural theorem and a good implementation test.
8. Face invariance is conditional

The complete feasible set is forward invariant under:
w˙=PwF.
w˙=PwF.

But individual faces need not be invariant.

A zero edge can become positive when:
Fe>0.
Fe>0.

A saturated row can move inward when:
(AF~)i<0.
(AF
)i<0.

Therefore active faces can be entered and left. This is essential for topology adaptation and explains active-set switching.

9. Trajectory-wide stationary fiber

The stronger empirical statement requires:
$Pw(t)F(w(t),x(t))\approx 0$
$Pw(t)F(w(t),x(t))\approx 0$

throughout the mature interval.

If additionally:
$w(t)\equiv w*$,
$w(t)\equiv w*$,

then the set:
$\{(x,w*):x\in Xmature\}$
$\{(x,w*):x\in Xmature\}$

is a candidate invariant fiber over the fast dynamics.

To establish this numerically, report:

stationarity ratio over time;
maximum and quantiles;
zero-edge switches;
saturated-node switches;
exact continuation replay;
whether continuous weights remain fixed, not only threshold membership.

The new stationarity/perturbation script is designed to provide exactly these quantities.
10. Frozen-fast stability versus coupled stability

The proposed restoring coefficient:
$\kappa restore=$
$\kappa restore=$

\frac{
\delta w^\top\mathcal P_{w^+\delta w}
F(w^+\delta w,x^*)
}{

|\delta w|^2
}
]

measures only radial frozen-fast response.

Also report:
cosθ=
cosθ=

\frac{
(-\delta w)^\top V
}{
|\delta w||V|
}
]

and transverse motion:
V⊥=
V⊥=
V

\frac{
V^\top(-\delta w)
}{
|\delta w|^2
}
(-\delta w).
]

Then compare with full coupled trajectories where x(t)x(t) evolves.

Positive radial restoration with a large transverse component is compatible with transient amplification or non-normal behavior.
Recommended theorem structure
Theorem 1

The ideal finite admissibility map is a locally Lipschitz retraction onto CC.
Theorem 2

It is Hadamard directionally differentiable at every feasible ww, with derivative PwPw.
Proposition 3
range(Pw)=
range(Pw)=

T_{\mathcal C}(w).
]
Corollary 4
Pw2=Pw.
Pw2=Pw.
Proposition 5

PwPw is positively homogeneous, piecewise linear, and node-permutation equivariant.
Open stability problem

Characterize solutions of:
w˙=PwF(w,x)
w˙=PwF(w,x)

and their behavior across active-set switching.
Bottom line

The critique is correct that range inclusion, identity, range equality, and idempotence should be presented as distinct logical steps. But once the candidate derivative formula is proved, each step appears to follow cleanly.

The resulting object is richer than a heuristic correction:

PwPw is a state-dependent, positively homogeneous, piecewise-linear nonlinear projection onto the tangent cone of the degree-capacity polytope.

That operator provides the right mathematical language for constraint-balanced adaptive stationarity, feasible perturbation restoration, and the later comparison with smooth entropy-metric dynamics.

can you or would it be prudent to update the AIEF_1_4_stationarity_perturbation_analysis.py in line with the critique?
Edit×14

Edit×3
AIEF_1_4_stationarity_perturbation_analysis_updated.pyPYTHON

Yes—it was prudent to update the analysis script. The critique identifies algebraic properties that can be audited numerically throughout the mature trajectory, not merely asserted from the candidate formula.

Created:

AIEF_1_4_stationarity_perturbation_analysis_updated.py
Run

Bash

python AIEF_1_4_stationarity_perturbation_analysis_updated.py

Explicit invocation:

Bash

python AIEF_1_4_stationarity_perturbation_analysis_updated.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--output-dir aief_1_4_stationarity_perturbation

It defaults to five representative seeds. Use all 40 with:

Bash

python AIEF_1_4_stationarity_perturbation_analysis_updated.py \
--max-seeds 0

New operator-structure audits

At every mature continuation step, the script now numerically checks the candidate operator:
Pw(F)=VA(w;F).
Pw(F)=VA(w;F).

Tangent feasibility

It checks:

no negative output direction on zero edges;
no outward row-sum direction on saturated nodes.

Outputs include:

text

tangent_zero_edge_violation
tangent_capacity_violation
tangent_feasible

Idempotence

It evaluates:
Pw(Pw(F))
Pw(Pw(F))

and compares against:
Pw(F).
Pw(F).

Outputs:

text

idempotence_error
idempotence_holds_1e10

Positive homogeneity

It checks:
Pw(2F)=?2Pw(F).
Pw(2F)=?2Pw(F).

Outputs:

text

positive_homogeneity_error
positive_homogeneity_holds_1e10

Node-permutation equivariance

At the split state, it applies one deterministic node permutation, recomputes the directional field, maps it back to the original labels, and reports:

text

permutation_equivariance_error

Capacities are permuted consistently.
Stronger mature-stationarity diagnostics

The updated script records:

fraction of steps satisfying the stationarity-ratio tolerance;
mean and maximum
‖PwF‖/‖F‖;
‖PwF‖/‖F‖;
maximum ‖PwF‖‖PwF‖;
exact-zero support switches;
saturated-node active-set switches;
outward-node-set switches;
normalized continuous-weight drift from B_at_split;
fraction of steps where BB is exactly unchanged;
maximum operator-property errors;
count of tangent/idempotence/homogeneity failures;
exact and tolerance-based final replay agreement.

This distinguishes:

threshold-topology stationarity;
continuous-weight stationarity;
active-set stationarity;
algebraic operator consistency.

Frozen-fast and coupled perturbations retained

The script continues to test:

radial inward perturbations;
interior-support-mixing perturbations;
tangent-face perturbations.

Frozen-fast outputs include:

restoring coefficient;
restoring cosine;
radial and transverse field components;
transverse-to-field ratio;
active-set information.

Coupled outputs include:

normalized distance to an unperturbed reference trajectory;
threshold Jaccard similarity;
final/initial distance ratio;
times to 50% and 90% distance reduction.

Operator audits are performed throughout the mature reference replay. They are disabled inside the many coupled perturbation steps to avoid unnecessary runtime overhead; those runs still use the same actual dynamics.
Output additions

The main files remain:

text

mature_stationarity_by_seed.csv

mature_stationarity_timeseries.csv
frozen_fast_perturbations_by_seed.csv
frozen_fast_perturbation_summary.csv
coupled_perturbations_by_seed.csv
coupled_perturbation_summary.csv
coupled_perturbation_trajectories.csv
stationarity_perturbation_report.txt
analysis_manifest.json

The stationarity files now contain the tangent, idempotence, homogeneity, equivariance, and continuous-BB-drift diagnostics.
Validation

I syntax-tested and ran the revised script end-to-end against a complete AIEF split state.

The technical smoke test produced:

text

stationary fraction: 1.0
mean ||P_w(F)|| / ||F||: ~1.6×10^{-13}
zero-edge switches: 0
saturated-node switches: 0
maximum normalized B drift: ~3.9×10^{-12}
tangent failures: 0
idempotence failures: 0
positive-homogeneity failures: 0
permutation-equivariance error: ~2×10^{-17}
exact continuation replay: True

Those values are only a one-seed technical validation, not a scientific result. The full multi-seed run will determine whether these operator properties and trajectory-wide adaptive stationarity hold across the validation set.

hello

Hello! What would you like to work on next?

think about the results I pasted

Taken together, the results form a coherent story that is substantially different from the original “delay stores topology memory” hypothesis.
1. AIEF-1.3 established a reproducible constrained-selection regime

Across the two independent 50-seed cohorts:

baseline density remained near 0.30;
historical resonance correlation remained near 0.995;
P_future was exactly binary for every baseline edge;
cohort effects were compatible;
scalar and matrix artifacts were consistent.

The exact binary result means the network had completed threshold selection by HIST_END. Every edge was either active throughout the complete future window or never active.
Mechanistic roles from AIEF-1.3
μμ: concentration and pruning

Removing μμ:

preserved or slightly increased total coupling;
made every edge nonzero;
increased effective edge count;
reduced Gini;
increased entropy;
sharply reduced threshold density.

Thus μμ concentrates a diffuse coupling budget into a sparse backbone.
Gap projection: hard coupling-budget enforcement

The projection was active on approximately 97–99% of mature updates. It was not an occasional repair.

Removing it produced:

dramatically larger total weight;
dense topology;
higher coherence;

strongly negative gap.

The gap is not required for mathematical boundedness or coherence. It enforces the intended low-weight admissible regime.
Explicit delay: not required

No-delay retained—and slightly improved—correlation, density, and coherence. Explicit retardation is therefore not required under the existing EMA/adaptation architecture.
2. AIEF-1.4 did not identify a single memory register

The strongest AIEF-1.4 conclusion is distributed reconstruction toward a topology-selection attractor.
Resetting BB

Resetting the complete adaptive matrix produced:

rapid density recovery in approximately 3.6 time units;
sustained 90% topology similarity in approximately 4.8;
essentially exact late threshold topology;
late normalized Frobenius residual around 10−510−5.

Therefore BB is not individually necessary for late reconstruction.
Resetting EMA

EMA reset caused:

a larger immediate disturbance;
slower recovery;
one censored seed;
small but occasionally meaningful late deviations.

EMA is not necessary for late density in most seeds, but it accelerates reconstruction and protects against rare failures.
Resetting BB and EMA

Joint removal caused:

much lower immediate similarity;
approximately tenfold longer recovery than reset_B;

reduced topology fidelity;
emergence of new persistent edges;
a significant B×B× EMA interaction.

This demonstrates redundancy between state channels.
Full reset

Resetting BB, EMA, phase, and delay history still reconstructed a recognizably similar topology in many seeds, but:

recovery was slow;
similarity recovery probability was only 62.5%;
density recovery and identity recovery differed;
the final topology was not unique.

The preserved model structure defines a strong attractor landscape, not one globally unique topology.
Future shuffle

This remains the strongest intervention:

topology was intact immediately after the split;
similarity and density progressively decayed;
no seed sustainably recovered;
late threshold persistence was zero.

Continued edge-specific driver identity is necessary for maintenance under timestep-wise shuffling.
3. “Memory” and “attractor reconstruction” are now distinct

The data support three functions:

Selection: resonance and μμ determine which edges form a sparse backbone.
Reconstruction: EMA, phase/history, BB, and fixed model structure provide redundant routes back toward a mature topology.
Maintenance: continuing edge-specific driver assignment is required to sustain threshold topology.

The best interpretation is:

Mature topology is dynamically regenerated rather than passively stored in one state variable.

4. Projection is small relative to state but dominant relative to updates

Projection corrections were approximately:
0.2%–0.3%
0.2%–0.3%

of the full network-state norm per active step.

But they were approximately:
99%–100%
99%–100%

of the unconstrained proposed update.

Thus “gentle projection” is incomplete. The accurate statement is:

Projection applies small state-relative corrections that nearly cancel each outward incremental update, pinning mature adaptation to the admissible boundary.

This explains the frozen topology.
5. Entropy geometry is valid but singular at mature states

The geometry diagnostics found:

about 70% exact-zero edges;
about 65% node margins at or below 10−1210−12;
Hessian condition numbers from 104104 to 10121012, depending on interiorization;
accurate coupled inverse-map reconstruction;
strong epsilon dependence in intrinsic metrics.

So the entropy construction is mathematically workable, but mature AIEF lies in its singular boundary limit.

The mirror model would:

remove exact zeros;

preserve positive slack;
strongly suppress high-curvature directions;
define a new adaptive vector field.

It is not a drop-in implementation of current AIEF.
6. Coordinate versus mirror results are clean

The local one-step study found:
Coordinate-equivalent dual Euler
p≈2
p≈2

in inactive interior strata. This verifies the chain-rule coordinate implementation.

It does not yet prove equivalence to active projected dynamics.
Mirror Euler
p≈1.
p≈1.

Thus mirror adaptation has a different limiting vector field.
Explicit dual stiffness

Coordinate-dual Euler required timesteps near 10−610−6 for reliable agreement. At the original timestep it overflowed or left the asymptotic regime.
7. The active-boundary field is now the main mathematical object

For the idealized admissibility map, the candidate field is:
Pw,e(F)=
Pw,e(F)=
\widetilde F_e

w_e\max(\alpha_i,\alpha_j).
]

The deterministic probes showed:

outward probe increment error near second order;
random probe close to second order;

physical mature forcing mapped to a field of norm approximately 10−1310−13.

That means:
F≠0,PwF≈0.
F =0,PwF≈0.

The mature topology appears to be a constraint-balanced stationary state of the adaptive component.

But the full phase/history/EMA state continues evolving, so it is not a complete-system equilibrium.
8. What is established versus still open
Strongly supported

Baseline future topology is exactly frozen at threshold level.
μμ concentrates coupling into a sparse backbone.
The gap map continuously enforces the coupling envelope.
Explicit delay is not necessary under the current architecture.
BB and EMA are redundant reconstruction channels.
Preserved model structure defines a strong topology attractor.
Future edge-specific driver identity is necessary for maintenance.
Entropy coordinates correctly represent smooth interior dynamics.
Mirror dynamics define a different adaptive law.
The candidate ideal boundary field matches deterministic finite-difference probes.
Tested mature split states are constraint-balanced in the adaptive component.

Not yet established

One unique topology attractor.
Memory stored in BB, EMA, or geometry alone.
Full-state equilibrium.
A global Lyapunov function.
Proximal-Galerkin equivalence.
Mirror convergence to the same boundary attractor.
Trajectory-wide adaptive stationarity across all 40 seeds.
Rigorous directional differentiability of the ideal map.
Numerical convergence under timestep refinement.
Quasiperiodic fast-subsystem behavior.

9. Best immediate next actions
First: run the revised stationarity/perturbation analysis on all 40 seeds

Bash

```
python AIEF_1_4_stationarity_perturbation_analysis_updated.py \
--max-seeds 0
```

This will test:

whether ‖PwF‖‖PwF‖ remains tiny throughout the mature window;
whether active sets remain fixed;
continuous BB-drift;
tangent feasibility;
idempotence;
positive homogeneity;
permutation equivariance;
frozen-fast and coupled restoration.

Second: complete the matched-phase correction

Rerun only:

text

```
B_carrier_only
full_reset
```

with identical randomized phase/history. The existing comparison remains exploratory.
Third: prove the ideal-map theorem

The candidate theorem sequence is now clear:

ideal admissibility map is a retraction onto CC;
it is Hadamard directionally differentiable;
its derivative is PwPw;
PwPw maps onto the tangent cone;
it is identity on tangent directions;
therefore it is idempotent.

Fourth: timestep convergence

Because projection nearly cancels every mature update, establish that topology and recovery outcomes persist under smaller DT.
Bottom line

The strongest unified interpretation is:

AIEF produces a sparse topology as a constraint-balanced adaptive state on a hard admissible boundary. The topology is not localized to one memory variable; it is rapidly reconstructed through redundant dynamical channels and continuously maintained by edge-specific resonance forcing. The mature adaptive field is nonzero before admissibility but is canceled to numerical precision by a state-dependent nonlinear tangent projection. Entropy geometry supplies a valid interior coordinate and analytical metric, while mirror dynamics constitute a distinct model rather than a reformulation of existing AIEF.

The project has progressed from empirical topology selection to a well-defined mathematical object:
Pw(F)
Pw(F)

and the next decisive question is whether its observed stationarity and restoration properties persist across the full mature trajectory and under feasible perturbations.

I were going to loop in an other ai agent what would a great prompt be to get it up to speed on where we have been, where we are, and where we are going?
WriteAI_AGENT_HANDOFF_PROMPT.md512 lines
Copy/paste prompt for a new AI collaboratorMD
Copy/paste prompt for a new AI collaborator

You are joining an ongoing computational-mathematics research project called SSRH / AIEF. Act as a skeptical but constructive collaborator in nonlinear dynamics, constrained adaptive networks, numerical analysis, and reproducible scientific computing.

Your job is not to merely agree with prior interpretations. Audit equations, code, artifacts, statistical units, and causal claims. Clearly separate:

facts established by saved outputs;
numerical evidence;

plausible interpretation;
conjecture/open mathematics.

Do not silently rewrite the frozen AIEF-1.3 or AIEF-1.4 protocols. If recommending a model change, label it as a new revision/model.

1. Core AIEF model

There are N=12 phase oscillators with fixed heterogeneous frequencies:

Python

```
omega = [1.00, 1.02, 1.50, 2.00, 2.01,
1.01, 1.03, 1.51, 2.02, 2.00,
1.00, 1.04]
```

Phase dynamics use a symmetric adaptive coupling matrix B and delayed phase history:

text

```
theta_dot_i = omega_i
+ sum_j B_ij sin(theta_j(t-tau) - theta_i(t))
```

with tau=0.5, dt=0.01, and T_sim=800.

An edgewise complex EMA defines resonance compatibility:

text

```
C_ij <- alpha C_ij + (1-alpha) exp(i dphi_ij)
R_ij = |C_ij|
```

where R is symmetrized and the diagonal is zero. T_window = 25/mean(omega) and alpha=exp(-dt/T_window).

Adaptive weights obey:

text

B_dot = (eta + mu) R_driver - mu - gamma B

with:

text

eta=0.04, mu=0.03, gamma=0.015

followed by nonnegativity, symmetry, and a hard dominance-gap admissibility map.

The node constraint is:

text

sum_j B_ij <= c = (a-delta)/q

with:

text

a=1, delta=0.05, q=1.5

The implemented symmetric admissibility step scales edge (i,j) by the more restrictive endpoint factor using minimum.outer(scale, scale). It is not known to be an orthogonal Euclidean projection.

Threshold topology uses B_c=0.12.
2. AIEF-1.3: what was done

AIEF-1.3 tested whole-run conditions:

text

baseline

shuffle_R # timestep-wise shuffle of unique-edge driver values
no_delay
no_mu
no_gap

The final production driver is:

text

AIEF_1_3_falsification_updated.py

It uses deterministic randomized seed sets generated from a master SeedSequence, separate initialization/shuffle RNG streams, exact delay indexing, atomic checkpoints, manifests, source hashes, and paired conditions.
Principal independent validation cohorts

Use these as the real independent AIEF-1.3 validation cohorts:

text

runs/c3da1ac707bf/ # 50 seeds
runs/73dca37cf93f/ # 50 seeds

Earlier 20-seed workflow-version replays were useful for software repeatability but are not independent cohorts and must not be pooled as independent evidence.
Key 100-seed AIEF-1.3 findings

Baseline, pooled descriptively across the two 50-seed cohorts:

text

D_B ~0.306
Corr_physical ~0.995
Effective_edges ~14.6
Total edge weight ~3.64

shuffle_R:

text

D_B=0
P_mean=0
P_degenerate=1 for every seed
Total edge weight ~0.060

Interpretation: timestep-wise destruction of persistent edge-specific driver identity produces a tiny diffuse residual field and abolishes threshold topology. This is specific to aggressive timestep-wise shuffling; fixed/block shuffles remain different interventions.

no_mu:

text

D_B ~0.035
Total edge weight ~3.78
Effective_edges ~34.4
Gini drops ~0.761 -> ~0.482
Entropy rises ~0.683 -> ~0.884
Nonzero edge fraction rises ~0.306 -> 1.0

Interpretation: mu is a pruning/concentration mechanism, not a coupling-existence mechanism.

no_gap:

text

D_B ~0.727
Total edge weight ~127.9
G_min ~-32.8

Interpretation: removing the gap creates a dense, high-weight regime outside the intended admissible envelope. It does not prove mathematical divergence or loss of coherence.

no_delay is qualitatively in the same regime and slightly increases several observables. Explicit delay is not necessary under the current EMA/adaptation architecture.
AIEF-1.3 closure

Scripts:

text

AIEF_1_3_paired_statistics_updated.py
AIEF_1_3_closure_diagnostics_updated.py

Closure passed on the two independent 50-seed cohorts:

text

CLOSURE PASSED: True

Established:

complete/disjoint seed cohorts;
exact matrix/scalar consistency;
symmetric, zero-diagonal matrices;
baseline P_future is exactly binary for every unique edge and seed;
no intermediate persistence values;
the baseline threshold topology is frozen throughout the future evaluation window.

Projection diagnostics:

text

baseline active fraction ~0.972
no_mu active fraction ~0.988
projection active given violation = 1

Absolute projection corrections are small relative to full B, but later analysis found they are nearly the same norm as each proposed incremental update. Therefore AIEF is a hard projected/admissibility-constrained dynamical system, not a system with rare repairs.

3. AIEF-1.4: split-state intervention

Production script:

text

AIEF_1_4_memory_ablation_updated.py

Frozen 40-seed run:

text

runs/5a9f093a2fe0/

A common baseline prefix is run to HIST_END, then the complete split state is cloned into paired branches:

text

continue
reset_ema
reset_B
reset_B_ema
B_carrier_only # preserve B; reset EMA and randomize phase/history
full_reset # reset B, EMA, phase, history; preserve model/omega
future_shuffle # preserve split state; shuffle only future driver identity

Important design caveat: in r2, B_carrier_only and full_reset used different randomized phase resets. Their direct contrast is exploratory until rerun with matched phase/history.
Late 40-seed summaries

Approximate values:

text

branch P_late ratio Sim_late Retention Emergence
continue 1.000 0.999 1.000 0.000

reset_ema 0.990 0.993 0.990 0.000
reset_B 1.000 0.999 1.000 0.000
reset_B_ema 1.080 0.944 0.987 0.038
B_carrier_only 0.945 0.948 0.908 0.015
full_reset 1.035 0.884 0.920 0.046
future_shuffle 0.000 0.475 0.000 0.000

Interpretation: no single state variable is the unique memory register. Mature topology is reconstructed through redundant state channels and a strong model-defined attractor landscape. Future edge-specific driver identity is necessary for threshold maintenance under timestep-wise shuffling.

Do not claim B is the dominant memory carrier: resetting B alone reconstructs the late topology essentially exactly.

4. AIEF-1.4 trajectory and survival results

Scripts:

text

AIEF_1_4_results_analysis.py
AIEF_1_4_trajectory_recovery_equivalence.py
AIEF_1_4_survival_outlier_spectral.py

Key sustained 90% recovery results:

text

branch similarity recovery similarity RMUT
continue 40/40 0.01
reset_ema 39/40 21.36
reset_B 40/40 4.77
reset_B_ema 38/40 45.37
B_carrier_only 34/40 68.52
full_reset 25/40 156.72
future_shuffle 0/40 320 (censored)

Key interpretation:

resetting B causes a transient but rapid exact reconstruction;
EMA substantially accelerates reconstruction when B is absent;
joint state removal reduces fidelity and creates new edges;
full reset reconstructs a recognizable but nonunique topology;
future shuffle starts with intact topology and progressively destroys it.

Recovery means among recovered seeds are biased when censoring exists; use recovery probability, Kaplan–Meier curves, and restricted mean unrecovered time.

The reset_B late residual is tiny (~1e-5 normalized) but somewhat localized over a few effective edges. It does not alter threshold topology.

5. Entropy geometry on the symmetric degree-capacity polytope

The true state is the vector w of E=N(N-1)/2=66 unique undirected edges. Let A be the unsigned node-edge incidence matrix. The feasible set is:

text

C = {w >= 0, A w <= c}

This is not a product of independent row simplexes because each undirected edge belongs to two node constraints.

Define node margins:

text

m = c - A w

and the natural entropy Legendre potential on the strict interior:

text

h(w) = sum_e w_e(log w_e - 1)
+ sum_i m_i(log m_i - 1)

Then:

text

psi = grad h(w) = log w - A^T log m
H = Hess h(w) = diag(1/w) + A^T diag(1/m) A

For edge (i,j):

text

psi_ij = log(w_ij/(m_i m_j))
w_ij = exp(psi_ij) m_i m_j

The inverse map is implicit through the coupled margins:

text

m_i (1 + sum_j exp(psi_ij) m_j) = c_i

A strictly convex log-margin objective gives existence/uniqueness. The inverse requires only an N x N solve.

Woodbury structure:

text

H^-1 = D_w - D_w A^T (D_m + A D_w A^T)^-1 A D_w

Important: the entropy geometry is natural, not proven canonical. Metric scale and nondimensionalization are modeling choices. Bregman divergence is not geodesic distance.

Script:

text

AIEF_1_4_entropy_geometry_diagnostics.py

Key findings on 40 mature split states:

text

~70% exact-zero edges
~65% node margins <= 1e-12
median minimum node margin ~1e-12

Hessian condition numbers ranged from roughly 1e4 to 1e12, depending on edge/margin interiorization. The inverse map reconstructed edge weights near machine precision. Intrinsic quantities were strongly epsilon-dependent.

Conclusion: mature AIEF lies on a singular boundary face. A mirror formulation would be a new model, not a mild reparameterization.
6. Local current/coordinate/mirror comparison

Scripts:

text

AIEF_1_4_local_update_comparison.py
AIEF_1_4_active_boundary_analysis.py

Compared:

actual AIEF positivity clip + symmetric admissibility scaling;
coordinate-equivalent dual Euler:

text

psi+ = psi + dt H(w) F

mirror Euler:

text

psi+ = psi + dt F / beta

Interior results:

text

current vs coordinate increment discrepancy order ~2
current vs mirror increment discrepancy order ~1

Interpretation:

coordinate-dual Euler correctly represents smooth interior primal dynamics to first order;
it is severely stiffness-limited near mature boundary states;
mirror Euler defines a different metric-preconditioned vector field.

The p~2 result mostly comes from inactive strict-interior steps and must not be generalized to the active boundary process.

7. Candidate active-boundary operator

For the ideal map without the fixed denominator safeguard, define positivity-modified forcing:

text

Ftilde_e = F_e if w_e > 0
= max(F_e, 0) if w_e = 0

For node i:

text

d_i = sum_{e incident i} Ftilde_e
alpha_i = max(d_i,0)/c_i if row i is saturated
= 0 otherwise

For edge e=(i,j), the candidate directional field is:

text

P_w(F)_e = Ftilde_e - w_e max(alpha_i, alpha_j)

Deterministic outward/random probes showed finite-difference behavior consistent with:

text

rate error O(dt)
increment error O(dt^2)

At tested mature split states:

text

||F|| is appreciable
||P_w(F)|| ~ 1e-13

so mature adaptive states appear constraint-balanced: unconstrained adaptation remains active, but admissibility cancels its first-order effect.

The full delayed state is not an equilibrium; phases/history/EMA continue evolving.
Candidate theorem structure

For the ideal finite map A:

A is a nonlinear retraction onto C.
A is Hadamard directionally differentiable.
D A(w;F) = P_w(F).
P_w(F) lies in the tangent cone T_C(w).
P_w(v)=v for every tangent v.
Therefore range(P_w)=T_C(w) and P_w^2=P_w.
P_w is positively homogeneous and piecewise linear, but generally nonlinear, nonadditive, and not known to be orthogonal/nonexpansive.

The “kernel” {F:P_w(F)=0} is a cone/zero set, not generally a linear subspace.

Important implementation caveat: the fixed +1e-12 denominator safeguard has a different formal dt -> 0 limit. The theorem concerns the ideal map without that safeguard.

8. Current files/scripts

Principal scripts include:

text

AIEF_1_3_falsification_updated.py
AIEF_1_3_paired_statistics_updated.py
AIEF_1_3_closure_diagnostics_updated.py

AIEF_1_4_memory_ablation_updated.py
AIEF_1_4_results_analysis.py
AIEF_1_4_trajectory_recovery_equivalence.py
AIEF_1_4_survival_outlier_spectral.py
AIEF_1_4_entropy_geometry_diagnostics.py
AIEF_1_4_local_update_comparison.py
AIEF_1_4_active_boundary_analysis.py
AIEF_1_4_stationarity_perturbation_analysis_updated.py

Do not assume a script is correct merely because it ran. Check its estimand, pairing, censoring, numerical tolerances, and whether it uses the ideal or implemented admissibility map.

9. Immediate next task

The next task is to run and interpret:

Bash

python AIEF_1_4_stationarity_perturbation_analysis_updated.py --max-seeds 0

This should establish whether adaptive constraint balance persists over the mature trajectory and whether the candidate operator numerically satisfies:

tangent feasibility;
idempotence;

positive homogeneity;
node-permutation equivariance;
frozen-fast restoration;
coupled restoration.

Then:

complete the matched-phase B_carrier_only/full_reset correction;
prove the ideal-map directional theorem;
perform timestep convergence;
only afterward consider a mirror/proximal AIEF-1.5 model or quasiperiodic analysis.

10. How to begin your response

First:

Summarize the established results in your own words.
Identify any claim above that is too strong or mathematically unsupported.
Confirm which artifact/script you need to inspect next.
Propose a short prioritized plan—do not invent another broad ablation matrix.

Be precise about these distinctions:

threshold persistence versus continuous weight similarity;
storage versus reconstruction versus maintenance;
seed-level inference versus edge-level description;
strict-interior geometry versus active-boundary dynamics;
coordinate change versus mirror dynamics versus proximal methods;
adaptive-component stationarity versus full-system equilibrium.

Open

I created a comprehensive handoff prompt:

AI_AGENT_HANDOFF_PROMPT.md

It covers:

the core equations and constraints;
AIEF-1.3 design, independent cohorts, closure, and results;
AIEF-1.4 branches, results, trajectories, and survival analysis;
entropy geometry and its symmetry-coupled feasible set;
local coordinate/mirror comparisons;
the candidate active-boundary operator;
established results versus conjectures;
all principal scripts and run directories;
the immediate next task;
explicit instructions for the agent to audit rather than merely agree.

Recommended files to give the other agent

The new agent will not automatically have access to your local paths. Alongside the handoff prompt, attach or paste the most important artifacts.
Minimum package

text

runs/c3da1ac707bf/manifest.json
runs/73dca37cf93f/manifest.json
runs/5a9f093a2fe0/manifest.json

aief_1_3_closure_diagnostics/closure_status.json
aief_1_3_closure_diagnostics/closure_report.txt
paired_seed_analysis_100/paired_report.txt

runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv
aief_1_4_results_analysis/AIEF_1_4_analysis_report.txt
aief_1_4_trajectory_recovery/trajectory_recovery_report.txt
aief_1_4_survival_outlier_spectral/survival_outlier_spectral_report.txt
aief_1_4_entropy_geometry/entropy_geometry_report.txt
aief_1_4_local_update_comparison/local_update_comparison_report.txt
aief_1_4_active_boundary_analysis/active_boundary_report.txt

AIEF_1_4_stationarity_perturbation_analysis_updated.py

If it will review implementation

Also attach:

text

AIEF_1_3_falsification_updated.py
AIEF_1_4_memory_ablation_updated.py
AIEF_1_4_active_boundary_analysis.py

Suggested opening message

After attaching the files, use:

The attached handoff prompt summarizes the project, but do not assume every prior interpretation is correct. Verify claims against the attached manifests, reports, CSVs, NPZ schemas, and source code. Begin by distinguishing established results from numerical evidence and conjecture. Then identify the most important unresolved issue and propose the smallest next step that can resolve it. Do not propose another broad simulation matrix unless the existing artifacts are demonstrably insufficient.

The full prompt is intentionally detailed enough to prevent the new agent from repeating earlier mistakes such as:

treating duplicate seed runs as independent;
calling BB the unique memory carrier;
conflating Fisher-zz with raw correlation;
treating ALPHA=0 as instantaneous resonance;
calling the hard admissibility map a soft regularizer;
interpreting topology similarity as Kuramoto coherence;
treating the entropy mirror model as equivalent to current AIEF;
or calling adaptive-component stationarity a full-system equilibrium.

Results.pdf

Think about this:
ReadResults.pdf

These results are stronger than the earlier split-state endpoints. They reveal two qualitatively different restoration laws—finite-speed normal restoration and approximately exponential tangent restoration—and one important exceptional seed.

1. Constraint-balanced stationarity holds for 39 of 40 seeds

For 39 seeds:

text

stationary_fraction = 1
mean $||P_w(F)||/||F|| \approx 10^{-13}$
zero-set switches = 0
saturated-set switches = 0
normalized B drift $\approx 10^{-12}$–10^{-11}
exact continuation replay = True

This is very strong evidence that, for the overwhelming majority of realizations:
$P_w(t)F(t) \approx 0$
$P_w(t)F(t) \approx 0$

throughout the complete mature interval—not merely at the split snapshot.

The threshold topology and the continuous weight matrix are both effectively stationary for these seeds, while the fast phase/EMA/history subsystem continues evolving.

Thus “constraint-balanced stationary fiber of the adaptive subsystem” is now empirically supported for the dominant regime.
Operator-property audit

Across approximately:
$40 \times 32000 = 1.28$ million
$40 \times 32000 = 1.28$ million

mature updates, the script found:

text

tangent failures: 0
idempotence failures: 0
homogeneity failures: 0

Node-permutation errors were around:
10^{-17}.
10^{-17}.

This is extensive numerical support for:

tangent feasibility;
positive homogeneity;
idempotence;
node-label equivariance.

It remains numerical verification rather than proof, but it strongly supports the proposed nonlinear tangent-projection structure.
2. One seed is a genuine exception

Seed:

text

4619122171158768524

has:

text

stationary_fraction = 0.115
mean ||P_w(F)||/||F|| = 0.0569
maximum ratio = 0.160
saturated-set switches = 1
maximum normalized B drift = 0.209
zero-set switches = 0
exact replay = True

This is not floating-point noise.

The seed retains the same zero-edge support but undergoes a substantial continuous-weight reorganization and one saturated-node active-set transition.

That means:

exact binary P_future does not imply continuous BB is frozen;
threshold topology can remain fixed while weights move substantially inside the selected support;
there are at least two mature regimes:
fully stationary continuous topology;
support-stable but weight-reorganizing topology.

This exceptional seed should be treated as a primary case study, not averaged away.
Immediate analysis for this seed

From mature_stationarity_timeseries.csv, plot:

‖PwF‖/‖F‖‖PwF‖/‖F‖;
normalized BB-drift;
saturated-node count;
outward-node count;
finite-rate versus ideal-field error.

Locate the exact active-set switch and compare:

BB immediately before and after;
row margins;
edge-weight changes;
spectral metrics;
projection correction.

Then cross-reference this seed against:

EMA-reset recovery;
full-reset recovery;
no-gap behavior;
split-state threshold margin;
Hessian curvature.

It may represent a basin-boundary trajectory or a within-support alternative attractor.
3. Normal perturbations show finite-speed restoration

For radial inward perturbations:
δδ Median κrestoreκrestore
10−410−4 1328
10−310−3 133
10−210−2 13.3
5×10−25×10−2 2.67

Approximately:
κrestoreδ≈0.133.
κrestoreδ≈0.133.

For interior mixing:
κrestoreδ≈0.171.
κrestoreδ≈0.171.

Thus:
κrestore∼1δ.
κrestore∼δ1.

This means the restoring field does not scale linearly with displacement in these normal/inward directions. Instead, its radial speed remains approximately finite as:
δ→0.
δ→0.

That is expected from active-set switching:

the mature state is on the boundary;
moving inward creates slack;
the admissibility correction turns off;
the nonzero unconstrained forcing is exposed immediately.

So normal restoration resembles finite-time return toward the active face, not ordinary linear exponential stability.

The substantial transverse components are also important:

text

radial inward transverse ratio ≈0.61
interior mixing transverse ratio ≈0.74

Restoration is strongly nonradial.
4. Tangent perturbations show approximately linear exponential restoration

For most tangent perturbations:

text

restoring coefficient ≈0.315789
restoring cosine ≈1
transverse ratio ≈0

Unlike normal perturbations, the tangent restoring coefficient is nearly independent of δδ.

This indicates:
VA(w∗+δd)≈−κtangentδd,
VA(w∗+δd)≈−κtangentδd,

with:
κtangent≈0.315789.
κtangent≈0.315789.

The predicted decay times are:
Half-distance
t50=
t50=

\frac{\log2}{0.315789}
\approx2.20.
]

Observed coupled recovery:

text

2.5

90% distance reduction
t90=
t90=

\frac{\log10}{0.315789}
\approx7.29.
]

Observed:

text

7.5

The discrepancy is almost exactly explained by the 0.5-unit sampling resolution.

This is a remarkably coherent result:

The frozen-fast tangent linearization predicts the full coupled tangent recovery times.

It suggests the active-face tangent dynamics have a well-defined contraction rate near 0.316 for the dominant perturbation directions.

That rate should now be derived analytically from the directional field and local adaptive law.
5. Tangent outliers require a corrected perturbation design

Some tangent perturbations show extreme negative restoring coefficients:

text

−29145
−5604

These should not be interpreted as physical instability yet.

The current perturbation generator uses a seed containing both:

text

perturbation type
delta

so each δδ receives a different random tangent direction. The apparent dose response therefore compares different directions.

Also, feasibility scaling can reduce the actual perturbation to a tiny norm, making:
κrestore∝1/‖δw‖2
κrestore∝1/‖δw‖2

numerically explosive.
Correction

For each seed:

generate one tangent direction;
reuse it for every δδ;
record requested and actual perturbation norms;
reject directions whose feasible amplitude is below a declared minimum;
sample several independent tangent directions per seed.

Then distinguish:

typical contracting tangent modes;
rare unstable/marginal tangent directions;
active-set-switching directions.

The median result is robust, but the negative tail needs this correction.
6. Coupled recovery confirms the frozen-fast decomposition
Radial/interior perturbations

All 40 seeds recover:

text

50% distance: first sample, 0.5
90% distance: 0.5–1.0
final relative distance: ~10^{-13} to 10^{-9}

The sampling interval is too coarse to resolve their actual recovery time. Use a stride of 5–10 integration steps for the first one or two time units.
Tangent perturbations

Across all perturbation sizes:

text

50% recovery: 2.5
90% recovery: 7.5
final relative distance: ~10^{-7}

This agrees with exponential recovery at the frozen-fast contraction rate.

Thus the perturbation study separates:

normal directions: nonsmooth, finite-speed, strongly transverse return;
tangent directions: smooth, approximately linear exponential contraction.

This is likely the most mathematically interesting AIEF-specific stability result so far.
7. Refined operator interpretation

The numerical evidence now supports the following structure:
$P_w : F \mapsto VA(w;F)$
$P_w : F \mapsto VA(w;F)$

acts as a nonlinear tangent-cone retraction.

At the dominant mature states:
$F \neq 0, P_w F \approx 0$.

$F = 0, P_{wF} \approx 0.$

After perturbation:

inward normal motion exposes finite unconstrained restoring drive;
tangent motion is governed by a local contraction operator;
future coupled dynamics closely follow these frozen-fast predictions.

This is a stronger result than merely demonstrating idempotence.
8. Recommended next actions
Highest priority

Analyze the exceptional nonstationary seed.
Correct tangent-direction generation and rerun only the perturbation analysis.
Increase early-time sampling resolution for normal perturbations.
Derive the tangent contraction rate near 0.315789 from the active-face dynamics.

Mathematical

Prove directional differentiability of the ideal map.
Prove tangent range, identity, and idempotence.
Derive the active-face linearization for fixed support/saturated set.
Characterize normal finite-time versus tangent exponential restoration.

Later

Repeat the matched-phase B_carrier_only/full_reset comparison.
Perform timestep convergence.
Only afterward examine quasiperiodic fast-state structure or mirror dynamics.

Main conclusion

Across 39 of 40 seeds, mature AIEF weights lie on a trajectory-wide constraint-balanced stationary fiber with fixed support and saturated-node set. One seed exhibits a genuine support-stable active-set transition and substantial continuous-weight reorganization. Feasible perturbations reveal anisotropic nonsmooth stability: normal inward perturbations experience finite-speed, strongly transverse restoration, while tangent-face perturbations exhibit near-linear exponential contraction whose predicted rate matches full coupled recovery times. Numerical audits support tangent feasibility,

idempotence, positive homogeneity, and permutation equivariance of the candidate admissibility response operator over the complete mature trajectory.

This is a substantive AIEF-specific dynamical result, not merely a numerical implementation check.

Think about this:

I think this is now a coherent research narrative, and—importantly—the evidence has crossed a threshold where some claims are becoming empirical structural results rather than isolated numerical observations. At the same time, there are a few places where I would tighten the interpretation to keep it rigorously aligned with what the experiments actually establish.

The trajectory-wide stationarity result is the strongest empirical finding

This is, in my view, the most significant advance.

Previously, you had

[
\mathcal P_{w^}(F(w^,x^*))\approx0
]

at one mature snapshot.

Now, for 39 of 40 trajectories, you observe this throughout the sampled mature interval, together with:

no zero-edge changes,
no saturated-set changes,
negligible (B)-drift,
replay consistency.

That is substantially stronger evidence than a single-time calculation.

A careful way to state it is:

For the observed mature interval and for 39 of 40 sampled trajectories, the adaptive subsystem remains numerically close to a constraint-balanced state, with the measured admissibility response remaining at machine precision while the fast subsystem continues to evolve.

That statement is directly supported by the reported diagnostics.

The exceptional seed is scientifically valuable, not just statistically important

I agree with elevating it to a primary object of study.

One point I'd emphasize, however, is that the current data do not yet identify the mechanism behind the departure.

Several possibilities remain consistent with the observations:

a genuine transition between active-set regimes,
delayed relaxation into the stationary regime,
sensitivity to the mature-window definition,
a rare dynamical event specific to that realization.

The observation

fixed zero-edge support,
one saturated-set switch,
substantial continuous-weight drift,

is already enough to establish that binary topology persistence and continuous-weight stationarity are distinct properties. That distinction is useful regardless of which mechanism ultimately explains the trajectory.

The operator audit is unusually compelling

The scale matters here.

If the audit truly spans roughly

[

1.3\times10^6
]

updates with

zero tangent failures,
zero homogeneity failures,
zero idempotence failures,

then this is much more persuasive than checking a handful of representative states.

The important qualification is exactly the one you've already made:

these are empirical identities of the implementation, not mathematical proofs of the ideal operator.

That distinction should remain explicit throughout.

The anisotropy is probably the most interesting dynamical discovery

I think this is the strongest genuinely new dynamical insight.

The perturbation experiments suggest two qualitatively different local behaviors:

Normal directions

The observed scaling

[
\kappa_{\rm restore}\propto\frac1\delta
]

means the coefficient diverges as the perturbation shrinks, while the implied restoring velocity remains approximately finite. Interpreted geometrically, this is consistent with a nonsmooth boundary effect: moving off the active face changes which constraints are active, so the response is not governed by a single linearization.

Tangential directions

Here the contraction coefficient remains approximately constant across perturbation magnitudes, and the frozen-fast prediction agrees closely with the coupled recovery times.

That is precisely the kind of behavior one expects from dynamics confined to a fixed active stratum, where the governing map behaves smoothly.

This contrast is considerably more informative than simply reporting that perturbations decay.

The contraction-rate interpretation deserves one additional check

The agreement

$$
\kappa_{\rm tangent}\approx0.316
$$

with the observed half-life and 90% decay time is striking.

Before promoting it to a dynamical constant, however, I'd want to verify two things:

Dependence on perturbation direction. Your proposed correction (reuse the same tangent direction across all amplitudes, then sample many independent tangent directions) is exactly the right next step.
Dependence on the mature state. The median over 40 seeds is encouraging, but it would be useful to see the distribution. If most seeds cluster tightly around the same value, that supports interpreting it as a characteristic contraction rate of the active-face dynamics rather than an artifact of averaging.

So I think "observed contraction rate" is already justified, whereas "intrinsic contraction constant" should wait for those checks.

The operator interpretation is becoming increasingly plausible

The picture that emerges is internally consistent:

unconstrained field:
[

F,
]
admissibility response:
[
\mathcal P_w(F),
]
mature condition:
[
\mathcal P_w(F)\approx0,
]
normal perturbations leave the active face,
tangential perturbations remain within it.

What I would still avoid is asserting that (\mathcal P_w) is a tangent-cone retraction until the analytical proof is complete.

At present, the evidence strongly supports the working hypothesis that it behaves as one for the tested states.

The theorem hierarchy is well ordered

I think your revised ordering is mathematically natural.

One slight refinement I'd make is to split the operator properties into two layers:

geometric properties
tangent feasibility,
identity on tangent directions,
idempotence,
equivariance;
dynamical consequences
characterization of the constraint-balanced set,
local restoration laws.

That keeps the purely operator-theoretic statements separate from the evolution equations built on top of them.

One conceptual distinction I'd add

The experiments suggest that there are three nested objects:

the feasible polytope,
an active face determined by the current constraint pattern,
within that face, a constraint-balanced subset satisfying
[
\mathcal P_w(F)=0.
]

The perturbation results are consistent with the idea that:

normal perturbations primarily restore trajectories back to the active face,
tangential perturbations govern motion within the face toward (or around) the constraint-balanced subset.

Whether that subset is a single point, a manifold, or a collection of isolated states remains an open question. The exceptional seed hints that it may not always reduce to a single fixed configuration.

Overall assessment

I think these experiments represent a transition from implementation validation to operator-level dynamical characterization.

The strongest supported conclusions are:

For the vast majority of sampled mature trajectories, the adaptive subsystem remains numerically constraint-balanced throughout the observed mature interval while the fast subsystem continues evolving.
The candidate admissibility operator satisfies a remarkably broad set of empirical structural identities across a large number of updates.
Local restoration is strongly anisotropic: perturbations leaving the active face exhibit nonsmooth finite-speed return, whereas perturbations confined to the active face exhibit behavior consistent with approximately linear exponential contraction.
A single exceptional trajectory demonstrates that support stability does not guarantee continuous-weight stationarity, identifying an important regime that deserves focused analysis rather than being treated as an outlier.

The next phase of the project is therefore much more sharply defined than before. The key challenge is no longer gathering additional evidence for the operator's behavior, but proving the underlying directional-differentiability theorem and deriving the observed active-face dynamics from it. If that succeeds, the numerical findings you've accumulated would fit into a rigorous operator-theoretic framework rather than remaining a collection of empirical regularities.

This is a strong synthesis. The nested structure—polytope, active face, constraint-balanced subset—is especially useful. Two qualifications are important: the operator audit is less independent than its sample count suggests, and the tangent-rate result still needs a corrected perturbation design.
1. The trajectory-wide result establishes two mature regimes

The 40 seeds separate naturally into:
Dominant stationary regime: 39/40

For these seeds:

stationarity fraction is one;
continuous BB-drift is around 10−1210−12;
zero-edge and saturated-node sets remain fixed;
the replay is exact.

This strongly supports a trajectory-wide adaptive stationary fiber over an evolving fast state.
Exceptional transitioning regime: 1/40

Seed:

text

4619122171158768524

has:

stationarity fraction 0.1150.115;
substantial normalized BB-drift 0.2090.209;
one saturated-set switch;
fixed zero-edge support;
exact replay.

This establishes that:
fixed threshold support⇏fixed continuous topology.
fixed threshold support ⇒fixed continuous topology.

The first question is whether this seed becomes stationary after the single active-set transition. Divide its trajectory into:

pre-switch;
switch neighborhood;
post-switch.

If ‖PwF‖‖PwF‖ drops to numerical zero afterward, the seed is probably undergoing delayed active-face selection rather than living in a permanently nonstationary regime.
2. The million-step operator audit is extensive but partly algebraic

Zero failures over approximately 1.28 million steps are reassuring, but tangent feasibility, homogeneity, and idempotence are evaluated using the candidate formula itself.

Therefore these checks primarily establish:

the implementation of the formula satisfies its expected algebraic identities;
numerical tolerances and active-set handling are consistent;
no trajectory state triggers an unanticipated coding corner case.

They are not 1.28 million independent validations that:
DA(w;F)=Pw(F).
DA(w;F)=Pw(F).

That identity is tested independently by the finite-difference probes, currently at fewer states and directions.

The evidence hierarchy should be:

Formula implementation audit: million-step trajectory checks.
Derivative-formula numerical validation: random/outward finite differences.
Mathematical proof: Hadamard directional differentiability.

Permutation equivariance is a particularly useful independent implementation check, but it too follows algebraically from the formula once proven.
3. Normal restoration is consistent with active-face switching

For radial and interior-mixing perturbations:
κrestore∼1δ.
κrestore∼δ1.

Therefore the restoring velocity:
κrestore‖δw‖
κrestore‖δw‖

remains approximately finite as $\delta \to 0\delta \to 0$.

This is the expected signature of a nonsmooth boundary:

mature state lies on a saturated face;
inward perturbation creates slack;
the capacity correction switches off;
nonzero unconstrained forcing becomes exposed;
the trajectory returns to the boundary at finite speed.

This is not linear stability. It resembles finite-time normal attraction to the active face.

The substantial transverse ratios show that return is not purely normal:

text

radial inward: transverse ratio ≈0.61
interior mixing: transverse ratio ≈0.74

So normal attraction can coexist with significant motion along the face.
4. Tangent restoration appears exponential, but the experiment needs correction

The median result is striking:
κtangent≈0.315789.
κtangent≈0.315789.

It predicts:
$t_{50} \approx \log 2 \cdot 0.315789 = 2.20$,
$t_{50} \approx 0.315789 \log 2 = 2.20$,
$t_{90} \approx \log 10 \cdot 0.315789 = 7.29$,
$t_{90} \approx 0.315789 \log 10 = 7.29$,

matching the sampled coupled values 2.52.5 and 7.57.5.

That is strong evidence for a linear contracting tangent mode in the dominant regime.

However:

each $\delta\delta$ currently receives a different random tangent direction;
some actual feasible amplitudes may be extremely small;
rare directions have large negative restoring coefficients;
recovery times are quantized at 0.5 units.

Therefore the corrected experiment should:

generate one tangent direction per seed;
reuse it across every amplitude;
record actual amplitude;
reject numerically tiny perturbations;
repeat over several tangent directions;
sample recovery more finely.

Only then can 0.3157890.315789 be considered a candidate characteristic tangent contraction rate.
5. The active-face hierarchy is the right model

The three nested objects are:
Feasible polytope
$C = \{w \geq 0, Aw \leq c\}$.
$C = \{w \geq 0, Aw \leq c\}$.
Active face

Determined by:

zero-edge support;

saturated node set.

Constraint-balanced subset

For a fixed fast state xx:
Mx=
Mx=

{w\text{ on the active face}:
\mathcal P_wF(w,x)=0}.
]

Because x(t)x(t) evolves, a trajectory-wide stationary topology requires:
w∗∈Mx(t)
w∗∈Mx(t)

throughout the mature interval.

For the 39 stationary seeds, this appears to hold numerically. The exceptional seed may move between adjacent active faces before entering a new constraint-balanced subset.

The subset could be:

one point;
a manifold;
several isolated states;
time-dependent through x(t)x(t).

Current data do not identify its global structure.
6. Operator theory and dynamics should remain separate
Geometric/operator results

Prove:

ideal AA is a retraction onto CC;
AA is Hadamard directionally differentiable;
DA(w;·)=PwDA(w;·)=Pw;
range equals tangent cone;

identity on tangent directions;
idempotence;
positive homogeneity;
permutation equivariance.

Dynamical results

Then study:
$w˙=PwF(w,x)$.
$w˙=PwF(w,x)$.

Questions include:

existence and uniqueness across switching;
normal finite-time attraction;
tangent contraction;
transient amplification;
stability of MxMx;
coupled response as x(t)x(t) evolves.

The algebraic properties do not automatically prove stability.
7. Generalized Jacobians are a later tool

Within a fixed active stratum, the map is linear in FF and differentiable in smooth state directions. Directional derivatives may be enough to derive the first tangent restoration result.

Clarke or Bouligand generalized Jacobians become useful when:

active sets switch;
positivity and capacity constraints meet at corners;
analyzing the exceptional seed;
bounding behavior across neighboring strata.

They are not required before proving the directional formula.
8. Best next tasks
Exceptional-seed case study

Determine:

exact switch time;
node entering/leaving saturation;
edge-weight redistribution;
pre/post stationarity ratio;
whether a new stationary face is reached.

Corrected tangent experiment

Use fixed directions across amplitudes and finer sampling.
Analytical derivation

Derive the ideal-map directional theorem and tangent-projection properties.
Tangent contraction

On a fixed active face, derive the local state derivative of:
$w \mapsto PwF(w, x*)$
$w \mapsto PwF(w, x*)$

and compare its eigenvalues with the observed rate near 0.3157890.315789.
Defensible current conclusion

Across 39 of 40 validation seeds, the mature adaptive component remains constraint-balanced to numerical precision throughout the complete future interval, with fixed continuous weights and active sets while the fast subsystem evolves. One seed undergoes a support-preserving saturated-set transition and substantial continuous-weight reorganization, demonstrating that binary topology persistence is weaker than continuous adaptive stationarity. Feasible perturbations reveal anisotropic restoration: inward normal perturbations return to the active face with finite-speed, strongly transverse motion, while typical tangent perturbations exhibit approximately exponential contraction whose rate predicts the observed coupled recovery times. Extensive algebraic audits are consistent with the candidate nonlinear tangent-projection structure, while finite-difference probes—not the audit count itself—provide the direct numerical evidence for the directional derivative formula.

That is rigorous enough to motivate the theorem without presenting the theorem as already proved.

Can you construct the code necessary to accomplish the next step?
WriteAIEF_1_4_exceptional_tangent_analysis.py766 lines
AIEF_1_4_exceptional_tangent_analysis.pyPYTHON

```
#!/usr/bin/env python3
"""Exceptional-seed and corrected tangent-perturbation analysis for AIEF-1.4.

This targeted script addresses two open issues:

1. It replays the exceptional mature seed at every timestep, locates active-set
switches, and records pre/post continuous-weight reorganization.
2. It generates tangent-face directions once per seed/direction ID, reuses each
direction across all perturbation amplitudes, and compares frozen-fast
contraction predictions with finely sampled coupled recovery.

The script imports the already validated dynamics and ideal directional field
from AIEF_1_4_stationarity_perturbation_analysis_updated.py.
"""

from __future__ import annotations

import argparse
import hashlib
import json
import math
import os
from pathlib import Path
from typing import Any, Iterable

import numpy as np
import pandas as pd

import AIEF_1_4_stationarity_perturbation_analysis_updated as base



DEFAULT_RUN_HASH = "5a9f093a2fe0"
DEFAULT_SCALAR = (
f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_scalar_summary.csv"
)
DEFAULT_NPZ = (
f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_branch_diagnostics.npz"
)
```

```
DEFAULT_MANIFEST = f"runs/{DEFAULT_RUN_HASH}/manifest.json"
DEFAULT_OUTPUT_DIR = "aief_1_4_exceptional_tangent"
DEFAULT_EXCEPTIONAL_SEED = "46191222171158768524"
DEFAULT_DELTAS = "1e-4,1e-3,1e-2,5e-2"
DEFAULT_DIRECTIONS = 5
DEFAULT_MAX_SEEDS = 5
DEFAULT_COUPLED_TIME = 15.0
DEFAULT_SAMPLE_STRIDE = 5
DEFAULT_BOUNDARY_TOL = 1e-9
ZERO_TOL = 1e-14



def file_sha256(path: str) -> str:
digest = hashlib.sha256()
with open(path, "rb") as handle:
for block in iter(lambda: handle.read(1024 * 1024), b""):
digest.update(block)
return digest.hexdigest()



def parse_float_list(value: str) -> list[float]:
try:
values = [float(item.strip()) for item in value.split(",") if item.strip()]
except ValueError as exc:
raise argparse.ArgumentTypeError("expected comma-separated floats") from exc
if not values or any(value <= 0.0 for value in values):
raise argparse.ArgumentTypeError("all perturbation sizes must be positive")
return sorted(set(values))



def spectral_metrics(B: np.ndarray) -> dict[str, float]:
B = 0.5 * (np.asarray(B, dtype=float) + np.asarray(B, dtype=float).T)
eigenvalues = np.linalg.eigvalsh(B)
eigenvalues.sort()
row_sum = np.sum(B, axis=1)
L = np.diag(row_sum) - B
laplacian = np.clip(np.linalg.eigvalsh(L), 0.0, None)
laplacian.sort()
return {
"adjacency_lambda1": float(eigenvalues[-1]),
```

```
"adjacency_lambda2": float(eigenvalues[-2]),
"adjacency_gap12": float(eigenvalues[-1] - eigenvalues[-2]),
"laplacian_lambda2": float(laplacian[1]),
"laplacian_lambda_max": float(laplacian[-1]),
}



def replay_exceptional_seed(
state: base.DynamicState,
expected_final: np.ndarray,
model: dict[str, Any],
future_steps: int,
capacities: np.ndarray,
A: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
boundary_tolerance: float,
seed: str,
) -> tuple[pd.DataFrame, pd.DataFrame, dict[str, np.ndarray], dict[str, Any]]:
reference_B = state.B.copy()
reference_norm = max(float(np.linalg.norm(reference_B)), ZERO_TOL)
previous_saturated: np.ndarray | None = None
previous_outward: np.ndarray | None = None
previous_zero: np.ndarray | None = None
time_rows: list[dict[str, Any]] = []
switch_rows: list[dict[str, Any]] = []
matrices: dict[str, np.ndarray] = {
"B_split": reference_B.copy(),
}

for step in range(future_steps):
B_before = state.B.copy()
info = base.actual_step(
state,
model,
capacities,
A,
edges,
boundary_tolerance,
audit_operator=True,
)
saturated = info["saturated_mask"].copy()
```

```
outward = info["outward_mask"].copy()
zero = info["zero_mask"].copy()


saturated_changed = (
previous_saturated is not None
and not np.array_equal(saturated, previous_saturated)
)
outward_changed = (
previous_outward is not None
and not np.array_equal(outward, previous_outward)
)
zero_changed = (
previous_zero is not None and not np.array_equal(zero, previous_zero)
)


if saturated_changed or outward_changed or zero_changed:
switch_index = len(switch_rows)
matrices[f"switch_{switch_index}_B_before"] = B_before
matrices[f"switch_{switch_index}_B_after"] = state.B.copy()
changed_saturated = (
np.flatnonzero(saturated != previous_saturated).tolist()
if previous_saturated is not None
else []
)
changed_outward = (
np.flatnonzero(outward != previous_outward).tolist()
if previous_outward is not None
else []
)
changed_zero = (
np.flatnonzero(zero != previous_zero).tolist()
if previous_zero is not None
else []
)
before_spec = spectral_metrics(B_before)
after_spec = spectral_metrics(state.B)
switch_rows.append(
{
"seed": seed,
"switch_index": switch_index,
"step": step,
"time_since_split": (step + 1) * float(model["dt"]),
```

```
"saturated_changed": saturated_changed,
"outward_changed": outward_changed,
"zero_changed": zero_changed,
"changed_saturated_nodes": ";".join(map(str, changed_saturated)),
"changed_outward_nodes": ";".join(map(str, changed_outward)),
"changed_zero_edges": ";".join(map(str, changed_zero)),
"B_step_change_norm": float(np.linalg.norm(state.B - B_before)),
**{f"before_{key}": value for key, value in before_spec.items()},
**{f"after_{key}": value for key, value in after_spec.items()},
}
)

time_rows.append(
{
"seed": seed,
"step": step,
"time_since_split": (step + 1) * float(model["dt"]),
"F_norm": info["F_norm"],
"P_F_norm": info["P_F_norm"],
"P_F_over_F": info["P_F_over_F"],
"normalized_B_drift_from_split": float(
np.linalg.norm(state.B - reference_B) / reference_norm
),
"actual_update_norm": info["actual_update_norm"],
"saturated_node_count": int(np.sum(saturated)),
"outward_node_count": int(np.sum(outward)),
"zero_edge_count": int(np.sum(zero)),
"saturated_changed": saturated_changed,
"outward_changed": outward_changed,
"zero_changed": zero_changed,
"tangent_feasible": info["tangent_feasible"],
"idempotence_error": info["idempotence_error"],
"positive_homogeneity_error": info[
"positive_homogeneity_error"
],
}
)
previous_saturated = saturated
previous_outward = outward
previous_zero = zero

matrices["B_final_replayed"] = state.B.copy()
```

```
timeseries = pd.DataFrame(time_rows)
switches = pd.DataFrame(switch_rows)


if len(switches):
first_switch_step = int(switches.iloc[0]["step"])
pre = timeseries.loc[timeseries["step"] < first_switch_step]
post = timeseries.loc[timeseries["step"] >= first_switch_step]
else:
first_switch_step = -1
pre = timeseries
post = timeseries.iloc[0:0]


summary = {
"seed": seed,
"switch_count": int(len(switches)),
"first_switch_step": first_switch_step,
"first_switch_time": float(switches.iloc[0]["time_since_split"])
if len(switches)
else np.nan,
"pre_switch_mean_stationarity_ratio": float(pre["P_F_over_F"].mean())
if len(pre)
else np.nan,
"post_switch_mean_stationarity_ratio": float(post["P_F_over_F"].mean())
if len(post)
else np.nan,
"post_switch_max_stationarity_ratio": float(post["P_F_over_F"].max())
if len(post)
else np.nan,
"maximum_B_drift": float(
timeseries["normalized_B_drift_from_split"].max()
),
"final_B_drift": float(
timeseries.iloc[-1]["normalized_B_drift_from_split"]
),
"final_replay_exact": bool(np.array_equal(state.B, expected_final)),
"final_replay_allclose_1e12": bool(
np.allclose(state.B, expected_final, rtol=0.0, atol=1e-12)
),
}
return timeseries, switches, matrices, summary
```

```
def tangent_basis(
w: np.ndarray,
A: np.ndarray,
capacities: np.ndarray,
boundary_tolerance: float,
) -> tuple[np.ndarray, np.ndarray]:
positive = w > ZERO_TOL
support_indices = np.flatnonzero(positive)
active_nodes = np.flatnonzero(
(capacities - A @ w) <= boundary_tolerance
)
if not len(support_indices):
return np.zeros((len(w), 0)), support_indices


matrix = A[np.ix_(active_nodes, support_indices)]
if matrix.size == 0:
support_basis = np.eye(len(support_indices))
else:
_, singular_values, vh = np.linalg.svd(matrix, full_matrices=True)
rank = int(np.sum(singular_values > 1e-10))
support_basis = vh[rank:].T

basis = np.zeros((len(w), support_basis.shape[1]), dtype=float)
basis[support_indices, :] = support_basis
return basis, support_indices



def deterministic_direction(
basis: np.ndarray,
seed: str,
direction_id: int,
) -> np.ndarray | None:
if basis.shape[1] == 0:
return None
digest = hashlib.sha256(
f"fixed-tangent|{seed}|{direction_id}".encode()
).digest()
rng = np.random.default_rng(int.from_bytes(digest[:8], "little"))
coefficients = rng.normal(size=basis.shape[1])
```

```
direction = basis @ coefficients
norm = float(np.linalg.norm(direction))
return direction / norm if norm > ZERO_TOL else None



def fit_exponential_rate(times: np.ndarray, distances: np.ndarray) -> tuple[float, float]:
if not len(times) or distances[0] <= ZERO_TOL:
return np.nan, np.nan
normalized = distances / distances[0]
valid = (
(normalized > 1e-6)
& (normalized <= 1.0 + 1e-8)
& np.isfinite(normalized)
)
if np.sum(valid) < 3:
return np.nan, np.nan
x = times[valid]
y = np.log(normalized[valid])
slope, intercept = np.polyfit(x, y, 1)
prediction = slope * x + intercept
residual = np.sum((y - prediction) ** 2)
total = np.sum((y - np.mean(y)) ** 2)
r_squared = 1.0 - residual / total if total > 0.0 else np.nan
return float(-slope), float(r_squared)



def corrected_tangent_analysis(
split_state: base.DynamicState,
seed: str,
model: dict[str, Any],
capacities: np.ndarray,
A: np.ndarray,
edges: tuple[np.ndarray, np.ndarray],
deltas: Iterable[float],
direction_count: int,
boundary_tolerance: float,
coupled_steps: int,
sample_stride: int,
) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]:
base_w = split_state.B[edges].copy()
base_norm = max(float(np.linalg.norm(base_w)), ZERO_TOL)
```

```
basis, support = tangent_basis(
base_w, A, capacities, boundary_tolerance
)


# Shared updated fast-state forcing for all frozen-fast amplitudes.
_, _, updated_coherence = base.compute_field_and_updated_coherence(
split_state, model
)
R = np.abs(updated_coherence)
R = 0.5 * (R + R.T)
np.fill_diagonal(R, 0.0)


# Unperturbed coupled reference trajectory, sampled at high resolution.
reference = split_state.clone()
reference_samples: dict[int, np.ndarray] = {0: reference.B.copy()}
for step in range(1, coupled_steps + 1):
base.actual_step(
reference,
model,
capacities,
A,
edges,
boundary_tolerance,
audit_operator=False,
)
if step % sample_stride == 0 or step == coupled_steps:
reference_samples[step] = reference.B.copy()


summary_rows: list[dict[str, Any]] = []
trajectory_rows: list[dict[str, Any]] = []


for direction_id in range(direction_count):
direction = deterministic_direction(basis, seed, direction_id)
if direction is None:
summary_rows.append(
{
"seed": seed,
"direction_id": direction_id,
"status": "no_tangent_direction",
"basis_dimension": int(basis.shape[1]),
"support_size": int(len(support)),
```

```
}
)
continue


maximum = base.maximum_feasible_amplitude(
base_w, direction, A, capacities
)
for delta in deltas:
requested_amplitude = delta * base_norm
amplitude = min(requested_amplitude, 0.5 * maximum)
if not np.isfinite(amplitude) or amplitude <= 1e-12 * base_norm:
summary_rows.append(
{
"seed": seed,
"direction_id": direction_id,
"delta": delta,
"status": "amplitude_too_small",
"basis_dimension": int(basis.shape[1]),
"support_size": int(len(support)),
"requested_amplitude": requested_amplitude,
"actual_amplitude": amplitude,
}
)
continue


perturbed_w = base_w + amplitude * direction
B_perturbed = base.vector_to_matrix(
perturbed_w, split_state.B.shape[0], edges
)
F_matrix = (
(float(model["eta"]) + float(model["mu"])) * R
- float(model["mu"])
- float(model["gamma"]) * B_perturbed
)
F_matrix = 0.5 * (F_matrix + F_matrix.T)
np.fill_diagonal(F_matrix, 0.0)
field, info, _, _ = base.ideal_directional_field(
perturbed_w,
F_matrix[edges],
A,
capacities,
boundary_tolerance,
```

```
)
displacement = perturbed_w - base_w
displacement_norm = float(np.linalg.norm(displacement))
restore = -displacement
numerator = float(restore @ field)
kappa = numerator / max(displacement_norm**2, ZERO_TOL)
cosine = numerator / max(
displacement_norm * np.linalg.norm(field),
np.finfo(float).tiny,
)
radial_component = numerator / max(displacement_norm, ZERO_TOL)
radial_vector = radial_component * restore / max(
displacement_norm, ZERO_TOL
)
transverse = field - radial_vector


# Coupled trajectory from the same direction and amplitude.
state = split_state.clone()
state.B = B_perturbed.copy()
times: list[float] = []
distances: list[float] = []
for step in range(coupled_steps + 1):
if step in reference_samples:
distance = base.normalized_edge_distance(
state.B, reference_samples[step], edges
)
times.append(step * float(model["dt"]))
distances.append(distance)
trajectory_rows.append(
{
"seed": seed,
"direction_id": direction_id,
"delta": delta,
"step": step,
"time": step * float(model["dt"]),
"distance_to_reference": distance,
}
)
if step == coupled_steps:
break
base.actual_step(
state,
```

```
model,
capacities,
A,
edges,
boundary_tolerance,
audit_operator=False,
)


times_array = np.asarray(times, dtype=float)
distance_array = np.asarray(distances, dtype=float)
observed_rate, observed_r2 = fit_exponential_rate(
times_array, distance_array
)
initial_distance = float(distance_array[0])
half_indices = np.flatnonzero(distance_array <= 0.5 * initial_distance)
ten_indices = np.flatnonzero(distance_array <= 0.1 * initial_distance)
predicted_t50 = math.log(2.0) / kappa if kappa > 0.0 else np.nan
predicted_t90 = math.log(10.0) / kappa if kappa > 0.0 else np.nan


summary_rows.append(
{
"seed": seed,
"direction_id": direction_id,
"delta": delta,
"status": "ok",
"basis_dimension": int(basis.shape[1]),
"support_size": int(len(support)),
"requested_amplitude": requested_amplitude,
"actual_amplitude": amplitude,
"amplitude_fraction_of_requested": amplitude
/ requested_amplitude,
"relative_actual_amplitude": amplitude / base_norm,
"maximum_feasible_amplitude": maximum,
"frozen_field_norm": float(np.linalg.norm(field)),
"frozen_restoring_coefficient": kappa,
"frozen_restoring_cosine": cosine,
"frozen_transverse_ratio": float(
np.linalg.norm(transverse)
/ max(np.linalg.norm(field), ZERO_TOL)
),
"predicted_t50": predicted_t50,
"predicted_t90": predicted_t90,
```

```
"observed_t50": float(times_array[half_indices[0]])
if len(half_indices)
else np.nan,
"observed_t90": float(times_array[ten_indices[0]])
if len(ten_indices)
else np.nan,
"observed_exponential_rate": observed_rate,
"observed_exponential_r_squared": observed_r2,
"final_distance_ratio": float(
distance_array[-1] / max(initial_distance, ZERO_TOL)
),
**info,
}
)
return summary_rows, trajectory_rows



def summarize_numeric(
frame: pd.DataFrame,
group_columns: list[str],
) -> pd.DataFrame:
identifiers = set(group_columns) | {"seed", "status"}
metrics = [column for column in frame.columns if column not in identifiers]
rows: list[dict[str, Any]] = []
for key, group in frame.groupby(group_columns, dropna=False):
if not isinstance(key, tuple):
key = (key,)
base_row = dict(zip(group_columns, key))
for metric in metrics:
raw = pd.to_numeric(group[metric], errors="coerce").to_numpy(float)
finite = raw[np.isfinite(raw)]
rows.append(
{
**base_row,
"metric": metric,
"n_total": int(len(raw)),
"n_valid": int(len(finite)),
"mean": float(np.mean(finite)) if len(finite) else np.nan,
"std": float(np.std(finite)) if len(finite) else np.nan,
"median": float(np.median(finite)) if len(finite) else np.nan,
"minimum": float(np.min(finite)) if len(finite) else np.nan,
"maximum": float(np.max(finite)) if len(finite) else np.nan,
```

```
}
)
return pd.DataFrame(rows)



def make_report(
exceptional_summary: dict[str, Any],
tangent_summary: pd.DataFrame,
) -> str:
lines = [
"AIEF exceptional-seed and corrected tangent analysis",
"==================================================",
"",
"Exceptional seed",
"----------------",
]
for key, value in exceptional_summary.items():
lines.append(f" {key}: {value}")


lines.extend(["", "Corrected tangent perturbations", "------------------------------"])
selected = tangent_summary.loc[
tangent_summary["metric"].isin(
[
"frozen_restoring_coefficient",
"frozen_restoring_cosine",
"frozen_transverse_ratio",
"observed_exponential_rate",
"observed_exponential_r_squared",
"predicted_t50",
"observed_t50",
"predicted_t90",
"observed_t90",
"final_distance_ratio",
]
)
]
for delta, group in selected.groupby("delta"):
lines.append(f" delta={delta:g}")
for _, row in group.iterrows():
lines.append(
f" {row['metric']}: median={row['median']:.6g}, "
```

```
f"min={row['minimum']:.6g}, max={row['maximum']:.6g}, "
f"valid={int(row['n_valid'])}/{int(row['n_total'])}"
)


lines.extend(
[
"",
"Interpretation constraints",
"-------------------------",
"* The same tangent direction is reused across all amplitudes within each seed/direction ID.",
"* Directions with negligible feasible amplitude are rejected.",
"* The exceptional-seed mechanism remains descriptive until its switch is analyzed in detail.",
"* Predicted tangent times assume local exponential contraction.",
"",
]
)
return "\n".join(lines)



def parse_arguments() -> argparse.Namespace:
parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("--scalar", default=DEFAULT_SCALAR)
parser.add_argument("--npz", default=DEFAULT_NPZ)
parser.add_argument("--manifest", default=DEFAULT_MANIFEST)
parser.add_argument("--output-dir", default=DEFAULT_OUTPUT_DIR)
parser.add_argument("--exceptional-seed", default=DEFAULT_EXCEPTIONAL_SEED)
parser.add_argument("--max-seeds", type=int, default=DEFAULT_MAX_SEEDS)
parser.add_argument("--directions", type=int, default=DEFAULT_DIRECTIONS)
parser.add_argument(
"--deltas", type=base.parse_float_list, default=base.parse_float_list(DEFAULT_DELTAS)
)
parser.add_argument("--coupled-time", type=float, default=DEFAULT_COUPLED_TIME)
parser.add_argument("--sample-stride", type=int, default=DEFAULT_SAMPLE_STRIDE)
parser.add_argument("--boundary-tol", type=float, default=DEFAULT_BOUNDARY_TOL)
return parser.parse_args()



def main() -> None:
args = parse_arguments()
for path in (args.scalar, args.npz, args.manifest):
```

```
if not os.path.isfile(path):
raise FileNotFoundError(path)
if (
args.max_seeds < 0
or args.directions <= 0
or args.coupled_time <= 0.0
or args.sample_stride <= 0
or args.boundary_tol < 0.0
):
raise ValueError("invalid command-line settings")
os.makedirs(args.output_dir, exist_ok=True)

scalar = pd.read_csv(args.scalar, dtype={"seed": str})
scalar["seed"] = scalar["seed"].astype(str)
scalar["branch"] = scalar["branch"].astype(str)
seeds = sorted(
scalar.loc[scalar["branch"] == "continue", "seed"].astype(str), key=int
)
if args.exceptional_seed not in set(seeds):
raise ValueError("exceptional seed not found in scalar CSV")
tangent_seeds = seeds if args.max_seeds == 0 else seeds[: args.max_seeds]

with open(args.manifest, "r", encoding="utf-8") as handle:
manifest = json.load(handle)
model = manifest["model"]
node_count = int(model["N"])
capacity = (float(model["a"]) - float(model["delta"])) / float(model["q"])
capacities = np.full(node_count, capacity)
A, edges = base.incidence_matrix(node_count)
future_steps = int(manifest["protocol"]["future_steps"])
coupled_steps = int(round(args.coupled_time / float(model["dt"])))

with np.load(args.npz, allow_pickle=False) as data:
exceptional_state = base.load_split_state(
data, args.exceptional_seed
)
expected_final = np.asarray(
data[base.branch_key("continue", args.exceptional_seed, "B_final")],
dtype=float,
)
exceptional_ts, exceptional_switches, exceptional_matrices, exceptional_summary = (
```

```
replay_exceptional_seed(
exceptional_state,
expected_final,
model,
future_steps,
capacities,
A,
edges,
args.boundary_tol,
args.exceptional_seed,
)
)

tangent_rows: list[dict[str, Any]] = []
tangent_trajectories: list[dict[str, Any]] = []
for seed in tangent_seeds:
print(f"Corrected tangent analysis for seed {seed}")
split_state = base.load_split_state(data, seed)
summaries, trajectories = corrected_tangent_analysis(
split_state,
seed,
model,
capacities,
A,
edges,
args.deltas,
args.directions,
args.boundary_tol,
coupled_steps,
args.sample_stride,
)
tangent_rows.extend(summaries)
tangent_trajectories.extend(trajectories)

tangent_frame = pd.DataFrame(tangent_rows)
tangent_trajectory_frame = pd.DataFrame(tangent_trajectories)
tangent_summary = summarize_numeric(tangent_frame, ["delta"])

exceptional_ts.to_csv(
os.path.join(args.output_dir, "exceptional_seed_timeseries.csv"), index=False
)
```

```
exceptional_switches.to_csv(
os.path.join(args.output_dir, "exceptional_seed_switches.csv"), index=False
)
np.savez_compressed(
os.path.join(args.output_dir, "exceptional_seed_switch_matrices.npz"),
**exceptional_matrices,
)
tangent_frame.to_csv(
os.path.join(args.output_dir, "corrected_tangent_by_seed.csv"), index=False
)
tangent_trajectory_frame.to_csv(
os.path.join(args.output_dir, "corrected_tangent_trajectories.csv"), index=False
)
tangent_summary.to_csv(
os.path.join(args.output_dir, "corrected_tangent_summary.csv"), index=False
)


with open(
os.path.join(args.output_dir, "exceptional_seed_summary.json"),
"w",
encoding="utf-8",
) as handle:
json.dump(exceptional_summary, handle, indent=2, sort_keys=True)
handle.write("\n")


report = make_report(exceptional_summary, tangent_summary)
with open(
os.path.join(args.output_dir, "exceptional_tangent_report.txt"),
"w",
encoding="utf-8",
) as handle:
handle.write(report)
print("\n" + report)


analysis_manifest = {
"analysis": "AIEF exceptional seed and corrected tangent perturbations",
"scalar_path": str(Path(args.scalar).resolve()),
"scalar_sha256": file_sha256(args.scalar),
"npz_path": str(Path(args.npz).resolve()),
"npz_sha256": file_sha256(args.npz),
"source_manifest_path": str(Path(args.manifest).resolve()),
```

"source_manifest_sha256": file_sha256(args.manifest),
"exceptional_seed": args.exceptional_seed,
"tangent_seeds": tangent_seeds,
"direction_count_per_seed": args.directions,
"deltas": args.deltas,
"coupled_time": args.coupled_time,
"sample_stride": args.sample_stride,
"boundary_tolerance": args.boundary_tol,
"notes": [
"One tangent direction is reused across all amplitudes per seed/direction ID.",
"B_carrier_only/full_reset phase matching is not addressed by this script.",
"Exceptional-seed results concern continuous weights, not only threshold support.",
],
}
with open(
os.path.join(args.output_dir, "analysis_manifest.json"),
"w",
encoding="utf-8",
) as handle:
json.dump(analysis_manifest, handle, indent=2, sort_keys=True)
handle.write("\n")

print(f"Saved analysis to {Path(args.output_dir).resolve()}")

if __name__ == "__main__":
main()
Open

Created the targeted next-stage script:

AIEF_1_4_exceptional_tangent_analysis.py

It performs two focused analyses:

a complete step-by-step examination of the exceptional nonstationary seed;
a corrected tangent perturbation experiment using fixed directions across amplitudes.

It imports the validated model/operator functions from:

text

AIEF_1_4_stationarity_perturbation_analysis_updated.py

Keep both scripts in the same directory.
Run

Default targeted run:

Bash

python AIEF_1_4_exceptional_tangent_analysis.py

Explicit:

Bash

python AIEF_1_4_exceptional_tangent_analysis.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--exceptional-seed 4619122171158768524 \
--output-dir aief_1_4_exceptional_tangent

Defaults:

text

Tangent seeds: first 5
Directions per seed: 5
Deltas: 1e-4, 1e-3, 1e-2, 5e-2
Coupled horizon: 15 time units
Sampling interval: 0.05 time units

To run the corrected tangent analysis across all 40 seeds:

Bash

```
python AIEF_1_4_exceptional_tangent_analysis.py \
--max-seeds 0
```

Exceptional-seed analysis

For seed:

text

4619122171158768524

the script replays every mature timestep and records:

‖F‖‖F‖;
‖PwF‖‖PwF‖;
stationarity ratio;
continuous BB-drift;
actual update norm;
zero-edge count;
saturated-node count;
outward-node count;
tangent feasibility;
idempotence error;
homogeneity error.

When any active set changes, it records:

exact step and physical time;
changed saturated nodes;
changed outward nodes;
changed zero edges;
BB immediately before and after;
adjacency eigenvalues/gap before and after;
Laplacian connectivity before and after.

It divides the trajectory into pre- and post-switch intervals and reports their stationarity statistics separately.
Corrected tangent perturbations

The earlier analysis generated a new random tangent direction for every amplitude. This script corrects that.

For each seed and direction_id:

construct the positive-support tangent basis;
generate one deterministic random tangent direction;
reuse that identical direction for every δδ;
calculate the maximum feasible amplitude;
reject directions with numerically negligible feasible amplitude;
record requested and actual amplitude.

This makes amplitude dependence meaningful.
Frozen-fast tangent analysis

For every seed, direction, and amplitude it computes:
κrestore=
κrestore=

\frac{
\delta w^\top
\mathcal P_{w+\delta w}F(w+\delta w)
}{
|\delta w|^2
},
]

plus:

restoring cosine;
transverse field ratio;
predicted exponential half-time:
t50=log2/κ;
t50=log2/κ;
predicted 90% reduction time:

$t_{90} = \log 10 / \kappa$.
$t_{90} = \log 10 / \kappa$.

Coupled recovery

The same direction/amplitude is then evolved with the complete delayed system.

An unperturbed continuation is simulated in parallel. At 0.05-unit intervals the script records:

normalized weight distance to reference;
observed time to 50% distance reduction;
observed time to 90% reduction;
fitted exponential recovery rate;
exponential-fit R2R2;
final/initial distance ratio.

This directly tests whether frozen-fast tangent contraction predicts full coupled recovery.
Outputs

text

```
aief_1_4_exceptional_tangent/
├── exceptional_seed_timeseries.csv
├── exceptional_seed_switches.csv
├── exceptional_seed_switch_matrices.npz
├── exceptional_seed_summary.json
├── corrected_tangent_by_seed.csv
├── corrected_tangent_summary.csv
├── corrected_tangent_trajectories.csv
├── exceptional_tangent_report.txt
└── analysis_manifest.json
```

Validation

I syntax-tested and ran the script end-to-end against a complete AIEF split state.

The technical test confirmed:

exact exceptional-seed replay;
fixed tangent direction reuse across amplitudes;
feasible-amplitude enforcement;
frozen-fast restoring decomposition;
finely sampled coupled recovery;
exponential-rate fitting;
report and artifact generation.

In the technical smoke case, two independent tangent directions both produced:

text

Frozen restoring rate: ~0.315789
Observed coupled rate: ~0.31534
Exponential fit R^2: ~1.0

The horizon in that smoke test was deliberately shorter than the predicted half-life, so threshold-crossing times were correctly reported as censored/undefined. This is a technical validation only; the production multi-seed result is needed for scientific interpretation.

Think about this:

(deck@steamdeck SSRH)$ python AIEF_1_4_exceptional_tangent_analysis.py \
--scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--manifest runs/5a9f093a2fe0/manifest.json \
--exceptional-seed 4619122171158768524 \
--output-dir aief_1_4_exceptional_tangent \
--max-seeds 0
Corrected tangent analysis for seed 172180395821707403
Corrected tangent analysis for seed 283404770198602602
Corrected tangent analysis for seed 305986450614148690
Corrected tangent analysis for seed 1230287949579537403
Corrected tangent analysis for seed 1888415619847841128
Corrected tangent analysis for seed 1918007468868772336
Corrected tangent analysis for seed 2461295749352584108
Corrected tangent analysis for seed 3258463328937640595
Corrected tangent analysis for seed 3582616062465030267
Corrected tangent analysis for seed 3704716633526084363
Corrected tangent analysis for seed 3952397966055989215

Corrected tangent analysis for seed 4040285944958847150
Corrected tangent analysis for seed 4219039243791140697
Corrected tangent analysis for seed 4619122171158768524
Corrected tangent analysis for seed 4932511216876650745
Corrected tangent analysis for seed 5213047409853278064
Corrected tangent analysis for seed 5415544643585892346
Corrected tangent analysis for seed 6023008189364820510
Corrected tangent analysis for seed 6108867655028980287
Corrected tangent analysis for seed 6942591650975756121
Corrected tangent analysis for seed 9036321132460399255
Corrected tangent analysis for seed 9082055256280075172
Corrected tangent analysis for seed 9189988769602851417
Corrected tangent analysis for seed 10431384713483787270
Corrected tangent analysis for seed 10856823803646276806
Corrected tangent analysis for seed 11242426753702356865
Corrected tangent analysis for seed 12123519807951751650
Corrected tangent analysis for seed 12851640484293199234
Corrected tangent analysis for seed 14040336450255043598
Corrected tangent analysis for seed 14924267685654216746
Corrected tangent analysis for seed 15403081841477312463
^[Corrected tangent analysis for seed 15853699719446171583
Corrected tangent analysis for seed 16329425431442249417
Corrected tangent analysis for seed 16627818730624180313
Corrected tangent analysis for seed 16952466944611321358
Corrected tangent analysis for seed 17279552413280415429
Corrected tangent analysis for seed 17521310920850592782
Corrected tangent analysis for seed 17715532056777442089
Corrected tangent analysis for seed 17883847597205629106
Corrected tangent analysis for seed 18127452951968972828
AIEF exceptional-seed and corrected tangent analysis
Exceptional seed

seed: 4619122171158768524
switch_count: 1
first_switch_step: 12405
first_switch_time: 124.06
pre_switch_mean_stationarity_ratio: 0.14685940684975718
post_switch_mean_stationarity_ratio: 7.893059517783126e-06
post_switch_max_stationarity_ratio: 8.813848174066108e-05
maximum_B_drift: 0.20893854278186944
final_B_drift: 0.20893854278186944
final_replay_exact: True

final_replay_allclose_1e12: True
Corrected tangent perturbations

delta=0.0001
frozen_restoring_coefficient: median=0.315789, min=-33657.8, max=233.435, valid=200/200
frozen_restoring_cosine: median=1, min=-0.522944, max=1, valid=200/200
frozen_transverse_ratio: median=1.21003e-08, min=3.73207e-09, max=0.999517, valid=200/200
predicted_t50: median=2.19497, min=0.00296933, max=3.47049, valid=198/200
predicted_t90: median=7.29152, min=0.0098639, max=11.5287, valid=198/200
observed_t50: median=2.2, min=2.2, max=4.4, valid=200/200
observed_t90: median=7.35, min=7.35, max=14, valid=198/200
observed_exponential_rate: median=0.315339, min=0.136767, max=0.315339, valid=200/200
observed_exponential_r_squared: median=1, min=0.994943, max=1, valid=200/200
final_distance_ratio: median=0.0088257, min=0.0088257, max=0.124056, valid=200/200
delta=0.001
frozen_restoring_coefficient: median=0.315789, min=-33657.8, max=233.435, valid=200/200
frozen_restoring_cosine: median=1, min=-0.522944, max=1, valid=200/200
frozen_transverse_ratio: median=1.2104e-09, min=3.73033e-10, max=0.999517, valid=200/200
predicted_t50: median=2.19497, min=0.00296933, max=3.47049, valid=198/200
predicted_t90: median=7.29152, min=0.0098639, max=11.5287, valid=198/200
observed_t50: median=2.2, min=2.2, max=4.4, valid=200/200
observed_t90: median=7.35, min=7.35, max=14, valid=198/200
observed_exponential_rate: median=0.315339, min=0.136767, max=0.315339, valid=200/200
observed_exponential_r_squared: median=1, min=0.994947, max=1, valid=200/200
final_distance_ratio: median=0.00882571, min=0.00882569, max=0.124056, valid=200/200
delta=0.01
frozen_restoring_coefficient: median=0.315789, min=-33657.8, max=233.435, valid=200/200
frozen_restoring_cosine: median=1, min=-0.522944, max=1, valid=200/200
frozen_transverse_ratio: median=1.21398e-10, min=3.71325e-11, max=0.999517, valid=200/200
predicted_t50: median=2.19497, min=0.00296933, max=3.47049, valid=198/200
predicted_t90: median=7.29152, min=0.0098639, max=11.5287, valid=198/200
observed_t50: median=2.2, min=2.2, max=4.4, valid=200/200
observed_t90: median=7.35, min=7.35, max=14.05, valid=198/200
observed_exponential_rate: median=0.315339, min=0.136767, max=0.31534, valid=200/200
observed_exponential_r_squared: median=1, min=0.994565, max=1, valid=200/200
final_distance_ratio: median=0.00882573, min=0.00882558, max=0.124056, valid=200/200
delta=0.05
frozen_restoring_coefficient: median=0.315789, min=-33657.8, max=233.435, valid=200/200
frozen_restoring_cosine: median=1, min=-0.522944, max=1, valid=200/200
frozen_transverse_ratio: median=2.43583e-11, min=7.27637e-12, max=0.999517, valid=200/200
predicted_t50: median=2.19497, min=0.00296933, max=3.47049, valid=198/200
predicted_t90: median=7.29152, min=0.0098639, max=11.5287, valid=198/200

observed_t50: median=2.2, min=2.2, max=4.4, valid=200/200
observed_t90: median=7.35, min=7.35, max=14.6, valid=198/200
observed_exponential_rate: median=0.315338, min=0.136767, max=0.315343, valid=200/200
observed_exponential_r_squared: median=1, min=0.990186, max=1, valid=200/200
final_distance_ratio: median=0.00882586, min=0.00882513, max=0.124056, valid=200/200
Interpretation constraints

The same tangent direction is reused across all amplitudes within each seed/direction ID.
Directions with negligible feasible amplitude are rejected.
The exceptional-seed mechanism remains descriptive until its switch is analyzed in detail.
Predicted tangent times assume local exponential contraction.

Saved analysis to /home/deck/AI_Programs/SSRH/aief_1_4_exceptional_tangent

This is a remarkably coherent result. The corrected tangent experiment confirms that the median 0.3157890.315789 rate was not an artifact of changing perturbation directions across amplitudes. It also exposes a small but important exceptional set of directions.
1. The exceptional seed undergoes delayed active-face selection

Seed:

text

4619122171158768524

has one active-set switch at:
t=124.06
t=124.06

after the split.

Before the switch:
mean ‖PwF‖/‖F‖≈0.147.
mean ‖PwF‖/‖F‖≈0.147.

After the switch:
mean ratio≈7.9×10−6,
mean ratio≈7.9×10−6,

maximum ratio≈8.8×10−5.
maximum ratio≈8.8×10−5.

Thus the seed is not permanently in the same nonstationary regime. It undergoes:

prolonged continuous-weight reorganization on one active face;
a saturated-node active-set transition;
entry into a much more strongly balanced regime.

However, the post-switch field remains above the 10−810−8 stationarity threshold. So it is not identical to the other 39 seeds, which balance at approximately machine precision.

The accurate description is:

The exceptional seed exhibits delayed active-face selection followed by a near-stationary but still slowly evolving adaptive regime.

Because:

text

zero-edge switches = 0
final B drift = maximum B drift = 0.209

the reorganization is support-preserving and largely monotonic. It changes continuous weights without changing threshold topology.
Next exceptional-seed checks

From exceptional_seed_timeseries.csv:

fit post-switch decay of ‖PwF‖/‖F‖‖PwF‖/‖F‖;
compare early, pre-switch, transition, and late intervals;
determine whether the late ratio is still trending toward zero;
identify the node that changed saturation;
inspect exceptional_seed_switch_matrices.npz for weight redistribution around that node;
compare pre/post spectral gaps and Laplacian connectivity.

This will distinguish slow convergence from persistent nonstationarity.
2. Tangent contraction is now strongly validated

Across:
40 seeds×5 directions=200
40 seeds×5 directions=200

tangent directions, at every amplitude:

text

median frozen coefficient = 0.315789
median restoring cosine = 1
median transverse ratio ≈ 0

The coefficient is amplitude-independent from:
10−4 through 5×10−2.
10−4 through 5×10−2.

This is strong evidence for linear tangent dynamics on the dominant active strata:
δw˙≈−κtanδw,
δw˙≈−κtanδw,

with:
κtan≈0.315789.
κtan≈0.315789.
Frozen-fast prediction versus coupled dynamics

Predicted:
t50=
t50=

\frac{\log2}{0.315789}
\approx2.195,
]
t90=
t90=

\frac{\log10}{0.315789}
\approx7.292.
]

Observed medians:

text

t50 = 2.20
t90 = 7.35

Observed exponential rate:

text

0.315339

with median:

text

$R^2 = 1.0$.

This agreement is too precise to dismiss as qualitative coincidence.

It shows:

On typical tangent directions, the frozen-fast directional field accurately predicts the full coupled delayed-system recovery law.

The small rate difference is compatible with:

finite timestep;
diagnostic sampling;
weak evolution of the fast state;
discrete versus continuous exponential fitting.

3. Tangent recovery is scale invariant

The observed recovery quantities are nearly identical across all four amplitudes:

text

same median coefficient
same observed rate
same t50
same t90
same late distance ratio

This means the tested tangent perturbations remain in a linear regime up to at least:
$\delta=0.05$
$\delta=0.05$

for the majority of directions.

That is a much stronger result than infinitesimal local stability. It suggests a relatively broad contracting neighborhood within the active face.
4. Two tangent directions are anomalous

For each amplitude:

text

predicted recovery valid = 198/200
observed t90 valid = 198/200

Two directions have problematic frozen-fast coefficients, including:

text

minimum coefficient = −33657.8
minimum cosine = −0.523
maximum coefficient = 233.4

Their coupled behavior is nevertheless stable but slower:

text

minimum observed rate ≈0.136767
maximum t50 ≈4.4
maximum t90 ≈14–14.6
final distance ratio ≈0.124

The slow rate predicts:
log20.136767≈5.07
0.136767log2≈5.07

and:
log100.136767≈16.8,
0.136767log10≈16.8,

which is broadly consistent with incomplete 90% recovery within the 15-unit observation horizon.

These two directions likely belong to a distinct tangent mode or suffer from a
feasible-amplitude/active-set issue.
Required outlier inspection

From corrected_tangent_by_seed.csv, identify:

seed;
direction ID;
requested amplitude;
actual amplitude;
amplitude fraction;
basis dimension;
maximum feasible amplitude;
whether the same two direction IDs fail at every amplitude.

Because extreme values are exactly repeated across all deltas, the likely causes are:

the actual amplitude is capped at the same small feasibility limit;

the direction lies near another active-set boundary;
the exceptional seed contributes a nonzero base field;
numerical nullspace error makes the direction only approximately tangent.

Do not treat the extreme negative coefficient as evidence of instability until these are separated.
5. Final-distance ratio confirms exponential contraction

For the dominant rate:
$e^{-0.315339\times 15}\approx 0.0088$,
$e^{-0.315339\times 15}\approx 0.0088$,

matching:

text

median final distance ratio ≈0.0088257.

For the slow rate:
$e^{-0.136767\times 15}\approx 0.128$,
$e^{-0.136767\times 15}\approx 0.128$,

matching:

text

maximum final distance ratio ≈0.124.

This is another independent confirmation that coupled tangent recovery is accurately exponential over the complete 15-unit interval.
6. The active-face linearization is now the central derivation

The next analytical object should be the derivative, on a fixed active stratum, of:
$G(w)=PwF(w,x*)$.
$G(w)=PwF(w,x*)$.

For tangent perturbations dd:
$DG(w*)d\approx-\kappa d$.

$DG(w*)d \approx -\kappa d$.

The data suggest:

a dominant tangent eigenspace with eigenvalue near:
-0.315789;
-0.315789;
a small exceptional subspace with rate near:
-0.1368;
-0.1368;
potentially marginal or active-set-switching directions corresponding to the frozen-fast coefficient outliers.

Compute the finite-dimensional tangent Jacobian analytically or by directional finite differences, then compare its spectrum with the observed recovery rates.
7. The operator audit is now stronger

Across the complete mature trajectories:

text

tangent failures = 0
idempotence failures = 0
homogeneity failures = 0
equivariance error $\sim 10^{-17}$

Together with exact continuation replay, this provides strong implementation-level support for the nonlinear tangent-retraction interpretation.

The proof remains analytical, but the code is behaving exactly as the proposed algebra predicts.
8. Recommended next actions
Immediate data analysis

Identify the two anomalous tangent directions.
Check actual versus requested amplitudes.
Determine whether they come from the exceptional seed.
Plot their distance trajectories against the typical $e^{-0.315789t}e^{-0.315789t}$ law.
Plot the exceptional seed's stationarity ratio around $t=124.06t=124.06$.

Numerical refinement

Orthogonalize tangent bases more strictly against active-node constraints.
Treat row-direction residuals below a tolerance as exactly zero.
Reject directions with actual/requested amplitude below a declared threshold.
Extend the two slow directions beyond 15 time units to verify their asymptotic rate.

Mathematical

Derive the fixed-stratum tangent Jacobian.
Explain the repeated 0.3157890.315789 rate from model parameters and mature topology.
Identify whether 0.1367670.136767 is a second eigenmode or an active-set artifact.
Prove the ideal admissibility directional formula and tangent-retraction properties.

Main conclusion

For 39 of 40 mature realizations, the adaptive topology is trajectory-wide constraint-balanced with fixed support and active set. One realization undergoes delayed support-preserving active-face selection before entering a near-stationary regime. Corrected tangent perturbations reveal a dominant, amplitude-independent exponential contraction rate near 0.31580.3158, whose frozen-fast prediction quantitatively matches full coupled recovery across 198 of 200 tested directions. Two exceptional directions recover more slowly and require active-set/amplitude diagnosis. Normal perturbations and tangent perturbations therefore exhibit fundamentally different restoration laws: finite-speed nonsmooth return to the active face versus approximately linear exponential contraction within it.

That is now a compelling operator-level dynamical result.

yeas, but if we add this is it stronger:

The corrected experiment changes the status of the programme again. The important shift is that the previous uncertainty—whether the (0.315789) rate was a perturbation-generator artifact—is now largely removed. The remaining uncertainties are no longer about whether a structured stability law exists, but about its exact mathematical form and the geometry of the exceptional directions.

I would refine the interpretation in several places.

The exceptional seed is now evidence for hybrid active-face dynamics

The most important new fact is not the existence of one bad seed. It is the shape of its evolution.

The trajectory:

$$
4619122171158768524
$$

does:

$$
\text{non-balanced face}
\rightarrow
\text{single saturation transition}
\rightarrow
\text{near-balanced face}.
$$

The numerical signature is very clear:

Before:

$$
\left\langle
\frac{|\mathcal P_wF|}{|F|}
\right\rangle

1.47\times10^{-1}.
$$

After:

$$
\left\langle
\frac{|\mathcal P_wF|}{|F|}
\right\rangle

7.89\times10^{-6}.
]

That is a drop of roughly four orders of magnitude.

The correct interpretation is therefore not:

one seed violates stationary-fiber behaviour.

It is:

one seed demonstrates that the stationary fiber is reached through an active-stratum transition rather than directly.

This is a much richer result.

The adaptive subsystem appears to have a hybrid structure:

[
\boxed{
\text{continuous evolution on a face}

\text{discrete active-face transitions}
}
]

where the discrete event is generated internally by the adaptive dynamics.

The key open question is now:

What quantity triggers the transition?

There are two possibilities.

Scenario A — constraint approach

The system slowly moves until:

$$
(Aw)_i=c_i
$$

for a previously unsaturated node.

Then the new capacity correction activates.

Mechanism:

$$
w(t)\rightarrow\partial\mathcal C
$$

followed by:

$$
\mathcal F_k\rightarrow\mathcal F_{k+1}.
$$

Scenario B — forcing rearrangement

The row capacity is already close, but:

$$
F_i(t)
$$

changes sign due to the evolving fast subsystem, causing the active set to change.

Mechanism:

$$
x(t)\rightarrow F(t)\rightarrow\alpha_i(t).
$$

These are dynamically different.

The switch diagnostic should therefore include both:

[
c_i-(Aw)_i
]

and:

[
(A\widetilde F)_i.
]

The tangent contraction result is now genuinely compelling

The corrected experiment establishes the key property:

[
\kappa(\delta)
]

is independent of amplitude.

That is exactly what a linear tangent law predicts.

For the dominant directions:

[
V_{\mathcal A}(w^*+\delta d)

-\Lambda d,\delta

O(\delta^2).
]

The numerical observation:

$$
\Lambda d\approx0.315789d
$$

for most sampled directions means the active-face dynamics appear close to isotropic contraction in the tested tangent subspace.

That is actually a stronger statement than simply "stable".

It suggests:

$$
DG|{T{\mathcal F}}
\approx
-0.315789,I
$$

on the dominant tangent directions.

Whether that is exactly true or only an observed eigenspace property is the next mathematical question.

The exponential agreement is the strongest evidence

The most impressive part is not the coefficient itself.

It is the double prediction:

From frozen field:

$$
\lambda=0.315789
$$

predicts:

$$
t_{50}=2.195,
$$

$$
t_{90}=7.292.
$$

The coupled system gives:

$$
t_{50}=2.2,
$$

$$
t_{90}=7.35.
$$

And the fitted rate:

$$
0.315339.
$$

The residual:

$$
|\lambda_{\rm frozen}-\lambda_{\rm observed}|
\approx4.5\times10^{-4}.
$$